Prezentarea cursului

Sisteme Concurente și Distribuite



Comunicarea

- Introducere
  - Comunicari sincrone si asincrone, tranzitorii si persistente
- Comunicarea la distanta intre procese
  - RPC si RMI
  - SOAP (in context SOA)



Arhitecturi Orientate pe Servicii

- SOA – Service-Oriented Archtecture
- Descrierea serviciilor, WSDL (Web Services Description Language)
- Publicarea si cautarea serviciilor, UDDI (Universal Description Discovery and Integration)
- Utilizarea serviciilor, WSDL - JAX-RPC (Java API for XML-Based RPC)
- Microservicii – fundamente – FaaS

Consistenţa si replicarea datelor

- Modele de consistenţă
  - centrate pe date.
  - centrate pe client.
- Gestiunea replicilor
  - Plasarea replicilor.
  - Propagarea actualizărilor.
  - Protocoale de consistenţă.
- Replicare activa
- Protocoale bazate pe cvorum
- Exemple: CODA, Bayou

Toleranţa la defectări

- Concepte de baza
  - sisteme de incredere
- Comunicarea fiabila client-server
- Grupuri tolerante la defectări.
  - Multicast fiabil si atomic
  - Tranzactii fiabile
- Recuperarea defectelor.
  - Checkpointing.
  - Logarea Mesajelor.



Securitatea sistemelor distribuite

- Securitatea in sisteme distribuite
- Construirea unui sistem distribuit sigur
  - Specificarea politicii
  - Proiectarea solutiei si alegerea mecanismelor
  - Evaluarea
- Autentificarea in SD
  - radius, Secure RPC, Kerberos

Securitatea sistemelor distribuite (2)

- Controlul accesului
  - Modele de securitate multinivel (Bell – LaPadulla, BIBA)
  - Modelul matricei de acces si derivate (Graham – Denning, Harrison-Ruzzo-Ullman, Take-Grant)
  - RBAC si variante (TBAC, TMAC, context-aware)
- Gestiunea cheilor pentru grupuri
  - protocoale pentru control centralizat, descentralizat si distribuit
  - delegarea

Retele P2P

- Sisteme P2P pentru distributie de continut
- Localizarea obiectelor si dirijarea
- Arhitecturi nestructurate
  - Hibrid descentralizate - Napster
  - Pur descentralizate - Gnutella
  - Partial centralizate - Kazaa
- Arhitecturi structurate
  - CAN
  - FreeNet
  - Chord

Cloud

- Caracteristici
- Clase de servicii
- Nivelul infrastructurii (IaaS)
  - Virtualizare
  - Provizionarea dinamica a resurselor
- Nivelul platformei (PaaS)
  - Sisteme de fisiere distribuite
  - Depozite de date
  - Framework-uri pentu aplicatii – MapReduce

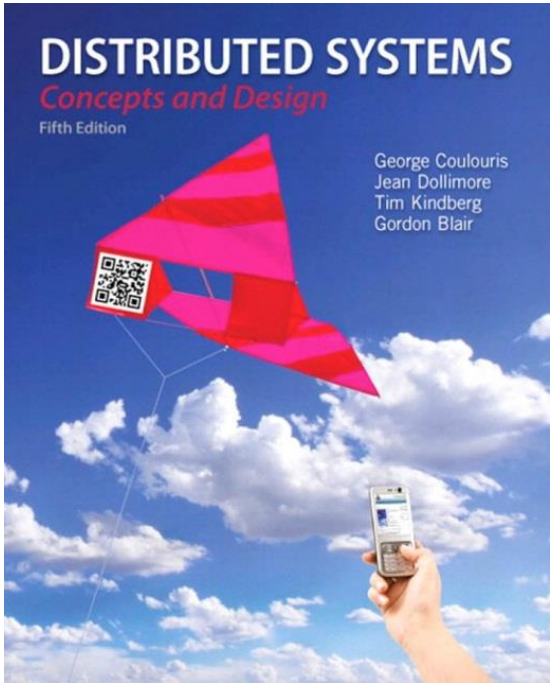
Sisteme Concurente și Distribuite

Introducere

Prima definitie

Un sistem distribuit este unul in care componente hardware si software localizate in calculatoare (diferite) conectate in retea comunica si isi coordoneaza actiunile doar prin transfer de mesaje.

Felul in care sistemele distribuite sunt construite depinde de *cerintele* aplicatiilor



DISTRIBUTED SYSTEMS. Concepts and Design, Fifth Edition
George Coulouris, Jean Dollimore, Tim Kindberg, Gordon Blair

WWW - Cerinte

Functie: Descarcare si prezentare informatii din pagini Web
Continut pagini - texte si multimedia

Cerinte

•Interfata utilizator simpla si intuitiva (usor de folosit)

- Scalabilitate - numar mare de utilizatori si de pagini

•Disponibilitate – functionare corecta chiar in prezenta *defectelor*

- Performanta rezonabila: timp de raspuns in limite acceptabile pentru perceptia umana (0.1 sec – 1 sec)

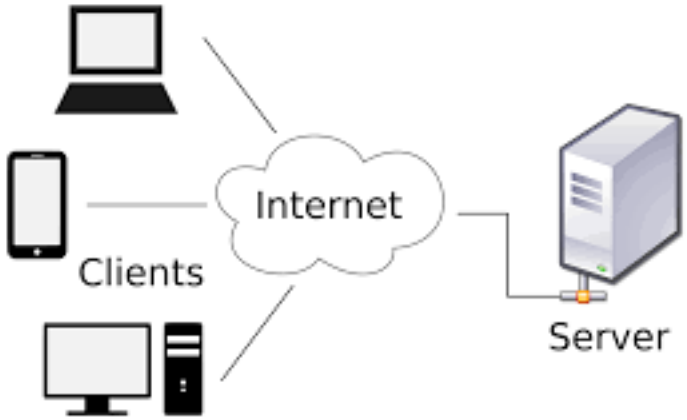
•Confidentialitatea si integritatea datelor

Soluția

Sistem distribuit cu doua tipuri de componente

clientul (browser) - interfata cu utilizatorul,

serverul - gazduirea si accesarea resurselor.



Bazat pe arhitectura REST (Representational State Transfer)

- model introdus in 2000 de Roy Fielding in teza sa de doctorat
- arhitectura client-server conceputa pentru sisteme distribuite hipermedia, in particular Web

Roy Fielding's dissertation, "Architectural Styles and the Design of Networkbased Software Architectures."

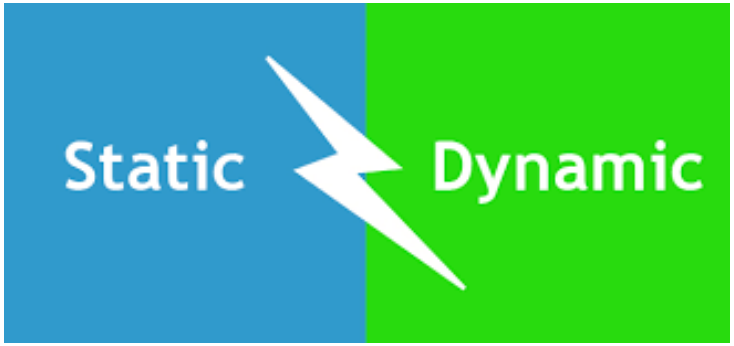
Continutul paginilor

Web-ul manevreaza texte si continut multimedia

- HTML (limbaj de marcaje) faciliteaza afisarea continutului la browser, nu si intelegerea continutului

Continutul furnizat de server poate fi

- static – preluat ca atare din fisiere
- dinamic – obtinut prin executia unui program
  - datele de intrare ale programului pot fi transmise de client prin cererea catre server

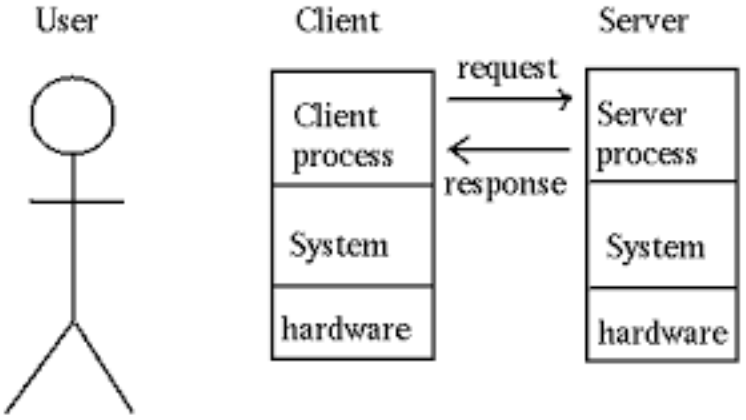


Modelul REST

Model

- sistem alcatuit dintr-o multime

de resurse localizate prin URL-uri



Functionare

- la o cerere a clientului (care include URL-ul unei resurse), sistemul returneaza o **reprezentare** a resursei
- la receptia reprezentarii, clientul **trece** intr-o noua **stare** a aplicatiei, in care are acces la alte URL-uri
- URL-urile interconecteaza reprezentarile resurselor, facilitand trecerea de la o stare la alta
- comunicarea client/server este reglementata de protocolul HTTP

Cereri si raspunsuri

Clientul poate cere descarcarea unei pagini, printr-o comanda GET care include URL-ul unei resurse



Serverul returneaza o **reprezentare** a resursei (pagina HTML)

Protocolul HTTP

HTTP este un protocol “stateless” ???

- serverul trateaza cererile independent una de alta
- dar clientul poate gestiona succesiunea cererilor trimise de el
- cererea si raspunsul contin toate informatiile necesare receptorului ca sa o poata intelege - sunt auto-descriptive

Avantaje ???

- scalabilitatea – serverul nu memoreaza starea si elibereaza rapid resursele folosite pentru o cerere
- creste vizibilitatea – un sistem de monitorizare gaseste intr-un singur mesaj toate informatiile

Dezavantaj ???

- multe informatii in fiecare mesaj → scad performanta

Conectivitatea si interoperabilitatea

- *conectivitatea* globala ???
  - multe resurse Web raspandite pe scara larga mai multor servere in Internet
  - accesibile prin orice browser
- *interoperabilitatea* client / server obtinuta prin ???

  adoptare de standarde (TCP, HTTP …)
  - ex. metode HTTP standard pentru acces la resurse
    - GET, POST, PUT, DELETE



Performanta

- *performanta* rezonabila: timp de raspuns 0.1 sec – 1 sec

– obtinut prin ???

replicarea serverelor

- reduce timpului de asteptare la server

distributia lor geografica

- reduce timpul de transport al cererii si raspunsului

– probleme ???

- consistenta replicilor pentru pagini dinamice





Disponibilitatea si securitatea



Disponibilitatea obtinuta prin: ???

- replicare
  - defectele sunt partiale – nu toate replicile sunt defecte
- corectarea erorilor prin retransmitere
- model HTTP simplu, “fara stare” → eroarea nu se propaga de la o cerere la alta

Confidentialitatea & *integritatea datelor* obtinute prin ???

- mecansime, algoritmi, protocoale de securitate
- *controlul accesului* la anumite resurse



Aplicatii bazate pe Web

- Folosesc Web-ul ca suport
  - browser: interfata utilizator uniforma si usor de intretinut
  - aplicatiile ruleaza “in spatele” serverului Web
- Aplicatia include componente (programe) distribuite, care ofera servicii bine definite
- Se folosesc doua *arhitecturi* consacrate
    - REST, REpresentational State Transfer
    - SOA, Service Oriented Architecture

Paradigma REST in aplicatii Web

- Servicii obtinute prin operatii (metode HTTP) asupra resurselor
  - PUT – creare resursa
  - DELETE – stergere resursa
  - GET – citire resursa
  - POST – modificare resursa
- cererile (parametrii) si raspunsurile pot fi reprezentate in XML (sau HTML) – diferenta?
  - XML descrie date structurate, procesabile de catre programe
  - HTML nu suporta comunicarea (semantică) intre programe
- procesarea unei aceleiasi operatii difera de la o resursa la alta, serviciile oferite fiind diverse
  - ex. amazon.com: comanda o carte, verifica starea comenzii curente etc.

Un exemplu – Depozit de mobila

Compania **Mobi** ofera un serviciu Web care permite clientilor

•sa obtina lista articolelor de mobilă

•sa obtina detalii despre un anumit articol

•sa faca o comanda

Serverul Web face disponibil un URL pentru lista articolelor

http://www.Mobi.com/articole

Pagina returnata la o invocare contine lista articolelor, intr-un format convenit cu clientul (de ex XML – clientul poate fi un program)

Pentru fiecare produs din lista, se specifica un ID si un URL, de ex

<Articol id="00345" xlink:href="http://www.Mobi.com/articole/00345"/>

Clientul poate folosi URL-ul pentru a obtine detalii despre acel articol

Descrierea articolului poate contine legaturi la alte detalii etc.

Pentru a primi o comanda, serverul face disponibil un URL

•Clientul creeaza un document conform cu o schema facuta cunoscuta de compania Mobi (de ex o schema XML – clientul poate fi un program)

•Clientul trimite documentul cu o metoda POST

•Serviciul Web raspunde trimitand un nou URL asociat cu comanda clientului

•Comanda devine astfel o resursa partajata intre client si furnizor, in forma unui serviciu Web

•Folosind URL-ul comenzii, clientul o poate regasi date, edita etc.

•Obs.: resursele sunt categorisite in functie de metodele pe care clientul le poate apela: GET pentru citire si POST, PUT, DELETE pentru modificare

REST - Model orientat pe resurse

- paradigma cerere / raspuns si absenta starii usureaza gestiunea relatiilor intre servicii Web
  - trateaza cererile independent una de alta
  - raspunde usor cererilor venite de la clienti diferiti
  - un serviciu Web poate fi replicat pe mai multe servere
- dezavantaj - modelul este orientat pe resurse (date)
  - clientii primesc intreaga stare a resursei nu o parte a ei (ca in cazul invocarii unor operatii)
- totusi, 80% din cererile de servicii Web la amazon.com sunt facute prin interfata REST

Referinta: Roger L. Costello Building Web Services the REST Way

http://www.xfront.com/REST-Web-Services.html

Gasiti aici descrierile XML ale continutului comenzilor si raspunsurilor din exemplul dat

SOA

- Un Serviciu software este construit ca o unitate de program de sine statatoare
- Fiecare serviciu este o colectie de operatii ce pot fi invocate, datele de intrare si rezultatele diferind de la o operatie la alta
- Serviciul poate comunica cu alte servicii software fara o cunoastere prealabila a acestora – cum e posibil?
  - Prin mecanisme de publicare, subscriere, notificare, descoperire (publish - subscribe - notify / discovery)
- Paradigma serviciilor software este adoptata in arhitecturi si tehnologii diferite
  - RPC, Java RMI, CORBA, DCOM
  - SOAP

e-Servicii

- Dezvoltate in mediul privat e-business si e-commerce si public e-government
- e-Serviciile furnizeaza bunuri care nu sunt tangibile
  - Exemplu: serviciul FedEx online de urmarire a pachetelor
- Ele raspund cerintelor si preferintelor utilizatorilor
  - presupun o intelegere intre furnizor si consumator
- Se bazeaza pe folosirea altor servicii mai simple – Exemple ?
  - de monitorizare a activitatii utilizatorilor si a contextului
  - vizualizare obiecte multimedia, analiza imaginilor,
  - baze de date,
  - motoare de cautare,
  - comunicare in timp real,
  - suport al activitatilor colaborative,
  - control al accesului, securitate
  - administrare etc.

e-Business si e-commerce

- e-Business
  - cuprinde gestiunea lantului de aprovizionare, prelucrearea comenzilor, gestiunea serviciilor pentru clienti, cooperarea cu partenerii si altele.
- e-Commerce
  - se refera la tranzactiile de vanzare/cumparare a produselor comercializate si intarirea relatiilor cu clientii si furnizorii

- Cerinte (se adauga celor de la Web si aplicatii Web)
  - scalabilitate: suporta traficul de varf
  - atomicitatea tranzactiilor
  - organizarea datelor: cautarea usoara in catalogul de produse
  - administrare simpla – modificari facute de ne-specialisti IT

Exemplu: Amazon.com

Contextul

- dimensiunea
  - platforma ofera servicii pentru mai multe site-uri web
  - implementata pe o infrastructura de zeci de mii de servere si componente de retea din mai multe centre de date
  - scalabilitatea suporta cresterea continua - este o proprietate importanta
- modul de operare
  - infrastructura cuprinde milioane de componente
  - in orice moment numarul de componente defecte este semnificativ
  - **tolerarea defectelor** este modul normal de operare

Referinta: G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall and W. Vogels.

Dynamo: Amazon's Highly Available Key-value Store

Cerinte operationale

Cerinte operationale stricte (care se regasesc si in alte platforme)

- siguranta (reliability) – cea mai mica intrerupere a serviciilor scade increderea clientilor si are impact financiar mare
- tratarea defectarilor – problema dificila deoarece sistemul este larg distribuit geografic

Solutii adoptate de Amazon

Obiectivul – gestiunea corecta a starii aplicatiei chiar in prezenta defectelor

- de ex. clientul sa poata vedea si adauga oricand obiecte in cosul de cumparaturi
  - serviciul de cumparaturi sa poata, oricand, citi si scrie in depozitul de date
  - datele sa fie disponibile in mai multe centre

Solutii

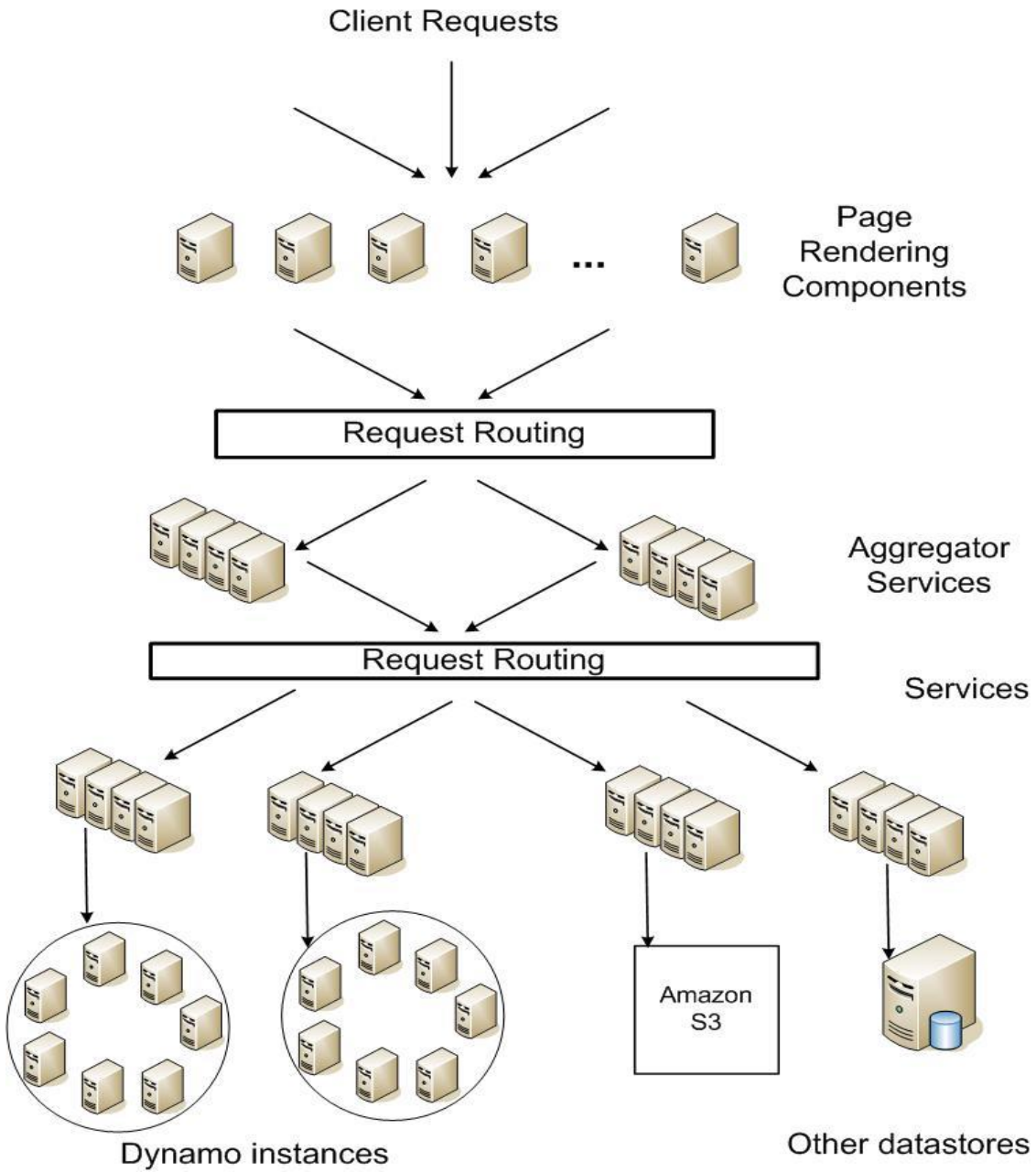
•tehnologii de depozitare distribuita a datelor

- Dynamo – folosit pentru scalabilitate si disponibilitate ridicata
- Amazon S3 (Simple Storage Service)

ATENTIE!

Modelele si metodele mentionate in aceasta prezentare vor fi studiate in detaliu pe parcursul semestrului !

Arhitectura distribuita orientata pe servicii



O cerere client poate fi tradusa in cereri catre peste 150 de servicii

Un serviciu poate utiliza diferite depozite de date dintr-o zona accesibila

Unele servicii fac agregarea raspunsurilor furnizate de alte servicii

Modelul de date

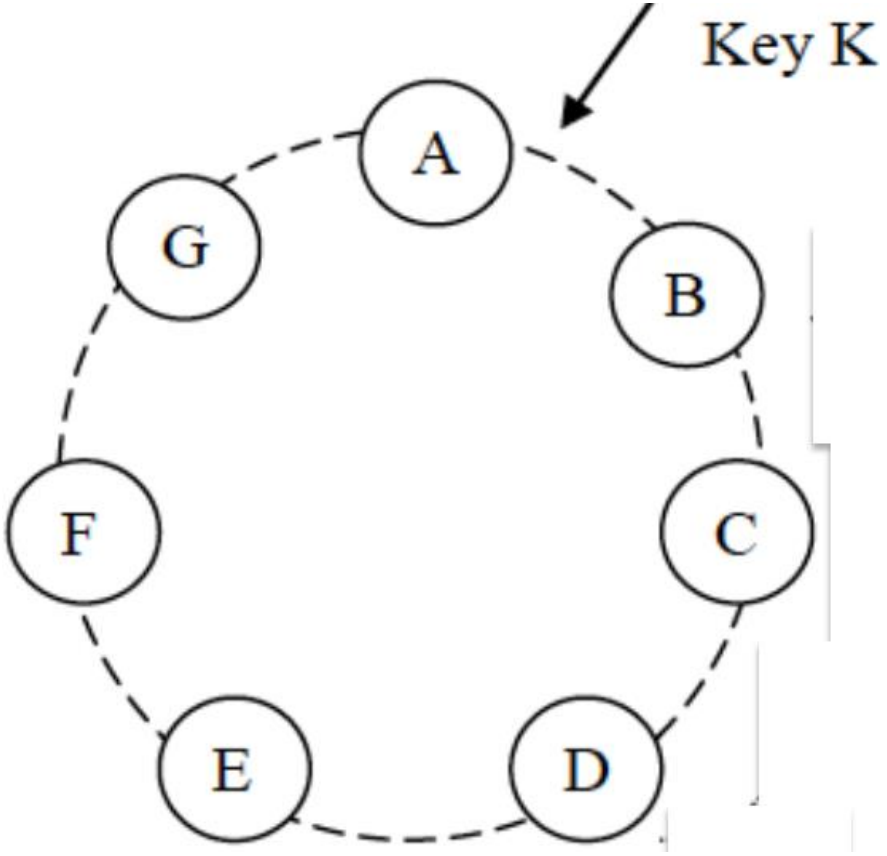
Tipic, sistemele de productie folosesc modelul relational

Amazon are un model simplu de date - cheie-valoare

- citiri si scrieri ale unor date identificate unic printr-o cheie
- suficiente pentru serviciile oferite, de ex. ???
  - lista celor mai buni vanzatori,
  - cosul de cumparaturi,
  - preferintele clientului,
  - gestiunea sesiunii,
  - nivelul vanzarilor,
  - catalogul de produse

Volume mari de date care trebuie **partitionate**

- datele sunt repartizate dinamic unui set de noduri (de stocare)
- foloseste o schema de hashing consistent cu spatiul valorilor organizat logic in inel
- fiecare nod are asociata o valoare din spatiu (pozitia nodului pe inel)
- fiecare data are asociata o valoare (din acelasi spatiu) egala cu hash-ul cheii (pozitia datei pe inel)
- data este stocata in nodul cu cea mai mica pozitie >= decat pozitia datei (cheia K in nodul B)

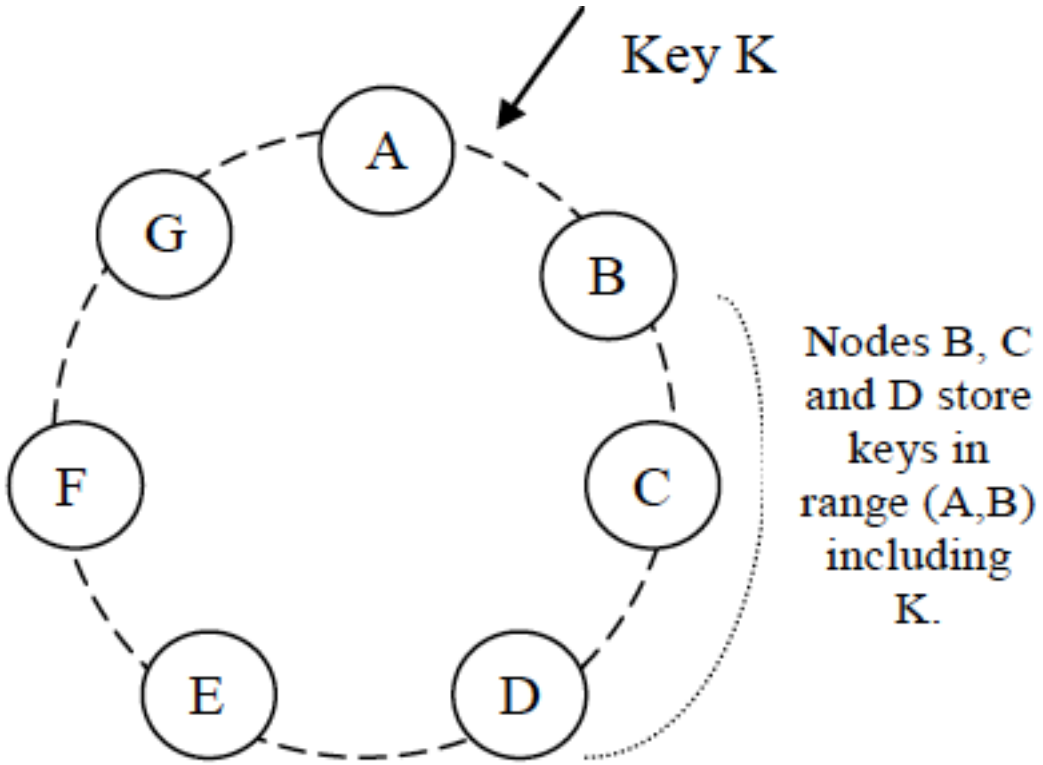


Replicarea

- modelul faciliteaza
  - o repartizare echilibrata a datelor pe diferite noduri
  - regasirea usoara a datelor in functie de cheie
  - adaugarea si eliminarea usoara a nodurilor

Replicarea

- fiecare data este replicata pe N noduri (N configurabil)
- schema: fiecare cheie este asignata unui coordonator (ex. K asign. nod B) care memoreaza datele corespunzatoare cheii si gestioneaza replicarea lor in alte noduri (ex. C si D)



Modelul de consistenta: eventual consistency

- nodurile de stocare a datelor sunt situate (de regula) in sisteme diferite – de ce?
- modificarea unei valori se face pe o replica (usor) accesibila
- modificarea este propagata asincron celorlalte replici
- dupa un timp, daca nu sunt alte modificari, replicile vor detine aceeasi valoare
  - modelul = consistenta "**in cele din urma**"
- in caz de defect actualizarea poate dura un timp indelungat
- dar, un apel PUT se intoarce la apelant inainte de actualizarea tuturor replicilor (operatiile "Add to Cart" nu trebuie rejectate sau uitate !)

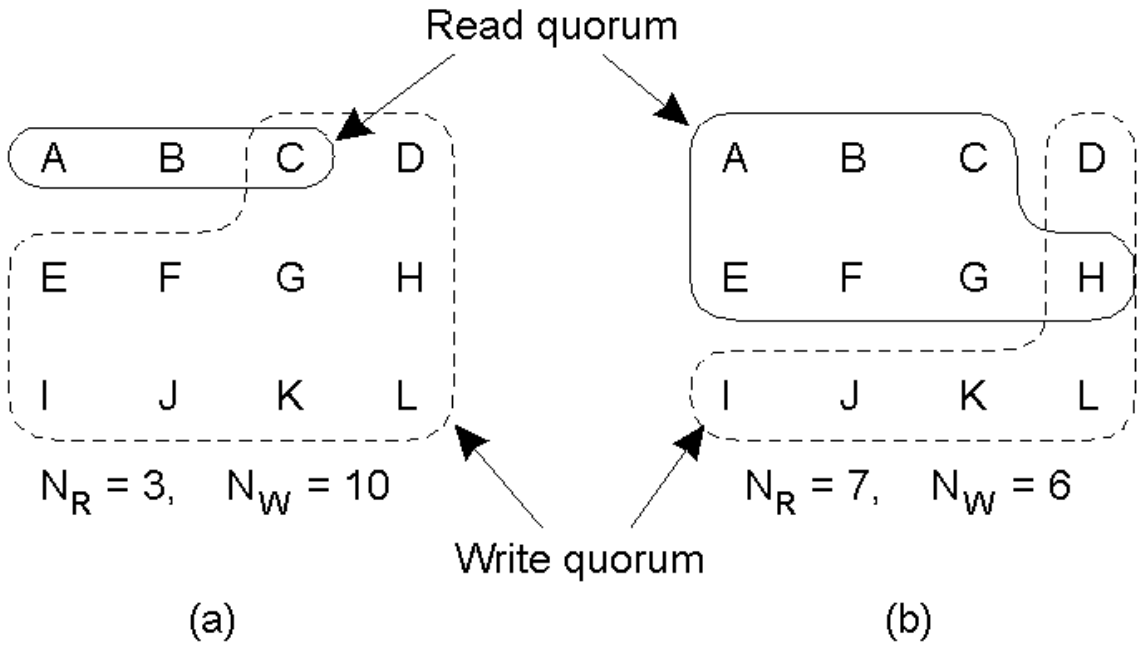
  → un apel GET ulterior poate accesa o replica neactualizata

Pastrarea consistentei

- Solutia = versionarea obiectelor
- pentru eficienta – modificarea se face pe cea mai recenta replica accesibila iar rezultatul este tratat ca o noua versiune a obiectului
  - pentru a descrie dependentele cauzale intre versiuni se folosesc ceasuri vectoriale
- reconcilierea cu celelalte versiuni se face de catre coordonator la un apel GET
  - conditionata de relatiile dintre replici !
  - daca replicile pastreaza mai multe versiuni independente cauzal, reconcilierea se faca la nivelul aplicatiei

Tratarea defectelor tranzitorii

- executia PUT si GET se face conform modelului de cvorum:
  - pentru N replici, operatia se termina cand
  - minimum N_W noduri participa la operatia de scriere (PUT) sau
  - minimum N_R noduri participa la operatia de citire (GET)
  - conditia este $N_R + N_W > N$
- asigura ca grupul nodurilor citite include cel putin un nod care are ultima versiune a obiectului



Tratarea defectelor permanente

- doua noduri pot schimba intre ele informatii despre seturile de chei comune
- foloseste un protocol de sincronizare descentralizata a replicilor bazat pe Merkle trees
  - frunzele sunt hash-uri de chei
  - celelalte noduri sunt hash-uri ale fiilor
- doua noduri schimba intre ele radacinile arborilor Merkle corespunzatoare unor seturi de chei comune
- determina diferentele (prin parcurgerea arborilor) si fac sincronizarea

Alte functionalitati

- detectia nodurilor defecte
- adaugarea si eliminarea dinamica a nodurilor de memorie

Modelul de performanta

- Serviciul pentru cosul de cumparaturi – raspunde intr-o zi la zeci de milioane de cereri rezultate din peste 3 milioane de checkout-uri
- Serviciul de gestiune a sesiunii gestioneaza mii de sesiuni active simultan
- Politica firmei este de a oferi servicii bune unui numar cat mai mare de clienti
  - ex.: un serviciu garanteaza un timp de raspuns sub 300 msec pentru 99.9% din cereri, la o incarcare de 500 de cereri pe secunda
  - procentul este determinat pe baza unei analize de cost
    - un procent mai mare ar determina o crestere semnifiativa a costului
    - pentru politica specificata - media / mediana / dispersia, folosite curent in alte sisteme, nu sunt potrivite

Cerintele altor categorii de servicii

- e-Banking:
  - Servicii electronice *robuste* si cat mai *complete*
  - fiabilitate (situri de rezerva)
  - confidentialitate si protectie
- e-Government
  - integrarea serviciilor conform unor modele de e-guvernare: G2C (Government to Citizens), G2B (Government to Businesses) etc.
- e-Health
  - toleranta la erori pentru transmisii audio/video, nu pentru EKG
  - mai multe canale de transmisie pentru domiciliu sau ambulanta
    - transmisie rapida in situatii de urgenta
  - protectia si confidentialitatea datelor despre pacienti
- m-Services
  - adaptarea serviciilor la context (ex. timp, locatie)
  - adaptarea la resursele reduse ale mobilelor
  - mentinerea continuitatii serviciului (in prezenta deconectarilor)

Cerinte comune aplicatiilor

- utilizare simpla
- interoperabilitate
- performanta ridicata
- scalabilitate
- disponibilitate - Livrarea serviciilor *oricand*, *oriunde*, si pe *orice echipament*, inclusiv pe mobile
- toleranta la defectari
- satisfacerea unor constrangeri de timp real
- controlul acesului
- confidentialitate, integritate, de incredere
- deschidere la extinderi sau re-implementari
- eficienta economica (cost redus)
- adaptarea la profilul clientilor

Alte arhitecturi distribuite

Cloud = Colectie (nor) de resurse (hardware, software …) distribuite in Centre de date, functionand dupa modelul client-server, pentru a furniza servicii de infrastructura, platforma, software etc., la cerere si contra cost

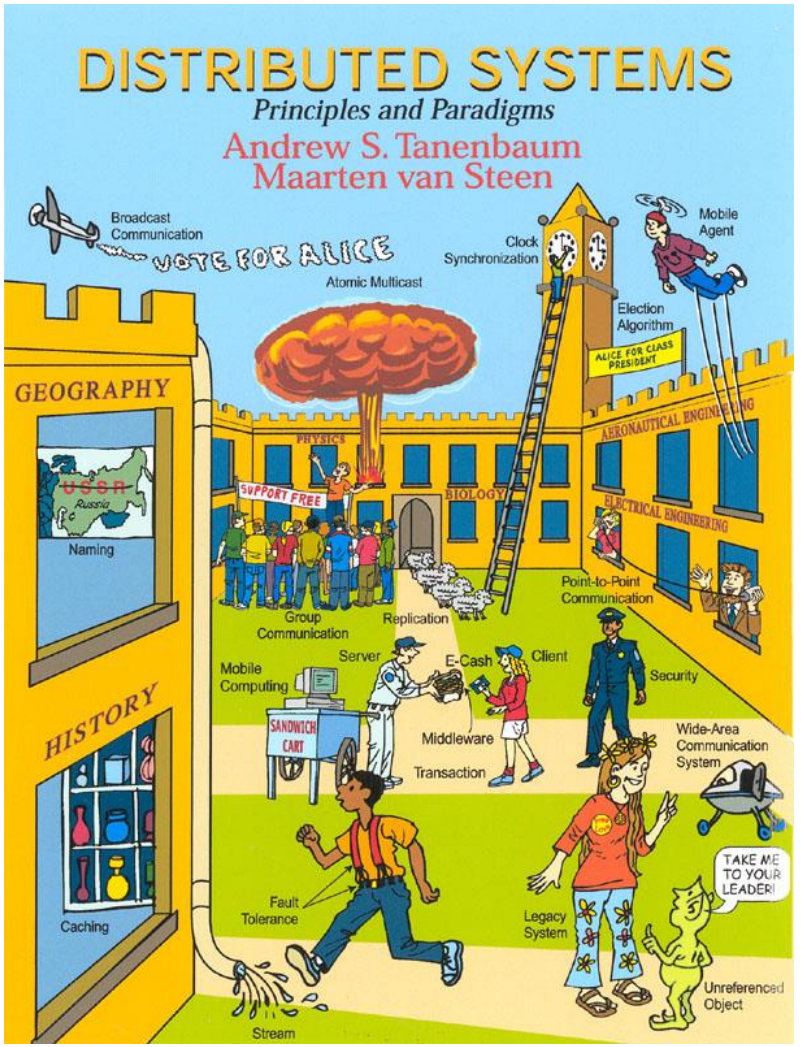
Grid = Colectie de resurse puse la dispozitie si folosite in comun de membrii unei Organizatii Virtuale (VO)

Retele Peer-to-Peer = colectie de resurse distribuite, controlate descentralizat de componente care sunt in acelasi timp clienti si servere

Sisteme distribuite - A doua definitie

- Un sistem distribuit este o colectie de calculatoare independente care apar utilizatorilor ca un singur sistem coerent.

Tanenbaum, van Steen
Prentice Hall 2007



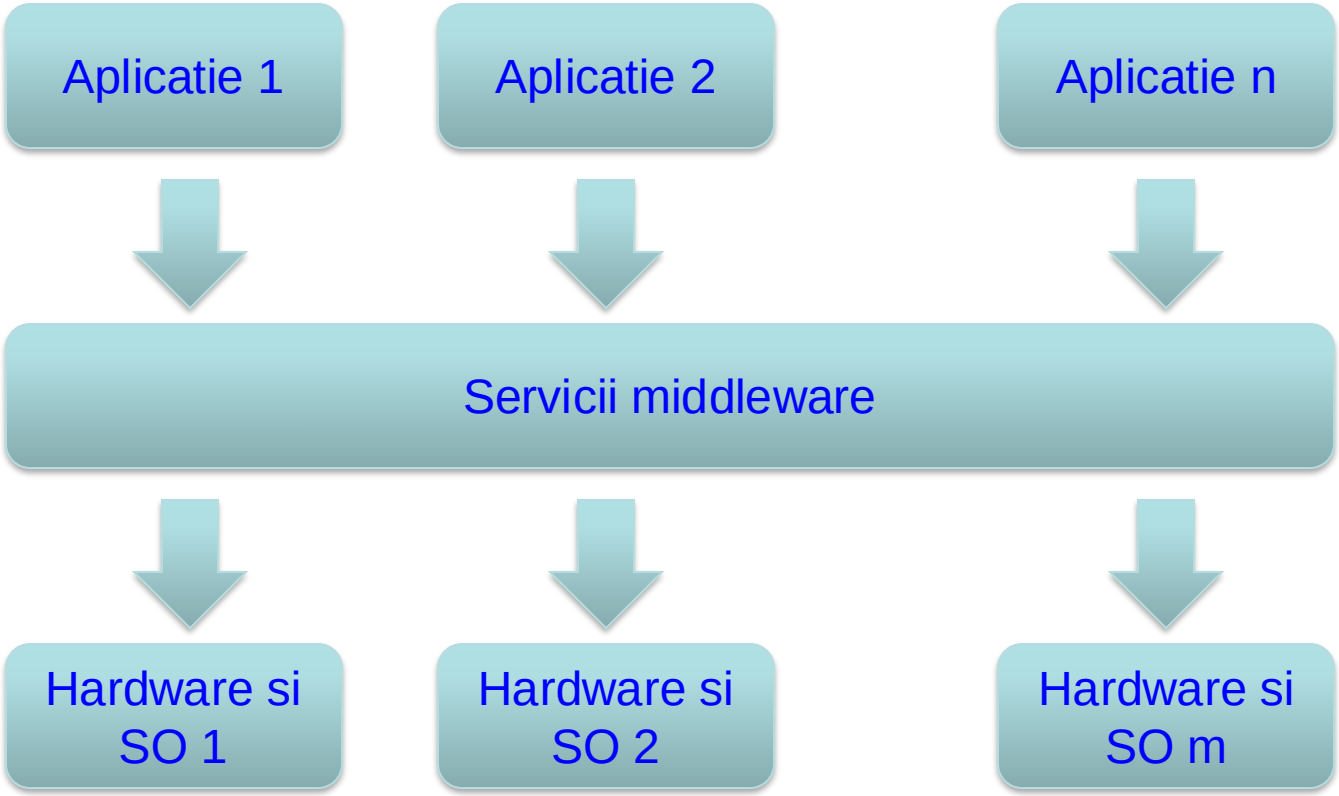
Avantajele sistemelor distribuite

- reducerea timpului de executie al aplicatiilor prin paralelizare si distributie,
- cresterea tolerantei la defectari
  - in SD defectele sunt partiale!
- specializarea functionala
  - exploatarea mai buna a particulaitatilor componentelor
- raportul cost / performanta mai redus,
- usurinta de acces la resurse indepartate,
- cresterea incrementala a sistemului.

O arhitectura a sistemelor distribuite

- Intr-un sistem distribuit, componentele aplicatiei, situate in sisteme diferite inter-opereaza prin intermediul retelei
  - un program situat intr-unul din sisteme are acces la programe si date din alte sisteme
- Platformele (hardware si sistem de operare ale) sistemelor individuale nu sunt construite pentru calculul distribuit
  - ele folosesc produse hardware si software “de gata”, comercializate publicului larg - Commercial off-the-shelf (COTS)
- Solutie:
  - includerea, intre nivelul aplicatiilor si cel al platformei, a unui nivel suplimentar numit *middleware*, care ofera serviciile cerute de aplicatiile distribuite, prin interfete (API-uri) standard.

Solutia middleware



Avantajele middleware

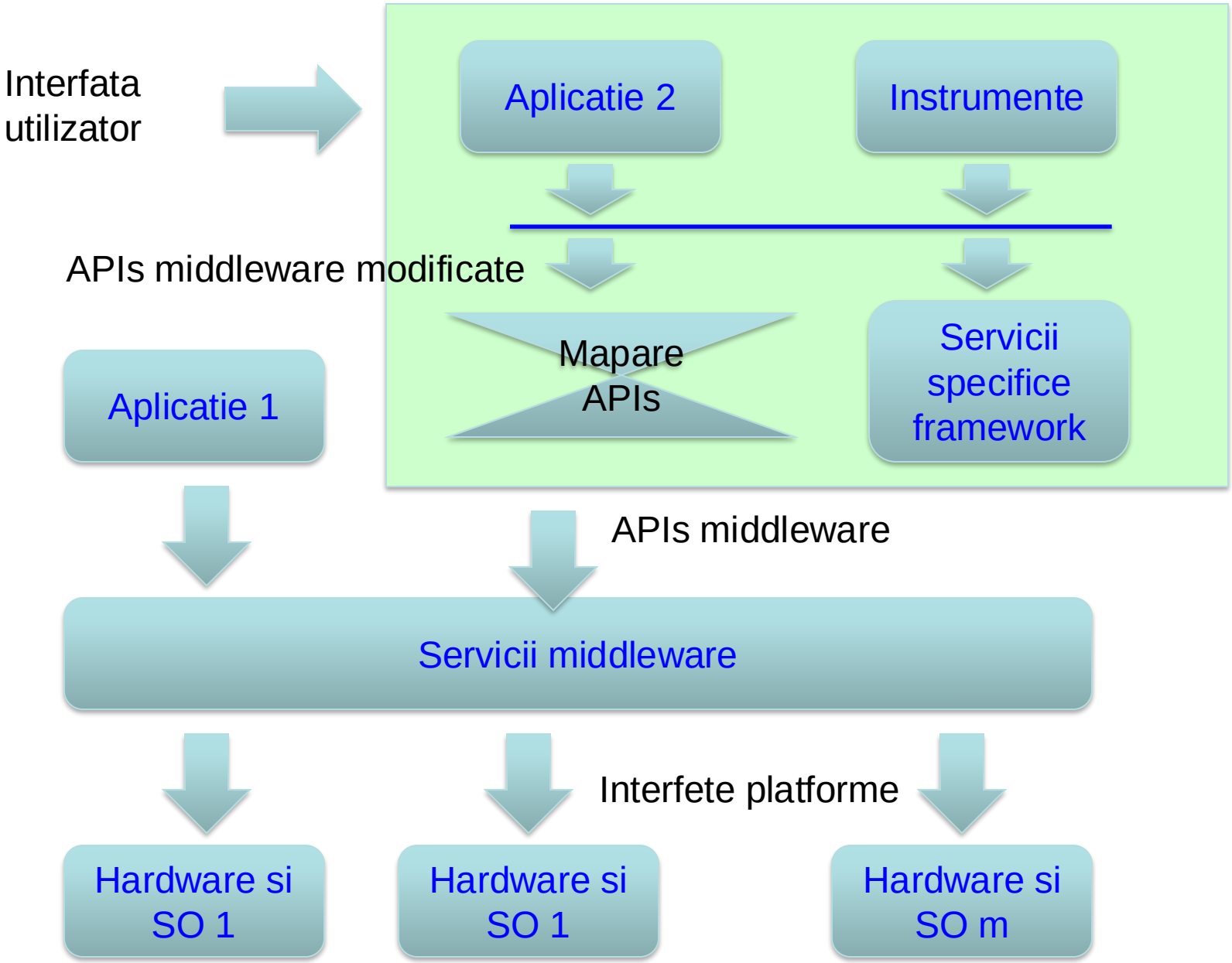
- ofera API-uri de nivel ridicat, independente de platforma (hardware si SO)
- permit dezvoltatorului de aplicatii sa se concentreze pe logica acestora
  - ii scuteste de rezolvarea problemelor de comunicare la distanta intre componente, acces concurent la resurse, tolerarea defectarilor, controlul accesului etc.
- suporta interoperabilitatea
- permite extinderea treptata a sistemului
- suporta portabilitatea

Categorii de servicii middleware

- Comunicarea intre procese
  - transmitere mesaje si apeluri la distanta (RPC, RMI, SOAP)
- Sincronizarea proceselor
  - tratarea concurentei
  - tranzactii distribuite
- Replicarea si consistenta datelor
- Tratarea defectelor si toleranta la defectari
- Mobilitatea
- Securitatea

Frameworks

- Serviciile sunt re-utilizabile
- efectul in cazul middleware:
  - efortul dezvoltatorului de aplicatii se muta de la dezvoltarea unor componente noi, la integrarea unora existente
- Cadrele de lucru (frameworks) faciliteaza dezvoltarea aplicatiilor
- Un cadru de lucru include
  - mapari pe serviciile middleware
  - servicii proprii framework-ului
  - instrumente pentru folosirea framework-ului
  - aplicatii


Interfata utilizator
Aplicatie 2
Instrumente
APIs middleware modificate
Mapare APIs
Servicii specifice framework
Aplicatie 1
APIs middleware
Servicii middleware
Interfete platforme
Hardware si SO 1
Hardware si SO 1
Hardware si SO m

END



Comunicarea in sisteme distribuite

Comunicari sincrone si asincrone, tranzitorii si persistente.

[Distributed Systems, A.S.Tanenbaum 2003]



Comunicarea in sisteme distribuite

- Comunicarea este bazata pe schimb de mesaje
- Nivelul transport este "cheia" ierarhiei
  - ofera o interfata uniforma (socket) pentru programatori
  - furnizeaza conexiuni sigure (ex. TCP)
  - suporta comunicarea multicast
  - limitare - mesajele sunt simple siruri de octeti (nu date structurate)
- Implementat ca TCP, UDP, RTP, T/TCP

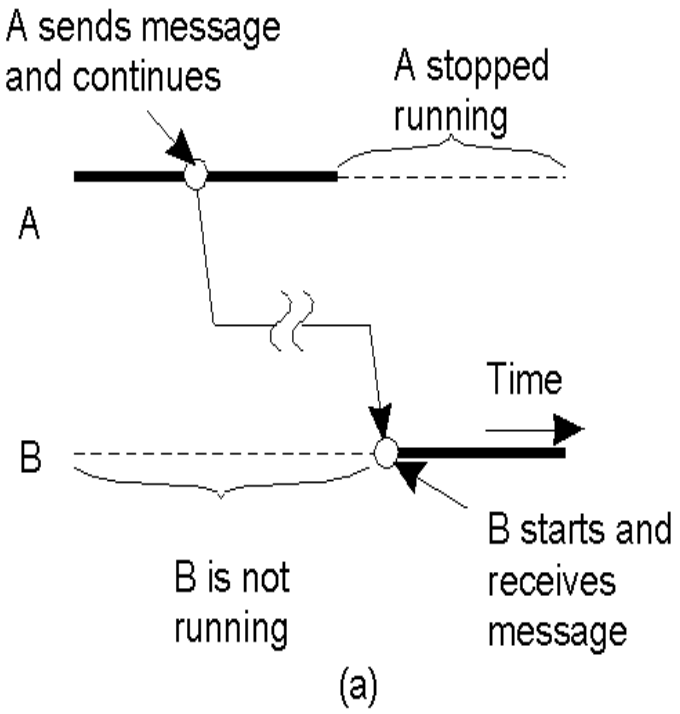


Modele de comunicare peste nivelul transport

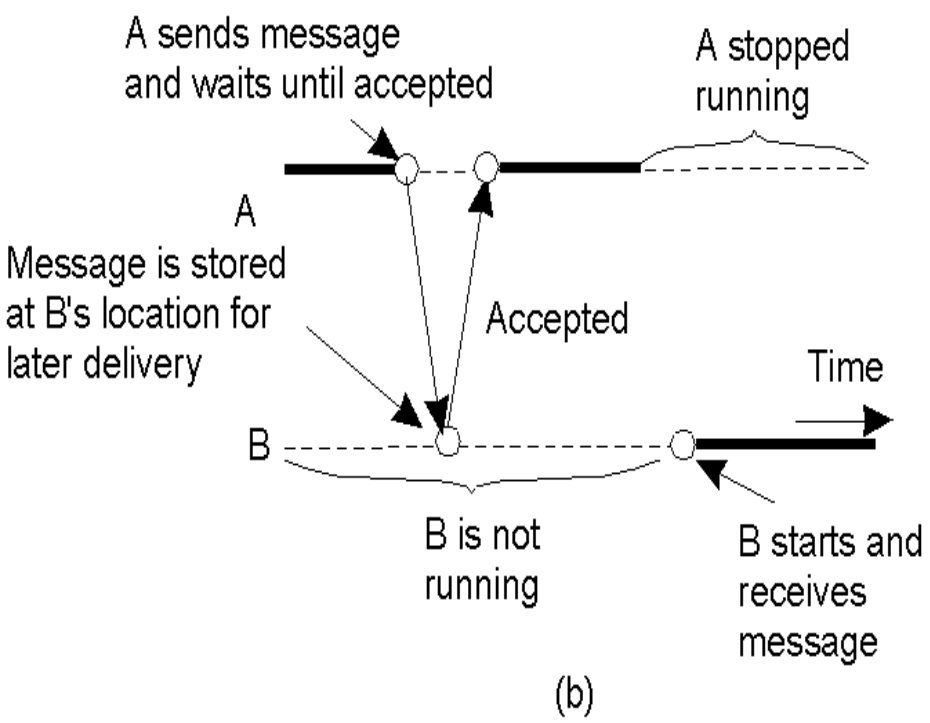
- Protocoale pentru **modelul REST** (HTTP)
- **Extensii ale apelului de procedurl**
  - apeluri de proceduri la distanta (RPC)
  - invocari de metode la distanta (RMI)
  - Simple Object Access Protocol (SOAP)
- **Alte forme ???**
  - comunicarea de grup
  - publish – subscribe (in sisteme bazate pe evenimente)
  - cozi de mesaje
  - multimi de tupluri (tuple spaces)
  - memorie distribuita partajata

Comunicare persistenta

asincrona (a)



sincrona (b)

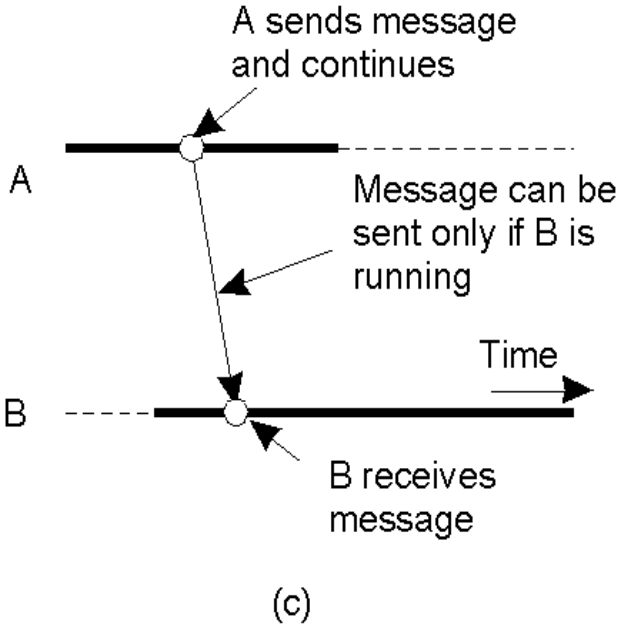


Un mesaj trimis este pastrat de middleware pana la livrarea lui la receptor

Comunicare tranzitorie

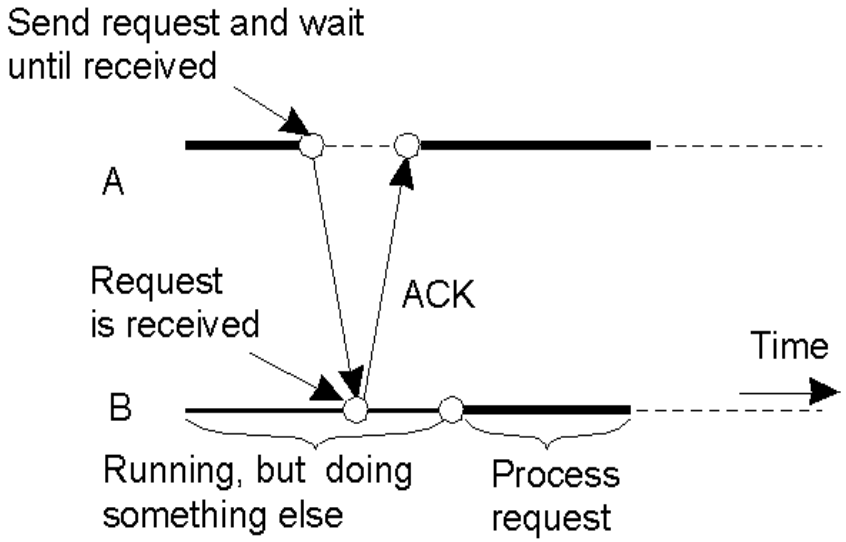
Un mesaj este pastrat de middleware doar cat timp aplicatiile transmitatoare si receptoare sunt in executie

asincrona



(c)

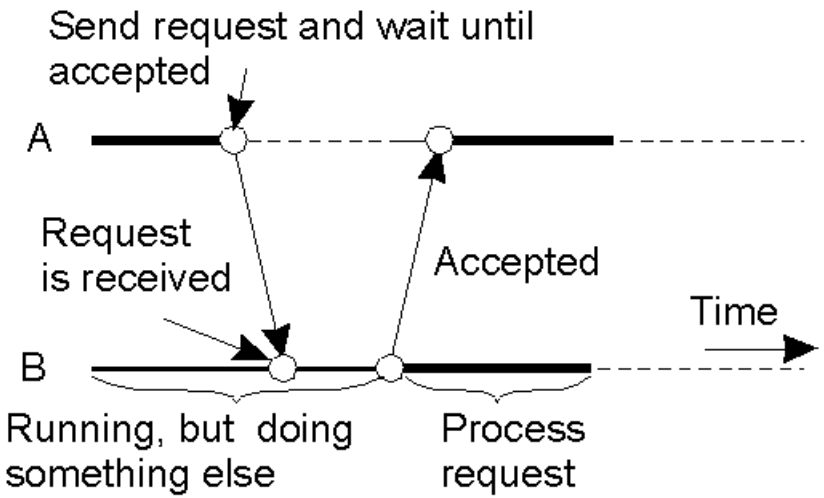
sincrona la receptie



(d)

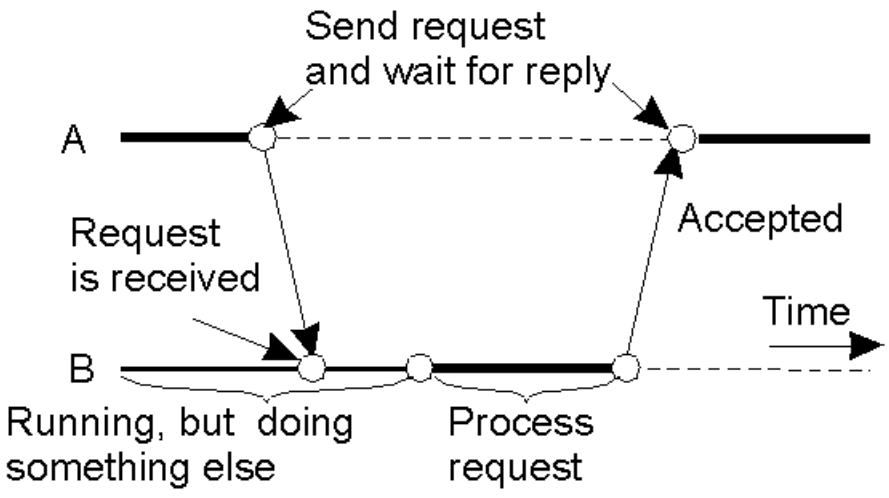
Comunicare tranzitorie

sincrona la livrare



(e)

sincrona la raspuns



(f)



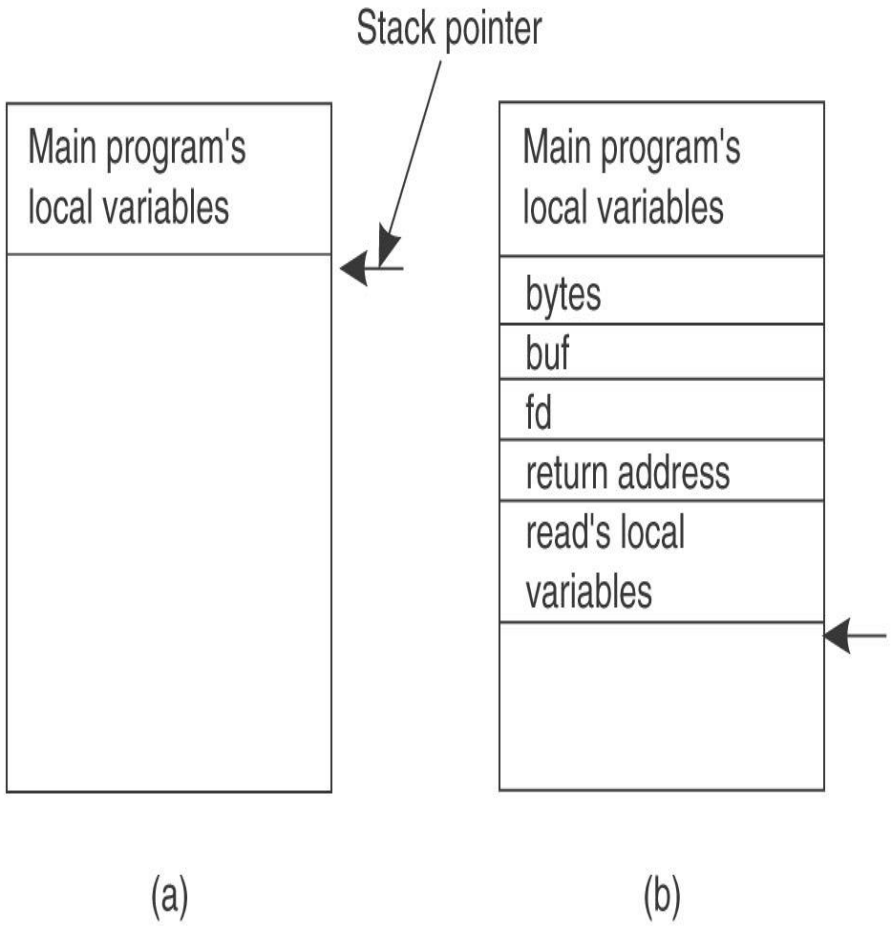
RPC – Remote Procedure Call

De ce apel la distanta ?

- Procedurile (subrutinele) locale au avantaje incontestabile ???
  - structureaza programele
  - permit reutilizarea codului (biblioteci)
  - reduc costul dezvoltarii si intretinerii programelor mari
  - foarte folosite in programarea secventiala
- Motivatia apelurilor la distanta
  - sa beneficieze de aceleasi avantaje si daca procedurile ruleaza in alte sisteme (la distanta)
- Problema
  - un sistem de calcul are incorporate mecansimele necesare apelului local, dar nu si pentru apeluri la distanta

Mecanisme apel de procedura conventional

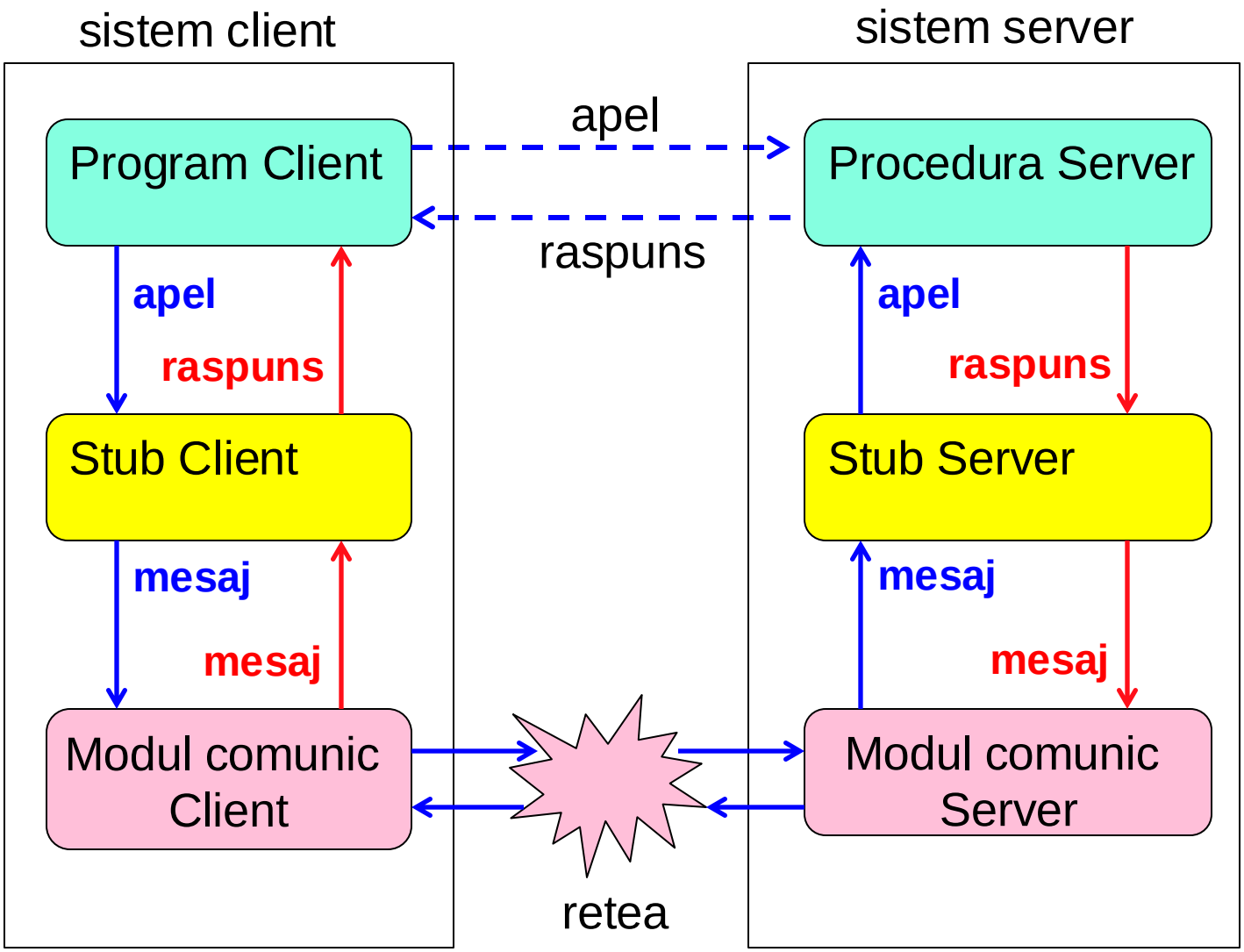
- count = read (fd, buf, bytes)
- la apel, programul principal pune pe stiva, in ordine, parametrii si adresa de intoarcere (fig. b)
- **read** transfera date in zona **buf**, intoarce rezultatul count, elimina adresa de intoarcere din stiva si da controlul programului
- programul elimina din stiva parametrii si continua executia
- scheme de pasare parametri
  - apel prin valoare (fd, bytes)
  - apel prin referinta (buf)
- alta schema posibila: "copy / restore"



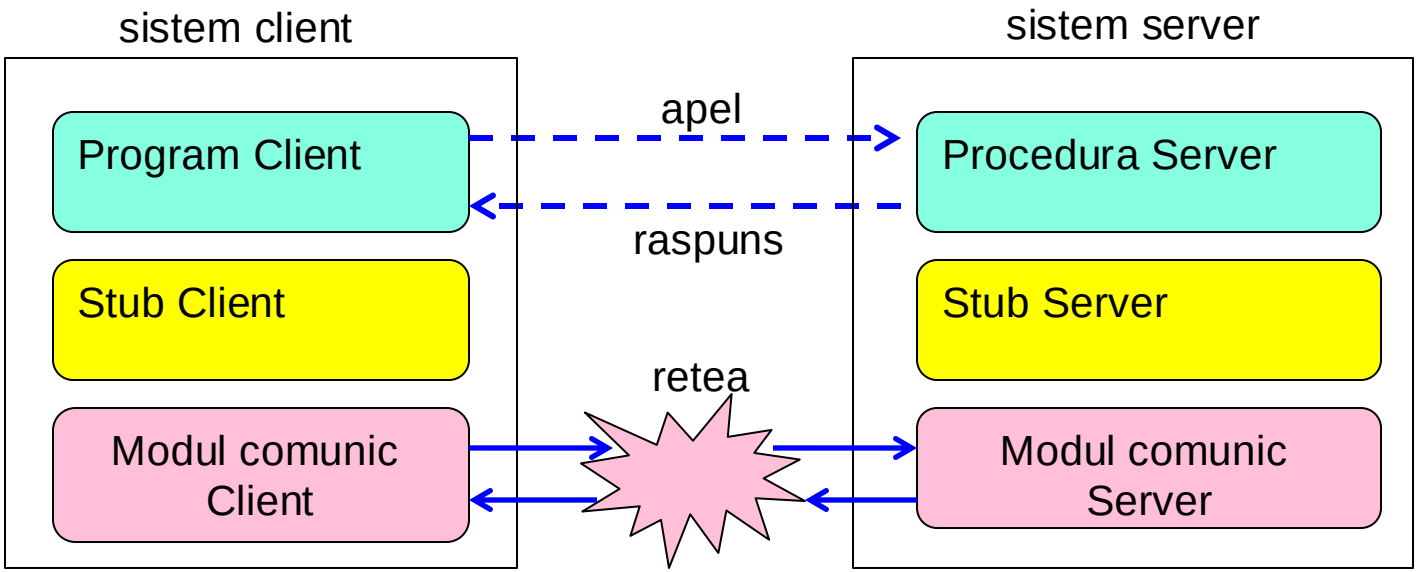
Apelul de proceduri la distanta (RPC)

- Apelantul si procedura ruleaza in sisteme diferite
- Comunicarea se face prin retea
- Vizeaza transparenta
  - aceeasi forma de apel pentru proceduri locale si la distanta
- RPC ofera o solutie completa pentru
  - localizarea procedurii
  - trimiterea apelului la distanta, executia procedurii si intoarcerea raspunsului
  - conversia reprezentarii datelor (daca reprezentarile difera de la un sistem la altul)

Arhitectura apelurilor la distanta



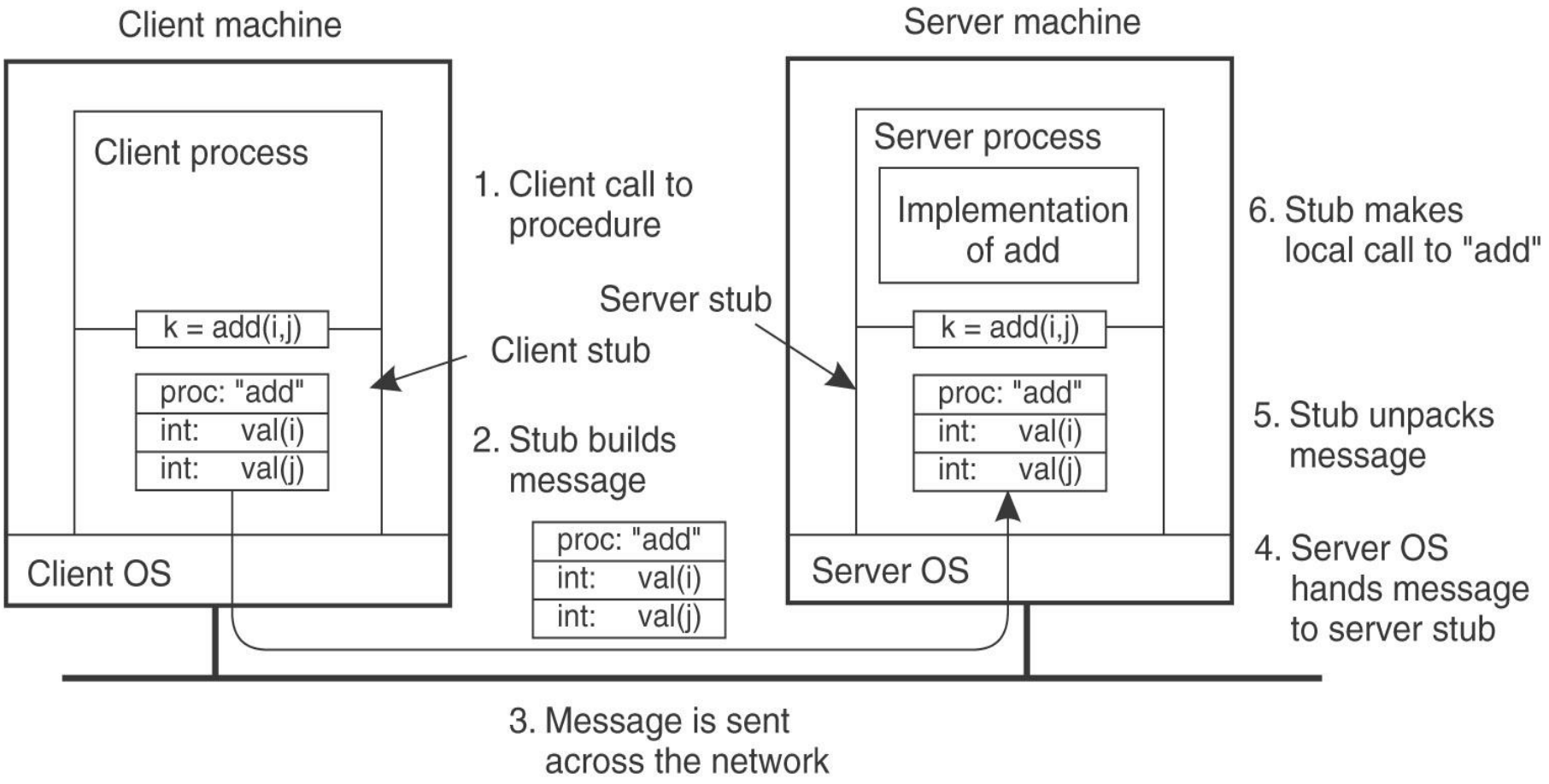
Arhitectura apelurilor la distanta



- Stub client este un proxy al procedurii server
  - accepta aceleasi apeluri dar le transforma in mesaje
- Stub server este un proxy al programului client
  - accepta aceleasi rezultate dar le transforma in mesaje
  - pentru ex. “count = read (fd, buf, bytes)” serverul umple un buf intern stub-ului

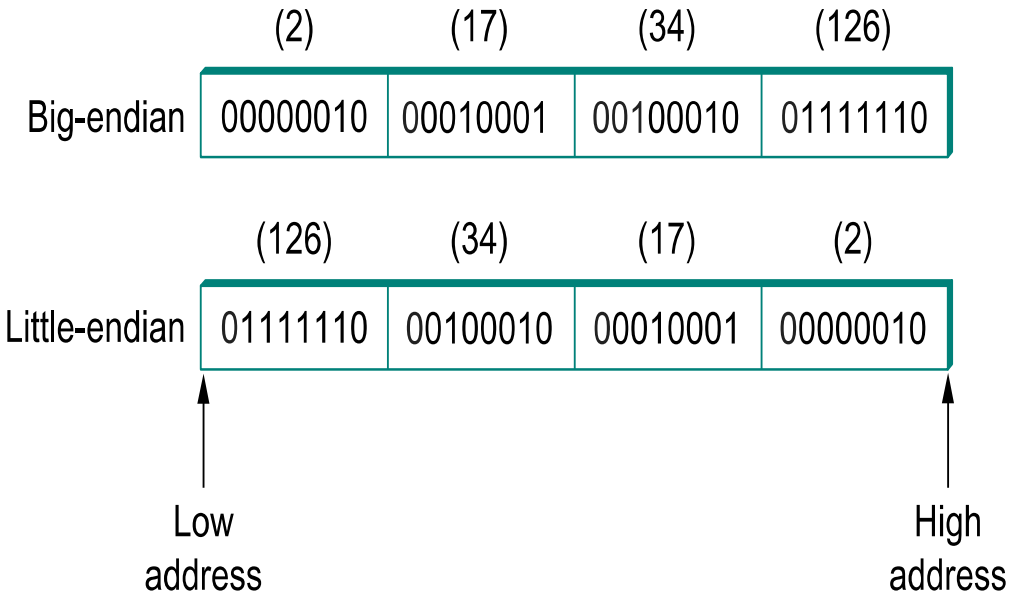
Formatul datelor pentru sisteme omogene

- reprezentarile parametrilor si ordinea sunt identice la client si server
- stub client transmite si numele procedurii (identifica una din procedurile rulate de server)



Probleme: Reprezentarea datelor

- big-endian (cel mai semnificativ bit este in octetul cu adresa mai mica) – adoptat in procesoarele MIPS® and PowerPC™
- little-endian (cel mai semnificativ bit este in octetul cu adresa mai mare) – adoptat in procesoarele Intel® x86
- o valoare intreaga necesita o conversie cand trece de la un tip de masina la altul



In figura: reprezentarea pe octeti a intregului **34 677** $_{10}$ = **0211227E** $_{16}$ in formele big-endian si little-endian

Transferul parametrilor

Marshalling – aranjarea parametrilor pentru transfer

Mecanisme diferite pentru pasarea parametrilor

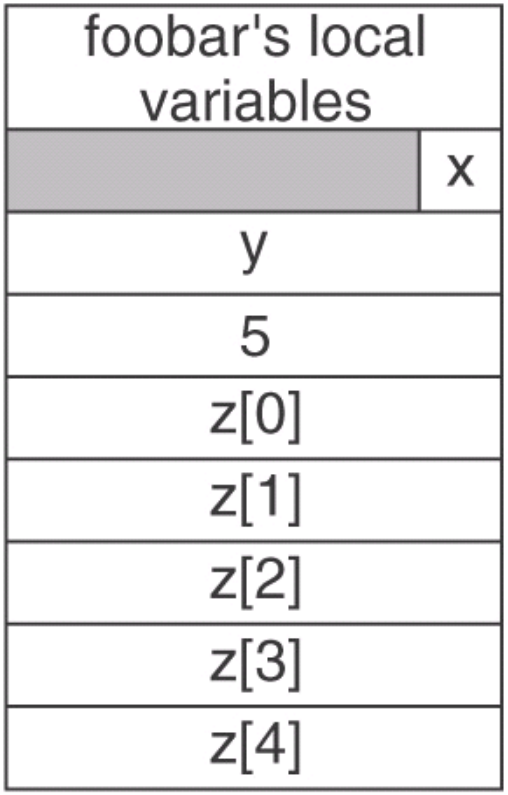
- Parametri valoare
  - adoptarea unor reprezentari standard pentru protocoalele RPC
    - ex. la Sun RPC reprezentarea XDR (eXternal Data Representation)
  - conversia datelor intre reprezentari interne din client / server si reprezentarea standard

Transferul parametrilor (2)

Parametri referinta – solutii diferite

- pentru structuri simple (arrays) – se transfera elementele tabloului
- copy / restore merge in anumite cazuri
- pentru eficienta (ex. transmitere intr-un singur sens) se specifica daca parametrii sunt in, out, in / out

```
foobar( char x; float y; int z[5] )
{
   ....
}
```



Transferul parametrilor (3)

Pointerii din structuri complexe nu pot fi gestionati usor

Incercari

- pentru structuri complexe (grafuri) care folosesc pointeri se face linearizarea structurii, transmiterea si apoi refacerea ei la celalalt capat
- transmiterea pointerilor si cod special pentru acces la datele din client
  - ex. cereri de la server catre client pentru a afla datele referite de pointeri

Solutia apelului la distanta

- specificarea unui **protocol** pentru comunicarea intre programul client si procedura server, care
  - descrie semnatura procedurii la distanta
  - descrie tipurile parametrilor apelurilor si ale rezultatelor invocarilor
    - folosite de un compilator care genereaza stub-urile client si server
  - faciliteaza conectarea intre client si server

Protocolul RPC reglementeaza

- formatul datelor si functiilor
  - reprezentatarea tipurilor si structurilor de date
    - folosesc tipuri specifice (XDR -eXternal Data Representation la Sun RPC)
    - sau standardizate (ASN.1—Abstract Syntax Notation)
  - descrierea (semnaturilor) procedurilor
    - Interface Definition Language, IDL
- regulile schimbului de mesaje (de ex. protocolul de transport folosit)

SUN RPC

Numit si ONC RPC (ONC – Open Network Computing)

Utilizat in scrierea NFS (Network File System)

Poate fi folosit peste TCP sau UDP la alegere

Are un limbaj de descriere a interfetei,

XDR (eXternal Data Representation)

Ofera un compilator asociat, rpcgen

XDR – eXternal Data Representation standard

Specificat in RFC 1014 - permite definirea

Procedurilor utilizabile de la distanta

Tipurilor asociate:

Constante, typedefs, structuri, uniuni, enumerari

Caracteristici

Mecanisme simple de identificare:

Interfata identificata prin **nr program** si **nr versiune**

O definitie de procedura include **semnatura** si **nr procedura**

Semnatura procedurii include

tip_rezultat nume_procedura tip_parametru_intrare

Initial - un singur parametru de intrare si un singur rezultat (restrictie eliminata)



Exemplu - Interfata RPC pentru rezervare loc avion

```
struct reserve_args {
  int flight_number;
  string passenger_name<>;
};

struct reserve_result {
  int ok;
  string seat<>;
};

program RESERVATION_PROG {
  version RESERVATION_VERS {
    reserve_result RESERVATION_RESERVE   (reserve_args) = 1;
  } = 1;
} = 400001;
```

Protocolul - Formatul mesajelor

Call	**Reply**
Xid	Xid
Call / reply	Call / reply
Rpc version	Accepted? (vs bad rpc version or auth failure)
Program #	Auth stuff
Program version	Success? (vs bad prog / proc #)
Procedure #	results
Auth stuff	
Arguments	

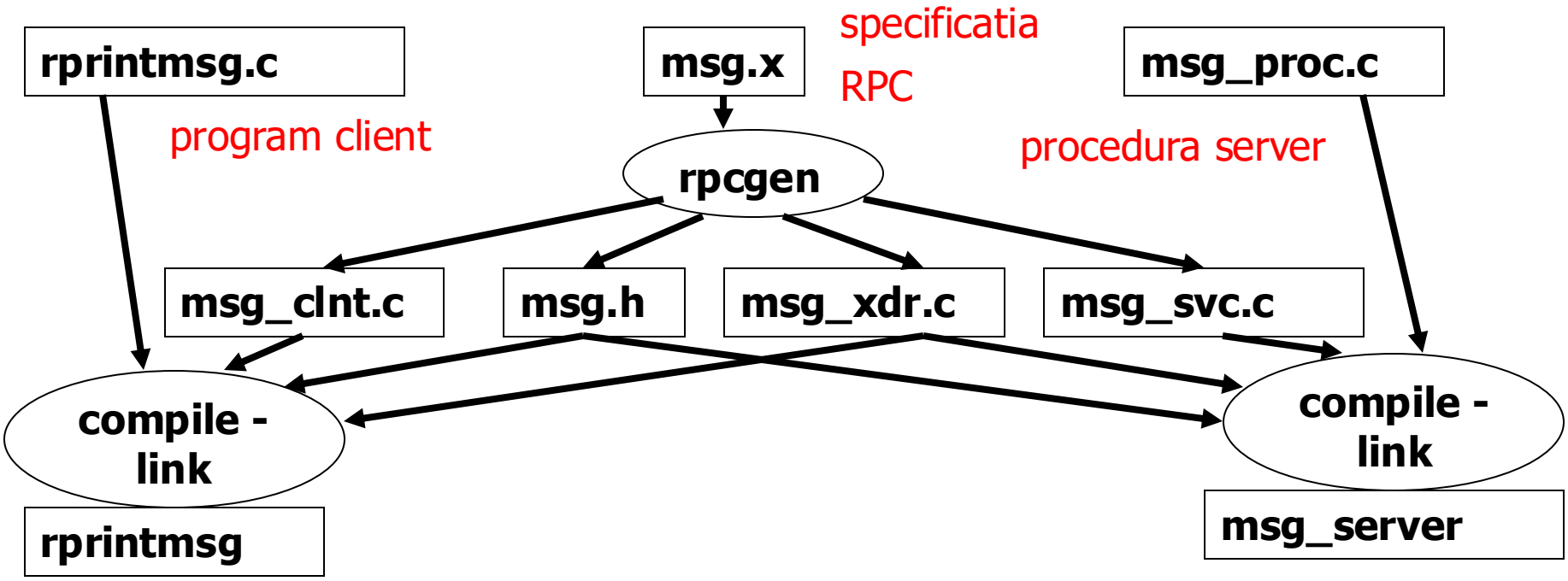
Reprezentarea datelor pentru XDR

int	complement fata de 2, 32 biti, little endian
unsigned int	32 biti little endian
float	32 biti: semn, exponent pe 8 biti, mantisa pe 23 biti
double	64 biti: semn, exponent pe 11 biti, mantisa pe 52 biti
string	lungime sir pe 4 octeti, n octeti sir plus r octeti pentru umplutura la multiplu de 4 octeti
tablou	lungime fixa: n elemente de la 0 la n-1
	lungime variabila: lungime pe 4 octeti urmata de n elemente de la 0 la n-1
struct	componentele in ordinea declararii
union	discriminant pe 4 octeti urmat de valoarea de pe varianta selectata
void	0 octeti

Compilatorul rpcgen

Pe baza specificatiei RPC (fisierul msg.x), rpcgen genereaza :

- procedurile din **stub client** (msg_clnt.c)
- procedurile din server: **main, dispecer, stub** (msg_svc.c)
- procedurile de **marshalling** si **unmarshalling** XDR (msg_xdr.c)
- fisier antet corespunzator definitiilor de proceduri si tipuri din msg.x (msg.h)



Transparenta? Modificari fata de apel local

Programele folosesc direct socketi si **functii** specifice care faciliteaza conectarea intre client si server

Clientul

- initializeaza interfata RPC cu ***clnt_create***
- care creeaza un **RPC handle** la programul si versiunea specificate, pe o gazda data;
  - parametrii includ:
    - nume server
    - nume si versiune program (generate de rpcgen in msg.h)
    - protocolul de transport utilizat
- apelata inaintea oricarei proceduri la distanta



Transparenta? Modificari fata de apel local

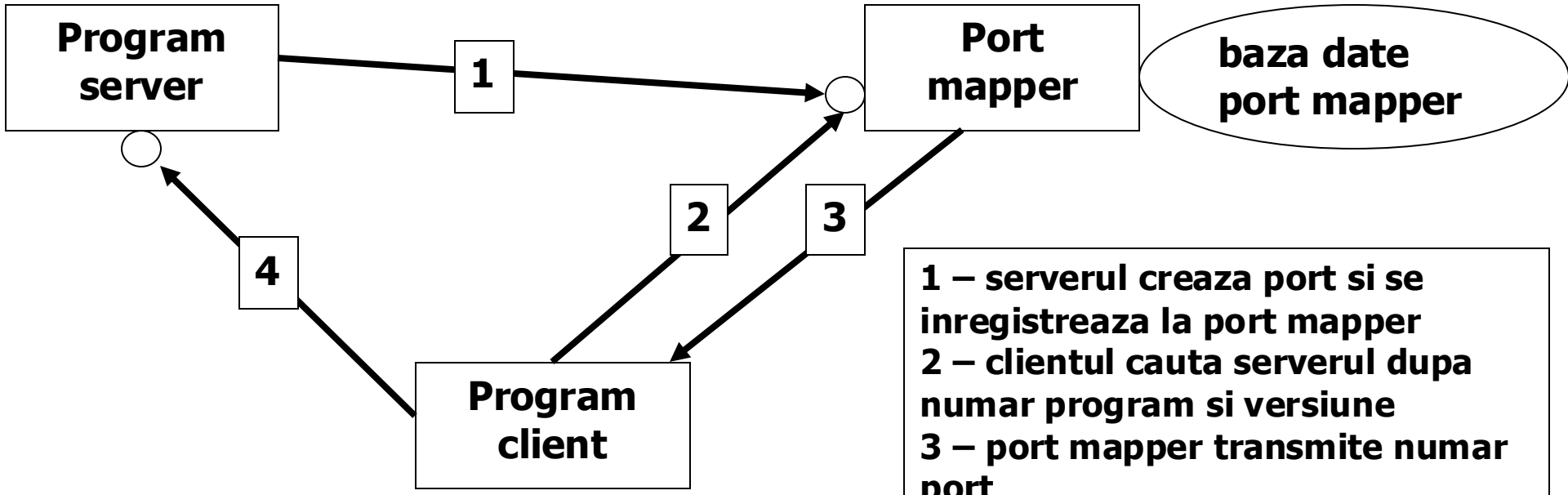
Serverul

- executa stub-ul si pune procesul in background
- creaza un socket si leaga un port local la socket
- apeleaza ***svc_register*** pentru a inregistra portul, numarul si versiunea programului (se contacteaza ***port mapper***)
- executa ***listen*** (asteapta cererile clientilor)

Binding

Foloseste **port mapper** aflat la un **port cunoscut** pe fiecare masina care ruleaza RPC.

La el se inregistreaza toate **serverele** de pe masina locala prin: **nr program, nr versiune, nr port**



DCE RPC

- Distributed Computing Environment / Remote Procedure Call
- Dezvoltat de Open Software Foundation (acum The Open Group)
- Proiectat pentru Unix si extins la alte OS (VMS, Windows)
- Defineste **servicii** client-server (incluse in DCE RPC)
  - Distributed file service
  - Directory service
  - Security service
  - Distributed time service

Obiective DCE RPC

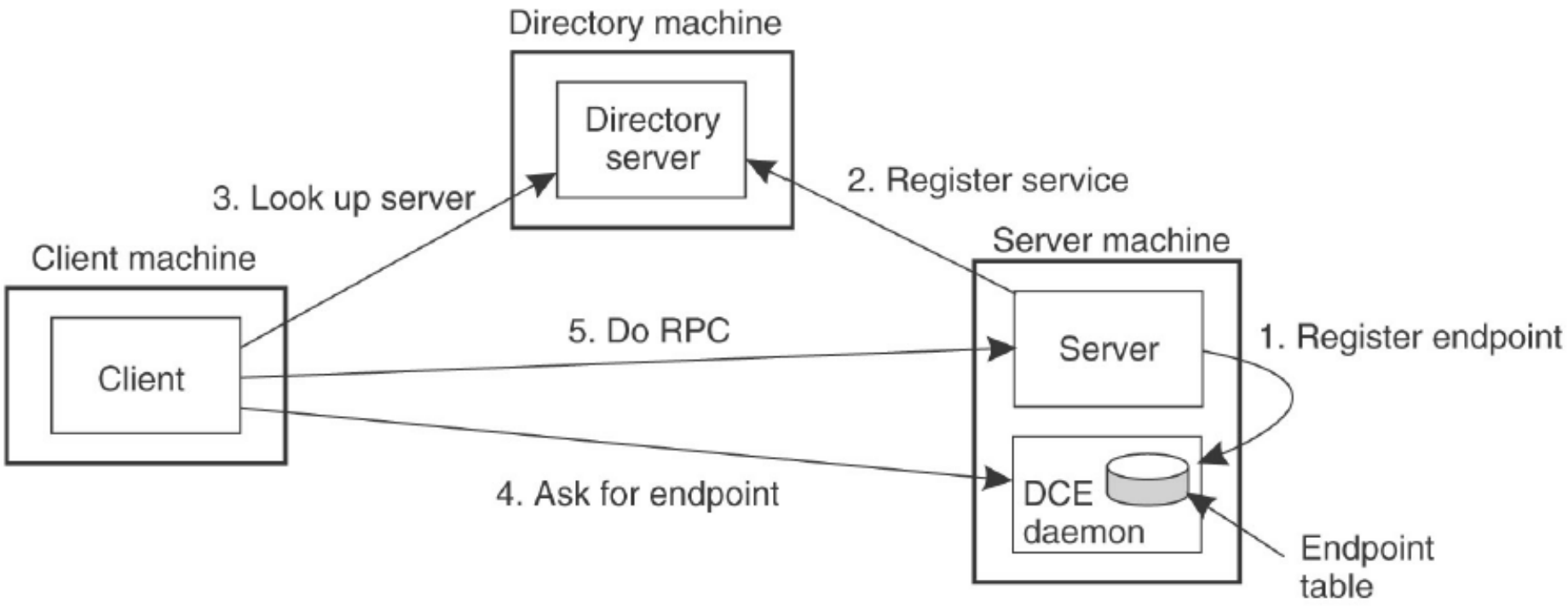
- Transport mesaje intre client si server
- Conversie tip date
- Transparenta locatiei
- Transparenta limbajului (ex. client Java, server C)
- Transparenta SO
- Transparenta hardware
- Transparenta protocoalelor de retea

Legare client - server in DCE

Legarea la server

Sun – programatorul trebuie sa cunoasca masina serverului

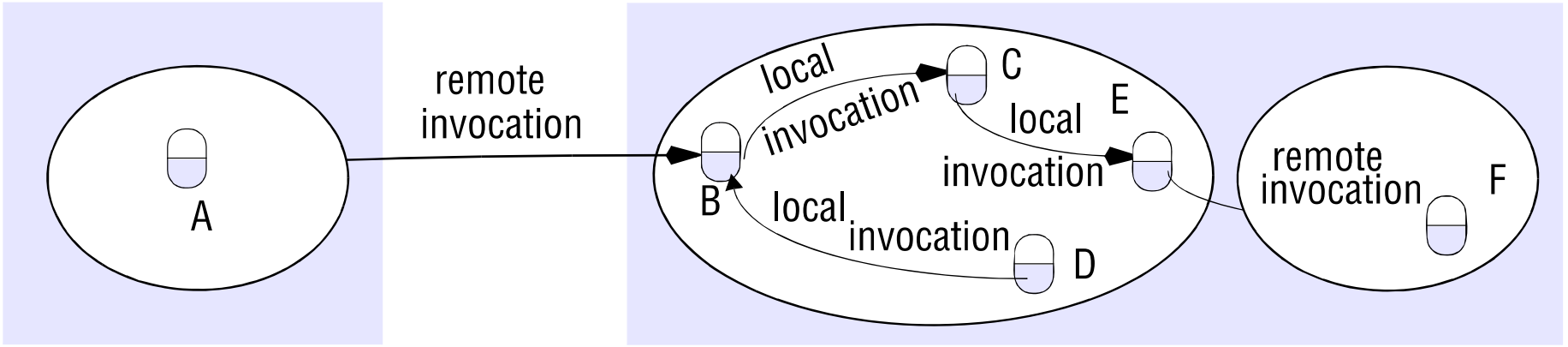
DCE – serverul inregistreaza endpoint (port) cu daemon local DCE si corespondenta {nume_server, masina} cu un server de directoare



Remote Method Invocation - RMI

Java RMI

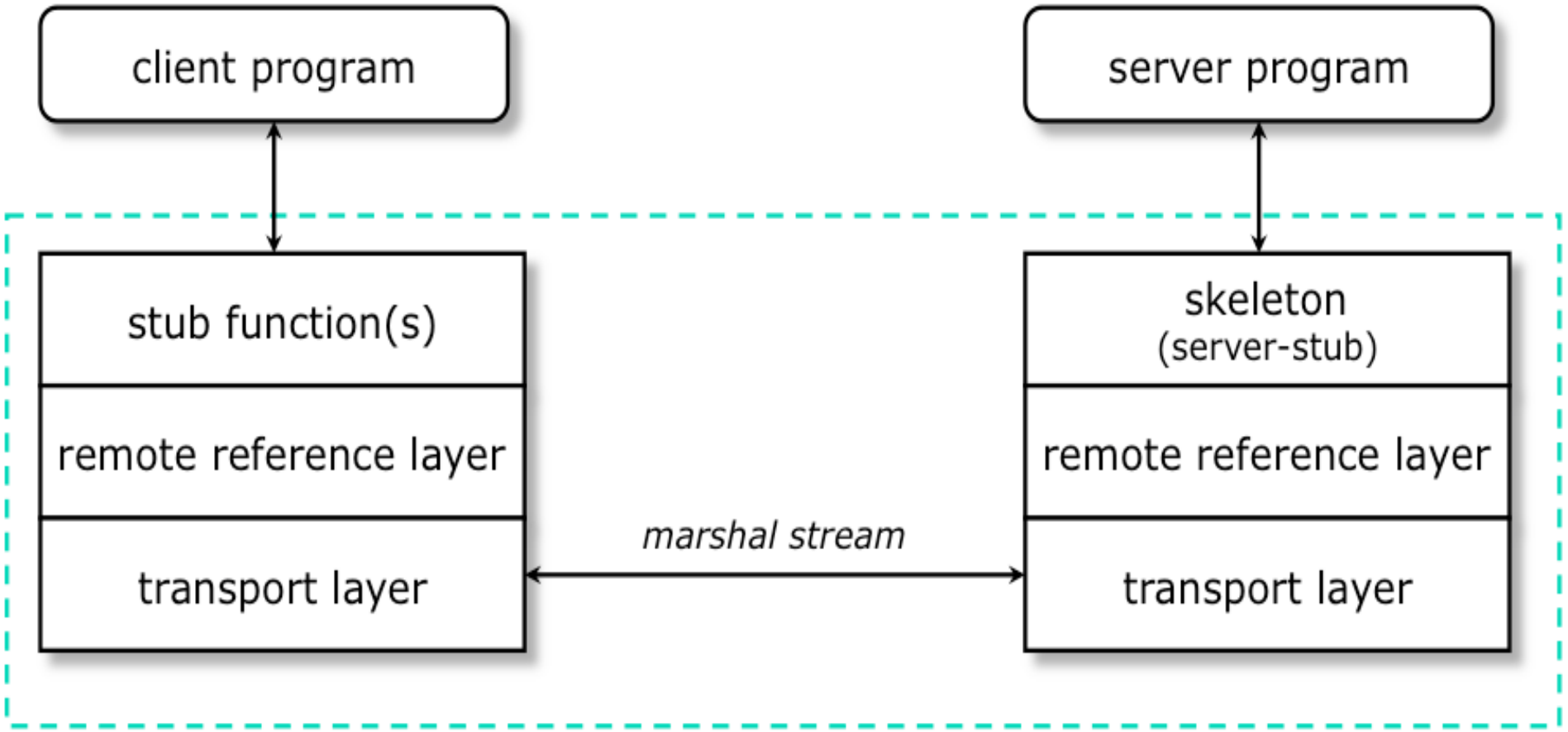
- Java RMI – sistem de obiecte distribuite integrate in Java
- Permite invocari ale metodelor obiectelor aflate in alte JVMs (Java Virtual Machines), deci in alte spatii de adrese (remote objects)
- Cu Java RMI un client poate obtine referinta unui obiect remote si poate invoca metodele acestuia ca si cum ar invoca metodele unui obiect local (din aceeasi masina virtuala)



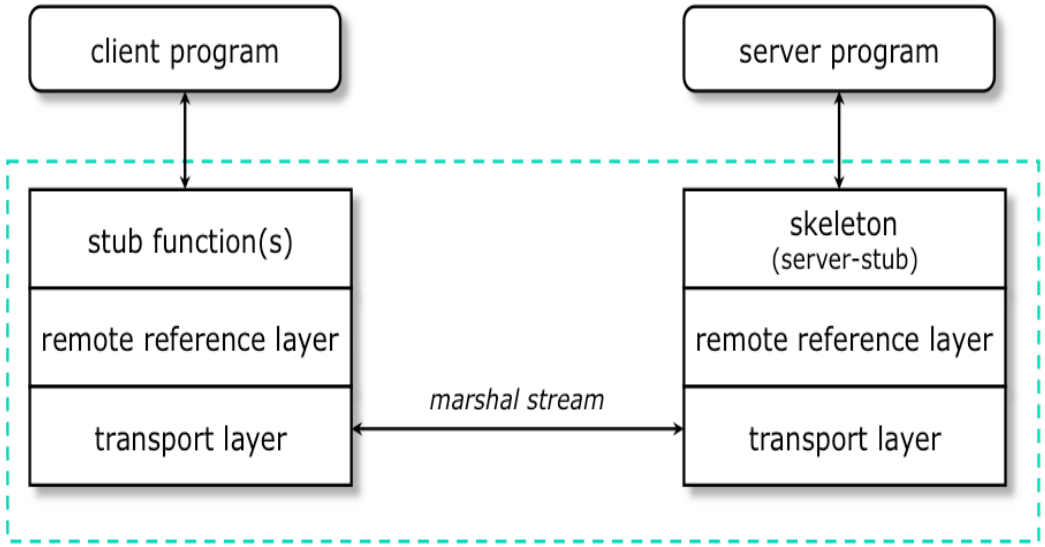
Arhitectura Java RMI

Include stub-urile client si server si modulele de comunicare cu functii similare RPC

si module pentru gestiunea referintelor remote



Rolurile Componentelor

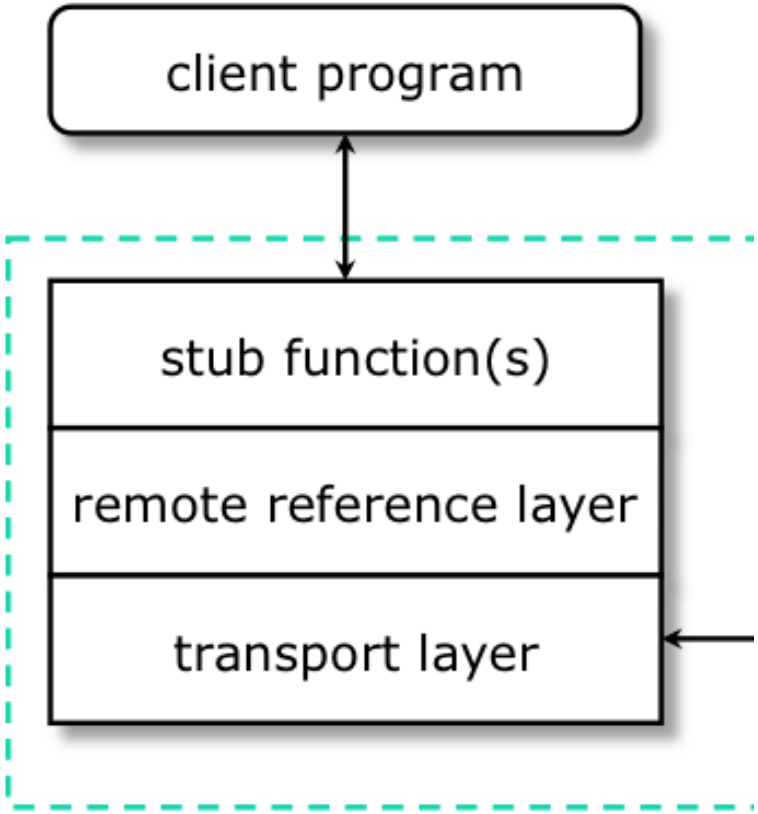


stub client

- stub-ul este un obiect local clientului, care implementeaza aceleasi metode ca obiectul remote
- un client initiaza o RMI apeland o metoda a obiectului stub
- stub-ul converteste (marshal) argumentele metodei intr-o forma serializata
- cere managerului de referinte remote sa transmita invocarea de metoda si argumentele obiectului remote
- asteapta apoi mesajul de raspuns, il converteste si returneaza rezultatul catre client

managerul de referinte remote

- o referinta remote contine
  - adresa de retea si portul (endpoint) serverului
  - un identificator local al obiectului remote in spatiul de adrese al serverului
- la client, managerul de referinte stabileste o conexiune de transfer cu serverul (folosind adresa de retea si portul)

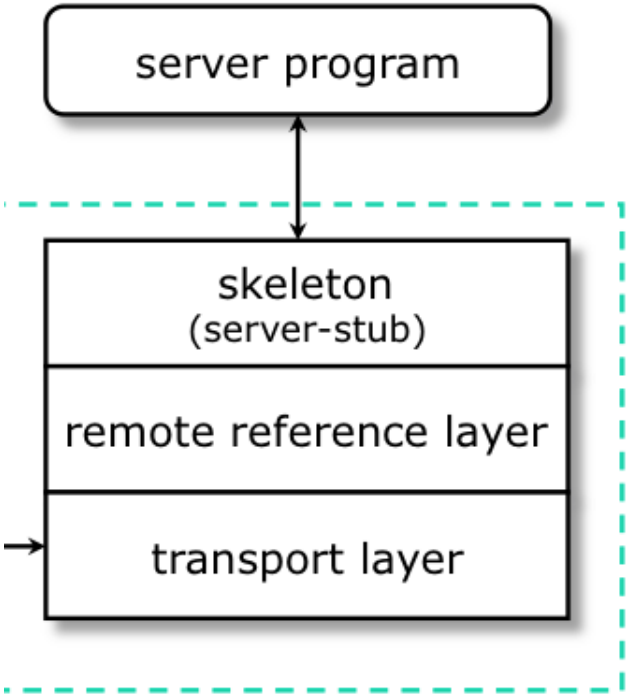


- construieste o cerere de transport pentru server, incluzand identificatorul local al obiectului remote
- paseaza cererea nivelului transport

managerul de referinte remote – la server

•la server, managerul de referinte primeste cererea de la nivelul transport

•o paseaza skeleton-ului corespunzator
identificatorului local al obiectului



skeleton la server

•converteste cererea remote intr-un apel de metoda corect (unmarshaling)

•invoca metoda obiectului remote

•Daca metoda genereaza un raspuns, trimiterea lui pe calea inversa la client reclama transformari similare la nivelele stub / skeleton si manager de referinte

Clase serializable

- Obiectele non-remote sunt pasate prin valoare; conditia este ca obiectul sa fie serializabil (sa implementeze interfata *Serializable*)
  - pentru transmitere, obiectul este serializat (marshaled) – reprezentat intr-o forma fara pointeri, ca o serie de octeti
  - forma originala a continutului se obtine prin de-serializare
  - nu toate obiectele sunt serializabile
    - ex. obiectele legate de platforma (descriptori de fisiere, socketi)
- Cand un obiect este pasat prin valoare, se creaza un nou obiect in procesul receptor
  - metodele obiectului creat pot fi invocate local
  - starea obiectului se poate schimba, putand deveni diferita de cea a obiectului din procesul transmitator

Descrierea unei clase remote

Descrierea unei clase remote are doua parti

- definirea interfetei
  - care sta la baza definirii clasei remote
- definirea clasei propriu zise – de fapt doua clase diferite
  - una la server - descrie implementarea interfetei
    - obiectele clasei exista in spatiul de adrese al serverului
    - un obiect poate contine si metode care nu apar in interfata si pot fi invocate local
  - alta la client
    - descrie functionalitatea proxy

Clase remote

- Un obiect remote este pasat ca parametru referinta
  - obiectele sunt identificate printr-o referinta (object handle) care poate fi
    - (ineficient ca volum transferat) un proxy (care reprezinta local obiectul remote - ale carui metode pot fi invocate local de un client) – trebuie sa fie serializabil
    - (mai eficient) o specificatie a claselor necesare pentru a construi stubul (proxy-ul) la client
  - obiectele trebuie sa ie serializabile

Un exemplu

Definitia interfetei HelloInterface.java

```
import java.rmi.*;
public interface HelloInterface extends Remote {
        // interfata HelloInterface extinde interfata Remote
        // implica includerea unei RemoteException
    public String say () throws RemoteException;
}
```

Clasa Remote Hello.java

```
import java.rmi.*;
import java.rmi.server.*;

public class Hello
 extends UnicastRemoteObject
                                  // permite obtinerea unei referinte
 implements HelloInterface {
  private String message;
  public Hello (String msg) throws RemoteException{
     message = msg;          // mesaj standard
   }
   public String say() throws RemoteException {
      return message;  // intoarce mesajul standard
   }
 }
```

Serverul (HelloServer.java)

```
import java.rmi.*;
import java.rmi.server.*;
public class HelloServer {
 public static void main (String argv[]) {
 try {
Naming.rebind ("Hello", new Hello ("Hello, world!"));
      System.out.println ("Hello Server is ready.");
    } catch (Exception e) {
      System.out.println ("Hello Server failed: " + e);
    }
 }
}
```

instantiaza obiectul server cu mesajul “Hello, world!”

inregistreaza serviciul cu RMIregistry

asociaza numele “Hello” cu obiectul remote

Clientul (HelloClient.java)

```
import java.rmi.*;
public class HelloClient {
 public static void main (String argv[]) {
  try {
// clientul apeleaza RMI registry pentru a afla referinta obiectului remote
// specifica un nume (Hello) in format URL:  //NumeMasina:port/NumeObiect
// Naming.lookup returneaza referinta (proxy hello)
    HelloInterface hello =
    (HelloInterface) Naming.lookup("//localhost/Hello");
// clientul invoca metoda “say” si afisaza rezultatul
    System.out.println (hello.say());
  }
  catch (Exception e) {
  System.out.println ("HelloClient exception: " + e);}
 }
}
```

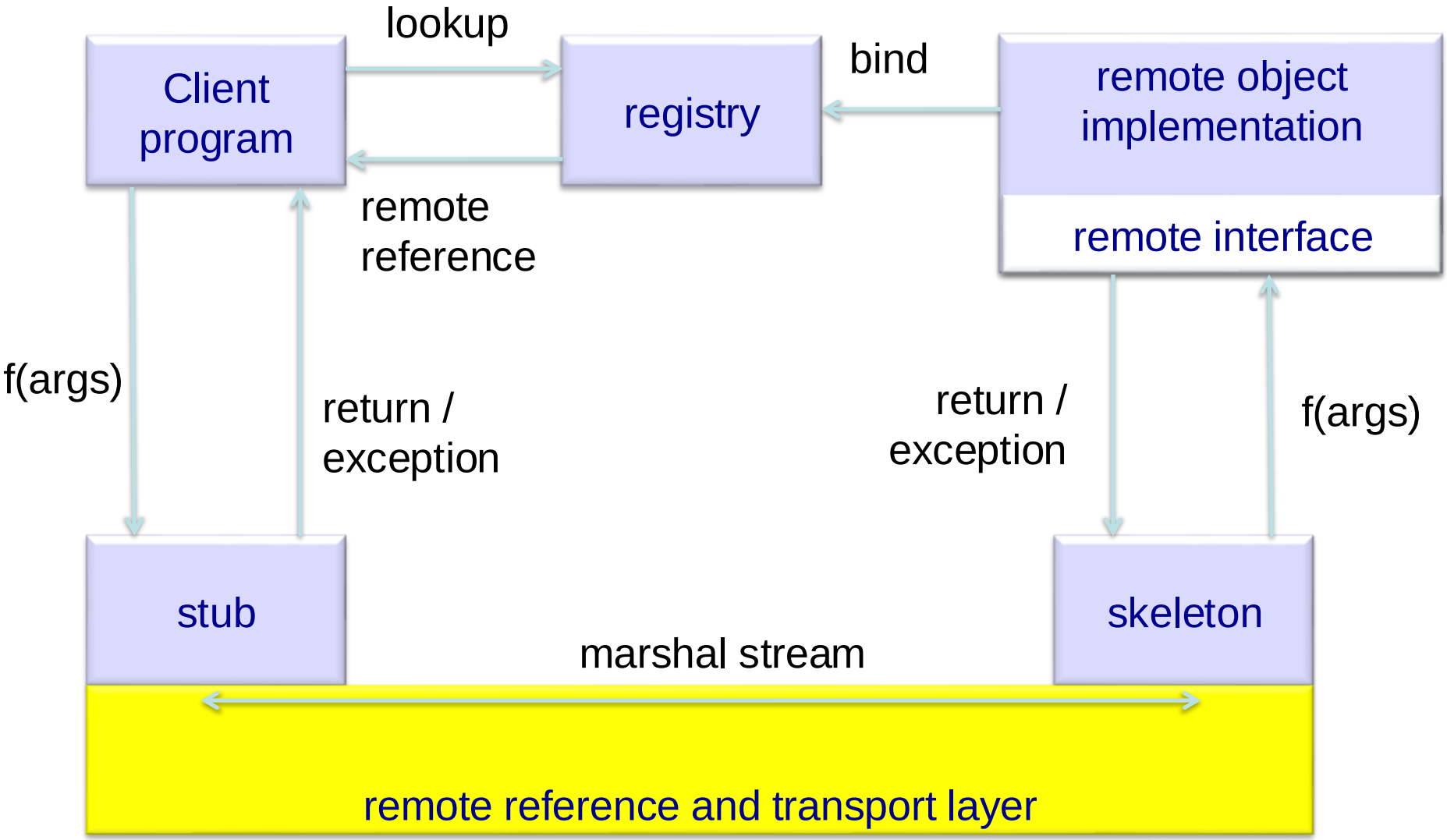
Pasii de compilare si executie

- Compilarea interfetei si implementarii
- Generarea stub client si skeleton server
- Porneste in background object registry (daca nu este in executie)
- Porneste in background serverul
- Executa clientul

Obs. in RMI, o interfata este implementata de doua clase

  - clasa remote – implementarea propriu zisa a serviciului
  - stub-ul client – care este un proxy de server
- Proxy include toate informatiile ce permit clientului sa invoce metodele obiectului remote
  - este serializabil (deci poate fi transmis de la un proces la altul)
- Cand se paseaza referinta obiectului server, daca clientul nu detine clasa proxy-ului, codul ei este descarcat automat

Configuratia de componente si invocari



Invocari dinamice de metode

- Invocarile pot fi generate si dinamic, la executie
  - utilitate – adaugarea unor servicii noi nu reclama re-compilarea aplicatiilor
  - apropie RMI de arhitectura orientata pe servicii
- Invocarile dinamice folosesc o forma generica

  invoke (obiect, metoda, in_param, out_param)

care include

referinta la obiectul remote

numele metodei si

parametrii

- Invocarile se genereaza la executie pe baza informatiilor primite de la un director de servicii

Despre interoperabilitatea RMI

RMI construit initial doar pentru Java

Nu a avut obiective legate de interoperabilitatea

- cu diverse sisteme de operare (precum CORBA)
- cu alte limbaje (precum SUN, DCE, CORBA)
- cu alte arhitecturi

Nu are nevoie de o reprezentare externa a datelor

- toate partile participante ruleaza o JVM

Beneficiu: simplitatea si designul clar



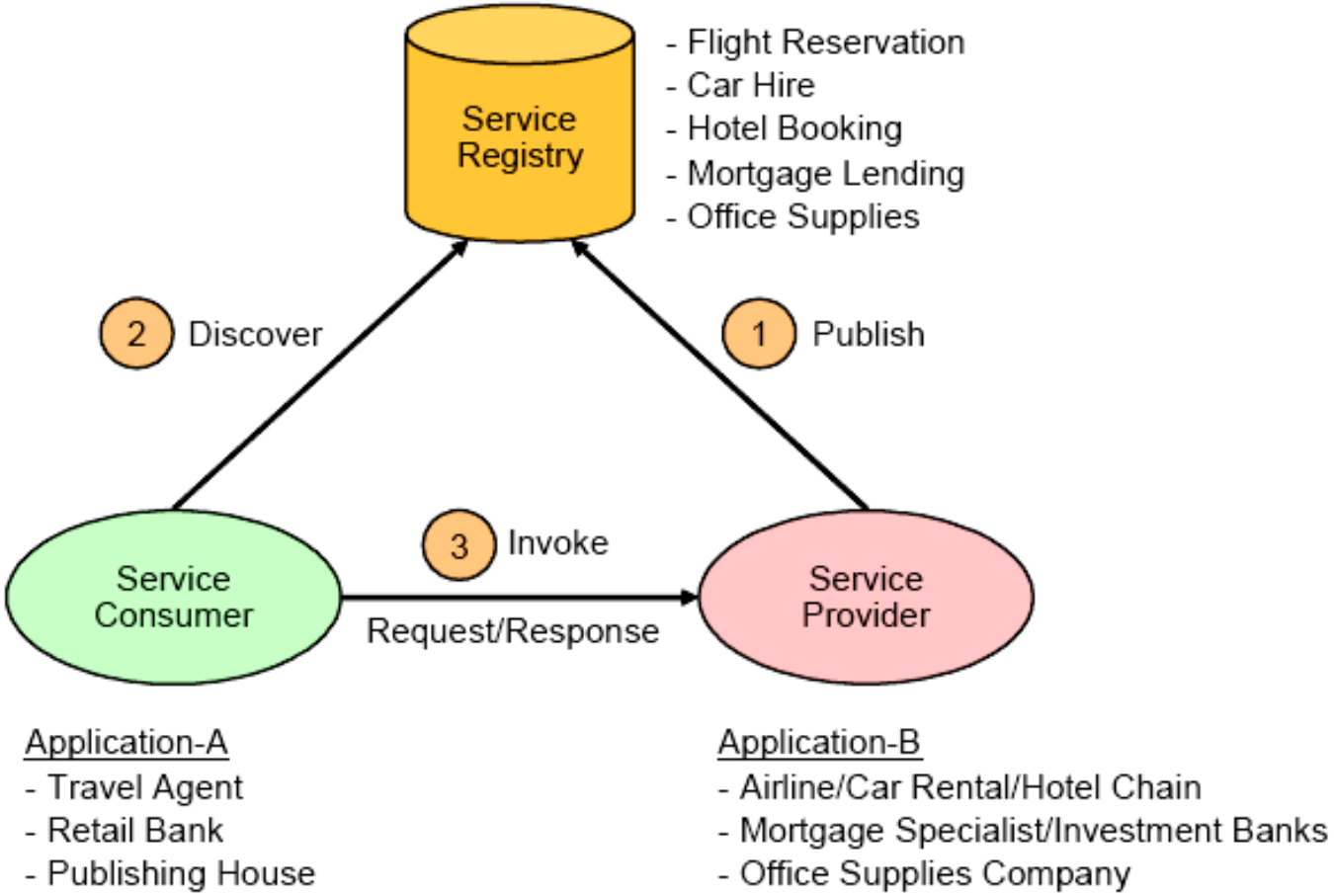
Microservice Architecture and Docker

Service-Oriented Architecture (SOA):

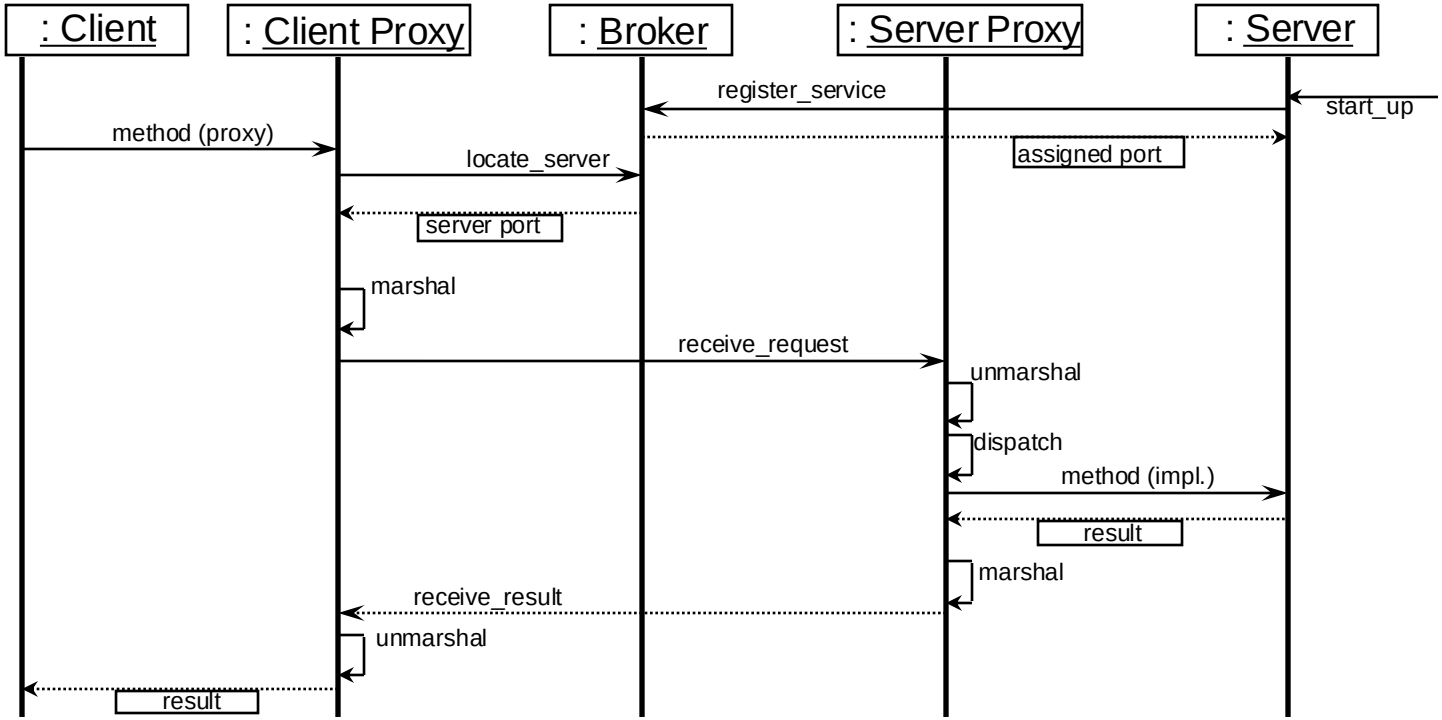
- *The policies, practices, frameworks that enable application functionality to be provided and consumed as sets of services published at a granularity relevant to the service consumer.*
- *Services can be*
  - *invoked,*
  - *published and*
  - *discovered,*

and are abstracted away from the implementation using a single, standards-based form of interface.

SOA Components and Operations



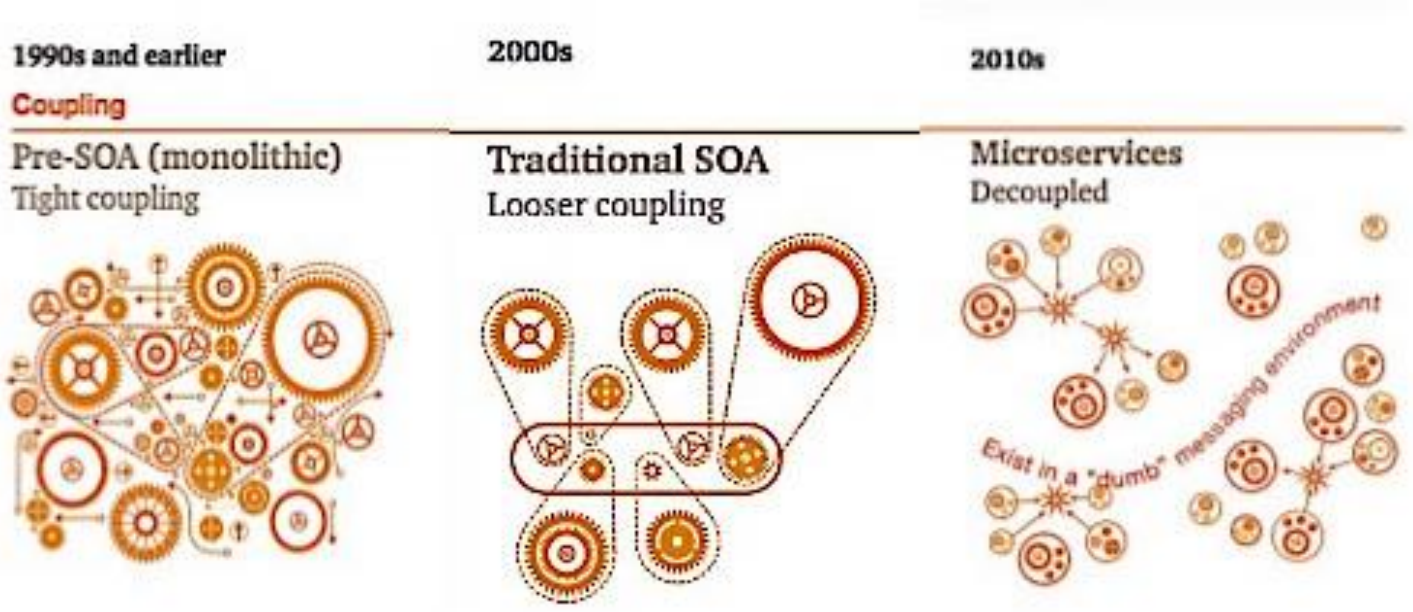
Broker Design Pattern Dynamics



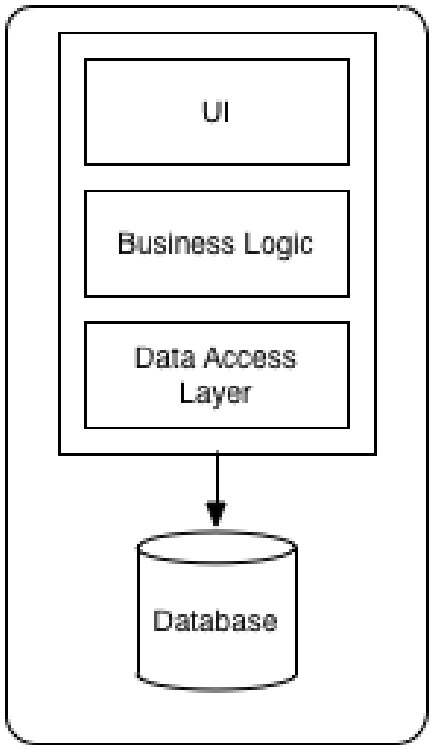
Broker tools provide the generation of necessary client & server proxies from higher level interface definitions



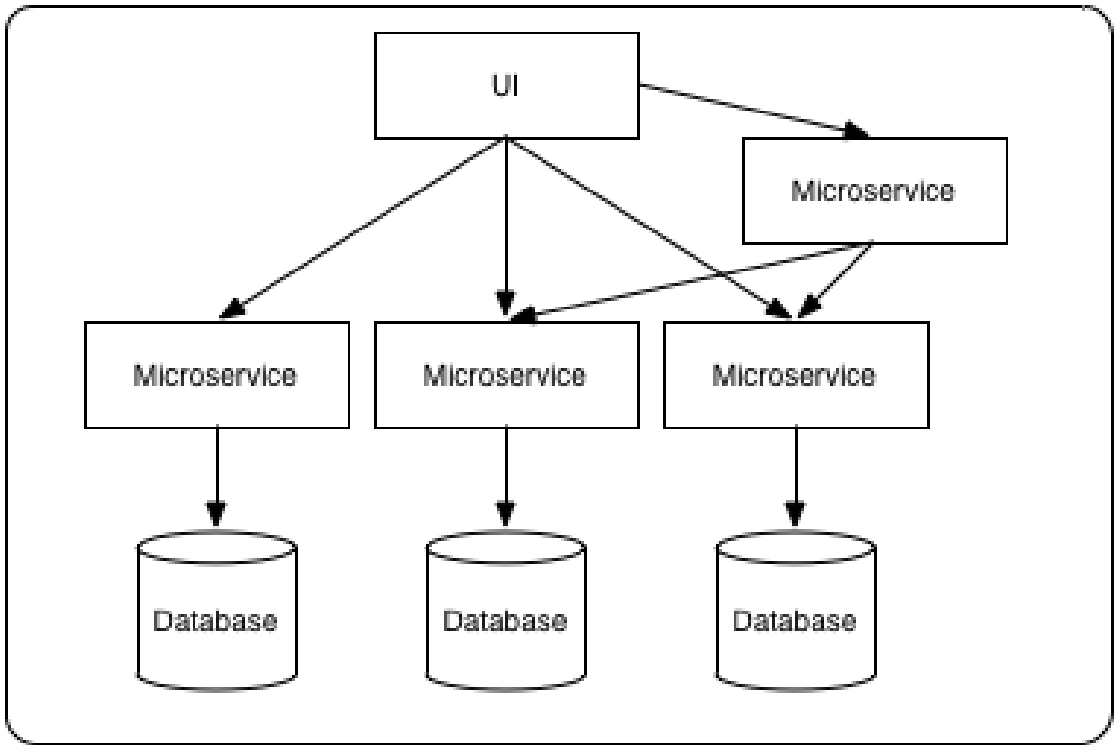
Microservices



Microservices

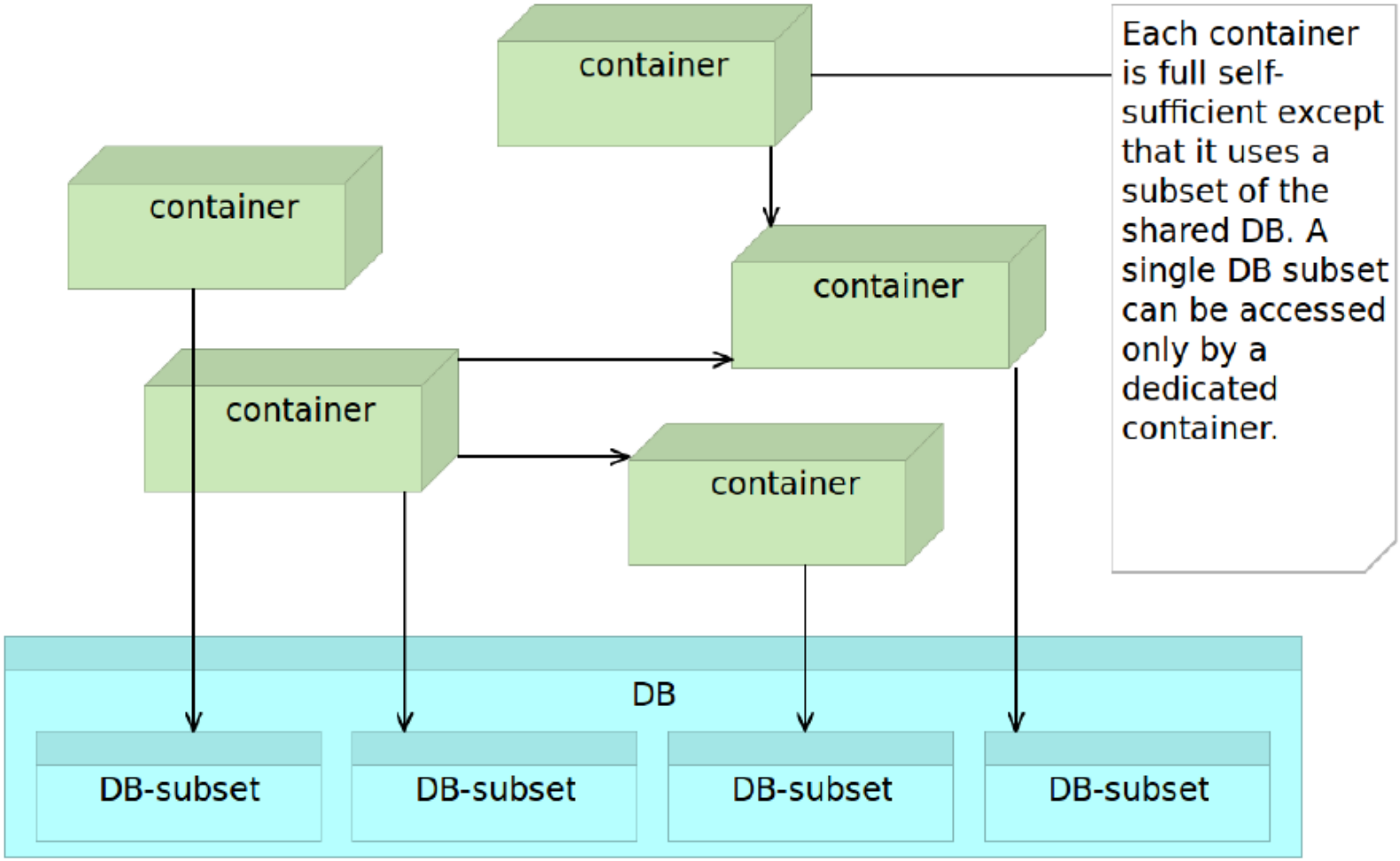


Monolithic Architecture



Microservices Architecture

Microservices Accessing the Shared Database



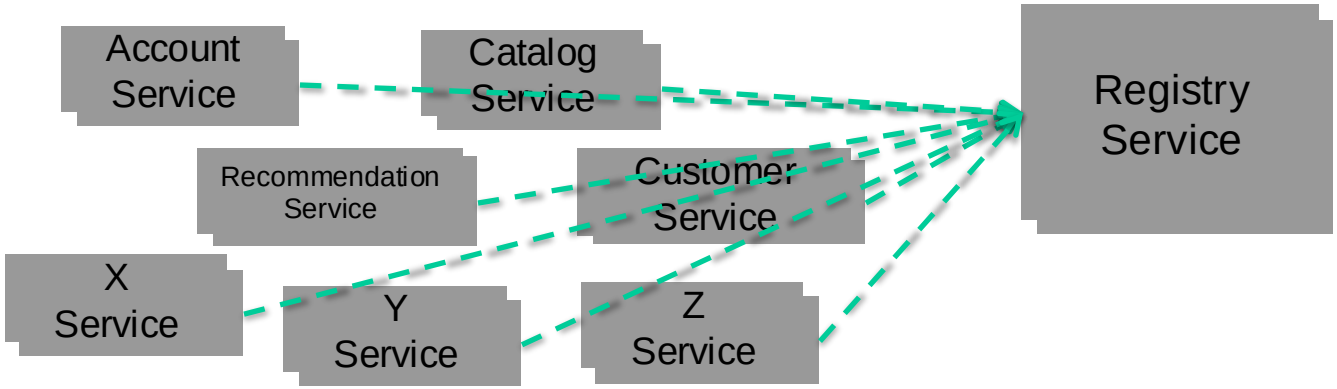
Microservices Characteristics

- Many smaller (fine grained), clearly scoped services
  - Single Responsibility Principle
  - Independently Managed
- Clear ownership for each service
  - Typically need/adopt the “DevOps” model



Service Discovery

- 100s of MicroServices
  - Need a Service Metadata Registry (Discovery Service)

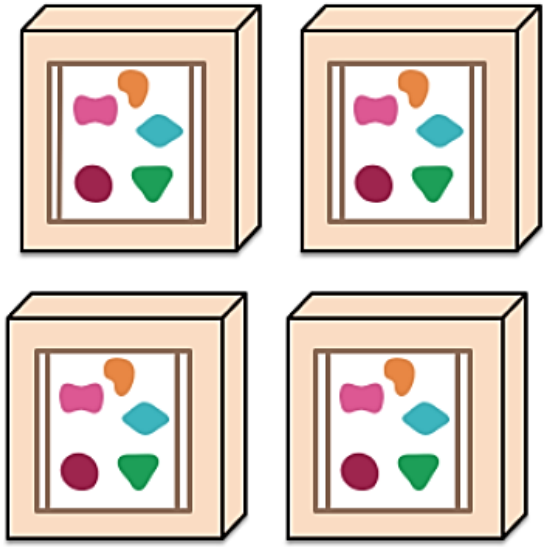


Microservices. Scalability

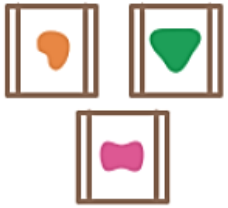
A monolithic application puts all its functionality into a single process...



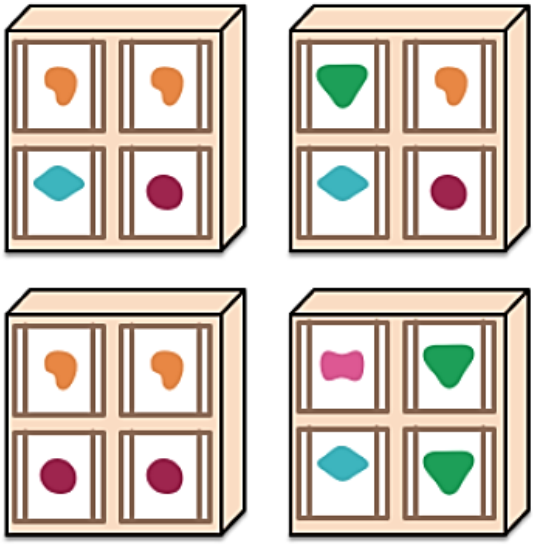
... and scales by replicating the monolith on multiple servers



A microservices architecture puts each element of functionality into a separate service...



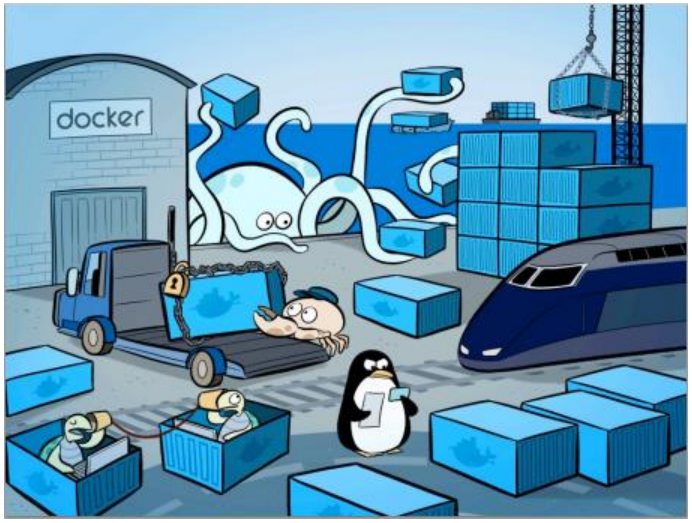
... and scales by distributing these services across servers, replicating as needed.



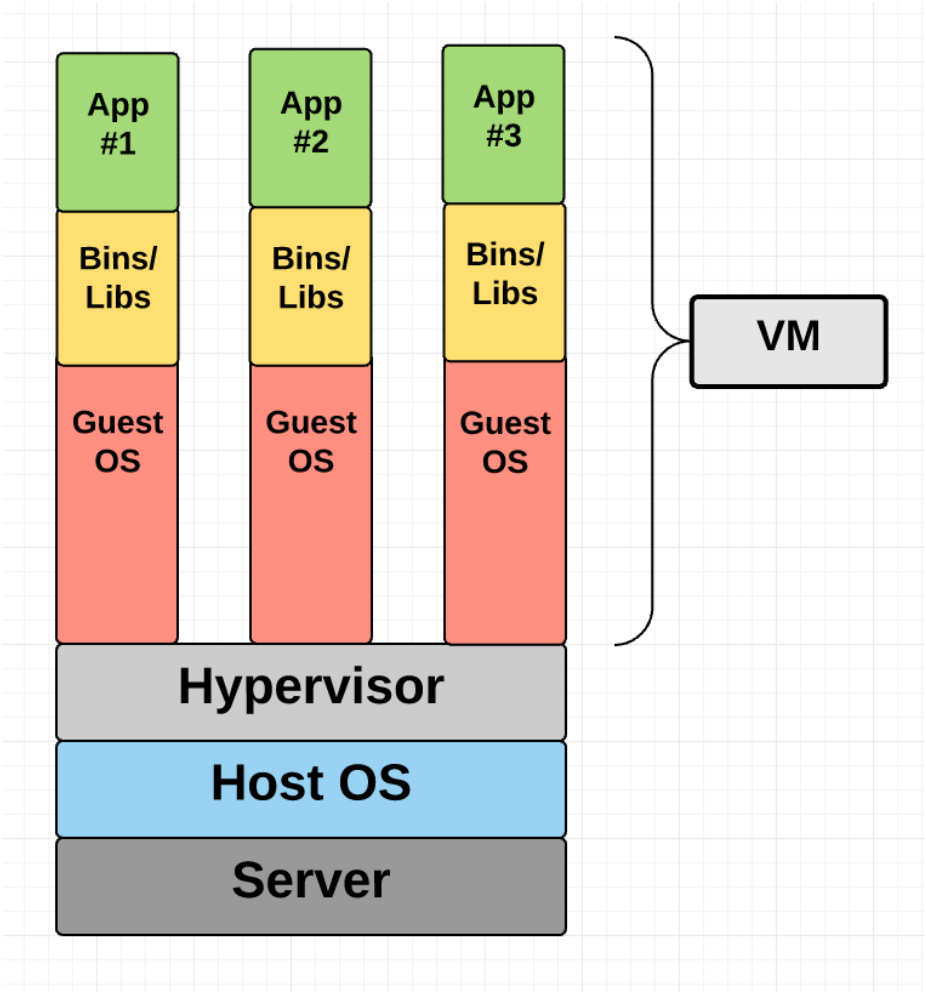
Docker: Containerization for Software



Docker

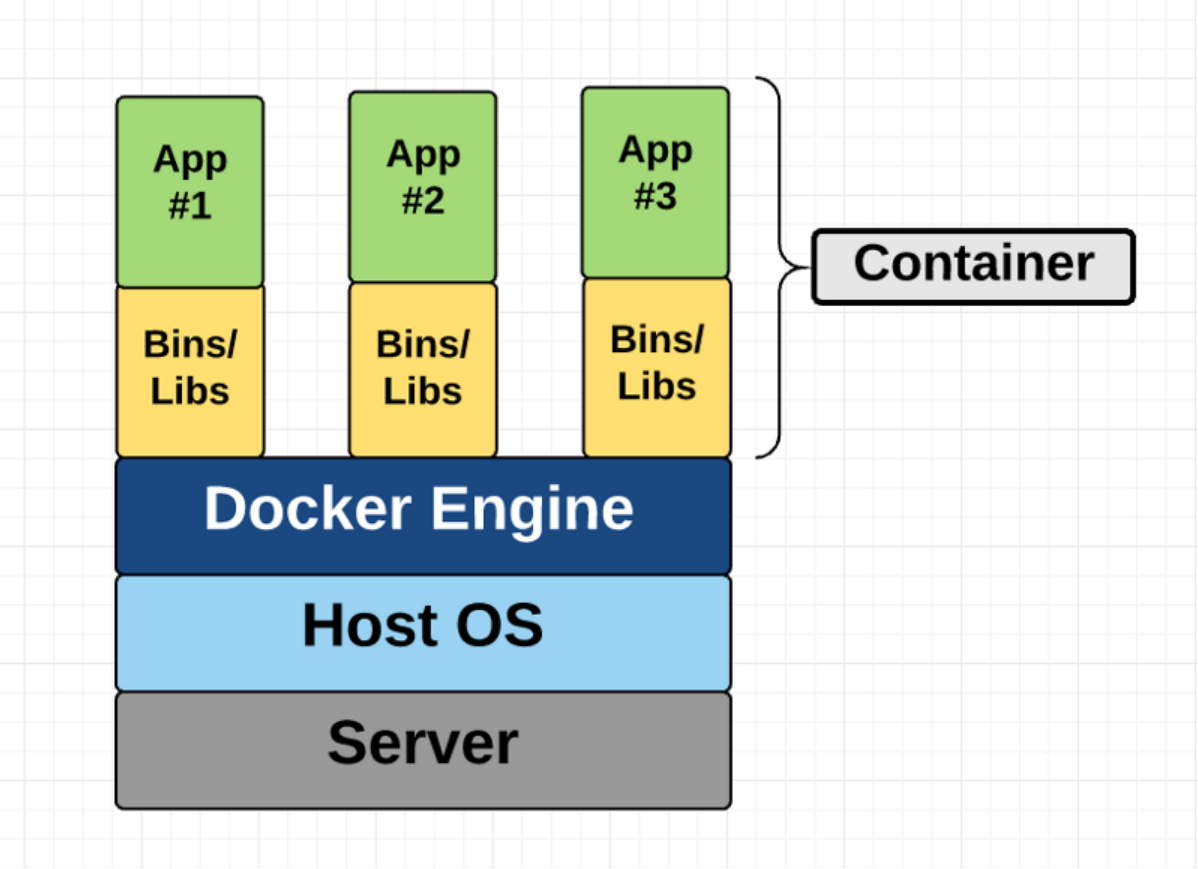


“Docker is an open platform for developers and sysadmins to build, ship, and run distributed applications”



Virtual Machine

Container


App #1
App #2
App #3
Bins/ Libs
Bins/ Libs
Bins/ Libs
Container
Docker Engine
Host OS
Server



So why Docker?

- Containers are far from new;
  - Google has been using their own container technology for years.
  - Others Linux container technologies include
    - Solaris Zones,
    - BSD jails, and
    - LXC, which have been around for many years.
- Docker is an open-source project based on Linux containers. It uses Linux Kernel features.



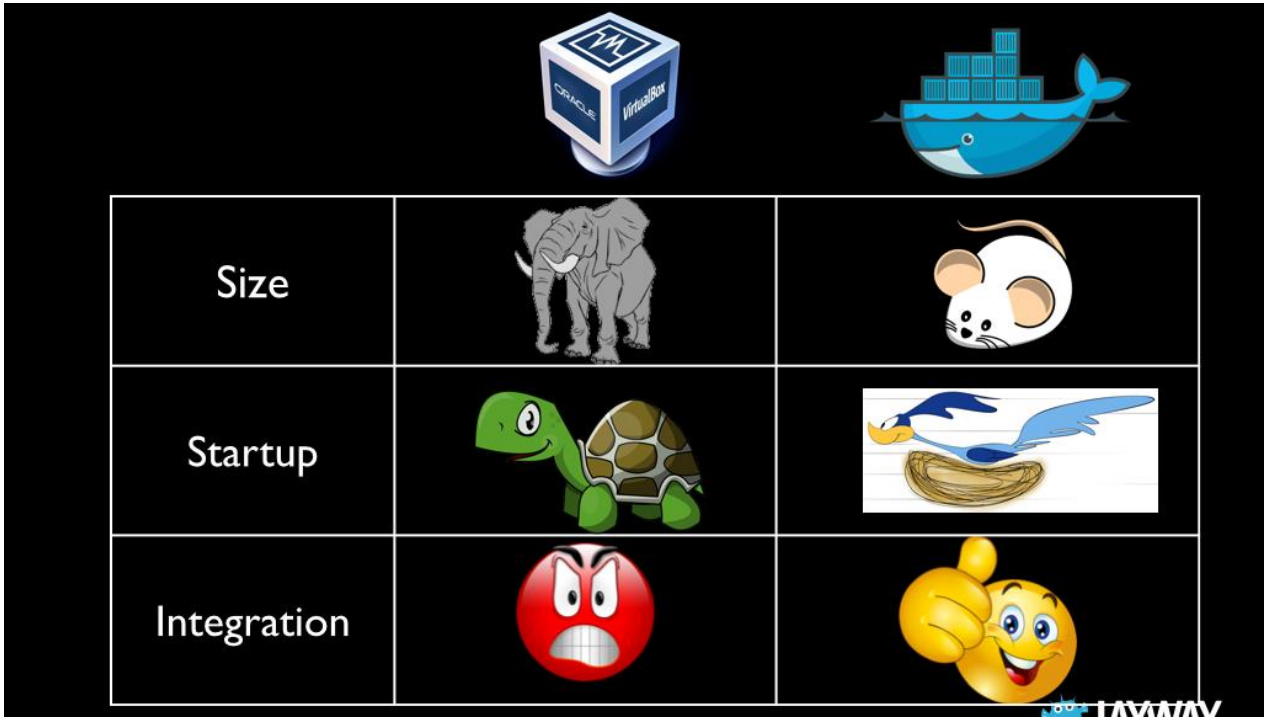
Docker Benefits

1. Local development environments can be set up that are exact replicas of a live environment/server.
2. It simplifies collaboration by allowing anyone to work on the same project with the same settings, irrespective of the local host environment.
3. Multiple development environments can be run from the same host each one having different configurations, operating systems, and software.
4. Projects can be tested on different servers.
5. It gives you instant application portability. Build, ship, and run any application as a portable container that can run almost anywhere.

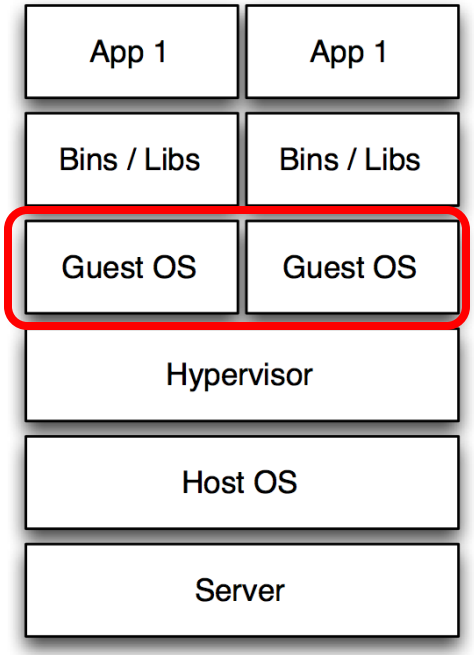
Why Docker?

- **Ease of use.** It allows anyone to package an application on their laptop, which in turn can run unmodified anywhere
  - The mantra is: “build once, run anywhere.”
- **Speed.** Docker containers are very lightweight and fast. Since containers are just sandboxed environments running on the kernel, they take up fewer resources. You can create and run a Docker container in seconds, compared to VMs which might take longer because they have to boot up a full virtual operating system every time.
- **Docker Hub.** Docker users also benefit from the increasingly rich ecosystem of Docker Hub, which you can think of as an “app store for Docker images.” Docker Hub has tens of thousands of public images created by the community that are readily available for use.
- **Modularity and Scalability.** Docker makes it easy to break out your application’s functionality into individual containers. With Docker, it’s become easier to link containers together to create your application, making it easy to scale or update components independently in the future.

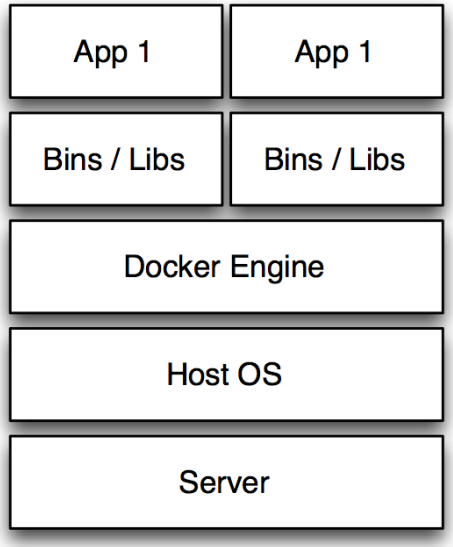
VM vs. Docker



VM vs. Docker (Containers)

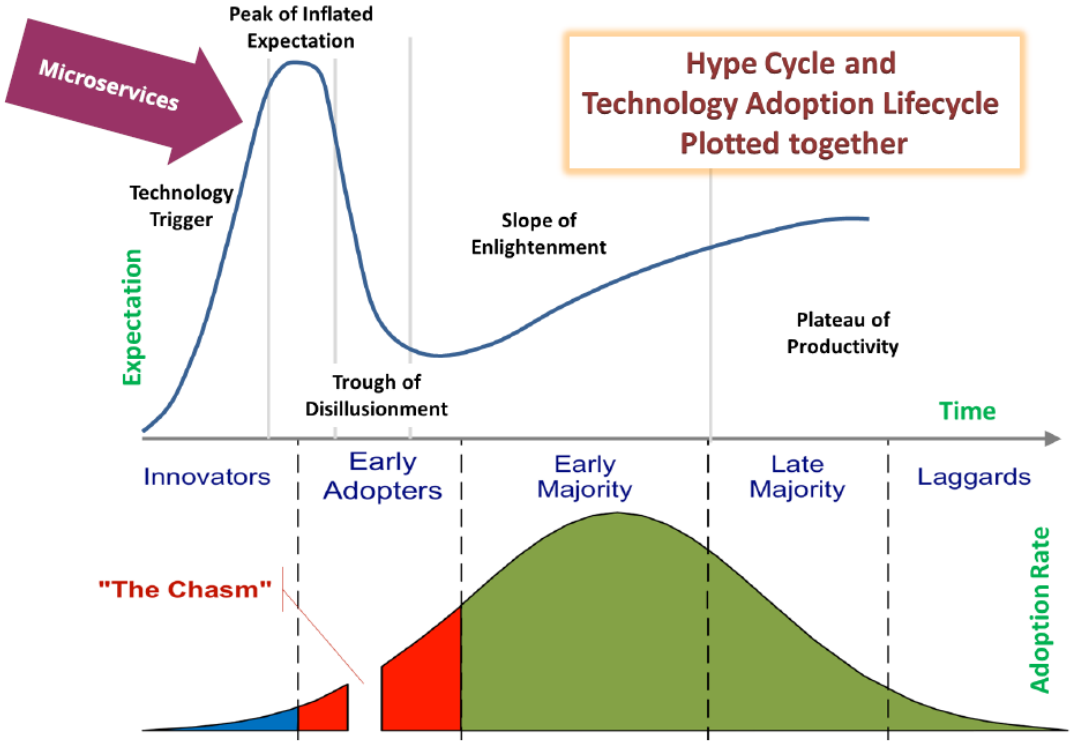


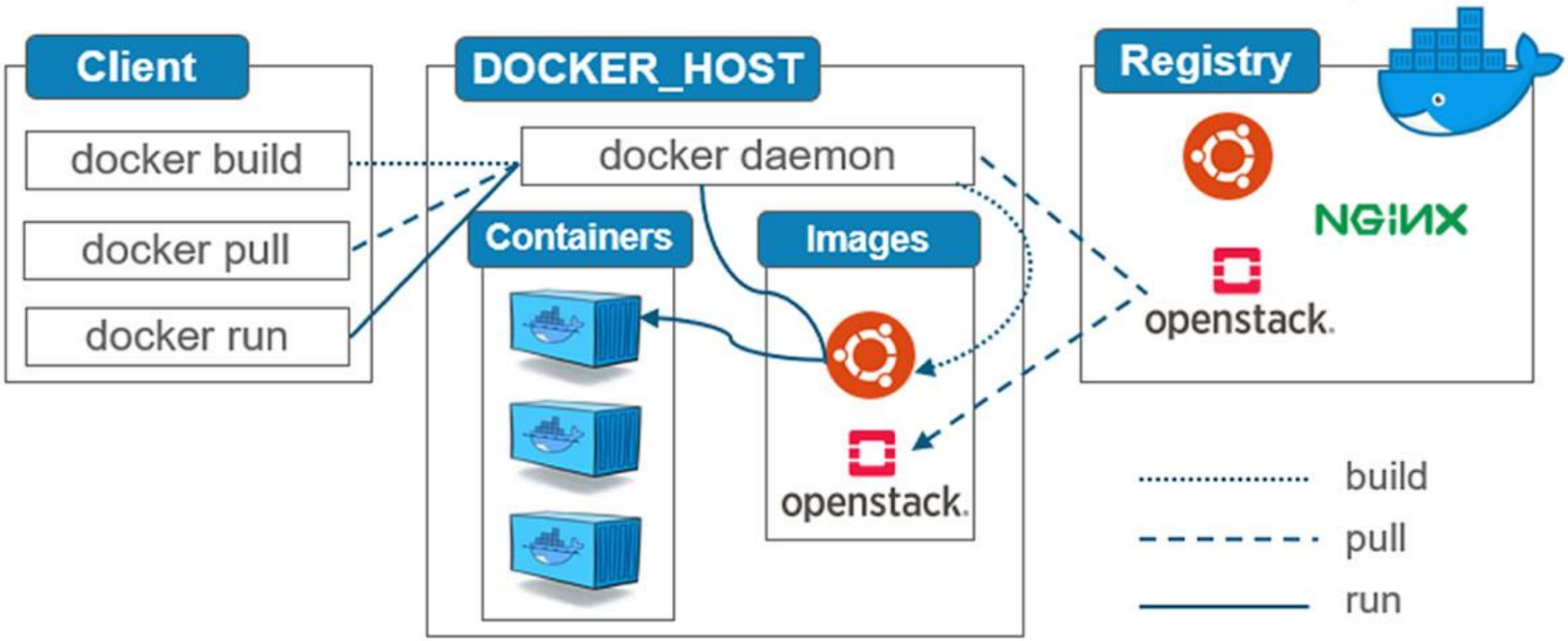
Virtual Machines

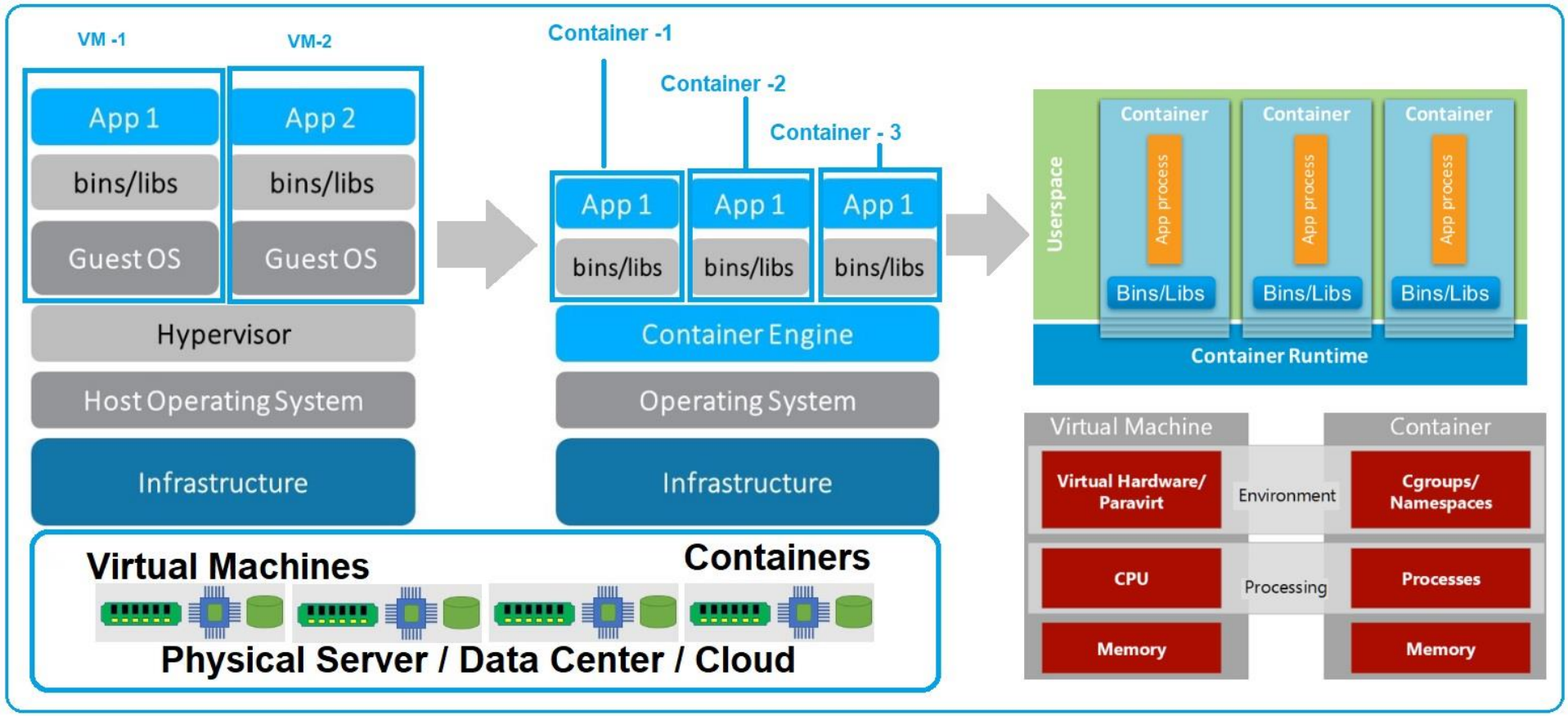


Docker

Gartner Hype Cycle




Client
docker build
docker pull
docker run
DOCKER_HOST
docker daemon
Containers
Images
openstack.
Registry
NGINX
openstack.
build
pull
run


VM -1
VM-2
App 1
App 2
bins/libs
bins/libs
Guest OS
Guest OS
Hypervisor
Host Operating System
Infrastructure
Container -1
Container -2
Container - 3
App 1
App 1
App 1
bins/libs
bins/libs
bins/libs
Container Engine
Operating System
Infrastructure
Virtual Machines
Containers
Physical Server / Data Center / Cloud
Userspace
Container
Container
Container
App process
App process
App process
Bins/Libs
Bins/Libs
Bins/Libs
Container Runtime
Virtual Machine
Container
Virtual Hardware/ Paravirt
Environment
Cgroups/ Namespaces
CPU
Processing
Processes
Memory
Memory

Arhitecturi orientate pe servicii

Web Services

Arhitecturi Orientate pe Servicii

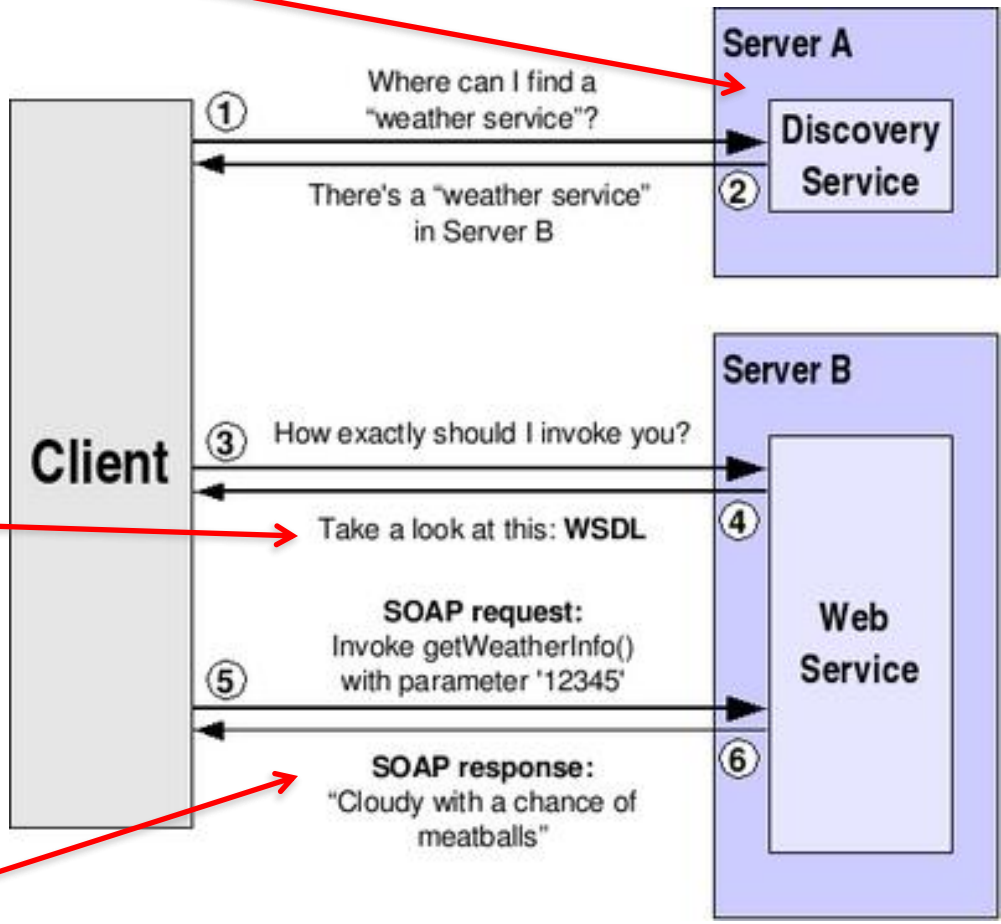
- Servicii Web
- Reprezentarea mesajelor, SOAP
- Descrierea serviciilor, WSDL
- Publicarea si cautarea serviciilor, UDDI
- Utilizarea serviciilor, WSDL - JAX-RPC

Elemente ale arhitecturii serviciilor Web

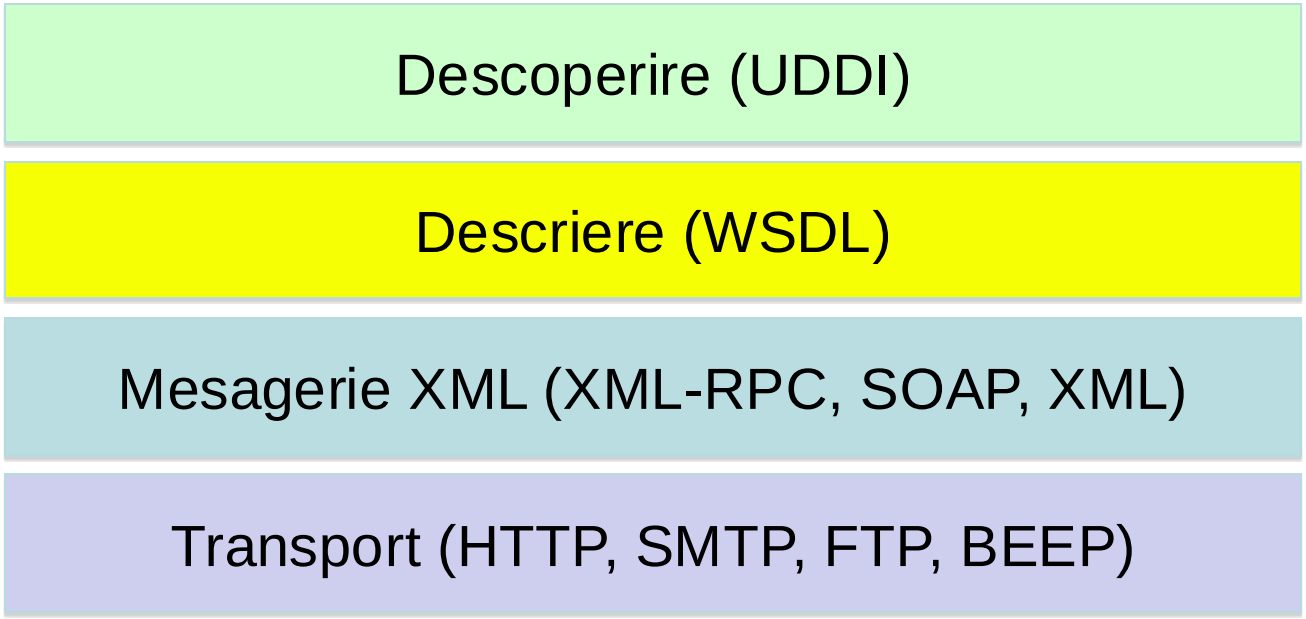
Descoperiea serviciilor Web se face printr-un director distribuit UDDI – Universal Description, Discovery, and Integration.

Serviciul WEB are o interfaţă descrisă într-un format care poate fi procesat de maşină: WSDL - Web Service Description Language.

Serviciile WEB sunt invocate în conformitate cu un protocol, de obicei SOAP - Simple Object Access Protocol (posibil alte protocoale).



Arhitectura orientata pe servicii



Stiva de protocoale incude functionalitati pentru

- transportul mesajelor intre aplicatii
  - BEEP (Blocks Extensible Exchange Protocol) – ofera securitate, tratarea erorilor, dialogul initial intre entitati (handshake)
- codificarea mesajelor intr-un format comun XML
- descrierea interfetei publice a unui serviciu
- evidenta serviciilor si mecanismul de publicare/subscriere

SOAP – Simple Object Access Protocol

- SOAP defineste o schema de utilizare a XML pentru
  - mesaje asincrone
    - ex. procesarea unei cumparaturi – dureaza mai mult si implica un schimb mai bogat de mesaje, inclusiv mesaje periodice de stare, detalii la finalul operatiilor etc.
  - cerere-raspuns – RPC
    - ex. verificarea online a unui card bancar
- Specifica
  - regulile de reprezentare a diferitelor tipuri de date din mesaje
  - alcatuirea mesajelor respectiv a cererilor / raspunsurilor
  - cum se folosesc HTTP, SMTP … pentru transportul mesajelor
- API-uri SOAP au fost implementate in diverse limbaje: Java, Perl, JavaScript, Pyton, C++, C#, Visual Basic

Cateva notiuni XML folosite in SOAP

Datele, reprezentate ca text, sunt intercalate cu marcaje (de asemenea textuale); marcajele permit aflarea tipului si structurii datelor

```
<?xml version="1.0"?>
  <angajat>
    <nume>Nicolae Ionescu</nume>
    <functia>contabil</functia>
    <id>123456789</id>
    <dataangajare>
      <ziua>5</ziua>
      <luna>iunie</luna>
      <anul>2013</anul>
    </dataangajare>
  </angajat>
```

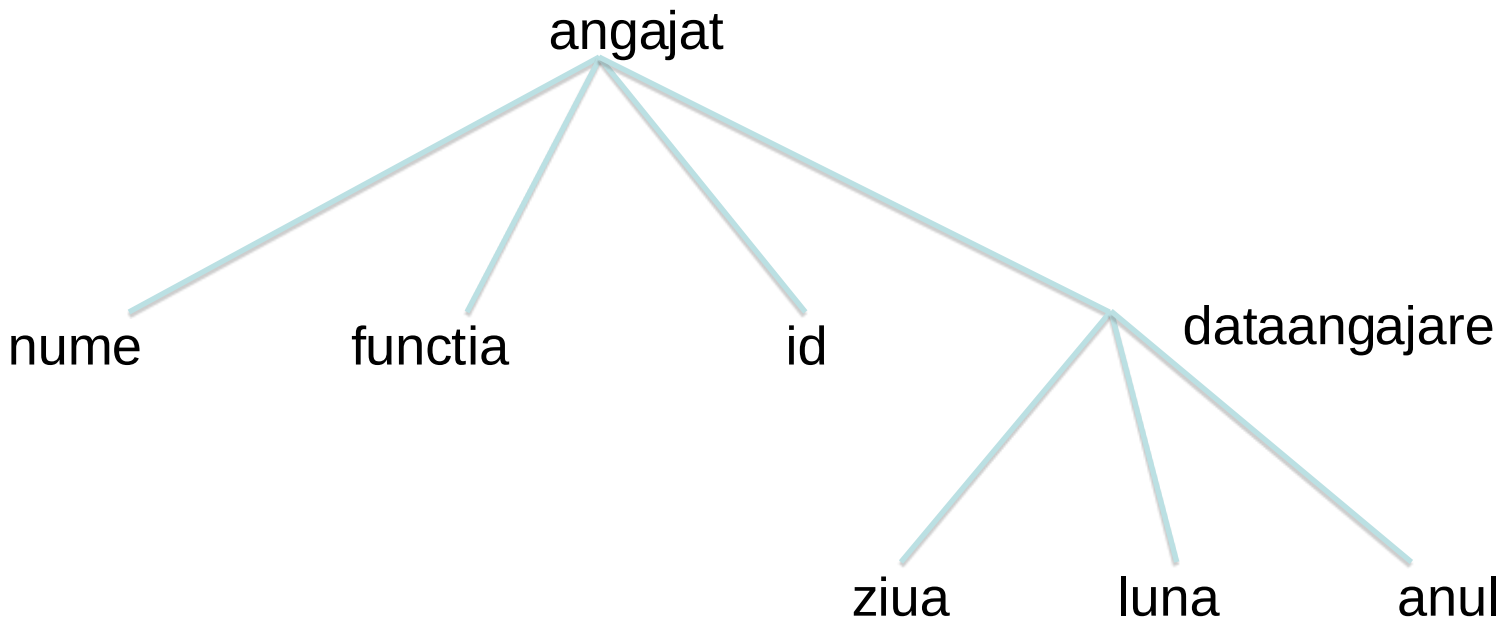
prolog – versiunea XML

element radacina (unic)

element inclus
nume = id
valoare 123456789

Modelul de date

- Descrierea unui angajat este structurata conform unui tipar (un arbore) si include mai multe elemente: nume, functia, un identificator unic si data angajarii
- dataangajare include trei elemente: ziua, luna, anul



Document XSD

```
<?xml version="1.0"?>
  <schema xmlns="http://www.w3.org/2001/
                                        XMLSchema">
    <element name="angajat">
      <complexType>
        <sequence>
          <element name="nume" type="string"/>
          <element name="functia" type="string"/>
          <element name="id" type="string"/>
          <element name="dataangajare">
            <complexType>
              <sequence>
                <element name="ziua" type="integer"/>
                <element name="luna" type="string"/>
                <element name="anul" type="integer"/>
              </sequence>
            </complexType>
          </element>
        </sequence>
      </complexType>
    </element>
</schema>
```

Schema de grupare a elementelor si tipurile lor sunt definite de un document XSD (XML Schema Document)

Defineste

- numele componentelor
- tipul lor
- ordinea elementelor,
- numarul minim si maxim de aparitii etc.

care permit “intelegerea” unui document XML de un program si de un operator uman

Spatiul de nume (XML namespace)

xmlns="http://www.w3.org/2001/XMLSchema”

In plus, nume de tipuri si **atribute** folosite pentru descrierea elementelor dintr-un document sunt definite in spatii de nume XML

O colectie de **nume** de tipuri si **atribute** (XML namespace) este referita printr-un URI (al fisierului care contine colectia)

Un namespace este specificat intr-un document XML printr-o pereche atribut=valoare

un document XML poate folosi mai multe spatii de nume

- un acelasi nume poate apare in spatii de nume diferite
- pentru evitarea ambiguitatilor se folosesc nume calificate

Doua spatii de nume cu marcajul comun “title”

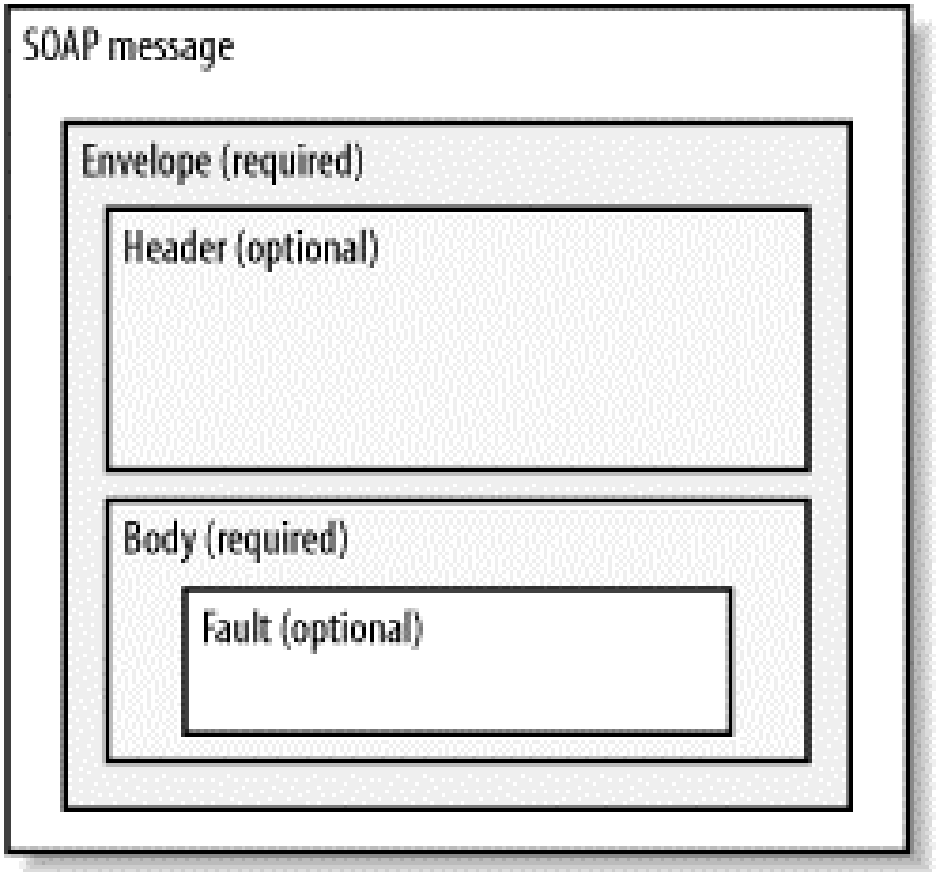
```
<?xml version"1.0"?>
<library-entry xmlns:authr="http://www.w3.org/TR/html4/"
                    xmlns:bk="http://www.w3schools.com/furniture">

<book>
<bk:title> Mythical Man-Month: Essays on Engineering </bk:title>
<pages>336</pages>
<isbn>0-201-83595-9</isbn>
< author>
<firstname>Frederick</firstname>
<middleinitial>P</middleinitial>
<lastname>Brooks</lastname>
<suffix>Jr</suffix>
<authr:title>Dr</ authr:title>
</author>
</book>
</library-entry>
```

numele calificate bk:title si authr:title au ca prefix identificatorii spatiilor de nume

Elementele principale ale unui mesaj XML SOAP

- Doua componente obligatorii: o *anvelopa* si *corpul* mesajului
  - anvelopa este singurul element al cererii; include
    - header – contextul pentru serviciu
    - body – documentul continand mesajul



O cerere SOAP

```
<?xml version='1.0' encoding='UTF-8'?>
<SOAP-ENV:Envelope
    xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:xsd="http://www.w3.org/2001/XMLSchema">
    <SOAP-ENV:Body>
        <ns1:getTemp
        xmlns:ns1="urn:xmethods-Temperature"
        SOAP-
        ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
            <zipcode xsi:type="xsd:string">10016</zipcode>
        </ns1:getTemp>
    </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Cererea are un singur element obligatoriu Envelope

care include elementul obligatoriu Body

Anvelopa

XML namespaces

xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"

mentioneaza versiunea folosita SOAP 1.1

diferita de 1.2: http://www.w3.org/2001/09/soap-envelope

xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance”
xmlns:xsd="http://www.w3.org/2001/XMLSchema">

specifica scheme de codificare a datelor si le leaga la prefixele xsi , xsd

xmlns:ns1="urn:xmethods-Temperature”

dezambiguizare identificatori din aplicatie specifici serviciilor disponibile public XMethods si leaga la prefix ns1

Corpul

Element Body

✧Folosit de regula pentru cerere si raspuns RPC

✧Cererea

```
<ns1:getTemp
xmlns:ns1="urn:xmethods-Temperature"
SOAP-ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
    <zipcode xsi:type="xsd:string">10016</zipcode>
</ns1:getTemp>
```

–apeleaza metoda getTemp

–cu parametrul zipcode

- avand tipul string
- si valoarea 10016

Raspunsul SOAP

```
<?xml version='1.0' encoding='UTF-8'?>
<SOAP-ENV:Envelope
    xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xmlns:xsd="http://www.w3.org/2001/XMLSchema">
    <SOAP-ENV:Body>
        <ns1:getTempResponse
        xmlns:ns1="urn:xmethods-Temperature"
        SOAP-
        ENV:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
            <return xsi:type="xsd:float">71.0</return>
        </ns1:getTempResponse>
    </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Elementul getTempResponse intoarce o valoare de tipul float
Valoarea (temperatura) este 71.0 grade Fahrenheit

Raspuns la eroare

```
<?xml version='1.0' encoding='UTF-8'?>
<SOAP-ENV:Envelope
    xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/"
    xmlns:xsi="http://www.w3.org/1999/XMLSchema-instance"
    xmlns:xsd="http://www.w3.org/1999/XMLSchema">
    <SOAP-ENV:Body>
        <SOAP-ENV:Fault>
        <faultcode xsi:type="xsd:string">
        SOAP-ENV:Client</faultcode>
        <faultstring xsi:type="xsd:string">
            Failed to locate method (ValidateCreditCard) in class
            (examplesCreditCard) at /usr/local/ActivePerl-5.6/lib/
            site_perl/5.6.0/SOAP/Lite.pm line 1555.
        </faultstring>
        </SOAP-ENV:Fault>
    </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

Raspunsul are un singur element, Fault

cu doua sub-elemente care specifica:
faultcode – clasa de erori Client – cererea contine o eroare (ex. nume metoda inexistenta)
faultstring – descrierea erorii este in limbaj natural

Tipuri de date

SOAP are un set de reguli pentru codificarea datelor

Pentru specificarea tipurilor de date, SOAP foloseste

- specificatiile elaborate de W3C (XML Schema)
- propriile conventii SOAP (ex. pentru tablouri si referinte)

Stilul de codificare este setat prin atributul SOAP-ENV:encodingStyle

SOAP 1.1 - http://schemas.xmlsoap.org/soap/encoding/

SOAP 1.2 - ttp://www.w3.org/2001/09/soap-encoding

Tipuri scalare

- includ string, float, double, integer
- sunt tipuri definite de XML Schema
- in ex. atributul xsi:type este setat la tipul xsd:double

```
<return xsi:type="xsd:double">54.99</return>
```

Tipuri compuse

Tablou

- Tipul xsi:type pentru tablou este un Array
- elementul arrayType specifica tipul elementelor si dimensiunile
  - ex. tablou de 10 elemente double

    arrayType="xsd:double[10]"
  - ex. tablou bi-dimensional de string-uri

    arrayType="xsd:string[5,5]"

Structuri

Pentru fiecare element este specificat un nume de acces si un tip

SOAP peste HTTP

- SOAP nu este legat de un protocol specific dar HTTP este cel mai popular
- Cererile sunt transmise prin HTTP POST (modificare resursa)

POST /perl/soaplite.cgi HTTP/1.0

Host: services.xmethods.com

Content-Type: text/xml; charset=utf-8

Content-Length: 538

SOAPAction: "urn:xmethodsBabelFish#BabelFish"

<?xml version='1.0' encoding='UTF-8'?>

<SOAP-ENV:Envelope …

valoarea specificata aici depinde de implementarea serverului;
in ex. urn necesar pentru accesul la serviciul Translation de la AltaVista BabelFish

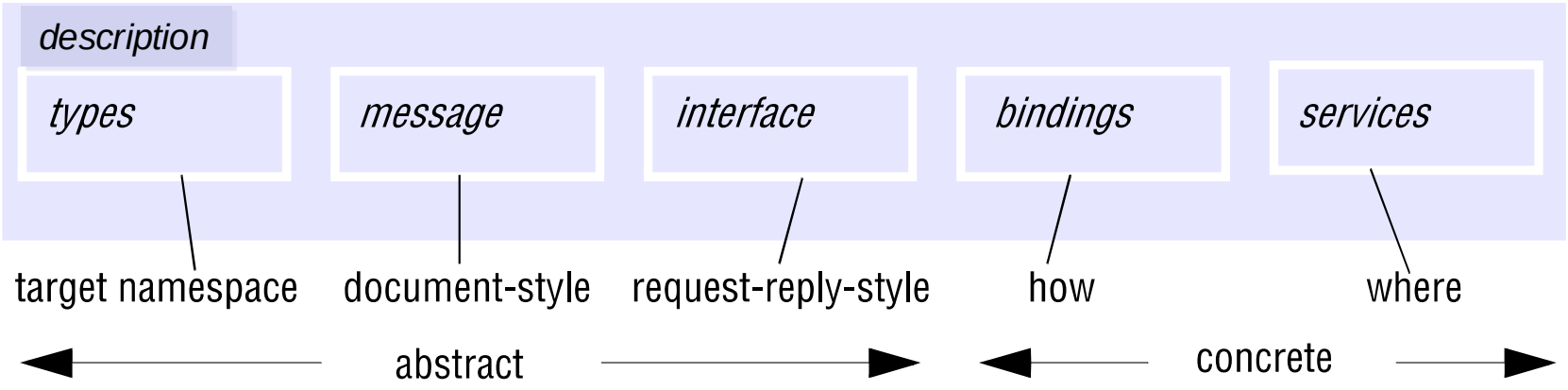
continutul este comanda sau raspunsul SOAP transportate

Servicii Web - Definirea interfetelor

- Programarea cu interfete separa interfata de implementare si ascunde heterogeneitatea
- O interfata de serviciu Web consta intr-o colectie de operatii ce pot fi executate de programe, obiecte, baze de date …
- Un document WSDL descrie un serviciu web precizand
  - operatiile oferite
  - tipurile datelor de intrare si rezultatelor fiecarei operatii
  - protocoalele folosite pentru comunicarea client-server
  - adresele la care serviciul poate fi accesat
- Se folosesc mai multe paradigme de comunicare – serviciile pot fi descrise in termenii
  - mesajelor schimbate intre client si server – stilul document
  - sau al operatiilor suportate de server – stilul request-reply

Elemente WSDL

The main elements in a WSDL description

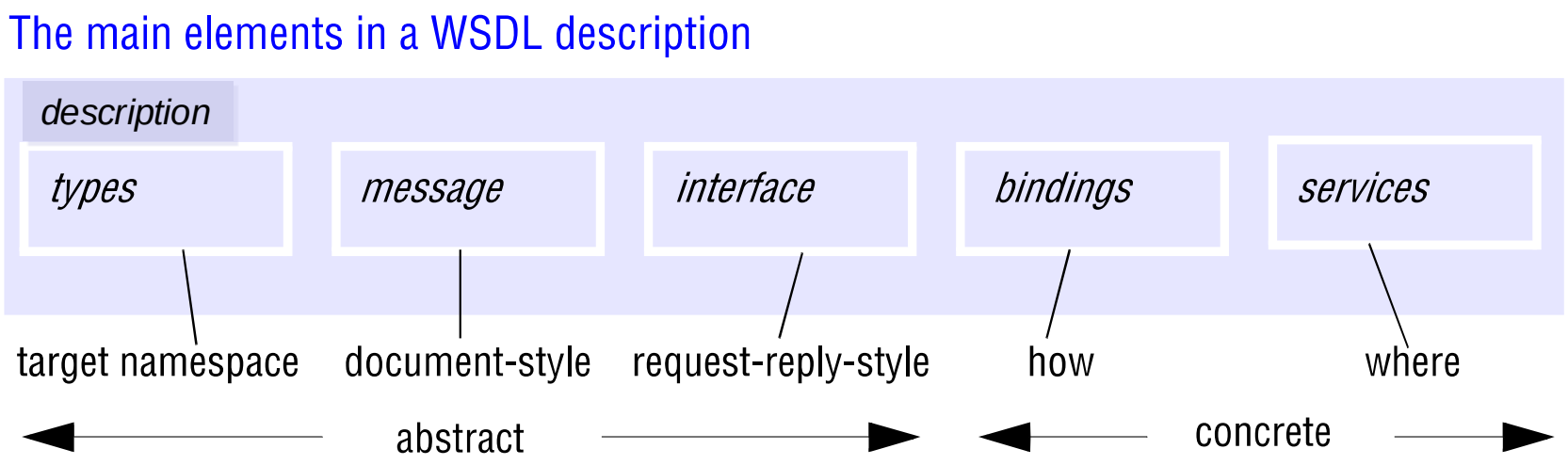


Descrierea WSDL este exhaustiva

- Include toate elementele serviciului care sunt **relevante pentru client**
- sunt folosite pentru a **genera codul proxy** la client

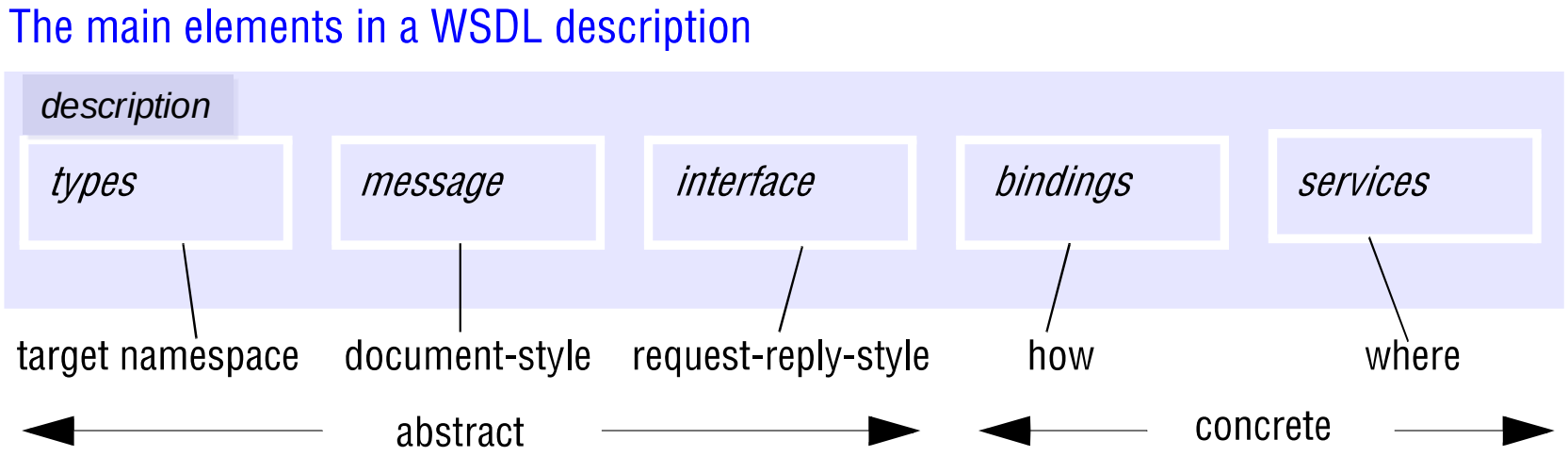
Descrierea are un element radacina (**description**) si elemente incluse, grupate in doua parti

Elemente WSDL – partea abstracta



- types – defineste tipurile folosite de serviciu
  - setul de nume definit in sectiunea types formeaza un spatiu de nume distinct, *target namespace*
- message - descrie mesajele schimbate specificand numele mesajelor, parametrii sau valorile returnate
- interface - operatiile serviciului Web

Elemente WSDL – bindings

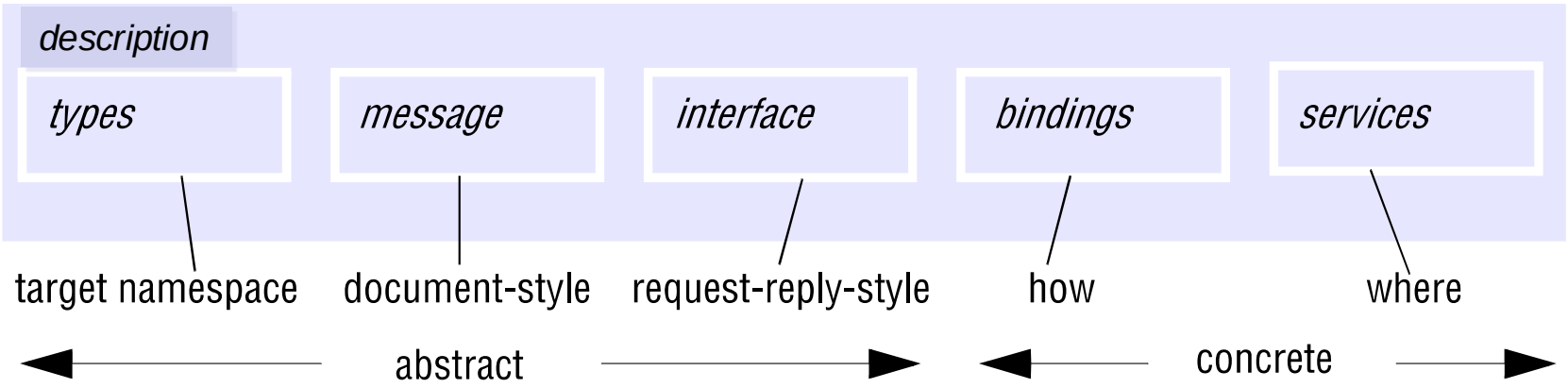


Specifica

- formatul mesajelor,
- protocolul de transport,
- tiparul schimbului de mesaje
  - in-out = cerere-raspuns
  - sau schimb de documente

Elemente WSDL – services

The main elements in a WSDL description



Specifica

- numele serviciului
- portul (endpoint) serviciului – care se refera la
  - binding-ul folosit
  - URL-ul serverului

Exemplu: serviciu Hello

La inceput sunt specificate spatiile de nume folosite in descrierea serviciului

<?xml version="1.0"?>

<wsdl:description

xmlns:wsdl=http://www.w3.org/ns/wsdl

- namespace pentru toate numele de elemente folosite in limbajul de baza WSDL

xmlns:wsoap= "http://www.w3.org/ns/wsdl/soap"

- nume si atribute din extensia SOAP

xmlns:hy="http://www.herongyang.com/Service/"

- introduce prefixul hy: care permite referirea spatiului de nume targetNamespace

targetNamespace="http://www.herongyang.com/Service/">

- namespace pentru toate numele definite in acest document

Definirea tipurilor

```
<types>
  <xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema"
   targetNamespace="http://www.herongyang.com/Service/”>
```

- *foloseste XML Schema in definirea tipurilor*

```
  <xsd:element name="Hello" type="xsd:string"/>
  <xsd:element name="HelloResponse" type="xsd:string"/>
```

- *definesc tipurile elementelor Hello si HelloResponce, ce vor fi incluse in targetNamespace*

```
  </xsd:schema>
</types>
```

- ***Hello** si **HellowResponse** sunt parametri de intrare / iesire ai operatiei Hello definite in sectiunea **interface** – slide urmator!)*
- *tipurile trebuie definite ca elemente singulare (nu ca secventa de elemente)*
- *tipul poate fi complex (elementul poate include sub-elemente)*

Definirea interfetei

```
<wsdl:interface name="helloInterface" >
    <wsdl:operation name="Hello"
    pattern="http://www.w3.org/ns/wsdl/in-out”
```

- *defineste operatia, “Hello” cu tiparul in-out*

```
    style="http://www.w3.org/ns/wsdl/style/iri">
```

- *mesajele pot fi serializate, conform stil IRI - Internationalized Resource Identifier*

```
    <wsdl:input messageLabel="In"
            element="hy:Hello" />
    <wsdl:output messageLabel="Out"
            element="hy:HelloResponse" />
```

- *operatia are un parametru **Hello** si un raspuns **HelloResponse** ale caror tipuri se gasesc in spatiul de nume **hy** (slide precedent)*

```
    </wsdl:operation>
</wsdl:interface>
```

Binding

<wsdl:binding name="helloBinding"

interface=”hy:helloInterface”

- *binding-ul helloBinding se refera la interfata helloInterface*

type="http://www.w3.org/ns/wsdl/soap"

- *formatul mesajului corespunde regulilor SOAP 1.2*

wsoap:protocol="http://www.w3.org/2003/05/soap/bindings/HTTP/">

- *protocolul de transport folosit este SOAP 1.2 peste HTTP*

<wsdl:operation ref=”hy:Hello"

wsoap:mep="http://www.w3.org/2003/05/soap/mep/soap-response"/>

- *in operatia Hello, tiparul schimbului de mesaje (MEP – Message Exchange Pattern) este in-out*

</wsdl:binding>

Descrierea serviciului

```
<wsdl:service name="helloService"
        interface=”hy:helloInterface”>
       <wsdl:endpoint name="helloEndpoint"
        binding=”hy:helloBinding"
        address="http://hello.example.com/2004/Service/HelloService”/>
</wsdl:service>
</wsdl:description>
```

Defineste numele serviciului helloService

cu interfata helloInterface

cu endpoint avand numele helloEndpoint

avand binding-ul curent helloBinding

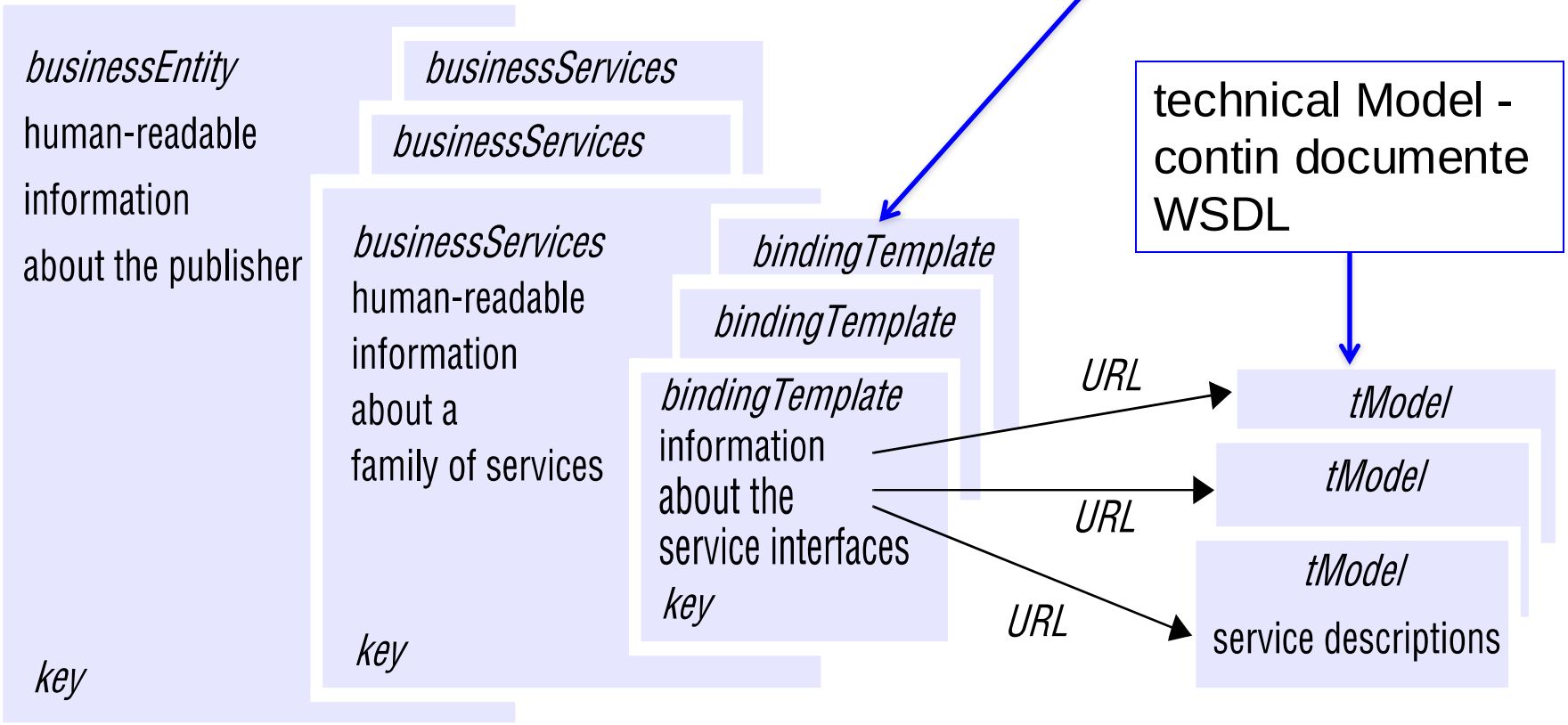
si adresa http://hello.example.com/2013/Service/HelloService

Universal Description, Discovery and Integration service (UDDI)

- Ofera servicii de nume si director
  - cautare dupa numele serviciului
  - cautare dupa atribute: categoria serviciilor
- Datele sunt organizate ca patru structuri

informatii despre adresa unui serviciu Web si referinte la descrierea serviciilor

technical Model - contin documente WSDL



Cautarea

- UDDI are un API pentru servicii de cautare
- doua seturi de query
  - gasirea unei entitati ce corespunde unei chei (get_xxx)

    get_BusinessDetail, get_ServiceDetail, get_bindingDetail, get_tModelDetail

  - gasirea unui set de entitati ce corespund unor criterii

    find_business, find_service, find_binding, find_tModel
- folosite in invocari succesive care ingusteaza aria de cautare
  - *find_business* – afla lista cu furnizorii
  - *find_service* – gaseste servicii care se potrivesc atributelor
  - se alege cheia pentru un *bindingTemplate*
  - se regaseste URL-ul documentul WSDL corespunzator
- UDDI ofera o interfata subscribe/notify care furnizeaza abonatilor informatii despre entitatile de interes

Publicarea

- UDDI are o interfata pentru publicarea si actualizarea serviciilor Web
- La prima publicare a unei structuri de date, i se asociaza o cheie in forma unui URI
  - ex. uddi:abcd1.net:123
- serverul UDDI devine proprietarul acesteia
- el difuzeaza informatia altor servere
- raspunsul la o cautare in UDDI este dat de un singur server, fara a interactiona cu alte servere din acelasi registru

Registre

- Un registru UDDI consta din unul sau mai multe servere, fiecare avand o copie a aceluiasi set de date
- Modificarile unei structuri de date se fac la serverul proprietar, cel la care aceasta a fost publicata prima data
  - proprietarul poate ceda acest drept unui alt server
- Schema de replicare
  - serverul care face modificarea notifica celelalte servere
  - acestea cer modificarile pentru actualizare
  - propagarea modificarilor foloseste vectori de timp

Model simplu – folosire usoara

  - toate modificarile unei structuri de date se fac la acelasi server care propaga apoi actualizarile
  - ca urmare, actualizarile sunt primite de un server in ordinea secventiala corecta
  - nu se impune vreo negociere intre actualizarile facute de servere diferite

Utilizarea WSDL

- documentele WSDL pot fi accesate prin URI-uri de clienti si servere
- generarea definitiilor WSDL se poate face prin instrumente cu interfata grafica
    - ocolesc structurile complexe ale WSDL
    - usureaza efortul utilizatorilor
  - ex. Web Services Description Language for Java Toolkit (WSDL4J) permite crearea, reprezentarea si manipularea documentelor WSDL
- definitiile WSDL pot fi generate in API-uri pentru diverse limbaje
  - ex. Java in JAX-RPC (Java API for XML based RPC)

JAX-RPC

API Java care permite crearea de servicii Web si accesul lor folosind standarde XML

- implementarea de referinta foloseste SOAP (ca protocol de aplicatie) si HTTP (ca protocol de transport)
- cererile si raspunsurile sunt transmise ca mesaje XML
- Dezvoltatorul poate ignora detalii XML despre serviciu (totusi, cunoasterea lor este recomandata pentru facilitatile avansate ale JAX-RPC – de ex. folosirea unor tipuri de date in afara celor suportate transparent de JAX-RPC)

Servicii si clienti

Dezvoltarea unui serviciu web in JAX-RPC incepe cu

- crearea unei **interfete Java** cu o metoda pentru fiecare operatie
- crearea unei **clase Java** care implementeaza interfata

Pentru **invocarea serviciului**

- JAX-RPC permite conversia automata de la descrierea Java a interfetei serviciului la descrierea WSDL
- din documentul WSDL, JAX-RPC poate genera codul Java pentru legarea clientului la server si apelul serviciului

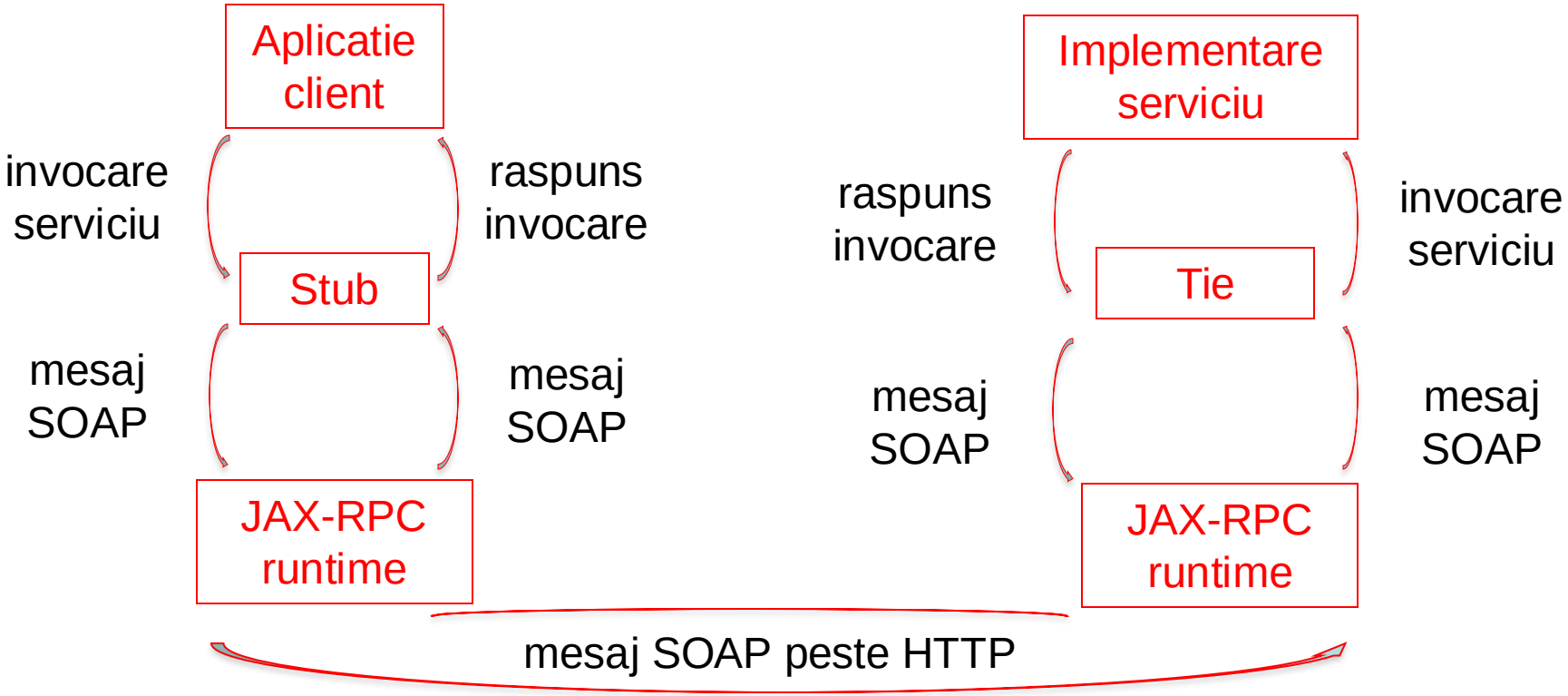
JAX-RPC runtime system

Invocarea serviciului de catre clienti ruland in alta masina virtuala (de regula in alta masina) reclama mecanisme speciale oferite de JAX-RPC runtime system



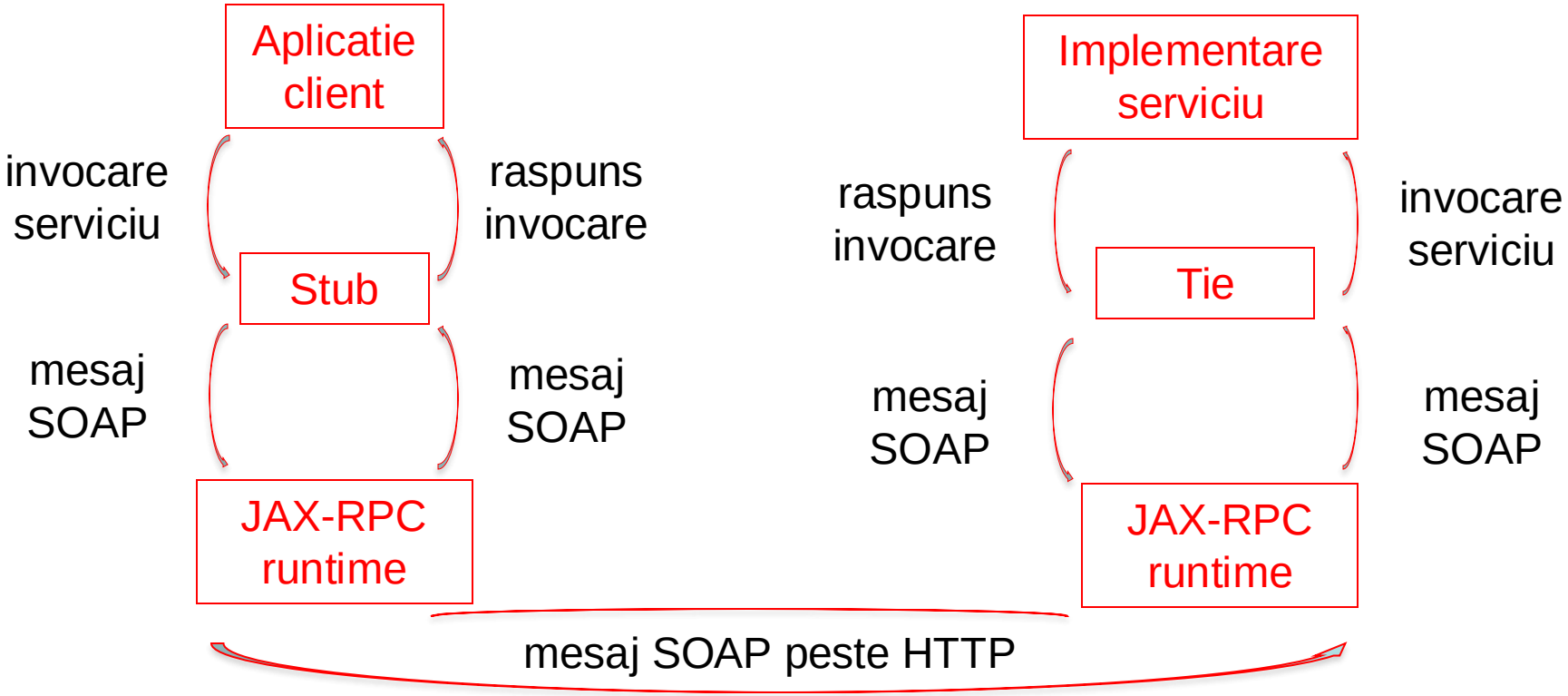
Stub

- O invocare este delegata de stub sistemului de runtime client care o transmite ca mesaj SOAP
- Raspunsul trimis de server ajunge la sistemul de runtime client si este pasat la stub; acesta il returneaza aplicatiei client ca rezultat al invocarii



Tie

- Similar, la server, mesajul SOAP primit ca urmare a invocarii unui client este convertit in apel de metoda de catre tie
- acesta extrage din mesajul SOAP numele metodei si parametrii, si invoca serviciul
- tie converteste rezultatul executiei metodei in mesaj SOAP de raspuns, returnat sistemului de runtime al clientului



Constructia si instalarea serviciului

Generarea stubs, ties, rutine de serializare si fisiere WSDL se face cu instrumente dedicate

De ex. implementarea in Sun Java System Application Server ofera instrumentul wscompile

- genereaza stubs, ties, serializatoare
- genereaza un document WSDL din descrierea serviciului
- genereaza interfata serviciului din fisierul WSDL
- genereaza modelul structurilor de date interne din descrierea serviciului sau WSDL

Pentru generare, wscompile foloseste un fisier de configurare care contine informatia necesara

Caracteristici JAX-RPC

- Functionalitati similare cu Java RMI
  - codul clientului apare ca facand invocari de metode ale unor obiecte locale (proxy-uri)
  - folosesc interfete pentru descrierea serviciilor
  - folosesc referinte / URI-uri la obiecte remote / resurse pentru a invoca servicii;
  - convertesc (marshaling/unmarshaling) invocari si raspunsuri la forme potrivite pentru transport prin internet – la JAX-RPC asigurate de Service/Client Runtime Environment

Caracteristici JAX-RPC

Diferente fata de Java RMI

• in JAX-RPC, mesajele intre client si server folosesc reprezentari XML → gama mai larga protocoale de transport (inclusiv HTTPS)

• interoperabilitate - JAX-RPC mai usor de folosit decat RMI/IIOP

  – descrieile XML folosite sunt textuale (usor de inteles de dezvoltatori)

• JAX-RPC suporta un set limitat de tipuri Java – corespunzator tipurilor de date din SOAP XML

Bibliografie

- Ethan Cerami, Web Services Essentials. Distributed Applications with XML-RPC, SOAP, UDDI & WSDL. O'Reilly 2002
- George Coulouris, Jean Dollimore,Tim Kindberg, Gordon Blair. Distributed Systems Concepts and Design, ed. 5, Addison Wesley
- Eric Armstrong,The JavaTM Web Services Tutorial, Copyright © 2003 Sun Microsystems, Inc.
- Kim Topley, Java Web Services in Nutshell, O'Reilly 2003
- http://www.inf.fu-berlin.de/lehre/SS03/19560-P/Docs/JWSDP/tutorial/doc/JAXRPC3.html



Consistenţa şi replicarea datelor în Internet

Bazat pe “Distributed Systems” de AS Tanenbaum

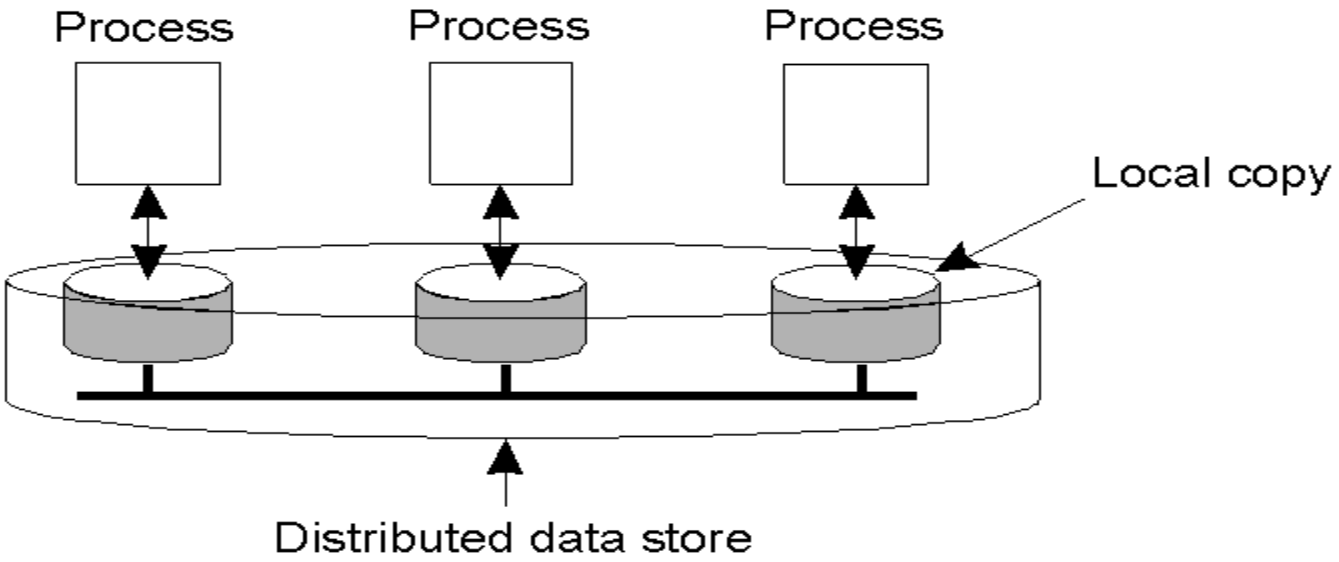
Motive pentru replicare

- Siguranta
  - Toleranta la defectari - sistemul continua sa ofere un serviciu corect chiar in cazul defectarii unei replici
  - Protectie mai buna impotriva datelor corupte (generali bizantini)
- Performanta
  - Timp de acces mai mic la replici apropiate de client
  - Incarcare mai mica a fiecarei replici
  - Disponibilitate mai mare
- Forme de replicare
  - Date – pastrarea unor copii ale datelor in mai multe calculatoare
  - Actiuni – executia mai multor replici identice ale unui proces
- Probleme: consistenta replicilor

Modele de consistenţă a datelor

- Consistenta stricta
  - modificarea unei replici se regaseste imediat in celelalte
  - greu de pastrat
- Solutie = constrangeri mai slabe de consistenta
- Categorii de modele de consistenta
  - centrate pe date
    - se aplica tuturor replicilor indiferent de utilizatorul care le foloseste
  - centrate pe client
    - consistenta replicilor apelate de un acelasi client

Modele de consistenta centrate pe date



- Organizare generala depozit de date distribuit si replicat
  - corespunde sistemelor distribuite actuale, inclusiv NoW, clustere …
- Operatii: **read** (copie locala), **write** (propagata la celelalte copii)
- Model de consistenta = contract intre proces si data store
- Ideal, read ar trebui sa intoarca rezultatul ultimei operatii write

Consistenţa Strictă

- Orice citire a unei date x intoarce valoarea rezultata din cea mai recenta scriere a lui x.

- Ex:

Doua masini A si B

Doua procese: Pa pe A si Pb pe B

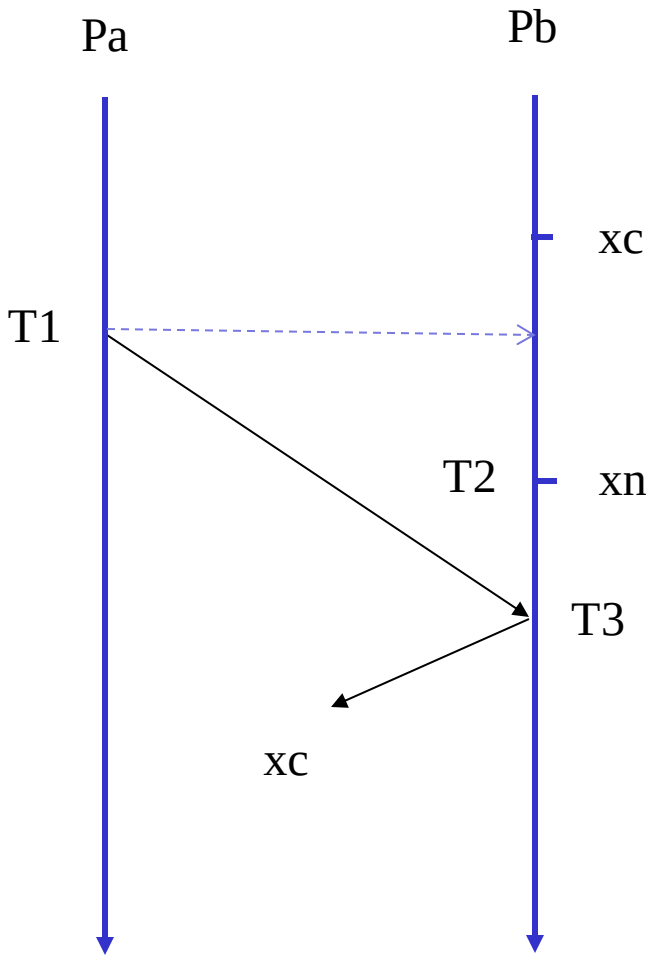
x memorat pe B (doar); valoarea curenta este **xc**

La T1, Pa citeste **x** - trimite un mesaj lui B sa citeasca x (la T1 valoare x = xc)

La T2>T1, Pb scrie **x**; noua valoare este **xn**

La T3>T2, mesajul de citire de la Pa soseste la B

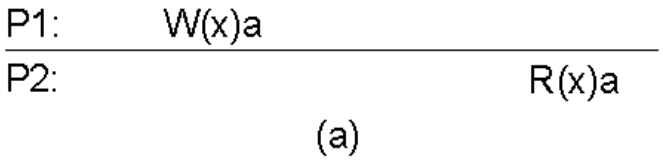
Consistenta stricta → B ar trebui sa intoarca **xc** (greu de implementat)



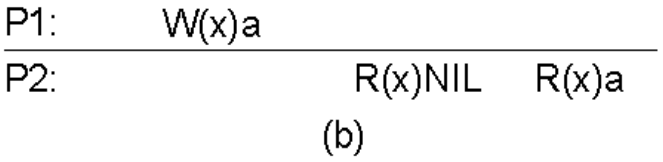
Consistenţa Strictă (2)

Se pastreaza ordinea absoluta globala in timp

Toate scrierile sunt instantaneu vizibile tuturor proceselor



(a)



(b)

W(x)a = scrie in x valoarea a R(x)a = citeste din x valoarea a

Axa orizontala este timpul fizic

Comportarea a doua procese operand pe acelasi item x

a) Memorie strict consistenta.

b) Memorie care nu este strict consistenta. DE CE ???

- Model greu de implementat
- ne-realist - Programele concurente nu sunt bazate pe timpul global sau pe viteza proceselor (ex. Producer / Consumer)
- Sunt necesare modele mai slabe

Consistenta Secvenţială

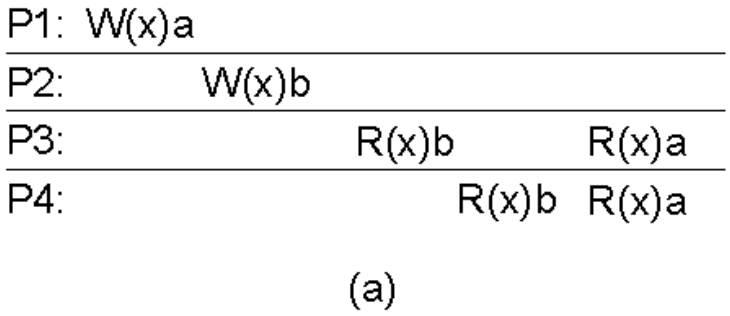
Rezultatul oricarei executii este acelasi cu cel obtinut daca
(1)operatiile (read, write) ale tuturor proceselor asupra depozitului de date sunt executate intr-o **secventa** oarecare si
(2)operatiile fiecarui proces individual apar in aceasta secventa in **ordinea** specificata de **programul** sau.

Orice intretesere valida a operatiilor read si write este acceptata

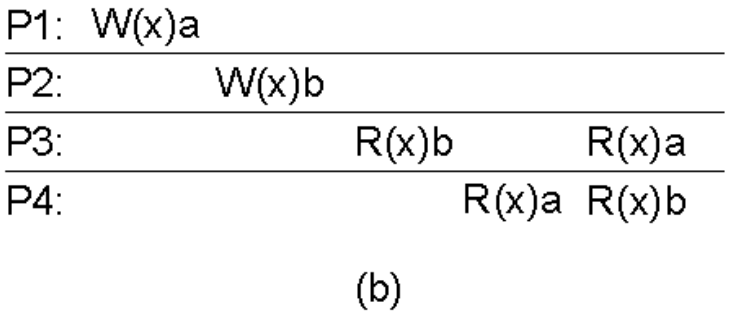
Toate procesele vad aceeasi intretesere de operatii

Operatiile nu au amprente de timp

Notatie: W(x)a – procesul scrie valoarea a in variabila x



(a)



(b)

Depozit **(a)** secvential consistent si **(b)** nu este secvential consistent. **DE CE ?**

Consistenţa Secvenţială – ordonari valide

x, y, si z sunt initializate la 0

Proces P1	Proces P2	Proces P3
x = 1;	y = 1;	z = 1;
print (y, z);	print (x, z);	print (x, y);

- Trei procese concurente.
  - assignment = write
  - print = doua read simultane

- Sunt posibile 90 ordonari valide diferite ale instructiunilor

  → 90 = 720 (=6!) / 8 **Interpretati formula !**

- Aplicatia care foloseste acest model ar trebui sa le accepte pe toate!

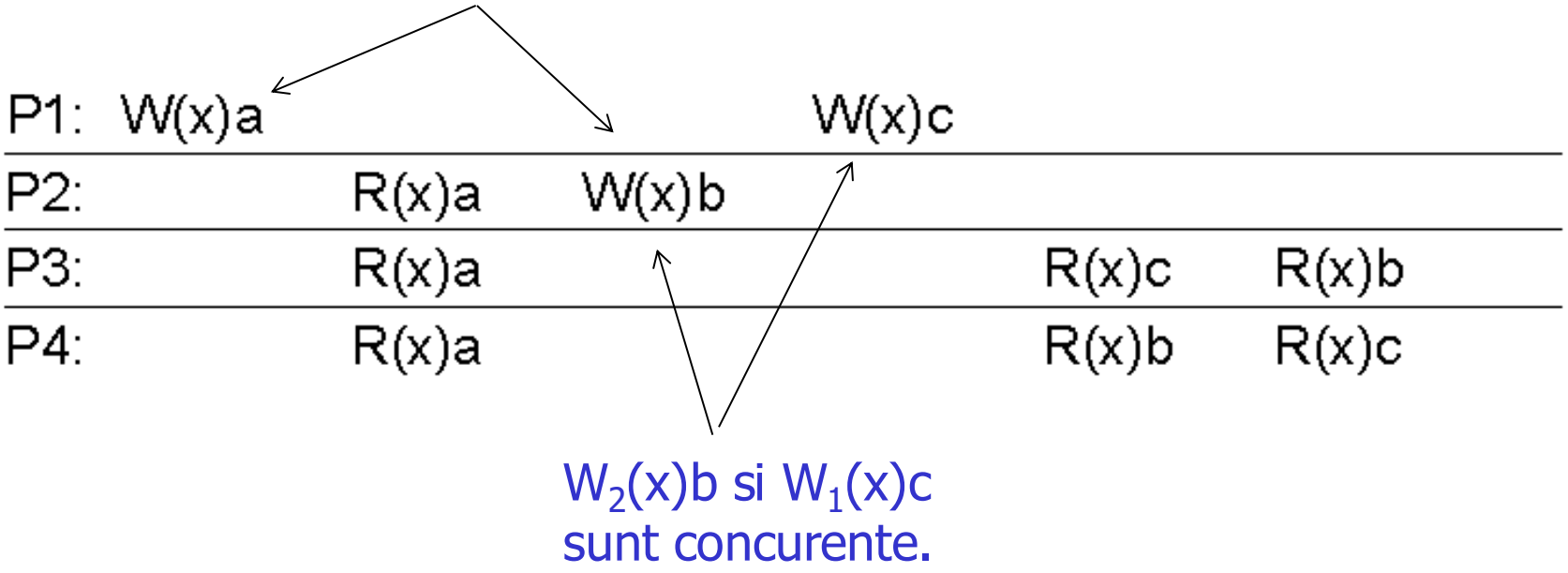
Consistenţa Cauzală (1)

- Doua categorii de operatii:
  1. Legate Cauzal (dependente cauzal)
     - ex:
       - procesul p executa (1) write x;
       - ulterior, procesul q executa (2) read x; (3) write y
       - (1) si (3) sunt legate cauzal
  2. Concurente

- Conditie necesara:
  Operatiile "write" care sunt potential legate cauzal trebuie sa fie vazute de toate procesele in aceeasi ordine. Scrierile concurente pot fi vazute in diferite ordini pe masini diferite.

Consistenţa Cauzală (2)

$W_1(x)a$ si $W_2(x)b$ sunt dependente cauzal;



$W_2(x)b$ si $W_1(x)c$ sunt concurente.

Aceasta secventa este permisa cu o memorie cauzal-consistenta, dar nu cu una secvential sau strict consistenta **DE CE ???**

Consistenta Cauzala (3)

P1: W(x)a				
P2:	R(x)a	W(x)b		
P3:			R(x)b	R(x)a
P4:			R(x)a	R(x)b

(a)

P1: W(x)a				
P2:		W(x)b		
P3:			R(x)b	R(x)a
P4:			R(x)a	R(x)b

(b)

a) Violare a consistentei cauzale. (Cele doua scrieri sunt legate cauzal.)

b) O secventa corecta de evenimente intr-o memorie cauzal-consistenta. (Cele doua scrieri nu mai sunt legate cauzal.)

Consistenta FIFO (1)

- Conditie Necesara:
  Scrierile facute de un singur proces sunt vazute de toate celelalte procese in ordinea in care au fost executate, dar scrierile din procese diferite pot fi vazute in ordini diferite de procese diferite.

- Usor de implementat
  - etichetand fiecare operatie write cu perechea (proces, numar secventa)

Consistenta FIFO (2)

P1: W(x)a						
P2:	R(x)a	W(x)b	W(x)c			
P3:				R(x)b	R(x)a	R(x)c
P4:				R(x)a	R(x)b	R(x)c

- O secventa valida de evenimente pentru consistenta FIFO
- Consistenta cauzala?

Consistenta FIFO (3)

Process P1	Process P2
x = 1;	y = 1;
if (y == 0) kill (P2);	if (x == 0) kill (P1);

Doua procese concurente.

- Consistenta FIFO: ambele procese pot fi omorate DE CE ???
  - deoarece scrierile sunt vazute in ordini diferite
- Prin contrast
  - Consistenta Secventiala: sase posibile intreteseri, nici una cu ambele procese omorate

DE CE ???

Gruparea operatiilor

- Datele partajate pot fi asociate cu o variabila de sincronizare.
- Un proces foloseste acquire si release pe variabilele de sincronizare
  - acquire – cand intra in sectiunea critica - datele protejate (replicile) de variabila de sincronizare sunt facute consistente
  - release - cand iese din sectiunea critica
- Model specific de folosire variabile de sincronizare
  - Fiecare variabila de sincronizare are un proprietar curent
  - Proprietarul poate modifica datele protejate de variabila de sincronizare, in mai multe sectiuni critice (ramane proprietar)
  - Un proces care vrea sa acapareze (acquire) trebuie sa faca o cerere proprietarului, devenind noul proprietar
  - Mod ne-exclusiv – Mai multe procese detin variabila doar pentru citirea datelor protejate

Consistenta la Intrare

P1:	Acq(Lx)	W(x)a	Acq(Ly)	W(y)b	Rel(Lx)	Rel(Ly)						
P2:							Acq(Lx)	R(x)a			R(y)NIL	
P3:									Acq(Ly)	R(y)b		

- Fiecare variabila partajata **x** este asociata cu o variabila de sincronizare (lock) **Lx**.
- Pentru ca un proces sa modifice date partajate, el trebuie sa capate accesul exclusiv la variabila de sincronizare care protejeaza acele date
  - P1 executa aquire pe x si y inainte de modificare
- Un acquire pe o variabila de sincronizare actualizeaza toate datele partajate protejate de acea variabila
  - P2 citeste corect pe **x** (s-a facut acquire)
  - dar nu si pe **y** (nu s-a facut acquire)

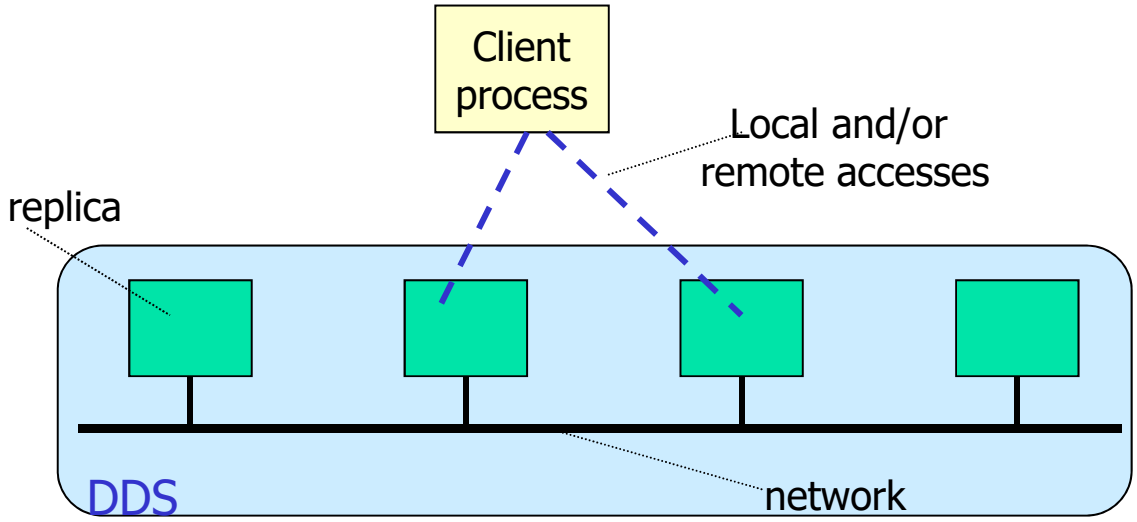
Modele de Consistenta Centrata pe Client

- Ofera garantii de consistenta pentru **un singur client**
  - efectul unei operatii depinde de istoria operatiilor clientului
  - nu se ofera garantii de consistenta pentru accesele intretesute ale diferitilor clienti
- Este un caz special: **lipsesc actualizarile simultane** ale datelor
- Exemple
  - Sisteme de baze de date – cele mai multe procese citesc din BD
  - DNS – pentru fiecare domeniu este asignata o autoritate care face actualizarile -> nu se fac actualizari simultane
  - WWW – paginile actualizate de webmaster sau de proprietar

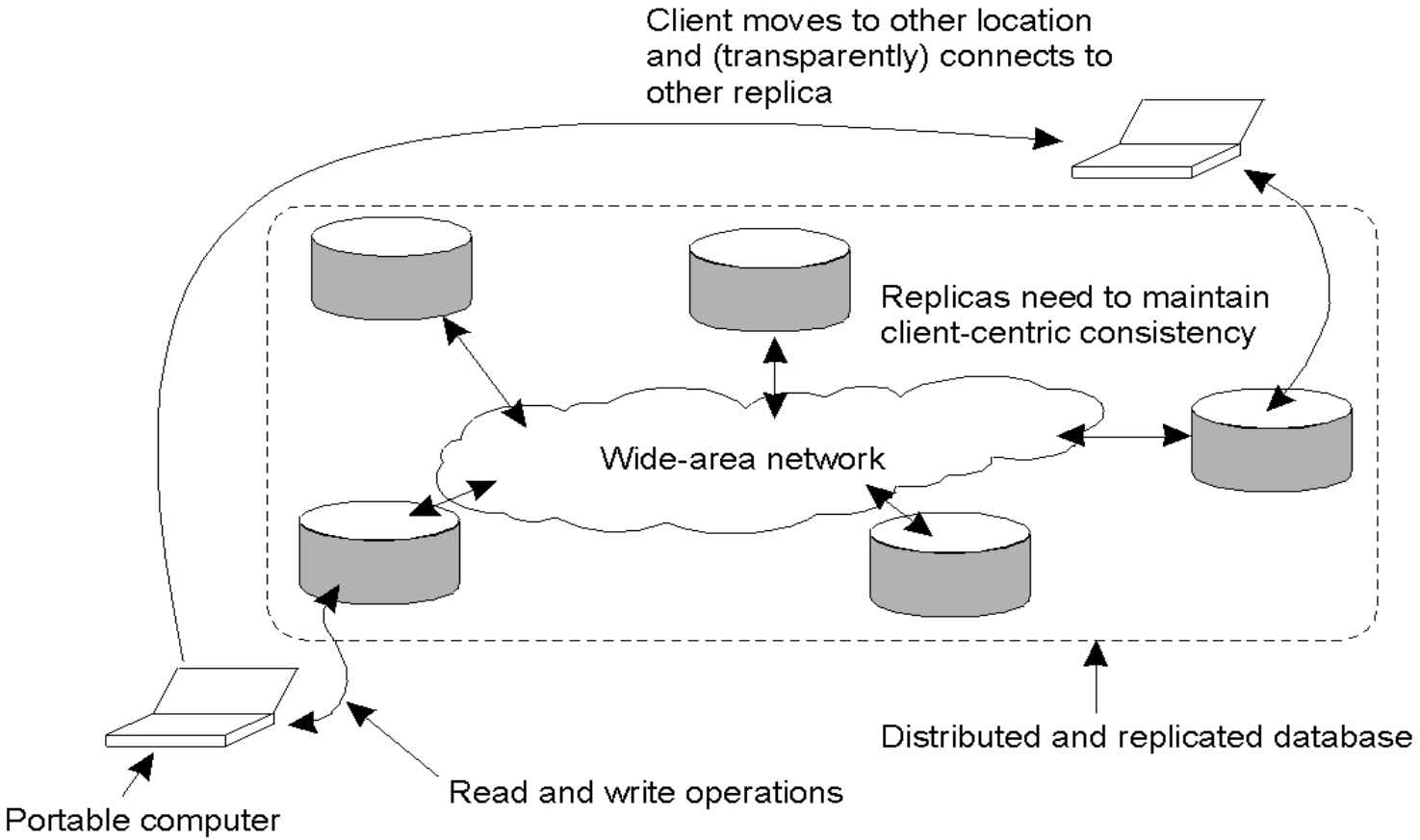
Consistenta "in cele din urma" (eventual consistency):

- in orice moment **pot exista replici neactualizate**
- in absenta indelungata a unor actualizari, toate **replicile vor deveni "in cele din urma" consistente**
- **conditie**: sa existe un mecanism de propagare a oricarei schimbari la toate replicile
- consistenta acceptabila cand clientul nu acceseaza replici diferite intr-un interval scurt de timp

Datele DDS (Distributed Data Store) vor deveni consistente "dupa un timp"



Consistenta Client-centric: un exemplu



- **Un utilizator mobil accesand diferite replici ale unei baze de date distribuite**
  - valorile citite si modificarile facute pe o replica sa fie regasite la cealalata replica
  - da garantii unui singur client privind consistenta acceselor la replici diferite ale acelui client

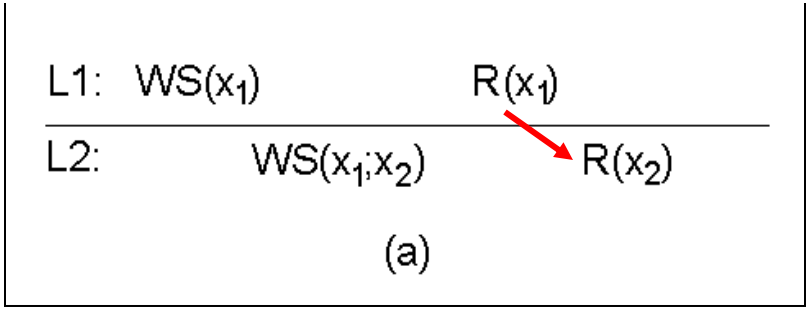
Citiri monotone (Monotonic Reads)

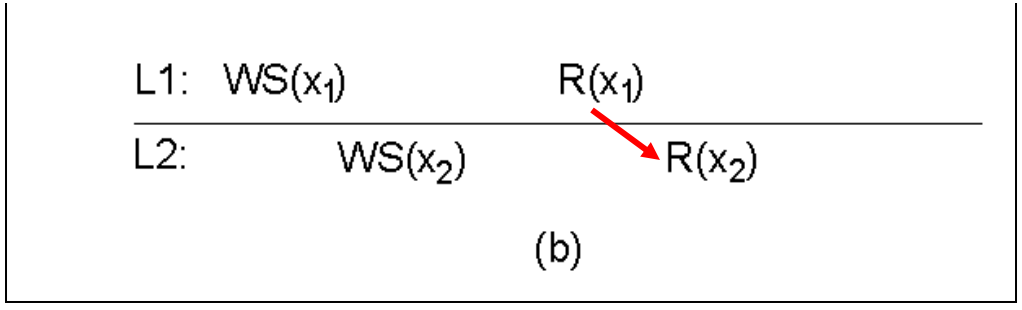
Exemplu: baza de date distribuita, pentru e-mail.

- Fiecare utilizator are o cutie postala distribuita si replicata pe mai multe masini
- Mail-urile pot fi inserate in diferite locatii
- Actualizarile se fac mai lent, cand este necesar (la cerere)
  - de ex. cand utilizatorul deschide cutia postala in alta locatie (acceseaza o alta copie)
- Presupunem ca utilizatorul citeste mesajele din cutia postala in locatia A si ulterior in B
- Citirea nu modifica cutia postala (de ex. nu sterge mesajele citite)
- Consistenta "Monotonic Reads" asigura ca mesajele citite de un proces in A vor fi regasite, de acelasi proces, la accesul in B

Citiri monotone (2)

Daca un proces citeste valoarea unei date x, orice citire succesiva a lui x de catre acel proces va returna acea valoare sau o valoare mai noua.





Operatiile *read* excutate de un singur proces *P* pe doua copii locale diferite ale aceleiasi baze de date.

Notatii: WS(x1) – Write Set - este seria de operatii de scriere pe x executate in locatia 1 de la initializare;

WS(x1;x2) => WS(x1) este parte a lui **WS(x2)**

a) Memorie **consistenta** "monotonic-reads" DE CE ???

- operatiile pe **x** in **L1** au fost propagate la **L2**

b) Memorie **ne-consistenta** "monotonic-reads"

Scrieri monotone (Monotonic Writes)

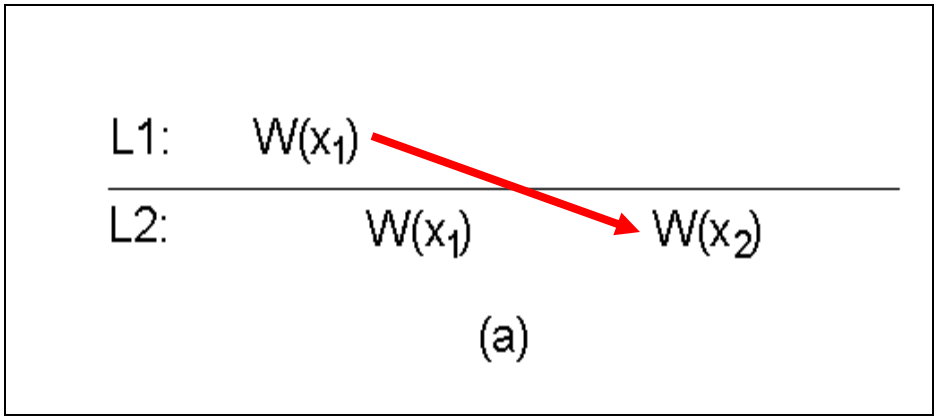
Exemplu: actualizarea unei biblioteci software

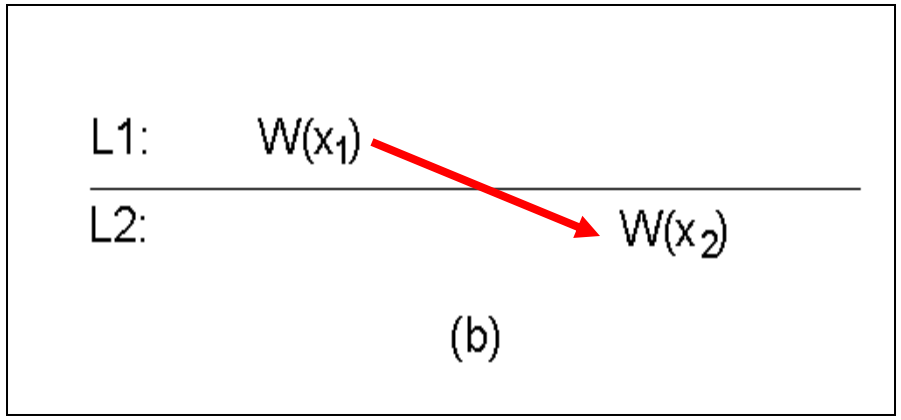
- actualizarea inseamna inlocuirea unei functii (sau a catorva functii), ducand la o noua versiune
  - schimba doar "o parte a valorii" bibliotecii
  - se face pe o replica
- o noua actualizare facuta pe o alta replica cere ca asupra ei sa se fi operat deja actualizarea anterioara
  - asigura obtinerea unei noi versiuni a bibliotecii
  - noua versiune incorporeaza toate actualizarile precedente
  - operatiile de scriere se aplica in aceeasi ordine tuturor replicilor

Scrieri monotone (2)

O operatie *write x* a unui proces este terminata inaintea oricarei operatii *write x* ulterioare a aceluiasi proces.

Obs. Seamana cu consistenta FIFO, dar se refera la un singur proces





Operatiile de scriere executate de un proces *P* pe doua copii locale diferite ale aceluiasi depozit de date

a) Un depozit consistent "monotonic-writes" - operatia W(x1) asupra lui x in L1, a fost propagata la L2 inainte de W(x2)

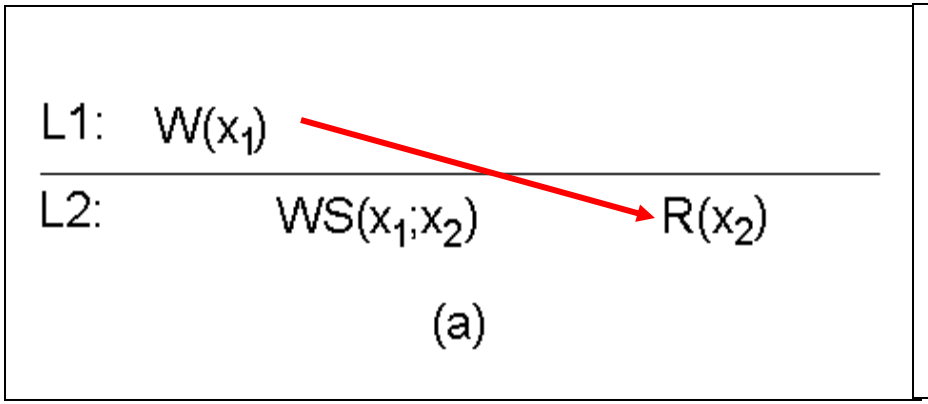
b) Un depozit ne-consistent "monotonic-writes".

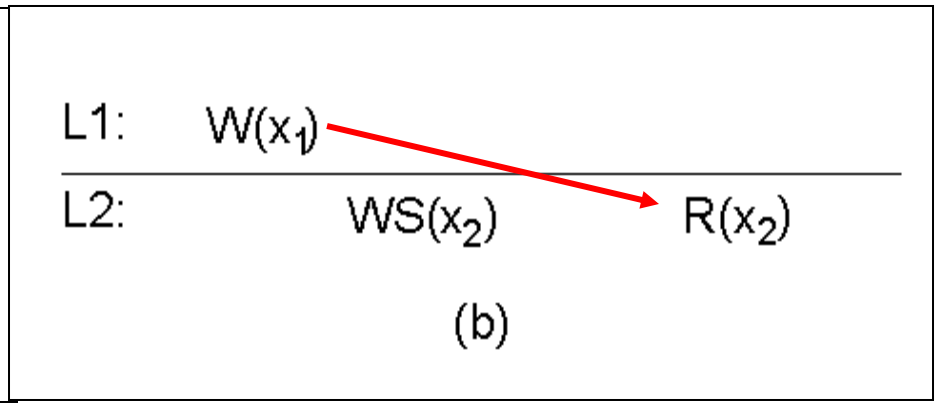
Citirea scrierilor proprii (Read Your Writes)

- **Exemplu**: actualizarea paginilor Web si observarea efectelor.
  - Actualizarea unei pagini Web are ca efect scrierea in fisierului pastrat de server
  - Citirea ulterioara a paginii se poate face dintr-o copie a fisierului pastrata de o replica a serverului
  - Read Your Writes asigura ca fisierul returnat de replica include modificarile facute anterior de utilizator
  - Cazul unei copii cache:
    - Daca browserul returneaza o copie din cache, Read Your Writes asigura invalidarea cache-ului la modificarea paginii Web si obliga la aducerea paginii actualizate de la server

Citirea scrierilor proprii (2)

Efectul unei operatii *write x* a unui proces P va fi totdeauna vazut de operatiile *read x* executate ulterior de acelasi proces P.





a) Un depozit care **ofera** consistenta "read-your-writes" (efectele scrierii lui x la L1 au fost propagate la L2 inainte de citire)

b) Un depozit care **nu ofera** consistenta "read-your-writes".

Scrierile urmeaza citirile (Writes Follow Reads)

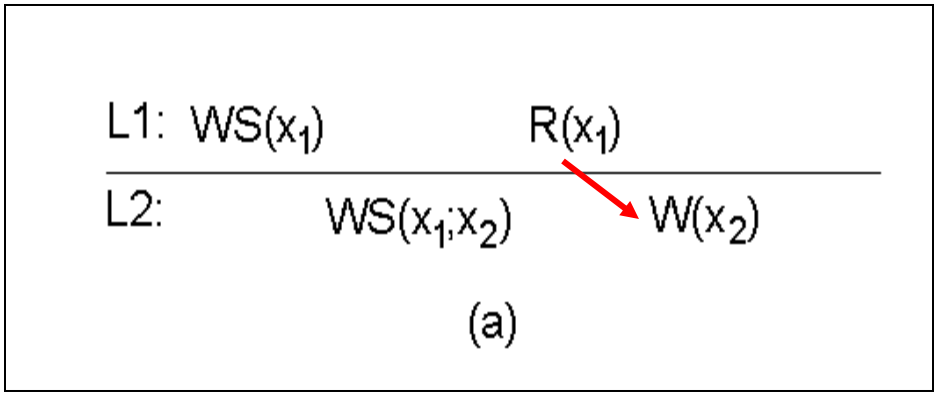
Exemplu: intr-un serviciu de stiri (newsgroup)

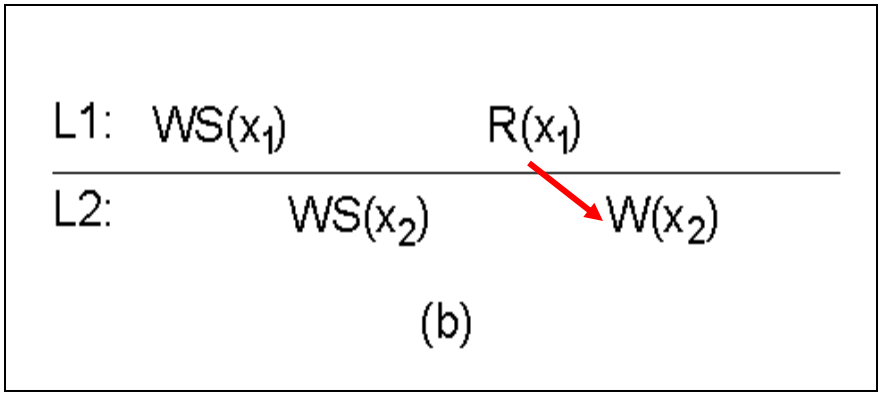
- un utilizator citeste un articol A
- apoi el reactioneaza si scrie (posteaza) un raspuns B
- consistenta "writes follow reads" garanteaza ca, in orice copie a newsgroup-ului, B va fi scris doar dupa ce A a fost scris de asemenea.

Efectul: un utilizator al unui serviciu de stiri intr-o retea nu vede o reactie la un articol fara sa vada si articolul la care se refera reactia

Scrierile urmeaza citirile (Writes Follow Reads)

O operatie *write x* executata de un proces P dupa o operatie prealabila *read x* a aceluiasi proces se executa garantat pe valoarea citita a lui *x* sau pe una mai recenta.





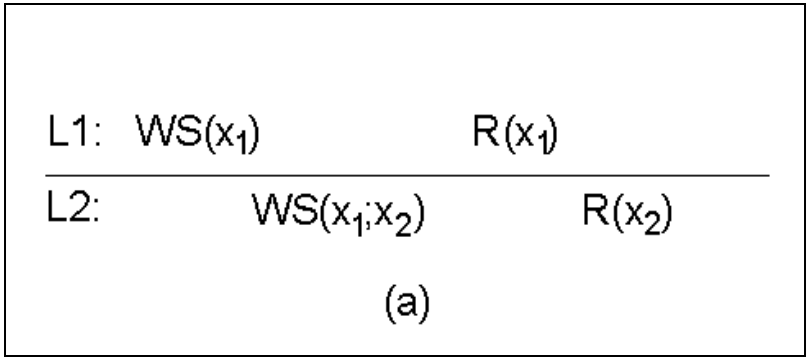
a) Un depozit care **ofera** consistenta "writes-follow-reads"

operatiile de scriere din L1, care au dus la valoarea citita in L1, apar in setul de scrieri de la L2 unde acelasi proces face apoi o scriere

b) Un depozit care **nu ofera** consistenta "writes-follow-reads".

Implementare "naiva" a consistentei centrate pe client

- Fiecarei operatii write i se asociaza un identificator global unic, de catre serverul care a initiat operatia (originea scrierii)
- Pentru fiecare client se pastreaza doua seturi de identificatori:
  - read set – identificatori ai operatiilor write relevante pentru operatiile read executate de client
    - in figura, operatiile de scriere relevante pentru citirea R(x1) sunt incluse in WS(x1)
    - la L2, scrierile relevante pentru R(x2) sunt operatiile din WS(x1;x2)
  - write set – identificatori ai operatiilor write executate de client





Implementare Monotonic Reads

- Clientul invoca operatia read la un server (replica) si ii paseaza un read set
- Serverul primeste read setul clientului si verifica daca toate operatiile write relevante au fost facute pe replica locala
- Daca nu
  - contacteaza celelalte servere (replici) si face actualizarea
  - eventual paseaza operatia read la serverul care a executat toate operatiile write din setul read al clientului
- Serverul adauga la read set propriile operatii write

- Actiuni similare sunt executate pentru celelalte modele

Probleme de implementare

Problema: seturile **read** si **write** pot deveni prea voluminoase

Solutiie:

- gruparea operatiilor in sesiuni de mai scurta durata (de ex. asociate unei aplicatii) si initializarea lor la inceputul fiecarei sesiuni
- utilizarea unor vectori de timp asociati operatiilor din fiecare sesiune
  - fiecare server Si tine un vector de timp STi asociat unei date,
  - fiecare element STi[j] este amprenta de timp a celei mai recente operatii write cu originea in serverul Sj, pe care Si a procesat-o
  - cand serverul i executa o operatie write, el ii asociaza o noua amprenta de timp pe care o inregistreaza in STi[i]

Probleme de implementare (2)

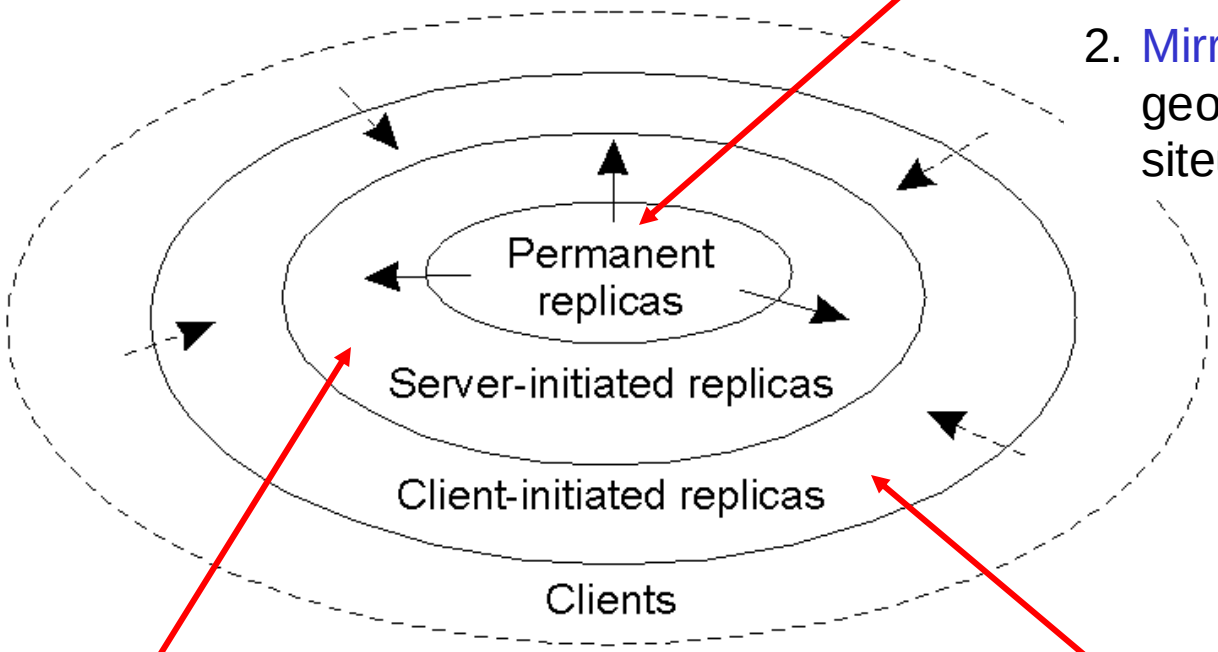
- un vector de timp este mai putin voluminos decat read set
- read set este inlocuit cu un vector de timp notat RS
- fiecare element RS[j] este amprenta maxima a operatiilor write cu originea in j relevante pentru read
- cand clientul invoca un read la serverul i, el trimite RS
- serverul verifica daca a procesat toate operatiile write inregistrate in RS
  - pentru **toti j** la care RS[j] > STi[j], serverul i preia de la Sj si executa operatiile write ne-executate anterior
- serverul actualizeaza eventual RS = max (RS, STi) si trimite raspunsul catre client impreuna cu noul RS



Gestiunea replicilor

- Plasarea replicilor
- Protocoale de propagare a actualizarilor
- Protocoale epidemice

Plasarea Replicilor



Replici permanente

1. Servere "replica" situate in acelasi sistem (cluster)
2. Mirroring – replici larg distribuite geografic - cererile sunt servite de siteul cel mai apropiat

Replici initiate de server pentru a imbunatati performanta

– replici apropiate de clienti

– replici pentru reducerea incarcarii unui server

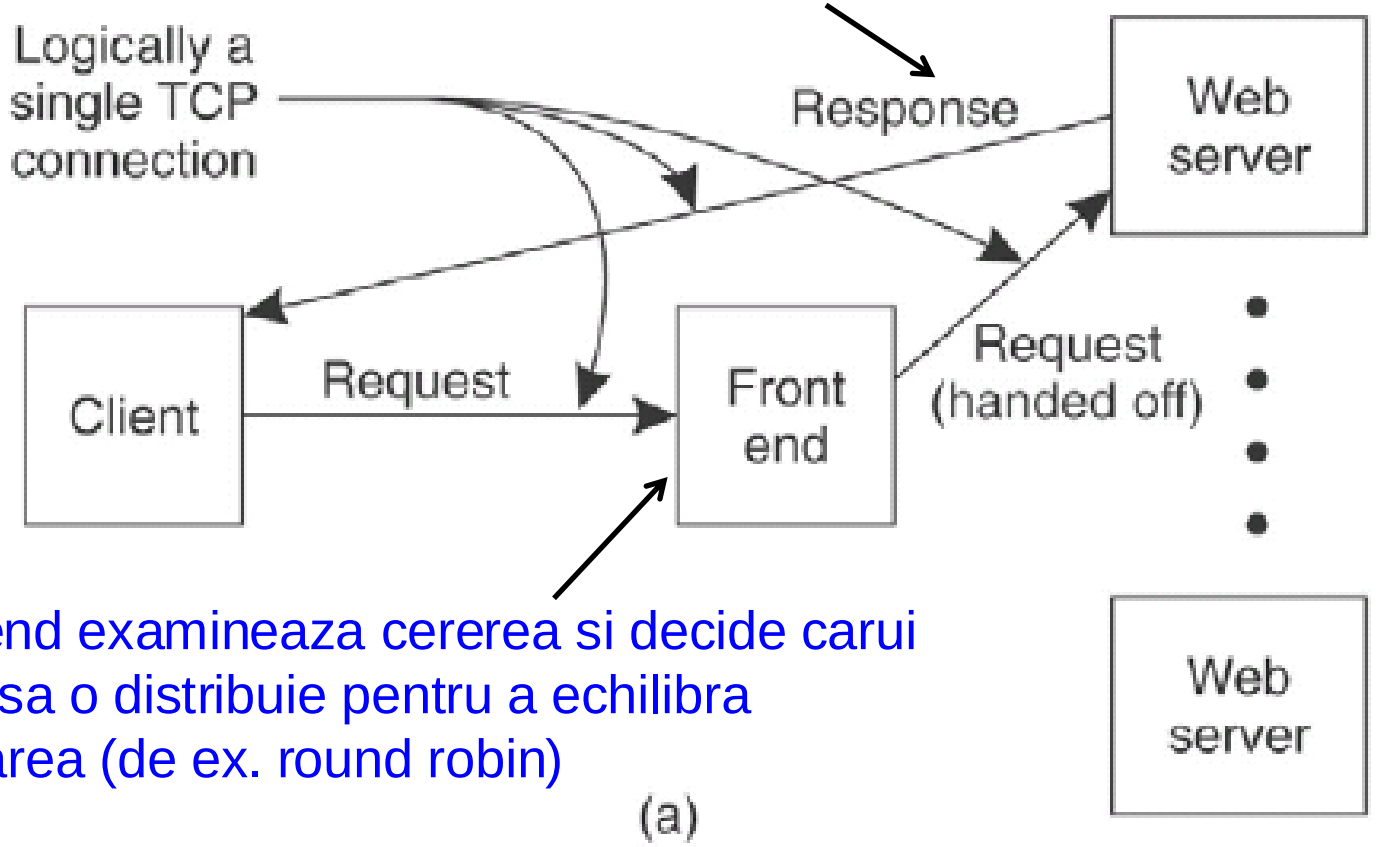
Replici initiate de clienti (caches)

- Utilizate pentru a reduce timp de acces la date
- Date pastrate pentru un timp limitat
- Cache-uri plasate la clienti
- Cache-uri partajate, in apropiere de clienti

Cluster de servere

Paginile sunt replicate pe un numar redus de servere din acelasi cluster

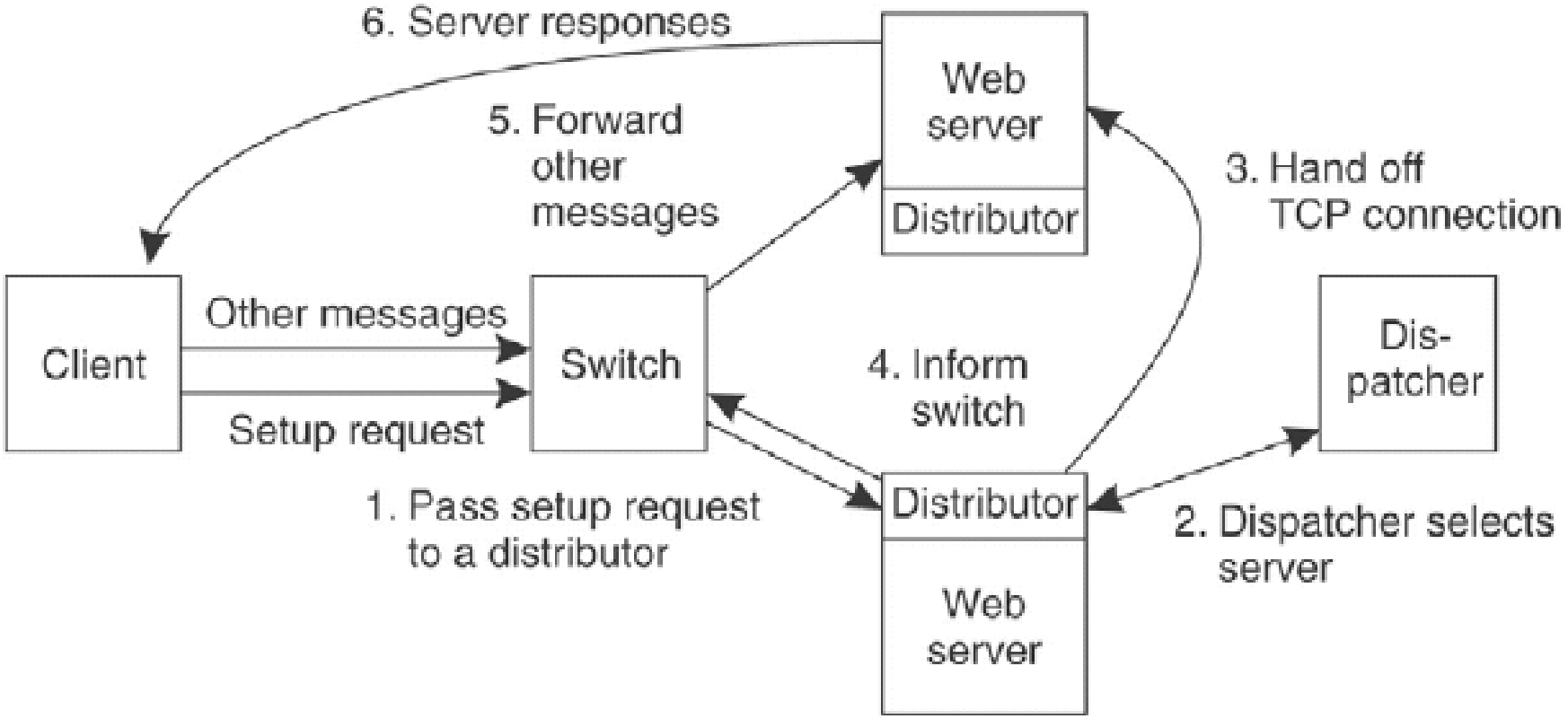
Raspunsul este trimis de serverul Web direct clientului (se foloseste TCP handoff protocol)



Front end examineaza cererea si decide carui server sa o distribuie pentru a echilibra incarcarea (de ex. round robin)

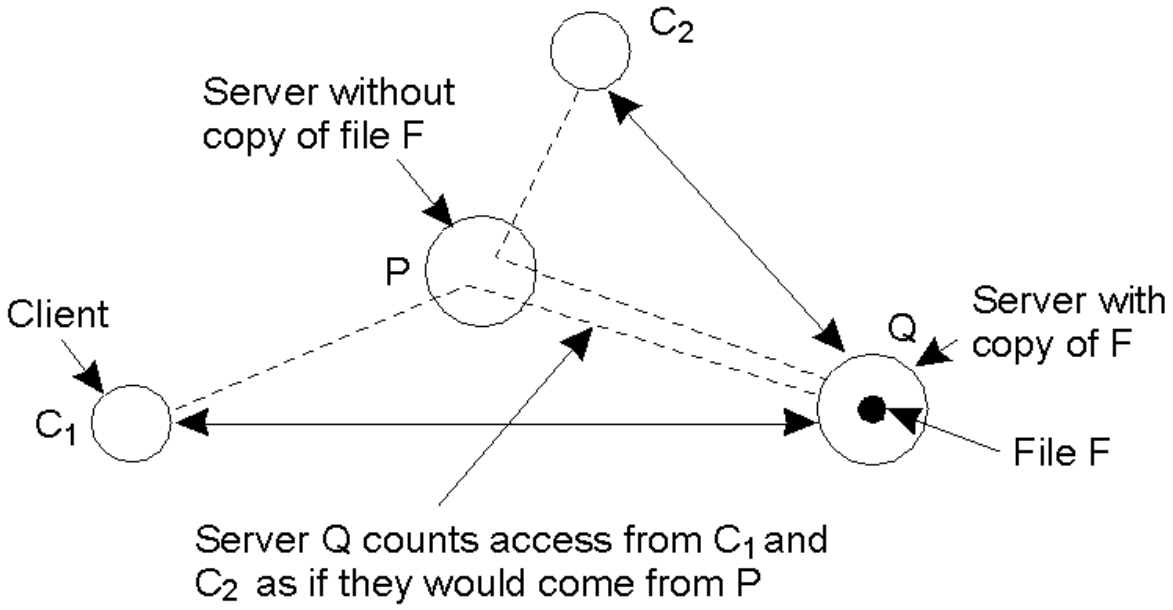
Cluster de servere (2)

Functia Front end este impartita intre un Switch de nivel transport, Distributori si un dispatcher



Replici initiate de server: Cand si unde?

- Fiecare server tine evidenta numarului de accese per fisier si a originii cererilor
- Politica de replicare (Rabinovich)
  - nr cereri pentru F < prag stergere **del(Q,F)** ➔ Q sterge fisier F
  - nr cereri pentru F > prag duplicare **rep(Q,F)** ➔ F replicat pe un alt server
  - nr cereri intre cele doua praguri ➔ Q poate decide mutarea lui F pe alt server (de ex. P)



Protocoale de Propagare

Ce se propaga?

- notificari
  - Utilizate de protocoalele de invalidare
    - replicile sunt informate ca datele nu mai sunt valide
  - Reclama largimi de banda reduse
- datele modificate
  - Utila pentru multe citiri: rata read / write este ridicata
  - Pot fi propagate doar log-urile modificarilor
- operatiile de actualizare
  - Replicare activa – re-executa operatiile asupra datelor
  - Cost comunicare redus dar consuma CPU la executie

Protocoale Pull versus Push

Push: actualizari transmise replicilor fara ca acestea sa ceara

- se aplica pentru replici permanente si "server initiated" care necesita un grad inalt de consistenta
- eficiente pentru multe citiri: rata read/write este ridicata

Pull: actualizari transmise la cerere

- folosite pentru cache-uri client
- eficiente pentru putine citiri: rata read/write este redusa

Exemplu - Comparatie intre protocoale push-based si pull-based in cazul sistemelor cu mai multi clienti si un singur server.

Probleme	**Push-based**	**Pull-based**
Starea replicilor pastrata la server	Lista replicilor si a cache-urilor client	Nimic
Mesaje transmise	update sau invalidare plus fetch&update ulterior	poll si fetch&update
Timp de raspuns la client	Imediat (sau timp de fetch&update)	Timp fetch&update

Protocoale de propagare hibride

- Bazate pe **Contract** – serverul trebuie sa transmita actualizari la client pe durata specificata in contract

  La expirare contract, actualizarile se trimit la cererea clientului

- Durata poate fi adaptata dinamic in functie de criteriile de contract (lease)

  – bazat pe vechime

    - contracte de durata pentru elemente cu sansa de a ramane nemodificate (ex. Web)

  – bazat pe frecventa de innoire

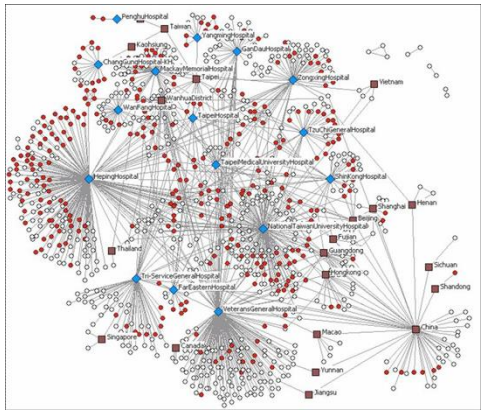
    - contracte de durata pentru clienti ale caror cache-uri trebuie innoite frecvent (pentru celelalte datele se trimit “la cerere”)

  – overhead-ul la server

    - serverul devine supraincarcat ➔ scade timpul de expirare al noilor contracte (ca urmare, scade numarul de clienti contractuali)

Protocoale de propagare epidemice

- Scop: propagarea actualizarilor la replici cu un numar cat mai mic de mesaje (evita propagarea prin inundare)
- Folosite in depozite de date consistente “in cele din urma”
- Presupunere – propagarea nu este dirijata central, dar
- este initiata la un singur server pentru a evita scrieri conflictuale
- Modelul propagarii se bazeaza pe cel al epidemiilor
- Categorii de servere



- ❑ Infectios – detine un update pe care vrea sa-l propage
- ❑ Susceptibil – inca ne-actualizat
- ❑ Eliminat – detine un update dar nu vrea sa-l propage

- Variante de propagare intre doua servere

- ❑ Push based – un server infectios trimite actualizari unui server susceptibil; mai bun pentru putini infectiosi (la inceputul propagarii)
- ❑ Pull based – un server susceptibil cere actualizari unui server infectios; mai bun pentru multi infectiosi

Protocoale de propagare epidemice (2)

Varianta gossiping (raspandirea zvonurilor)

- Vizeaza evitarea propagarii catre noduri deja actualizate
- Serverele infectioase aleg aleator serverele pentru propagare
- Interesul serverului de a propaga actualizarile scade pe masura ce intalneste servere actualizate deja
- Nu garanteaza actualizarea tuturor replicilor
  - se combina cu anti-entropie pentru a completa distributia

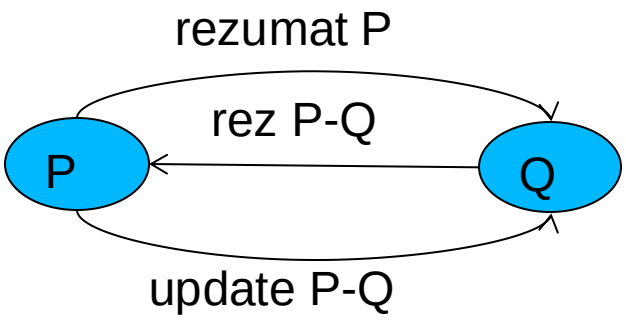




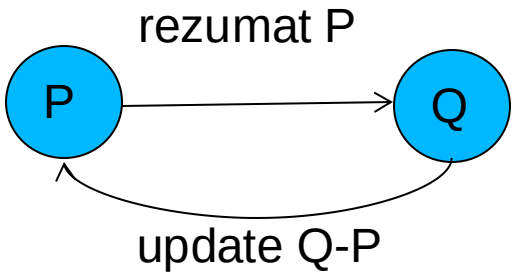

Protocoale de propagare epidemice (3)

Varianta “anti-entropie” (entropie = masura a dezordinii)

- Un server P alege aleator un alt server Q si actualizeaza replicile cu el
- Protocoalele folosite in actualizare
  - **push** - P trimite propriile modificari lui Q
  - **pull** – P preia de la Q noile modificari
  - **push-pull** – P si Q trimit unul altuia modificarile recente



push



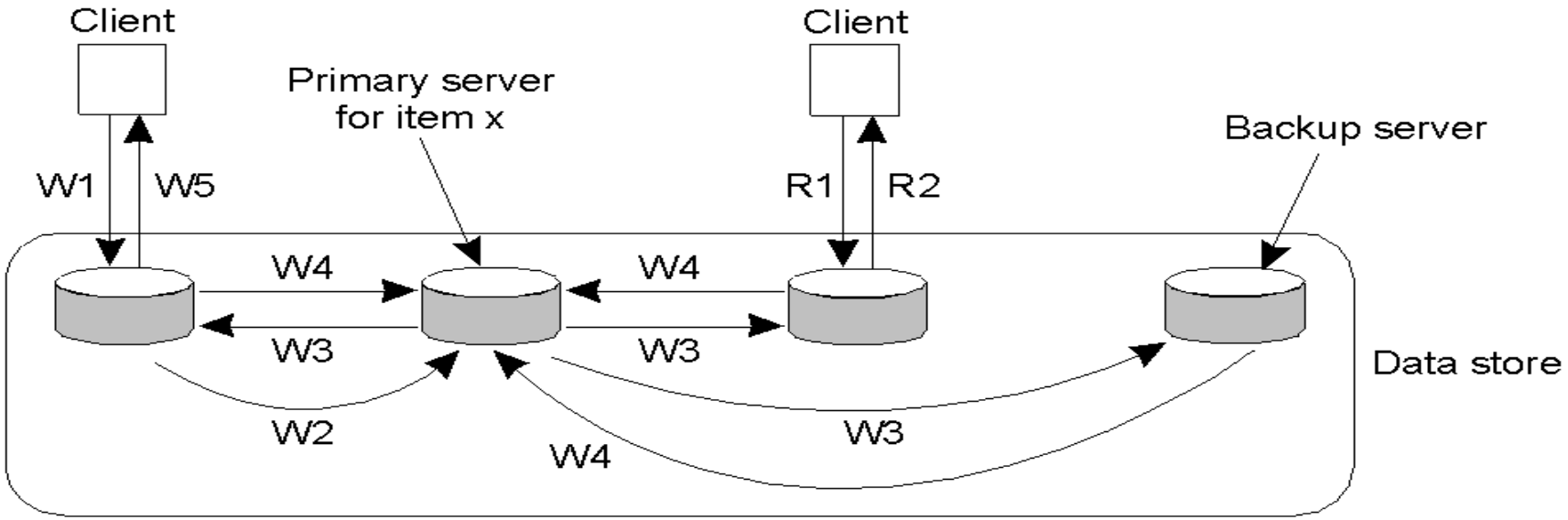
pull

Protocoale de consistenta

- Un protocol de consistenta descrie o implementare a unui model de consistenta
- Protocoale bazate pe o **copie primara**
  - scriere la distanta (Remote-write)
  - scriere locala (Local-write)
- Protocoale cu **scriere** replicata
  - replicare activa
  - protocoale bazate pe cvorum
- Protocoale de coerenta a **cache**-urilor

Protocoale Remote-Write

Protocolul primary-backup: scrierea este blocanta

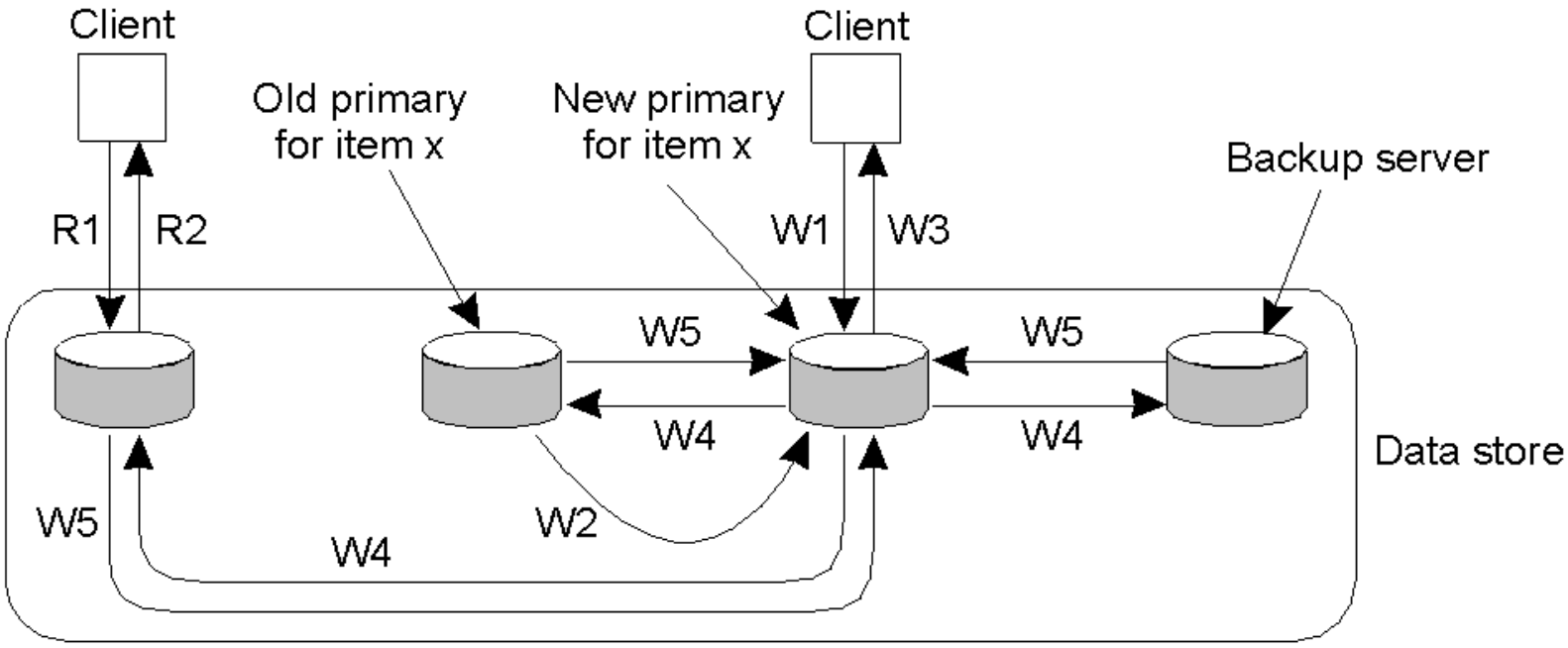


W1. Write request
W2. Forward request to primary
W3. Tell backups to update
W4. Acknowledge update
W5. Acknowledge write completed

R1. Read request
R2. Response to read

- Implementeaza **consistenta secventiala** (copia primara poate ordona toate operatiile de scriere primite) – celelalte replici vad scrierile in aceeasi ordine
- se poate si scriere non-blocanta (acknowledge dupa actualizarea locala)
  - alte procese ar putea sa nu vada noua valoare ci una mai veche

Protocoale Local-Write



W1. Write request
W2. Move item x to new primary
W3. Acknowledge write completed
W4. Tell backups to update
W5. Acknowledge update

R1. Read request
R2. Response to read

Protocol "primary-backup" in care copia primara migreaza la procesul care vrea sa faca actualizarea

Replicare activa

Fiecare replica are asociat un proces care executa operatiile de actualizare.

Operatiile trebuie executate in aceeasi ordine in toate replicile
Solutie: mecanism de **multicast total ordonat**

In cazul replicarii **obiectelor**, doar acest mecanism nu este suficient

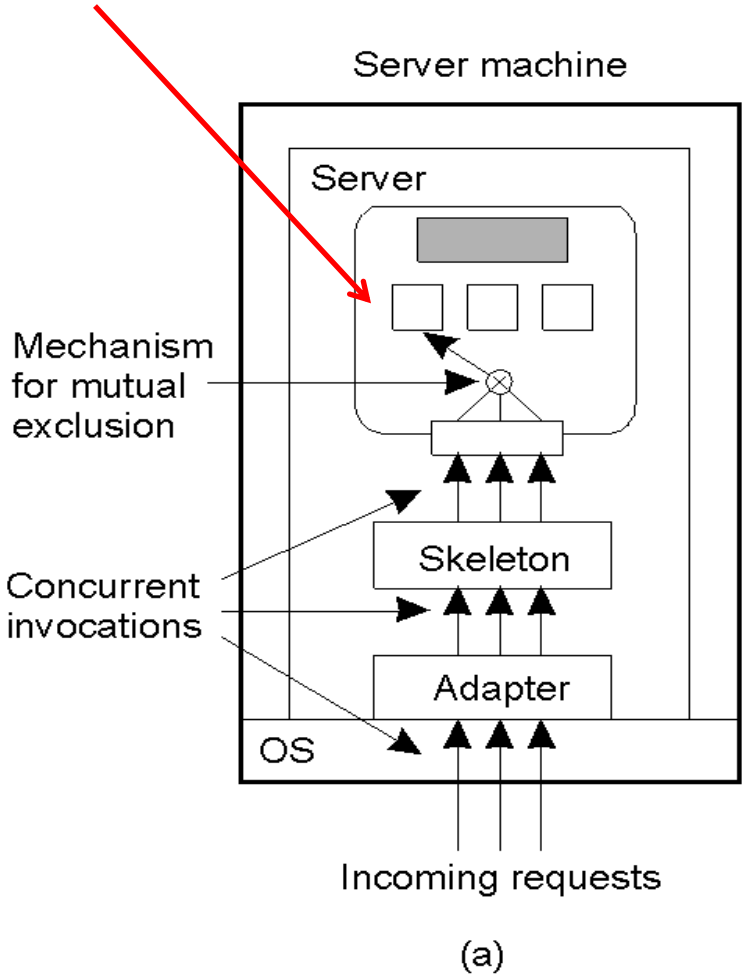
Doua probleme pentru obiecte:

1. Prevenirea executiei concurente a invocarilor aceluiasi obiect
2. Asigurarea executiei in aceeasi ordine a operatiilor in diferite replici

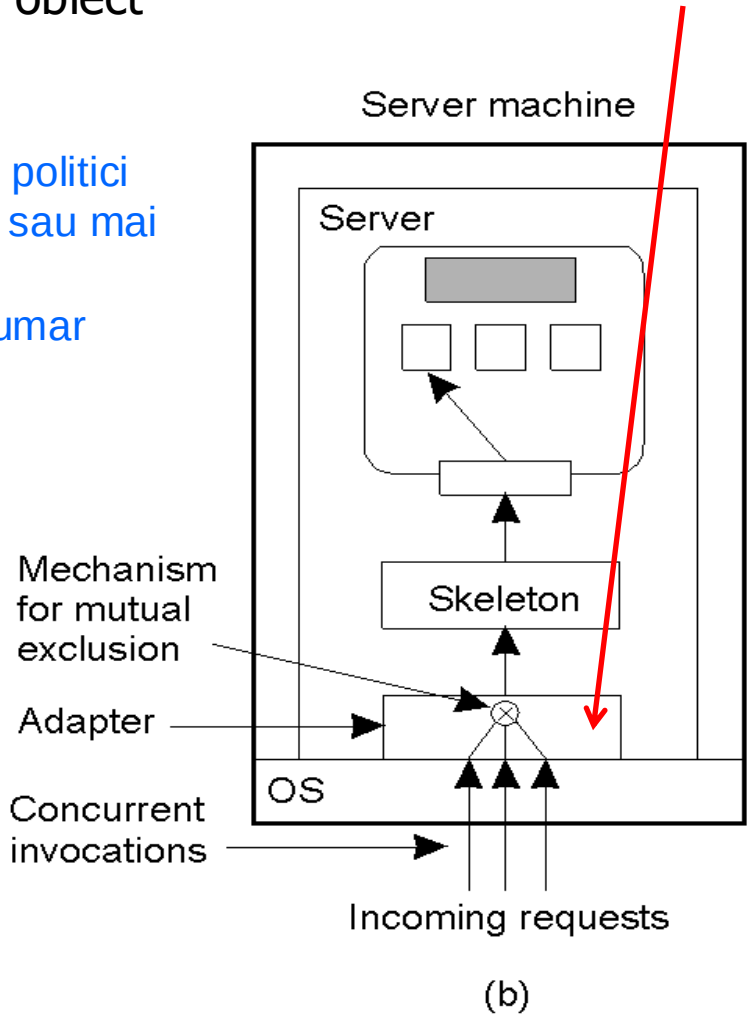
Controlul accesului concurent la un obiect

Obiectul insusi manevreaza accesul concurent – metode *synchronized*

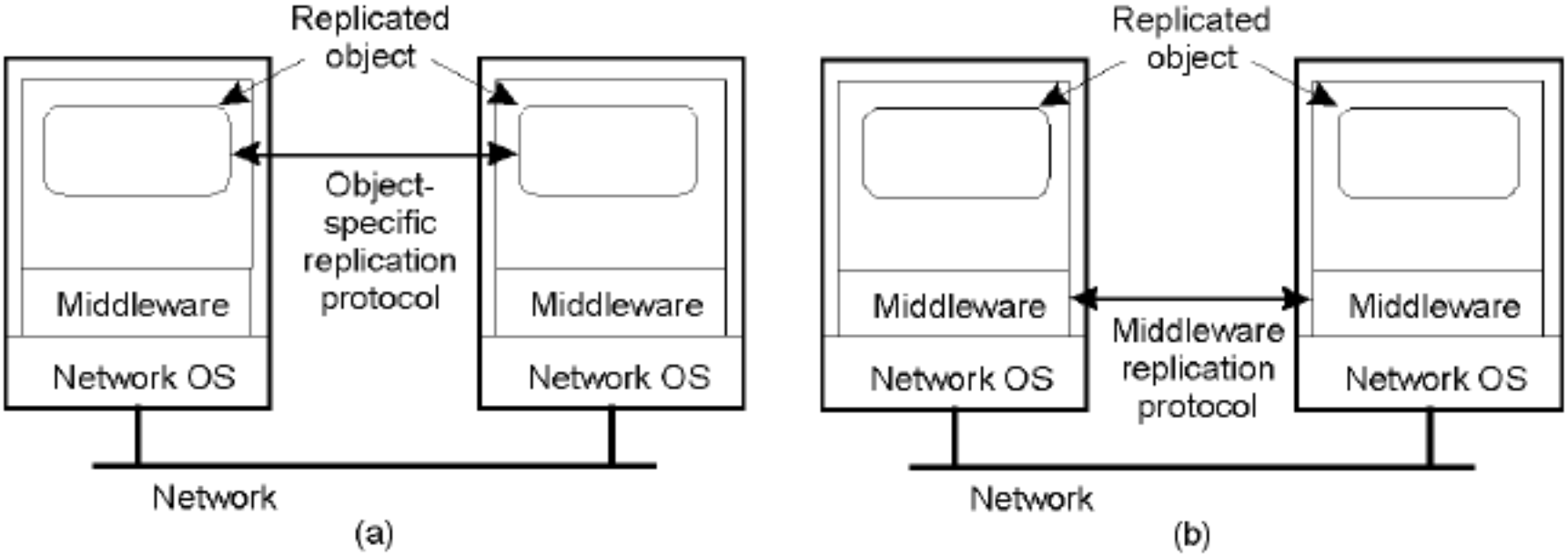
Adaptorul asigura ca invocarile concurente nu lasa obiectul intr-o stare corupta – de ex. asigura un thread per obiect



- Adaptor – politici
- un thread sau mai multe
- limitare numar threaduri



Invocari concurente ale obiectelor replicate



- **Problema**: aceleasi modificari de stare sa apara la obiectele replicate
- **Solutia**: invocarile sa se faca in **aceeasi ordine** la toate replicile
- Doua **implementari** posibile:
  a) Obiectele sunt "constiente" de replicare (Globe)
  b) Sistemul distribuit (middleware) face gestiunea replicilor (Piranha)
     - de ex. ordonarea totala a invocarilor la toate replicile

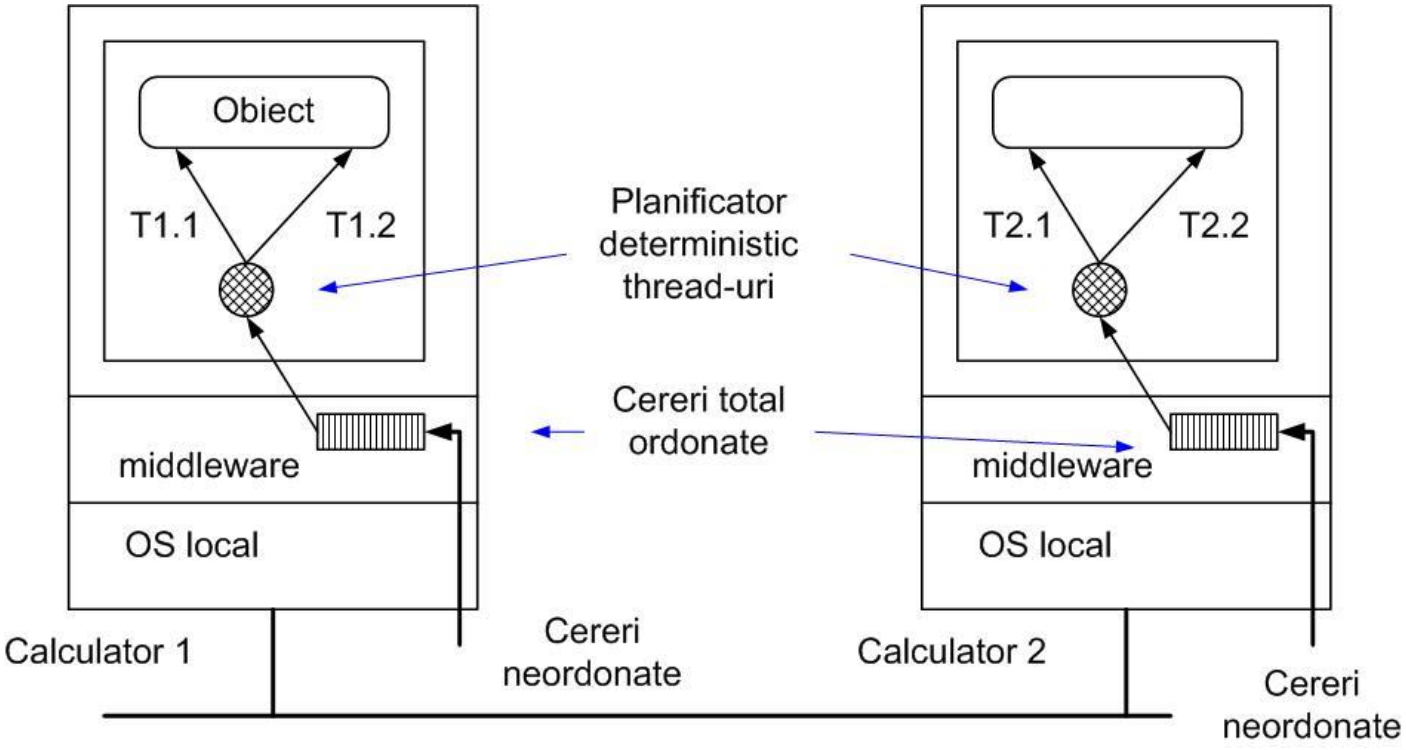
Obiectele sunt "constiente" de replicare

Folosirea unei scheme primary-based la nivel de aplicatie

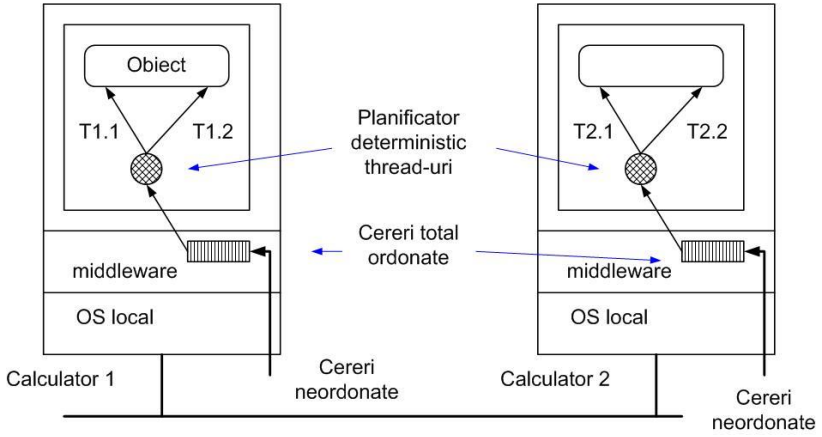
- serializeaza cererile
- neajuns - efort crescut al dezvoltatorului de aplicatii

Implementare consistenta in middelware

- Multicast total ordonat pentru invocari
- In plus - trebuie asigurat si ca threadurile trateaza cererile in ordinea corecta

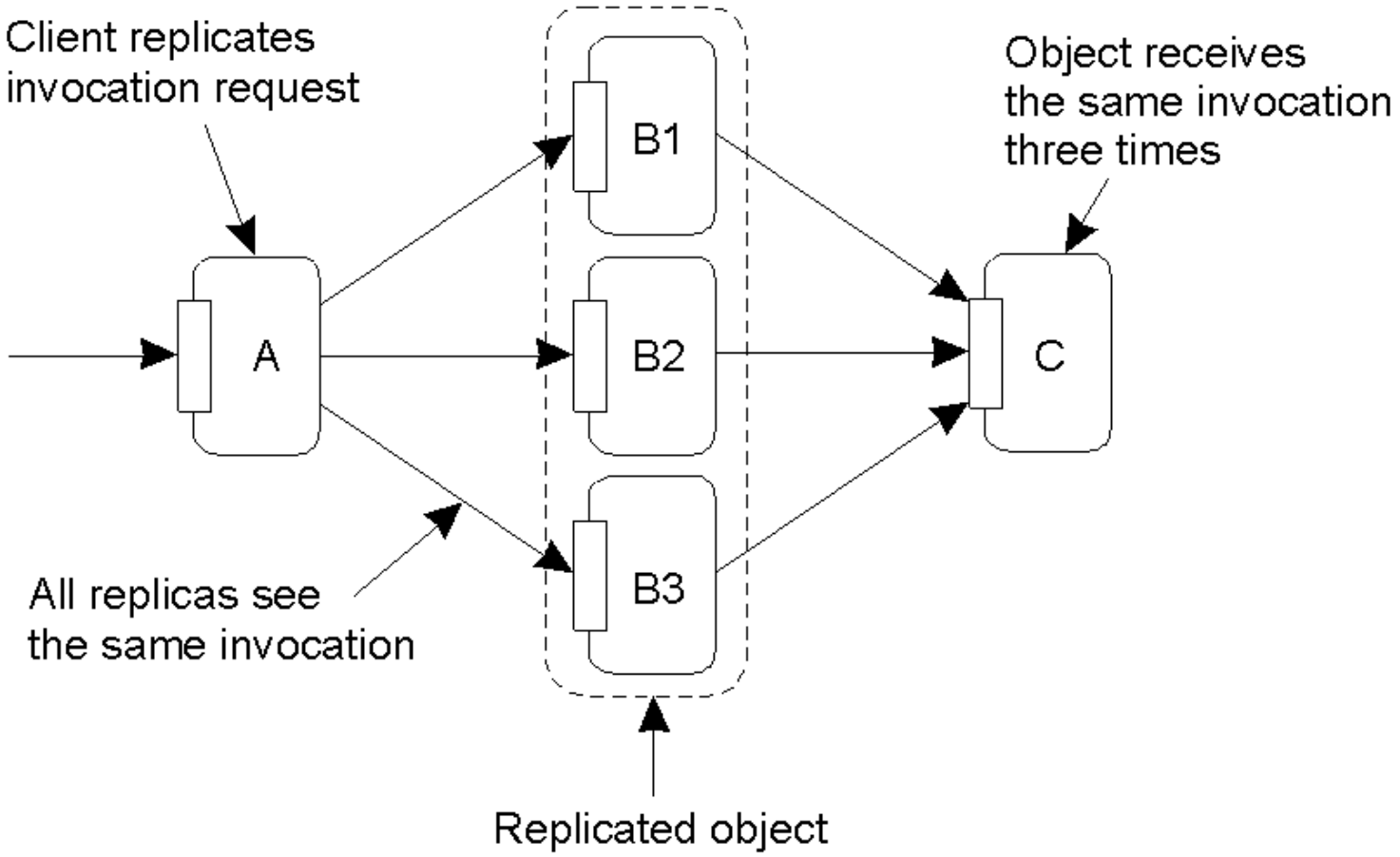


- Functionare servere multithread
  - preiau o cerere si o paseaza unui thread disponibil
  - trec la urmatoarea cerere
  - **planificatorul** de threaduri aloca procesorul threadurilor executabile –ordinea **nu** este aceeasi in toate replicile



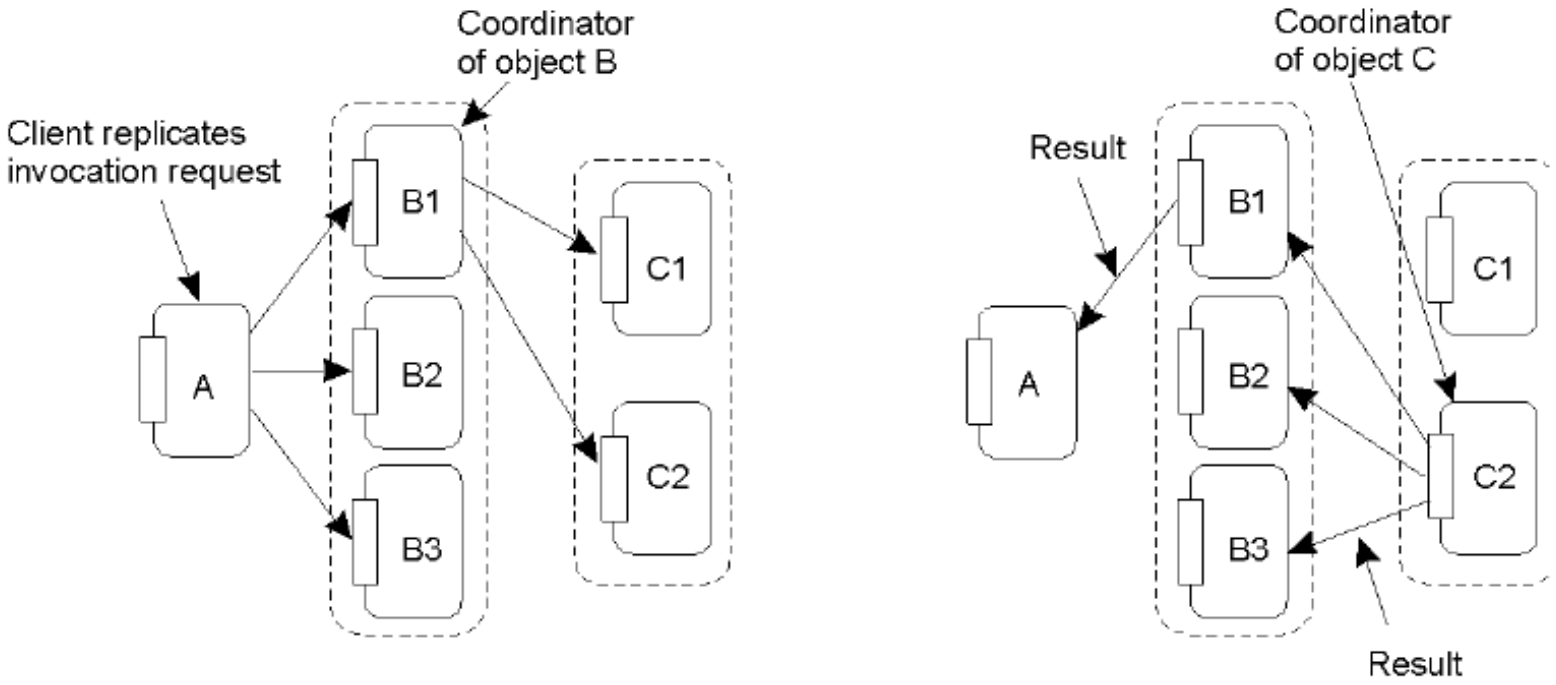
- Solutie: planificator determinist
  - foloseste o varianta de replica primara
    - replica primara determina ordinea threadurilor
    - se sincronizeaza cu celelalte replici prin informatii de context transmise de replica primara
    - se bazeaza pe mecanismul de lock-unlock pentru a porni thread-urile in ordinea stabilita
  - dezavantaj - frecvente comunicari intre replici - ineficient

Invocari Replicate



- Problema: la C ajung invocari replicate; doar una este necesara

Communicatii constiente de replicare



Solutie: bazata pe transmiterea multicast a invocarilor/raspunsurilor

- replicile asigneaza acelasi identificator invocarilor
- doar coordonatorul transmite invocarea
- toate replicile primesc copii ale aceluiasi raspuns

Alternativa: identificarea si eliminarea invocarilor duplicate la un obiect

Protocoale bazate pe cvorum

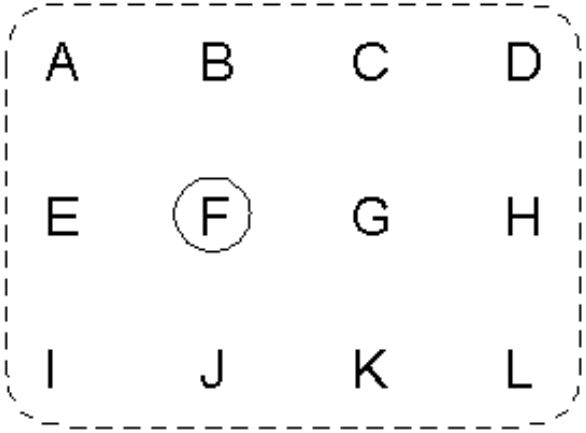
Modelul clasic de gestiune a operatiilor read si write in cazul replicarii datelor este ROWA (Read One, Write All)

In acest caz:

- numarul de replici accesate la read este $N_R = 1$
- numarul de replici accesate la write este $N_W = N$ (egal cu numarul total de replici)

Solutia este convenabila atunci cand frecventa operatiilor read este semnificativ mai mare decat a operatiilor write

In alte cazuri, se pot alege scheme in care numarul de replici citite si numarul de replici scrise sunt ambele mai mari ca 1



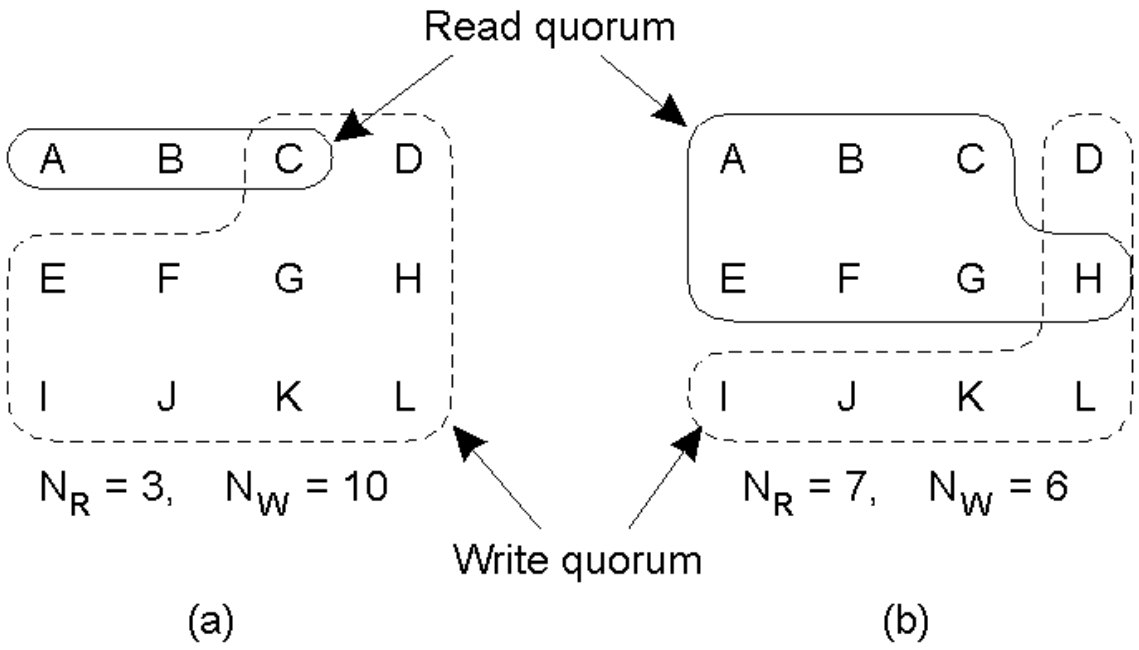
$N_R = 1, \quad N_W = 12$

Protocoale bazate pe cvorum (2)

In alegerile ilustrate aici, N_R si N_W trebuie sa respecte anumite reguli

Se foloseste un numar de versiune pentru fiecare replica

La un write, cele N_W replici capata acelasi numar de versiune.

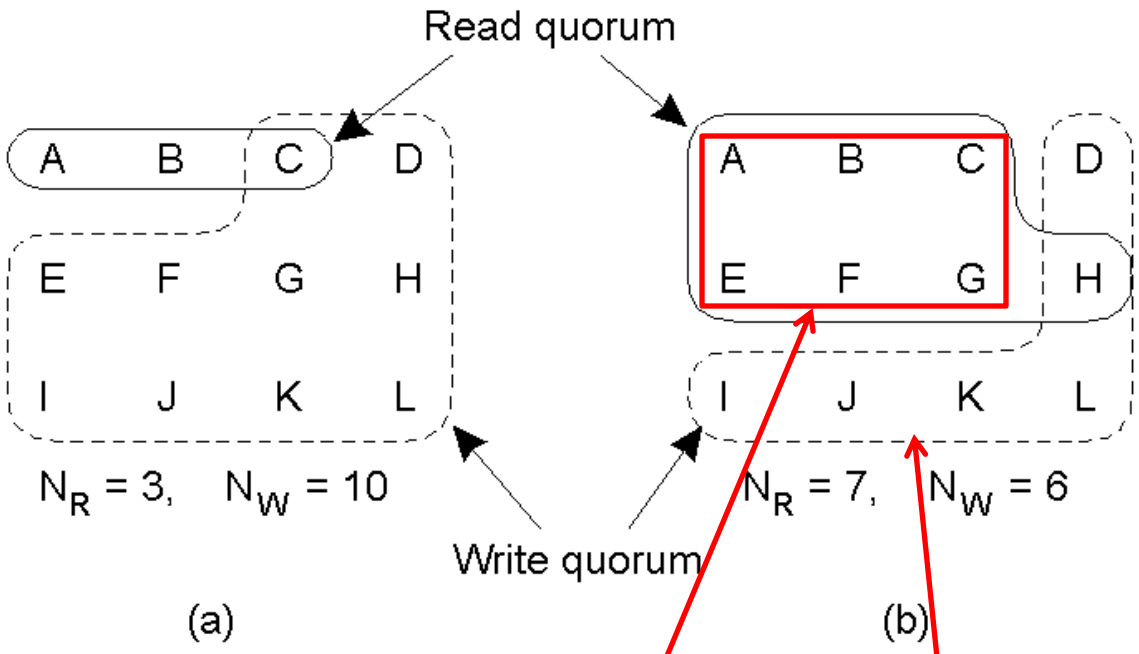


(a) Alegere corecta a seturilor (**forumurilor**) de citire si scriere

$$N_R + N_W > N$$

Oricare ar fi alegerea cititorului, in cvorumul de citire intra cel putin o resursa care are ultima versiune

Protocoale bazate pe cvorum (3)



(b) Alegere ce poate conduce la conflicte write-write

Ex. un proces alege setul de scriere {A,B,C,E,F,G} iar altul {D,H,I,J,K,L} putand rezulta replici cu acelasi numar de versiune dar valori diferite Solutie ???

Eliminarea conflictelor impune $\mathbf{N_W > N/2}$

Concluzii ROWA

- (a) Nr+Nw>N – Nu avem scrieri in timpul citirilor
- (b) Nw>N/2 – cel putin un writer

Aplicatie: partajarea fisierelor in CODA

Carnegie Mellon University

Abordarea CODA

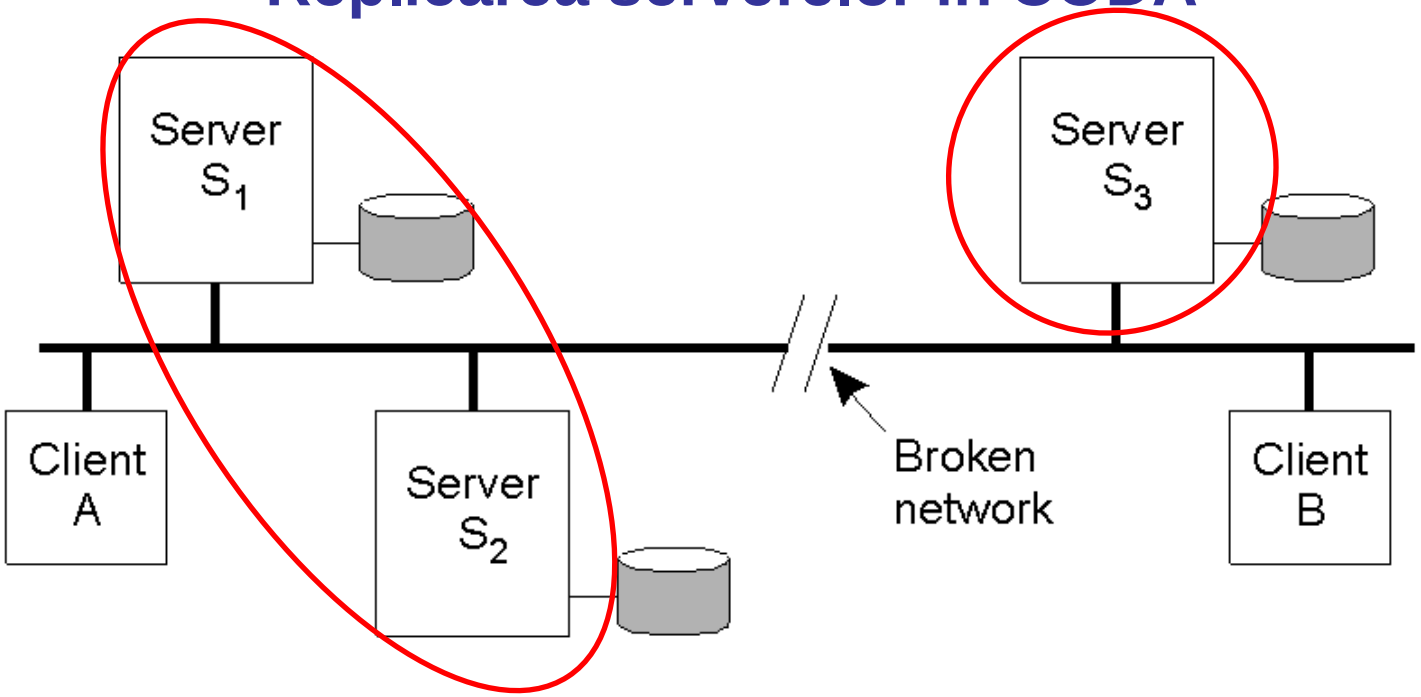
•cand un client deschide un fisier f, o copie a lui f este transferata la client si serverul inregistreaza ca clientul A are o copie a lui f

•mai multi clienti pot deschide f pentru read

•un singur client poate deschide f pentru write

- cand termina sesiunea de actualizare, clientul va trimite serverului fisierul actualizat
- serverul va trimite un mesaj de invalidare clientilor care citesc fisierul
- acestia pot ignora mesajele si pot continua sa lucreze pe copiile locale, neactualizate – justificare ???
  - o sesiune este tratata ca o tranzactie – procesul care citeste poate considera ca sesiunea lui s-a terminat inainte de modificarea lui **f**

Replicarea serverelor in CODA



Unitatea de replicare este **volumul** (colectie de fisiere)

- **VSG** (Volume Storage Group) = grupul de servere care au copia volumului
- **AVSG** (Accessible Volume Storage Group) = serverele la care un client are acces

In figura {S1, S2} este AVSG pentru clientul A
{S3} este AVSG pentru clientul B

CODA foloseste un protocol **ROWA** ptr. consistenta unui volum replicat

- la read, clientul foloseste un fisier dintr-un volum accesibil
- la write, un singur client modifica un fisier dintr-un volum accesibil; la sfarsitul sesiunii, el transfera fisierul, in paralel, celorlalte volume accesibile

La partitionare retea (defectare)

- doi clienti pot lucra pe AVSG-uri diferite si pot modifica replici diferite ale aceluiasi fisier
- la refacerea retelei, se propaga modificarile la toate replicile din WSG
- problema: modificari inconsistente

Solutie detectie inconsistente

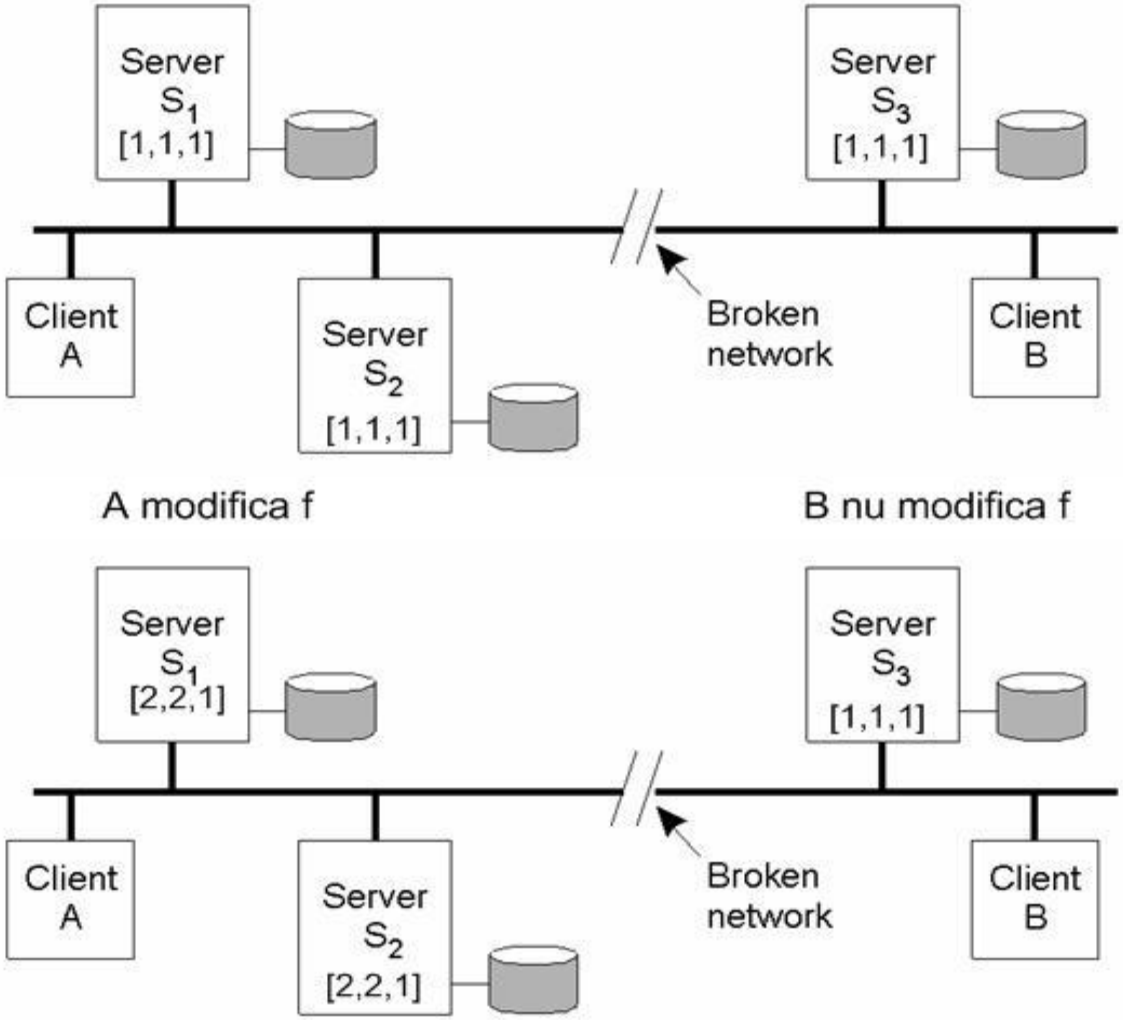
- CODA pastreaza pentru fiecare fisier **f** si server **k** un vector de versiune (Coda Version Vector) $\mathbf{CVV_k(f)}$
- fiecare element $\mathbf{CVV_k(f)[j]}$ este numarul, pastrat de **k**, al versiunii curente a lui **f** pe serverul **Sj**
- dupa rezolvarea defectului de partitionare a retelei, se compara CVV-urile si se detecteaza conflictele

Fara conflict

[1,1,1] = numerele de versiune ale unui fisier, corespunzatoare serverelor S1, S2, S3 in aceasta ordine

doua CVV nu sunt in conflict daca sunt egale sau unul este <= decat celalalt

Modificarile facute de Clientul A si inregistrate de S1 si S2 vor fi acceptate de S3 dupa refacerea retelei

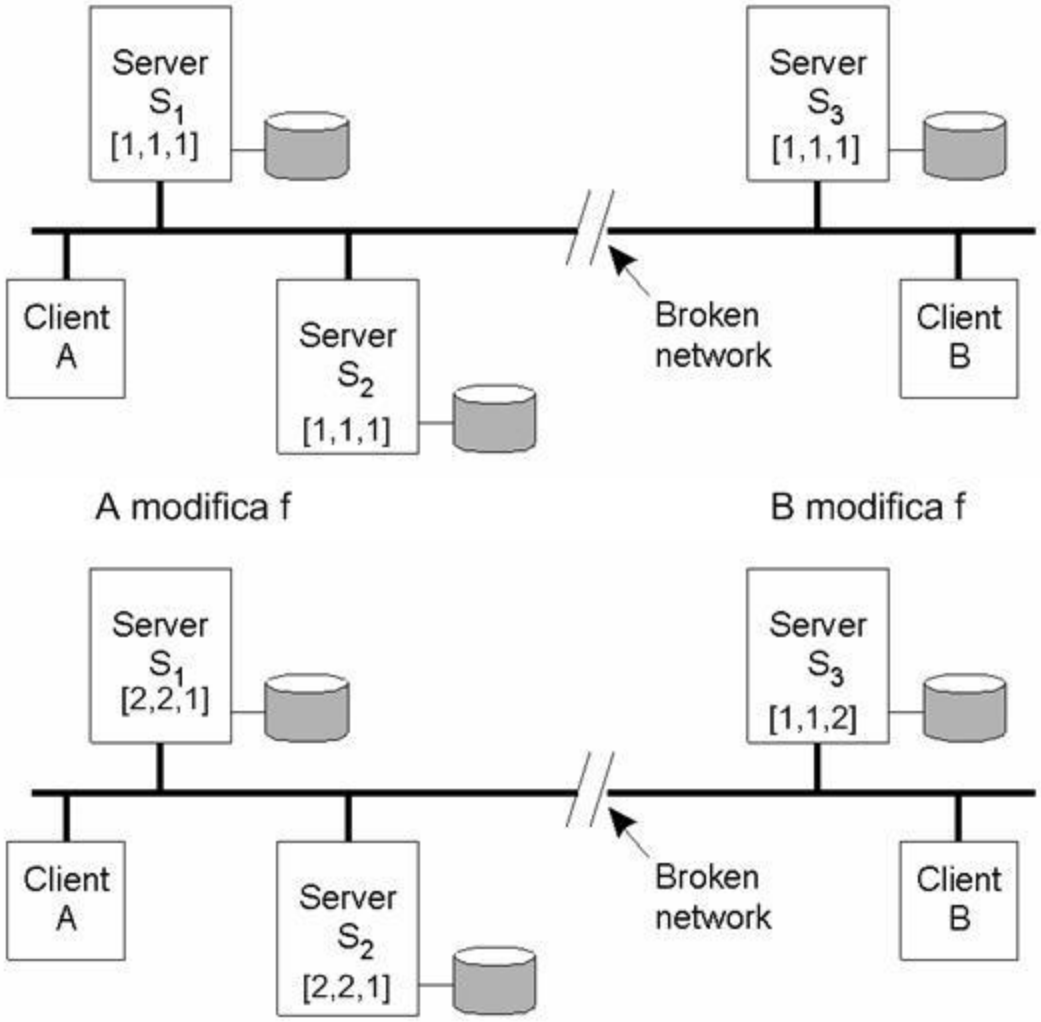


Conflict

- f este modificat de ambii clienti, A si B
- cele doua CVV, [2,2,1] si [1,1,2], sunt in conflict

Rezolvare

- solutie dependenta de aplicatie, posibil automata
- solutie manuala

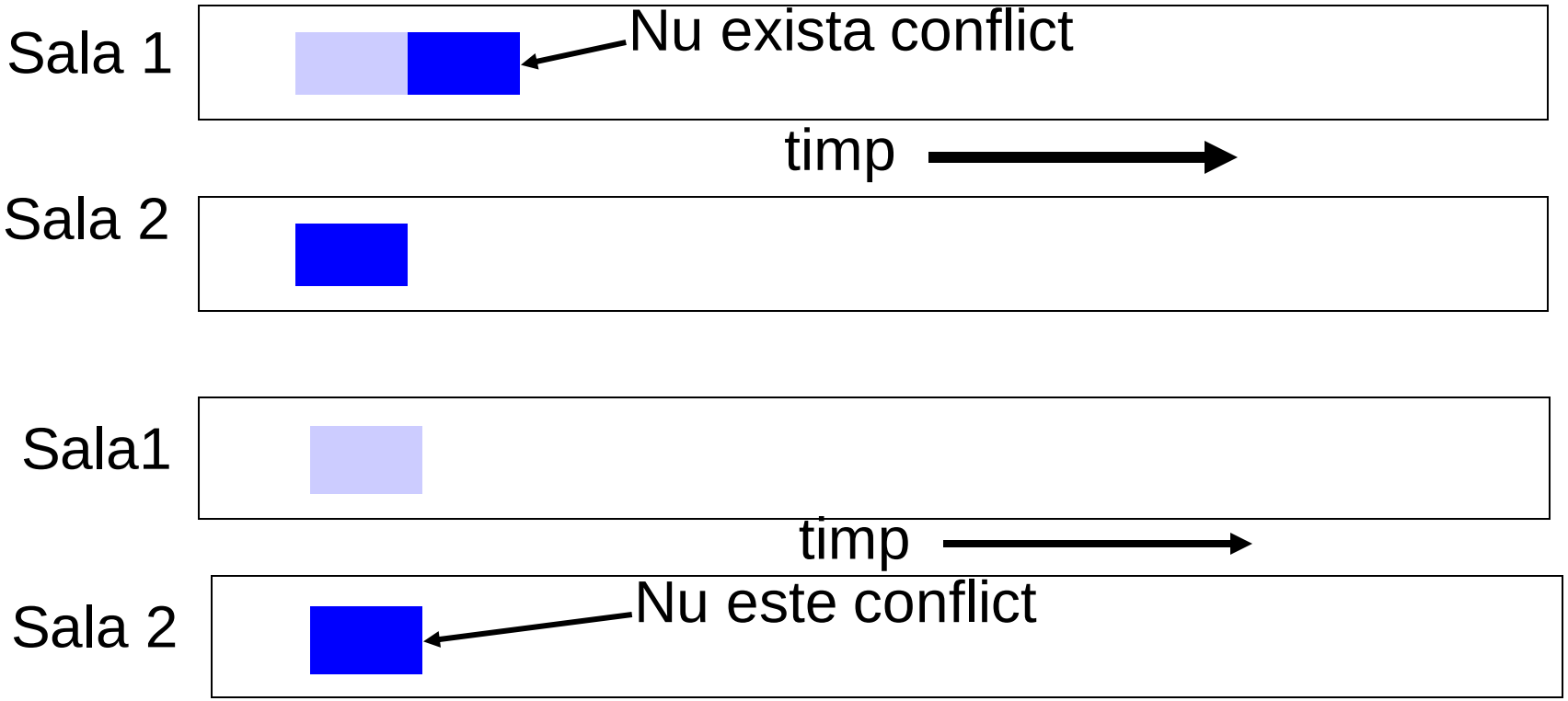


Replicare si Consistenta in Bayou

- Modelele de consistenta centrate pe client isi au originea in Bayou
  - Bayou = Baza de date distribuita peste o retea cu conectivitate nesigura
- Scopul Bayou: gestiunea bazelor de date total replicate in orice numar de site-uri
- Modelul de consistenta este relaxat
  - accesul si actualizarea datelor se poate face oriunde (access-update-anywhere)
- Replicarea este ne-transparenta
  - Aplicatiile sunt implicate in detectia si rezolvarea conflictelor
    - aplicatiile stiu mai bine cum sa trateze inconsistentele
    - Dezavantaj – implementare mai complicata a aplicatiei

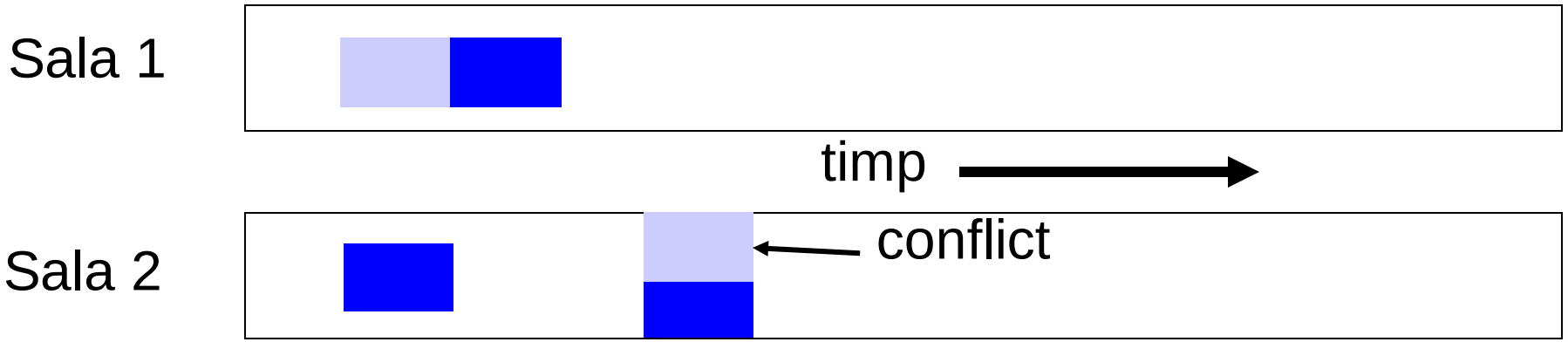
Aplicatie: Planificarea unei sali de conferinte

- Presupunem ca sunt disponibile doua sali de conferinte de aceeasi capacitate.
- Daca doua persoane rezerva sala la ore diferite sau rezerva sali diferite la aceeasi ora nu exista conflict.



Planificarea unei sali de conferinte

- Daca doua persoane rezerva aceeasi sala la o aceeasi ora exista un conflict



Implicarea aplicatiilor

- Pentru fiecare rezervare, se poate face lock pe toata baza de date
  - Nu este necesar daca nu exista conflict
  - In caz de conflict, exista mai multe alternative
    - Se muta o rezervare in cealalta sala
    - Daca cealalta sala este rezervata, se muta rezervarea la o alta ora acceptabila

Concluzii

- Aplicatiile stiu mai bine cum sa rezolve conflictele
- Trebuie oferita interfata care suporta cooperarea intre aplicatii si managerii datelor

Detectia Conflictelor: Controlul dependentelor

Detectia conflictelor dependente de aplicatii se face in Bayou la fiecare write, prin controlul dependentelor (dependency_check)

- controlul consta dintr-un query furnizat de aplicatie si rezultatul asteptat
- controlul se face de serverul invocat, pe replica curenta de date
- se detecteaza un conflict daca rezultatul produs nu este cel asteptat

In caz de conflict, operatia write nu se executa,

iar serverul apeleaza o procedura de rezolvare a conflictului (mergeproc)

Exemplu (descris in continuare)

- operatia write incearca sa rezerve o sala de intalniri pentru un interval de o ora




```
Bayou_Write(
update = {insert, Meetings, 12/18/95, 10:00am, 60min, “Project Meeting: Kevin”},
dependency_check = {
   query = “SELECT key FROM Meetings WHERE day = 12/18/95
      AND start < 11:00am AND end > 10:00am”,
   expected_result = EMPTY},
mergeproc = {
   alternates = {{12/18/95, 12:00pm},{12/19/95, 11:30am}};
   newupdate = {};
   FOREACH a IN alternates {
     # check if there would be a conflict
     IF (NOT EMPTY ( SELECT key FROM Meetings WHERE day = a.date
                  AND start < a.time + 60min AND end > a.time))
          CONTINUE;
     # no conflict, can schedule meeting at that time
     newupdate = {insert, Meetings, a.date, a.time, 60min, “Project Meeting: Kevin”};
     BREAK;
}
IF (newupdate = {}) # no alternate is acceptable
   newupdate = {insert, ErrorLog, 12/18/95, 10:00am, 60min, “Project Meeting: Kevin”};
RETURN newupdate;}
)
```

Comentarii

Procesul este executat automat de server ca parte a operatiei write

Forma: <update><dependency check><mergeproc>

Operatia write: insert este o actualizare a rezervarilor salilor de conferinte

update = {insert, Meetings, 12/18/95, 10:00am, 60min, “Project Meeting: Kevin”},

Preconditia de executie: daca e satisfacuta se aplica update

dependency_check = {
query = “SELECT key FROM Meetings WHERE day = 12/18/95
AND start < 11:00am AND end > 10:00am”,
expected_result = EMPTY},

Comentarii (2)

Procedura de rezolvare: aplicata daca preconditia nu este satisfacuta

- cod interpretat (stil SQL) care genereaza un update
- verifica mai multe rezolvari alternative

```
mergeproc = {
    alternates = {{12/18/95, 12:00pm},{12/19/95, 11:30am}};
    newupdate = {};
    FOREACH a IN alternates {
        # check if there would be a conflict
        IF (NOT EMPTY ( ...
```

Rezolvare automata Imposibila: conflictul este semnalat, urmand a fi rezolvat de utilizator.

```
IF (newupdate = {})      # no alternate is acceptable
    newupdate = {insert, ErrorLog, 12/18/95, 10:00am, 60min, “Project
        Meeting: Kevin”};
```

Executia op. write si inconsistenta replicilor

O operatie write (actualizare) poate fi adresata de un client oricarui server care contine o replica a datelor corespunzatoare

- ea este executata local
- si este propagata altor replici, care o executa imediat ce o primesc
- intarzierile de propagare difera de la o replica la alta → doua replici pot avea valori diferite datorita unor op. write diferite sau executate in ordini diferite

Bayou foloseste un protocol anti-entropie de reconciliere intre replici care asigura ca, in cele din urma

- serverele adopta aceeasi ordine a operatiilor write
- ca urmare, toate replicile vor avea acelasi continut de date
- Bayou **nu poate garanta** o limita a intarzierii de reconciliere a operatiilor write



Sistemul Bayou – log-uri si baza de date

Sistemul de stocare (server) pastreaza pentru fiecare replica

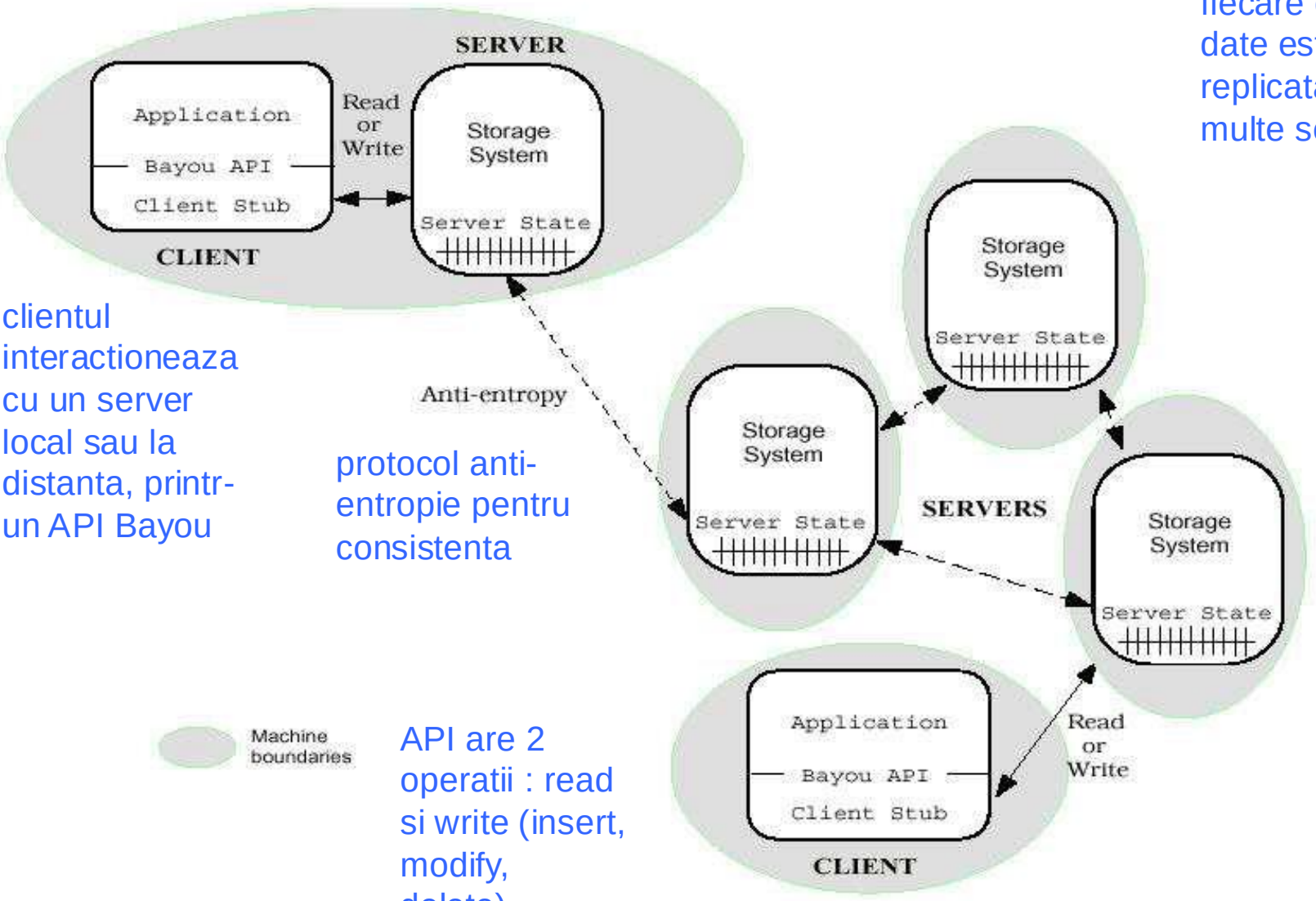
- un log ordonat de operatii write (scrieri)
  - care include toate operatiile write primite de la aplicatie (clienti) sau de la alte servere
- o baza de date al carei continut rezulta din aplicarea ordonata a scrierilor

Asigurarea consistentei “in cele din urma” (eventual consistency) se bazeaza pe

- ordonarea **globala** a tuturor operatiilor write
- folosirea unor proceduri deterministe pentru detectia si rezolvarea conflictelor la write

Modelul sistemului Bayou

fiecare colectie de date este total replicata pe mai multe servere



clientul interactioneaza cu un server local sau la distanta, printr-un API Bayou

protocol anti-entropie pentru consistenta

API are 2 operatii : read si write (insert, modify, delete)

Ordonarea partiala a operatiilor write

- Cand un server primeste un write de la un client, el ii asigneaza un numar de ordine / amprenta de timp – accept-stamp
  - poate fi un contor a carui valoare creste de la un write la altul
- Operatiile write acceptate de la client se transmit altor servere
  - impreuna cu accept-stamp si cu identificatorul serverului care i-a asignat numarul de ordine - (accept-stamp, server_ID)
- Accept-stamps ordoneaza partial setul operatiilor write primite de un server, de la client sau de la alte servere
  - se poate spune ca write A precede write B cand ambele au fost acceptate de acelasi server si write A a fost acceptat inaintea lui write B

Ordonarea totala – operatii stabile si tentative

- Ordinea partiala nu este suficienta pentru asigurarea consistentei
- Pentru a obtine consistenta “eventual” se foloseste un proces anti-entropie care stabileste ordinea corecta a operatiilor write, pe care trebuie sa o respecte toate replicile
  - procesul stabileste operatiile stabile (committed) - a caror pozitie in log nu se mai modifica in procesul anti-entropie
  - si operatiile a caror ordine se poate modifica, numite tentative
  - logul operatiilor write pastrat pentru fiecare replica contine operatiile stabile in ordinea numerelor de secventa comise, urmat de operatiile tentative in ordinea amprentelor accept-stamp
- Operatiile tentative trebuie sa poata fi anulate si re-executate
  - deoarece serverul executa imediat operatiile write primite, ordonarea totala poate schimba ordinea operatiilor tentative si cere anularea efectelor unora din operatii si re-executia lor in ordinea corecta

Protocolul anti-entropie de baza

Protocolul permite ca doua severe sa puna de acord (compatibilizeze) continutul log-urilor lor folosind numerele de ordine accept-stamps

- se bazeaza pe schimbul de operatii write intre replici

Propagarea operatiilor write este constransa de proprietatea prefix

- un server R care detine o scriere Wi initial acceptata de serverul X, detine toate scrierile acceptate de X inainte de Wi
- simplifica evidenta operatiilor de scriere cunoscute de R, folosind un vector de versiune R.V avand o intrare pentru fiecare server
- astfel, intrarea din R.V pentru serverul X, notata R.V(X) contine doar accept-stamp maxim al operatiilor write acceptate de X despre care R are cunostinta

Algoritmul anti-entropie

Algoritmul actualizeaza serverul R cu operatiile write detinute de S si necunoscute de R, folosind vectorul R.V de versiuni al lui R

```
anti-entropy(S,R) {
   Preia R.V de la serverul R
    #trimite toate op write necunoscute de R
   w = primul write in S.write-log
   while (w) do
     if R.V(w.server-id) < w.accept-stamp
        then
           # w este nou pentru R
            SendWrite(R,w)
        w = urmatorul write in S.write-log
}
```

Procesul de actualizare a serverului S de catre R este identic

Stabilizarea operatiilor - comiterea

Obtinerea operatiilor write stabile in loguri permite executia lor in aceeasi ordine in toate replicile (ops. stabile nu trebuie re-executate!)

Bayou foloseste un primary-commit protocol pentru a determina cand un write devine stabil

- o replica este desemnata primara
- ea “stabilizeaza” (commit) operatiile write
  - asociaza operatiei un numar de secventa comis CSN (Commit Sequence Number)
- operatiile write stabile sunt total ordonate



Log-ul ordonat al operatiilor

Cu CSN, informatia asociata unei operatii write devine

(CSN, accept-stamp, server_ID)

Prin conventie, numarul CSN asociat unei operatii tentative este infinit ∞

Log-ul ordonat al operatiilor (dupa comitere) are doua parti

- lista scrierilor stabile total ordonata crescator dupa CSN
- lista scrierilor tentative partial ordonata dupa accept-stamps

CSN este propagat celorlalte servere folosind o extensie a algoritmului anti-entropie

La propagare se trimit in ordine operatiile write stabile, apoi operatiile write tentative



Extensia algoritm anti-entropie

```
anti-entropy(S,R) {
   Preia R.V de la serverul R
    #Trimite toate op write comise pe care R nu le stie
    if R.CSN < S.CSN then
       w = primul write comis despre care R nu stie
       while (w) do
           if w.accept-stamp < R.V(w.server-id) then
                # R are op write, dar nu stie ca ea este comisa
                SendCommitNotification(R, w.accept-stamp,w.server-id, w.CSN)
           else SendWrite(R,w)
           end
           w = urmatorul write comis din S.write-log
       end
    end
```

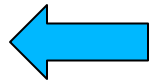
Algoritmul este executat de serverul S pentru a actualiza serverul R

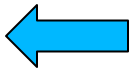
S.CSN este cel mai mare numar de secventa comis cunoscut de S

El reprezinta concis toata partea comisa a log-ului deoarece

- operatiile comise sunt ordonate dupa numerele de secventa comise
- propagarea lor se face in aceasta ordine

```
anti-entropy(S,R) {
   Preia R.V de la serverul R
    #Trimite toate op write comise pe care R nu le stie
    if R.CSN < S.CSN then
       w = primul write comis despre care R nu stie
       while (w) do
           if w.accept-stamp < R.V(w.server-id) then
               # R are op write, dar nu stie ca ea este comisa
               SendCommitNotification(R, w.accept-stamp,w.server-id, w.CSN)
           else SendWrite(R,w)
           end
           w = urmatorul write comis din S.write-log
       end
    end
```





In prima parte a algoritmului, S trimite lui R operatiiile write comise pe care R nu le stie, in doua feluri

- notificari de comitere pentru operatiile w pe care R le are dar nu stie ca sunt comse (w.accept-stamp < R.V(w.server-id)
- operatiile in intregime, pentru writes pe care R nu le are




```
anti-entropy(S,R) {
   Preia R.V de la serverul R
    #Trimite toate op write comise pe care R nu le stie
    if R.CSN < S.CSN then
       w = primul write comis despre care R nu stie
       while (w) do
            if w.accept-stamp < R.V(w.server-id) then
                 # R are op write, dar nu stie ca ea este comisa
                 SendCommitNotification(R, w.accept-stamp,w.server-id, w.CSN)
            else SendWrite(R,w)
            end
            w = urmatorul write comis din S.write-log
       end
    end
    w = primul write tentativ
    #acum trimite writes tentative
    while (w) do
          if R.V(w.server-id) < w.accept-stamp then
                SendWrite(R,w)
          w = urmatorul write din S.write-log
    end
}
```

In partea a doua a algoritmului, se trimit operatiile tentative pe care R nu le are

Proprietatile solutiei

Anti-entropie incrementala

- reconcilierea intre doua replici poate fi intrerupta (defecte de retea)
- reluarea nu reclama re-transmiterea operatiilor deja actualizate

Replicile pot trunchia orice prefix din partea stabila (operatii comise)

- Implicatie: o replica nu mai are toate operatiile pentru reconciliere incrementala cu o alta replica
- Rezolvare: se mentin informatii de identificare a partii trunchiate
- daca este cazul, se transfera intreaga baza de date pentru actualizarea altei replici

Exemplu cu trei servere

Trei servere cu vectorii de versiune initiali

P	A	B
P.V = [0, 0, 0]	A.V = [0, 0, 0]	B.V = [0, 0, 0]

Un vector de versiuni contine accept-stamp-urile maxime ale replicilor

Elementele vectorilor corespund, in ordine, replicilor P, A, B

Serverele scriu independent

P adauga log-ului operatiile acceptate
Fiecare intrare contine <CSN, accept-stamp, server_ID>

inf = ∞

P	A	B
<inf, 1, P>	<inf, 2, A>	<inf, 1, B>
<inf, 4, P>	<inf, 3, A>	<inf, 5, B>
<inf, 8, P>	<inf, 10, A>	<inf, 9, B>
[8, 0, 0]	[0, 10, 0]	[0, 0, 9]

vectorul de versiuni al replicii P
dupa acceptarea unor operatii write

P si A fac un schimb anti-entropie

P	A	B
<inf, 1, P>	<inf, 1, P>	<inf, 1, B>
<inf, 2, A>	<inf, 2, A>	<inf, 5, B>
<inf, 3, A>	<inf, 3, A>	<inf, 9, B>
<inf, 4, P>	<inf, 4, P>	
<inf, 8, P>	<inf, 8, P>	[0, 0, 9]
<inf, 10, A>	<inf, 10, A>	
[8, 10, 0]	[8, 10, 0]	dupa schimb





<inf, 1, P>	<inf, 2, A>	
<inf, 4, P>	<inf, 3, A>	
<inf, 8, P>	<inf, 10, A>	
[8, 0, 0]	[0, 10, 0]	inainte de schimb

P comite unele scrieri

P comite 3 operatii

P	A	B
<1, 1, P>	<inf, 1, P>	<inf, 1, B>
<2, 2, A>	<inf, 2, A>	<inf, 5, B>
<3, 3, A>	<inf, 3, A>	<inf, 9, B>
<inf, 4, P>	<inf, 4, P>	
<inf, 8, P>	<inf, 8, P>	[0, 0, 9]
<inf, 10, A>	<inf, 10, A>	
[8, 10, 0]	[8, 10, 0]	

< inf, 1, P>
< inf, 2, A>
< inf, 3, A>
<inf, 4, P>
<inf, 8, P>
<inf, 10, A>

[8, 10, 0]

P si B fac un schimb anti-entropie

P	A	B
<1, 1, P>	<inf, 1, P>	<1, 1, P>
<2, 2, A>	<inf, 2, A>	<2, 2, A>
<3, 3, A>	<inf, 3, A>	<3, 3, A>
<inf, 1, B>	<inf, 4, P>	<inf, 1, B>
<inf, 4, P>	<inf, 8, P>	<inf, 4, P>
<inf, 5, B>	<inf, 10, A>	<inf, 5, B>
<inf, 8, P>		<inf, 8, P>
<inf, 9, B>	[8, 10, 0]	<inf, 9, B>
<inf, 10, A>		<inf, 10, A>
[8, 10, 9]		[8, 10, 9]
↑		↑
<1, 1, P>		<inf, 1, B>
<2, 2, A>		<inf, 5, B>
<3, 3, A>		<inf, 9, B>
<inf, 4, P>		
<inf, 8, P>		[0, 0, 9]
<inf, 10, A>		
[8, 10, 0]		

P comite mai multe operatii write

P
<1, 1, P>
<2, 2, A>
<3, 3, A>
<inf, 1, B>
<inf, 4, P>
<inf, 5, B>
<inf, 8, P>
<inf, 9, B>
<inf, 10, A>
[8, 10, 9]



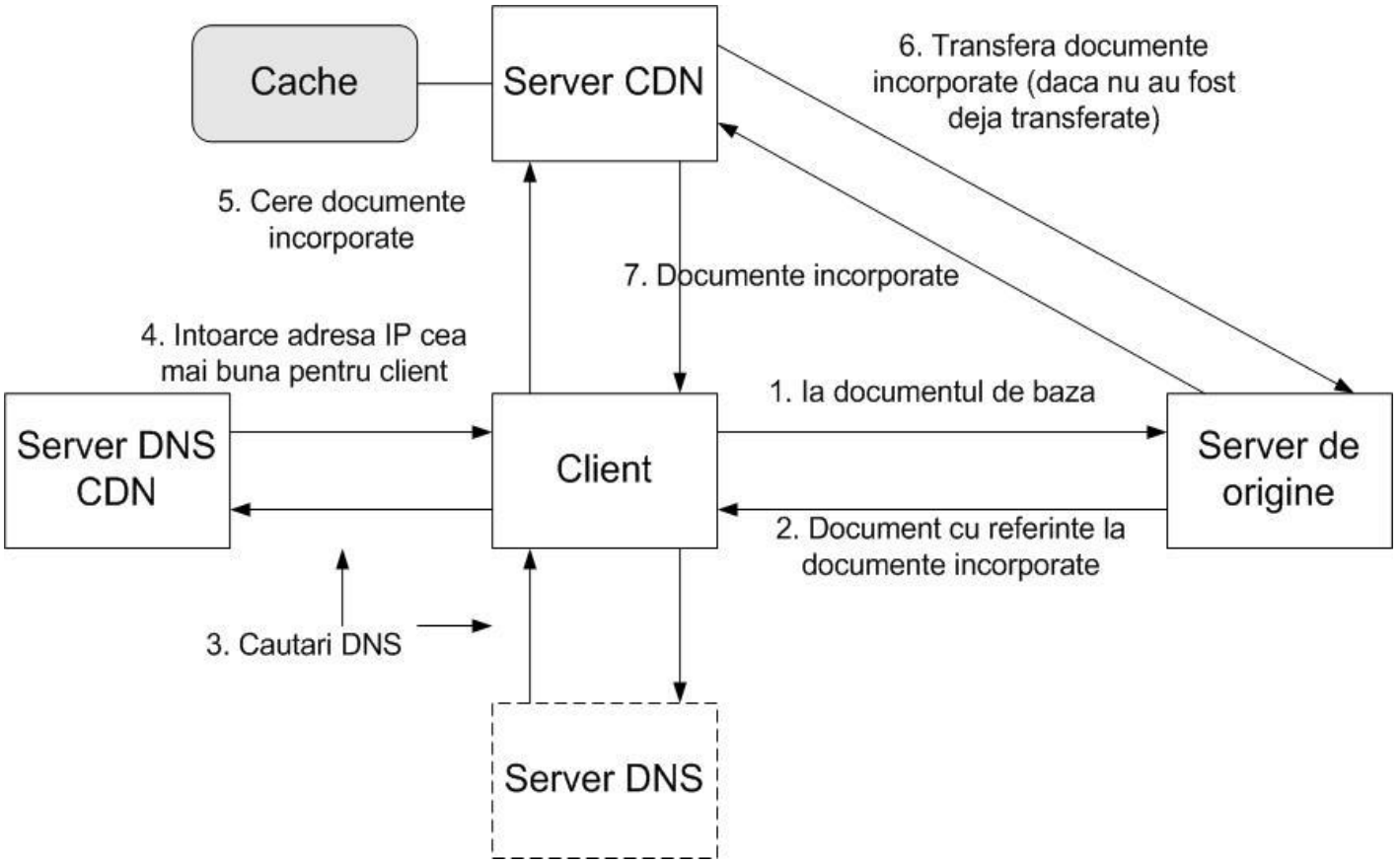
P
<1, 1, P>
<2, 2, A>
<3, 3, A>
<4, 1, B>
<5, 4, P>
<6, 5, B>
<7, 8, P>
<inf, 9, B>
<inf, 10, A>
[8, 10, 9]

Replicare – CDN – Content Delivery Network

- Un document Web poate consta dintr-o pagina care include referinte la alte documente (imagini, video etc.)
- Pentru afisarea intregului document, browser-ul trebuie sa descarce documentele incluse
- Daca aceste documente se schimba rar in timp, are sens replicarea lor
- O solutie: servere Akamai CDN
  - foloseste peste 12 000 servere CDN
- Functionare
  - fiecare document inclus este referit, de regula, printr-un URL
  - in Akamai se foloseste un URL modificat → virtual ghost care
    - este o referinta la un server in CDN
    - contine, in plus, numele DNS al serverului de origine

Replicare – CDN – Content Delivery Network

pas 4 - DNS CDN returneaza adresa unui server CDN care:
- este apropiat geografic de client
- are o «incarcare» redusa (nu este aglomerat)

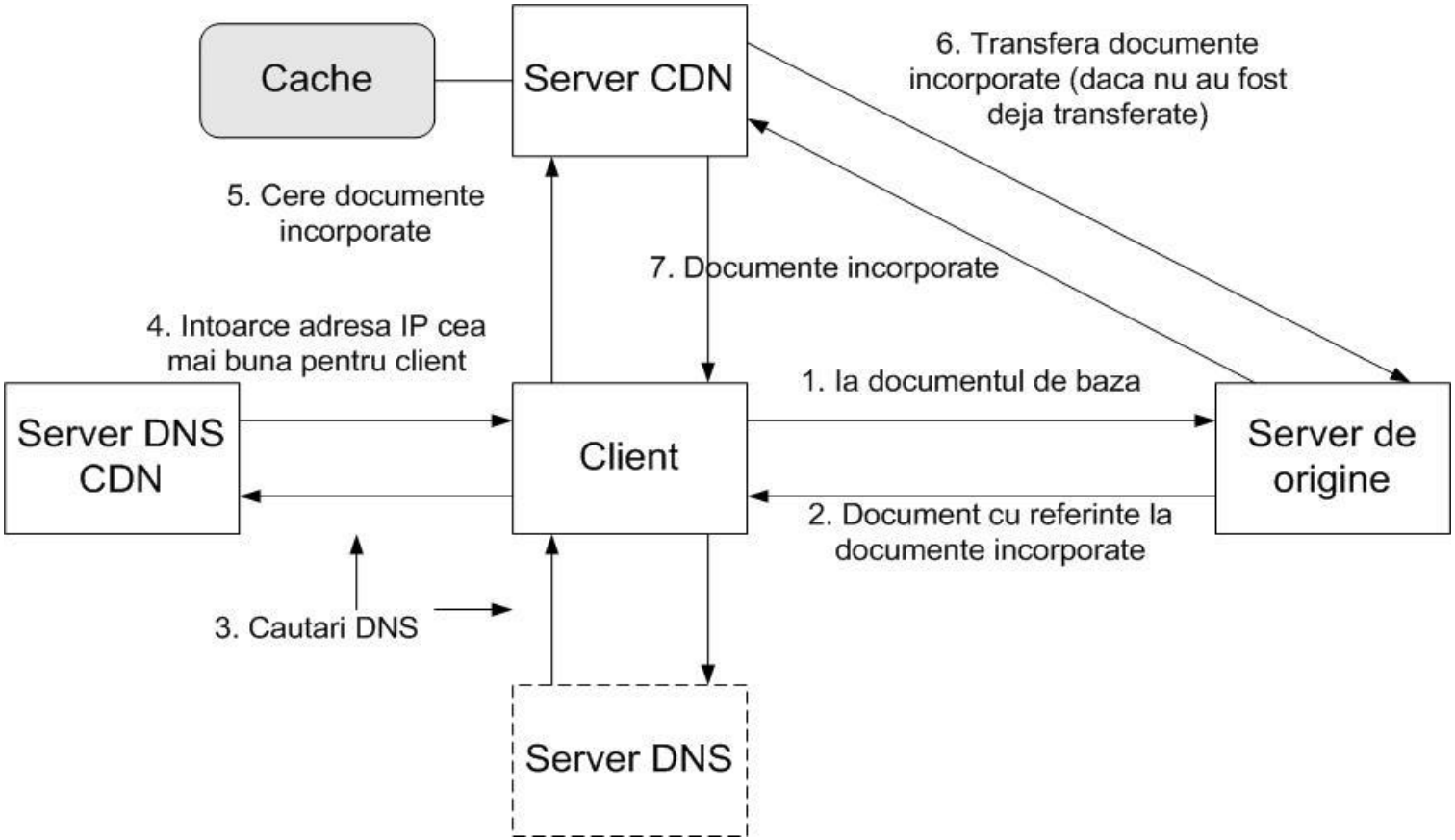


pas 3 – numele DNS este rezolvat de un server DNS la un server DNS CDN

pas 1 – cere o pagina
pas 2 - Referinta virtual ghost include un nume DNS (ex. ghosting.com)

Solutia Akamai

- pas 5 - clientul cere documentul
- pas 6 - daca serverul CDN nu are inca resursele, le descarca folosind adresa serverului de origine
- pas 7 - returneaza documentul cerut



Solutia Akamai

Rezolvarea legaturii la CDN apropiat

- se cauta dupa adresa IP a clientului intr-o baza de date care mentine o harta a Internet-ului
- alternativa: se da aceeasi adresa IP mai multor servere CDN, urmand ca cel mai apropiat sa fie gasit prin politica de rutare "calea cea mai scurta »

Consistenta

- identificatorul documentului incorporat se schimba odata cu continutul
- noul document nu este gasit in serverul CDN si este descarcat de la serverul de origine

Bibliografie

- A.S. Tanenbaum, M. van Steen (2007). Distributed Systems. Principles and paradigms, Ed.2, Prentice Hall
- L. Shklar, R. Rosen (2003) Web Application Architecture. John Wiley & Sons
- Fielding, R. (2000). Architectural styles and the design of network-based software architectures. Ph.D. thesis, University of California, 2000, Retrieved April 12, 2009 from http://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm.
- Karin Petersen, Mike J. Spreitzer, Douglas B. Terry, Marvin M. Theimer and Alan J. Demers (1997) Flexible Update Propagation for Weakly Consistent Replication, Computer Science Laboratory, Xerox Palo Alto Research Center, Palo Alto, California 94304 U.S.A.
- Ion Stoica, CS 268 Classic Distributed Systems: Bayou and BFT, Retrieved October 2015 from http://www.cs.berkeley.edu/~istoica/classes/cs268/06/notes/20-BFTx2.pdf



Toleranţa la defectări

Concepte generale
Comunicarea fiabila client-server
Multicast fiabil si atomic
Tranzactii fiabile
Recuperarea

Bazat pe "Sisteme distribuite" de A.S. Tanenbaum

Terminologie

- O ***specificatie*** descrie comportarea ideala a unui *sistem* la *interfetele* sale.
- **Defect, Greseala** (Fault): este **cauza** unei erori
- **Eroare** (Error): stare interna care poate conduce la esec (nu neaparat!); nu este vizibila la interfata
- **Esec, incapacitate** (Failure): **deviere** de la specificatia interfetei
- Dependenta cauzala

  Defect ➔ Eroare ➔ Esec
- **Exemplu**:

  praful determina reducerea turatiei unui ventilator care va produce mai putin aer si va determina supraincalzirea sursei de alimentare; o componenta arde si sursa se opreste **cum se aplica dependenta cauzala pe acest exemplu ???**
- sistem ventilatie:

  praf = cauza; reducere turatie = eroare; mai putin aer = esec
- sursa de alimentare

  esec ventilatie = cauza; supraincalzire = eroare; oprire functionare = esec

Sisteme de incredere

- **Increderea** (Dependability)
  - Un sistem de incredere (dependable system) respecta specificatia chiar in prezenta defectarilor

- **Atribute** ale sistemelor de incredere ce inseamna ???
  - Disponibilitatea (Availability)
    - gata de a fi folosit imediat (la un moment dat)
  - Fiabilitatea (Reliability)
    - cat timp functioneaza continuu fara esec (failure)
  - Siguranta (Safety)
    - temporar poate functiona incorect fara efecte catastrofice
  - Mentenabilitatea (Maintainability)
    - cat de usor poate fi reparat

Tipuri de defecte si esecuri

Tipuri de defecte

tranzitorii – se produc o data si dispar

intermitente – se produc, dispar si reapar

permanente

Tipuri de esecuri	Descriere comportare server
Cadere (Crash)	Un server se opreste dintr-o data
Omisiune *receptie* *transmisie*	Nu raspunde cererii Nu a primit cererea Nu trimite raspunsul (desi serverul l-a pregatit)
Timing	Raspunde dupa trecerea timpului specificat
Raspuns *valoare* *tranzitie*	Raspunde incorect Valoarea din raspuns este gresita Deviaza de la fluxul de control corect
Arbitrar (Bizantin)	Produce raspunsuri arbitrare la momente arbitrare

Tehnici de tratare a defectelor

- **Prevenire**
  - proiectare formala, controlul calitatii
  - injectare defecte si testare.
- **Detectie**
  - se foloseste schimbul regulat de mesaje intre vecini
  - procesele intreaba vecinii despre starea lor (are you alive?)
  - procesele comunica vecinilor starea lor (I'm alive!)
- **Recuperare**
  - checkpoint-restart
  - log-uri de mesaje
- **Prezicere**
  - folosind date de monitorizare
- **Tolerare**
  - furnizarea serviciului chiar daca apar defecte (**mascarea** defectelor)

Mascarea esecurilor

- Replicare => toleranta la defectari
- Protocoale
  - Primary based
    - o replica primara (leader) coordoneaza toate operatiile de scriere
    - foloseste alegere leader pentru a trata un esec al replicii primare
  - Replicated write
    - replicare activa sau
    - cvorumuri

- Sistem **k-fault tolerant**
  - mascheaza k procese defecte

- Problema: cat de mare trebuie sa fie grupul (n)?
- Solutie:
  - n = k+1 esecuri de tip crash
  - n = 3k+1 esecuri bizantine (solutie Lamport)

Esecuri bizantine

Solutia are la baza generalii bizantini

toti locotenentii loiali «primesc » acelasi ordin de la comandant (loial sau neloial)

Pentru **mesaje orale** (nesemnate), mai mult de 2/3 din generali trebuie să fie loiali.

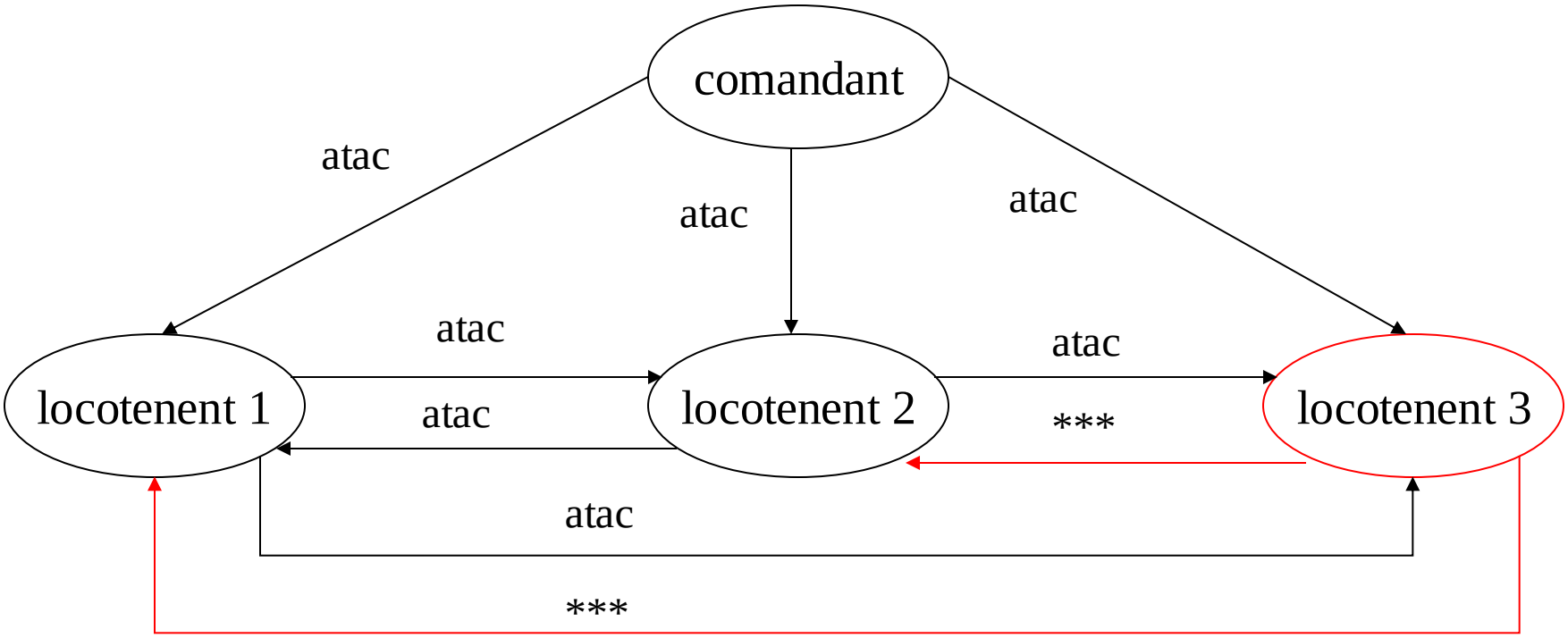
Cu mesaje orale şi un trădător

nu există o soluţie pentru doar trei generali

există soluţie pentru patru generali.

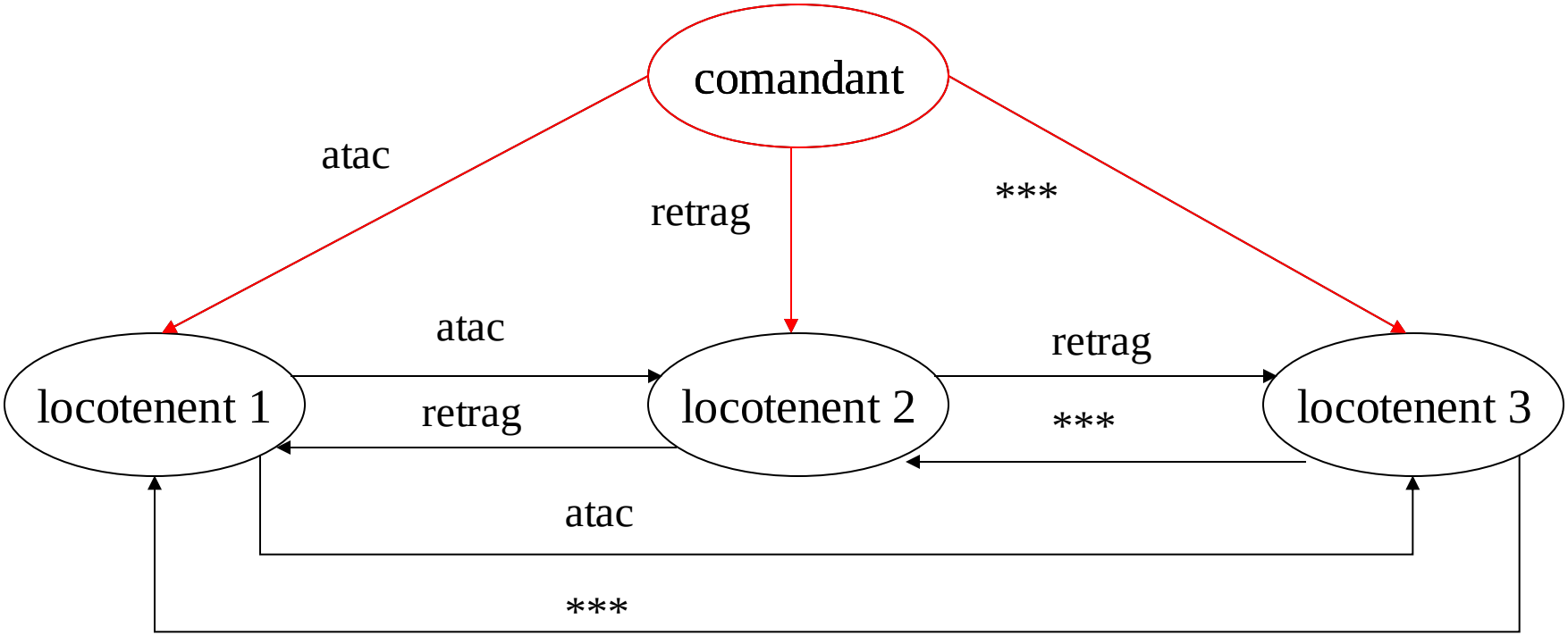
Există soluţie pentru un trădător între 4 generali

Scenariul 1. Comandantul este loial, iar decizia este **atac**.



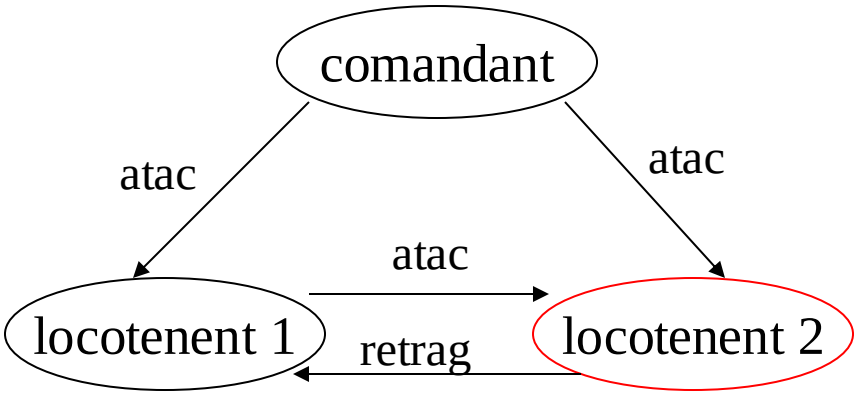
Cei doi locotenenti loiali selecteaza "atac" pe baza principiului majoritatii

Scanariul 2. Comandantul este neloial şi transmite mesaje diferite locotenenţilor.



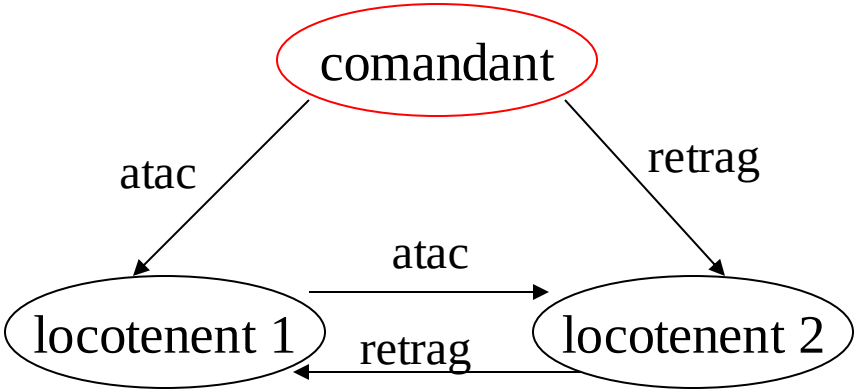
Locotenentii loiali au acelasi set de valori (atac, retrag, ***) si iau aceeasi decizie

Nu există soluţie pentru 1 trădător şi 3 generali



Scenariul 3. Grupul conţine trei generali. Alg. locotenenti loiali iau decizie determinista, de ex. dau prioritate comandantului

a) locotenentul 2 este neloial → generalii loiali iau aceeasi decizie



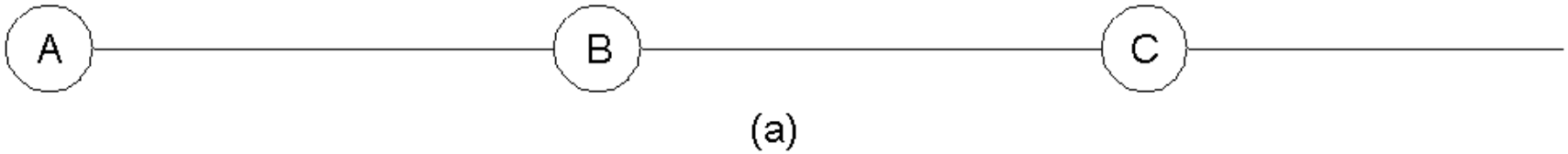
b) comandantul este neloial → generalii loiali iau decizii diferite

Se poate generaliza la n generali (demo constructivă)

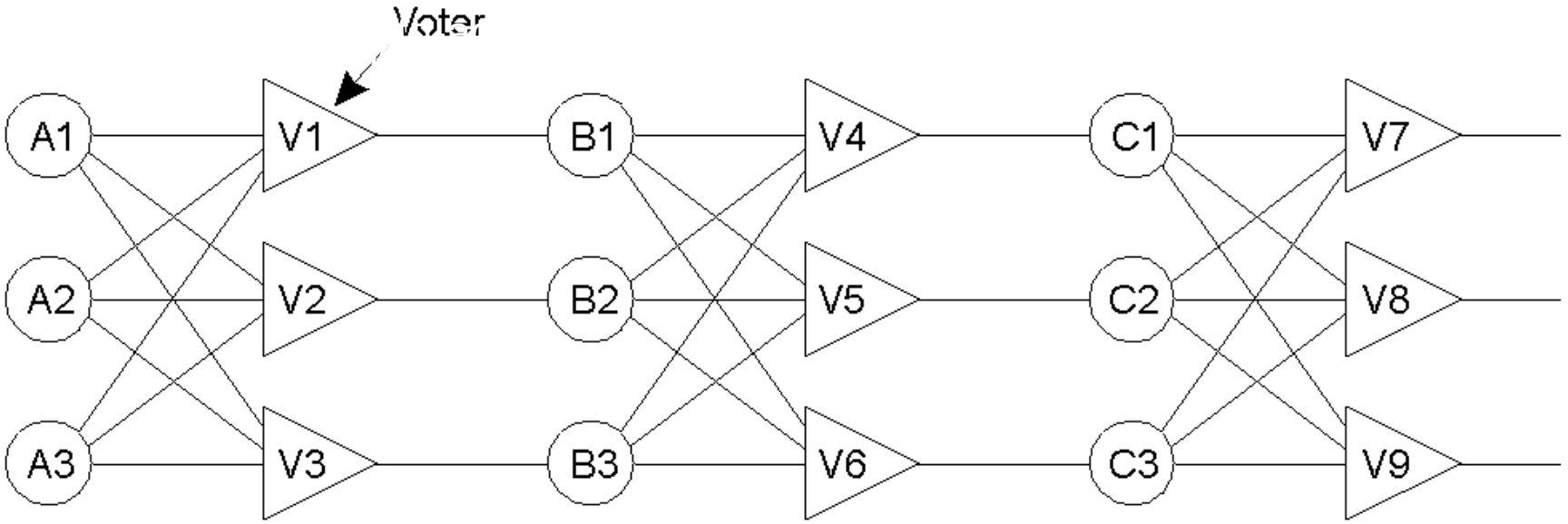
Mascarea esecurilor prin redundanta

Redundanta Modulara Tripla.

- fiecare echipament este triplat
- dupa fiecare tripleta este inclus un ansamblu de echipamente de votare



(a)

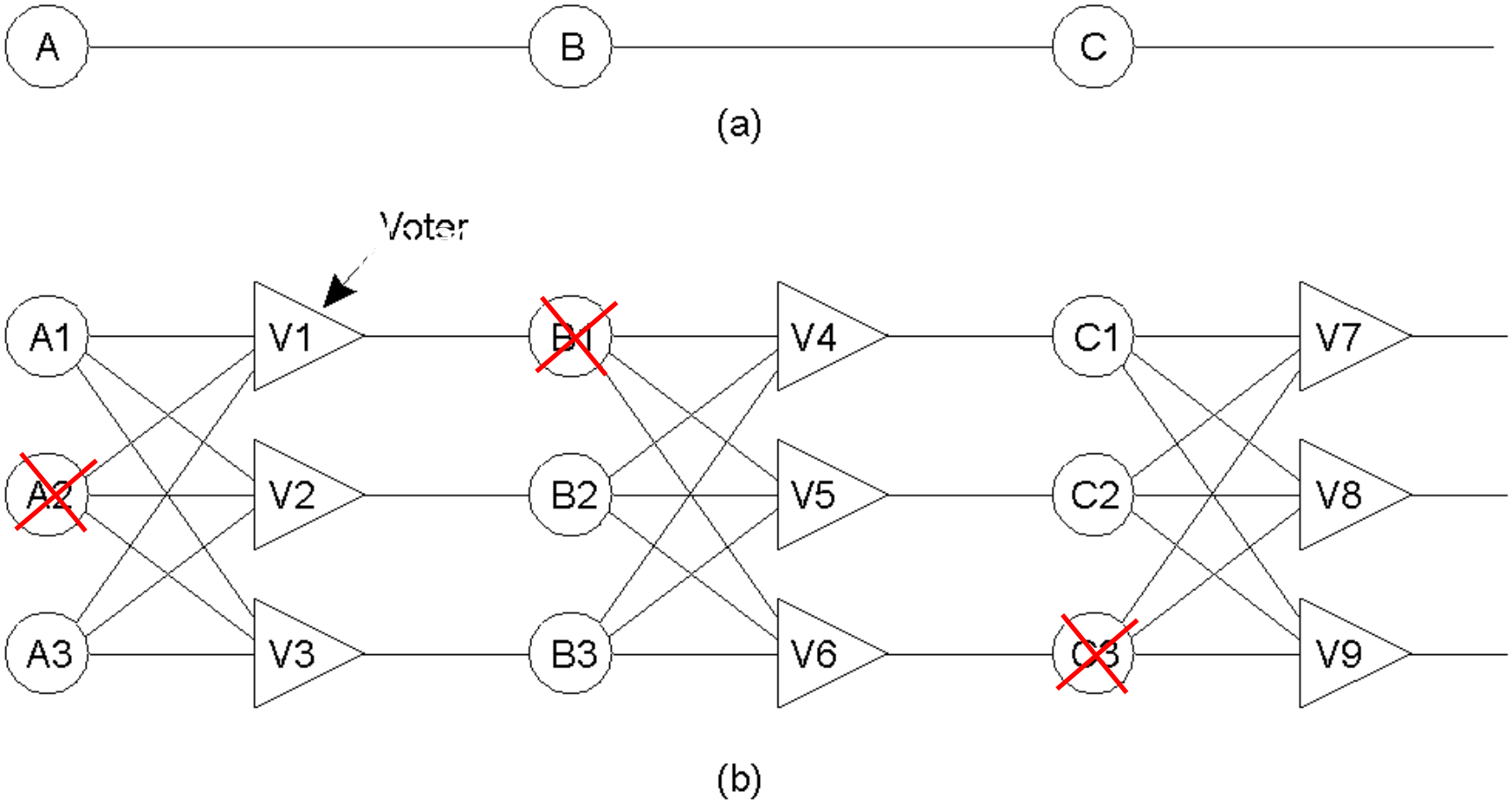


(b)

Mascarea esecurilor prin redundanta

Redundanta Modulara Tripla.

- suporta defectarea unei componente din trei, la fiecare nivel



Fiabilitatea apelurilor de proceduri la distanta

Solutii in caz de eroare			Semantica invocarii
retransmite *request*	Filtrare duplicate	Re-executa procedura sau Retransmite *reply*	
Nu	Ne-aplicabil	Ne-aplicabil	← Maybe
Da	Nu	Re-executa procedura	← At least once
Da	Da	Retransmite *reply*	← At most once

- Maybe – clientul nu poate spune daca executia s-a facut sau nu
- At least once – rezultate eronate la re-executia procedurii
  - daca operatiile nu sunt idempotente
- At most once – clientul primeste rezultat sau eroare

Semantici RPC in prezenta defectelor

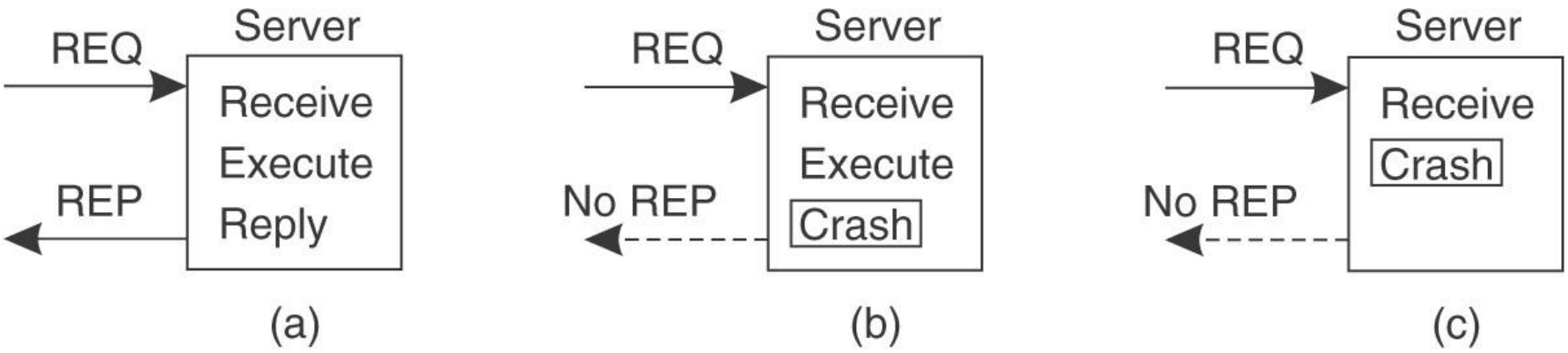
- **Scopul RPC**
  - ascunderea localizarii: apelul la distanta arata ca unul local
  - problema – aparitia defectelor

- Clientul nu poate localiza serverul
  - Server defect sau o versiune noua, necunoscuta de client
  - Solutii
    - Provoaca exceptie (Java) sau foloseste handler de semnalizare (in C)
  - Neajunsuri
    - nu orice limbaj are exceptii sau handler de semnalizare
    - distruge transparenta

Semantici RPC in prezenta defectelor

- Mesaj de cerere pierdut
  - Solutie
    - Timer in OS client
    - Retransmite cererea
    - La mai multe timeout-uri – mesaj de eroare catre client
  - Neajuns
    - Merge daca mesajul a fost cu adevarat pierdut
    - Altfel, serverul trebuie sa faca diferenta intre original si copie

Caderea Serverului



a – normal
b – crash dupa executie
c – crash inainte de executie

cand apare un timeout, clientul nu poate diferentia intre b si c
solutii pentru client ???

- repeta cererea pana reuseste (at least once)
- semnaleaza eroarea (at most once)
- nimic (maybe)
- solutie teoretica? (exactly once)

Exactly once – Strategii server

- Aplicatia - Client-server pentru tiparire text
- Strategii server
  - trimite Mesaj ACK (eveniment M) inainte de a spune Printer sa tipareasca (eveniment P); ordinea este M → P
  - trimite mesaj ACK dupa tiparire text; ordinea este P→ M

- Combinatii posibile (C este eveniment Crash datorita caruia operatiile intre paranteze nu se mai pot executa):

Strategia M → P	Strategia P→ M
M P C	P M C
M C (P)	P C (M)
C (M P)	C (P M)

Exactly once - Strategii client

- *Server anunta ca a cazut si acum este iar operational*
- Strategie client de retransmitere cerere
  - mereu
  - niciodata
  - doar cand nu s-a primit mesaj confirmare cerere de la server
  - doar cand s-a primit mesaj confirmare cerere de tiparire

Combinatii (2)

Client	Server					
	Strategy M → P			Strategy P → M		
Reissue strategy	MPC	MC(P)	C(MP)	PMC	PC(M)	C(PM)
Always	DUP	OK	OK	DUP	DUP	OK
Never	OK	ZERO	ZERO	OK	OK	ZERO
Only when ACKed	DUP	OK	ZERO	DUP	OK	ZERO
Only when not ACKed	OK	ZERO	OK	OK	DUP	OK

OK = Text is printed once
DUP = Text is printed twice
ZERO = Text is not printed at all

pentru nici o perche de strategii client/server nu avem 3 OK-uri
concluzia: nici o combinatie de strategii client si server corecta

Pierdere mesaje reply

Solutie

- timer
- retransmitere cerere

Dezavantaje

- merge la operatii idempotente
- pentru celelalte
  - clientii asociaza cererilor numere de secventa si serverul tine evidenta lor
  - serverul trebuie sa tina evidenta raspunsurilor date clientilor
  - in plus, un bit in mesaj poate distinge cererea originala de copii
    - serverul trateaza imediat originalul (originalul este mai sigur)
    - retransmisia cere mai multa atentie

Cadere Client

- Clientul face o cerere si cade inainte de primirea raspunsului
  → Calcule orfane – produc rezultate pe care nu le asteapta nimeni
- Probleme
  - consum inutil resurse
  - fisiere blocate (lock)
  - sincronizare cu client re-boot-at
- Solutii
  - exterminare
    - fiecare invocare se memoreaza mai intai intr-un log
    - la re-boot-area clientului, orfanii sunt exterminati pe baza informatiilor din log
  - probleme:
    - inregistrarea fiecarui apel este costisitoare
    - descendentii orfanilor (grandorfans) – pot fi greu sau imposibil de localizat
    - retea partitionata – nu se poate face exterminarea

- înnoire (epoci)
  - timpul este impartit in epoci succesive
  - la rebootare, un client difuzeaza tuturor masinilor un mesaj de epoca noua
  - la receptia lui, toate calculele din epoci anterioare sunt distruse
  - raspunsurile din epoci anterioare care totusi sosesc sunt distruse la client
- înnoire lina (localizare proprietar)
  - la receptia mesajului de epoca noua, o masina incearca identificarea proprietarului pentru calculele facute pentru client
  - exterminarea se face daca proprietarul nu este gasit
- expirare
  - fiecare RPC are alocata o cuanta de timp T standard
  - daca nu poate termina, cere o alta cuanta
  - la repornire dupa crash, clientul asteapta un timp T (pentru distrugerea orfanilor)

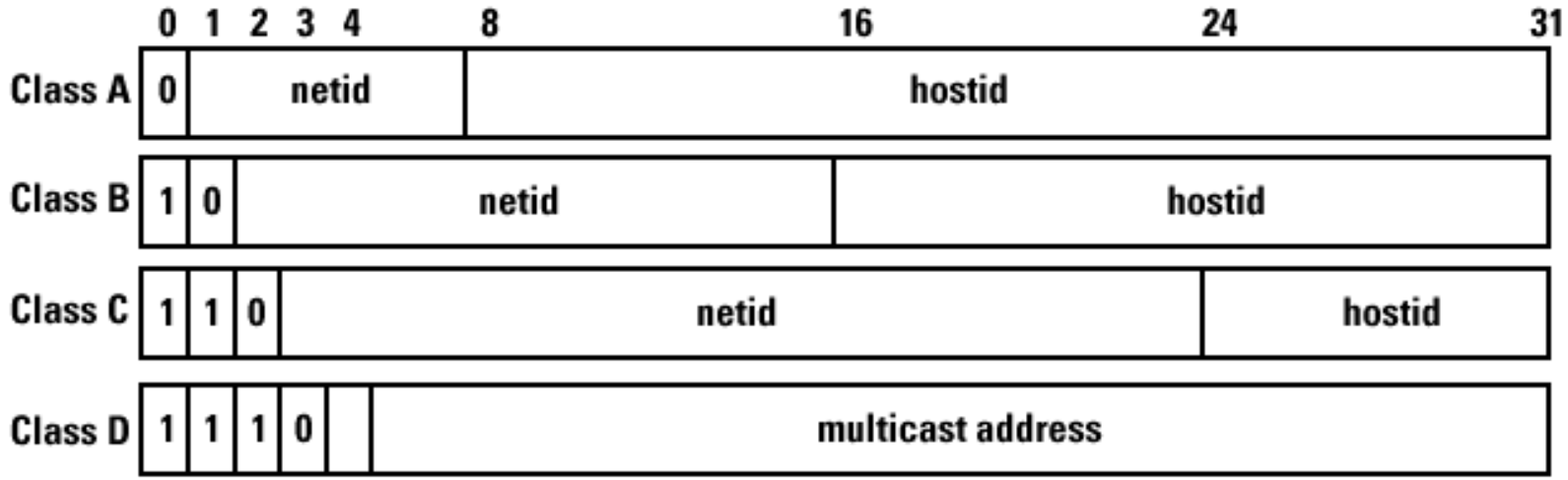
Comunicare de grup fiabila

- Multicast fiabil: un mesaj trimis unui grup de procese trebuie livrat fiecarui membru al grupului
- Simplificare
  - **grupul este stabil** (procesele nu «cad» (nu crash) si nu se alatura grupului in timpul comunicarii)
  - consideram doar **defecte de comunicare**
- Exemple de utilizare
  - Tranzactii fiabile si ordonate - **Uniform Reliable Group Communication Protocol** (URGC)
  - Multicast articole noi pe Mbone – **Muse**
  - Transmitere fisiere - **Multicast File Transfer Protocol** (MFTP)
  - Evidenta pachetelor transmise intr-un server de logging – **Log-Based Receiver-reliable Multicast** (LBRM)

Multicast IP

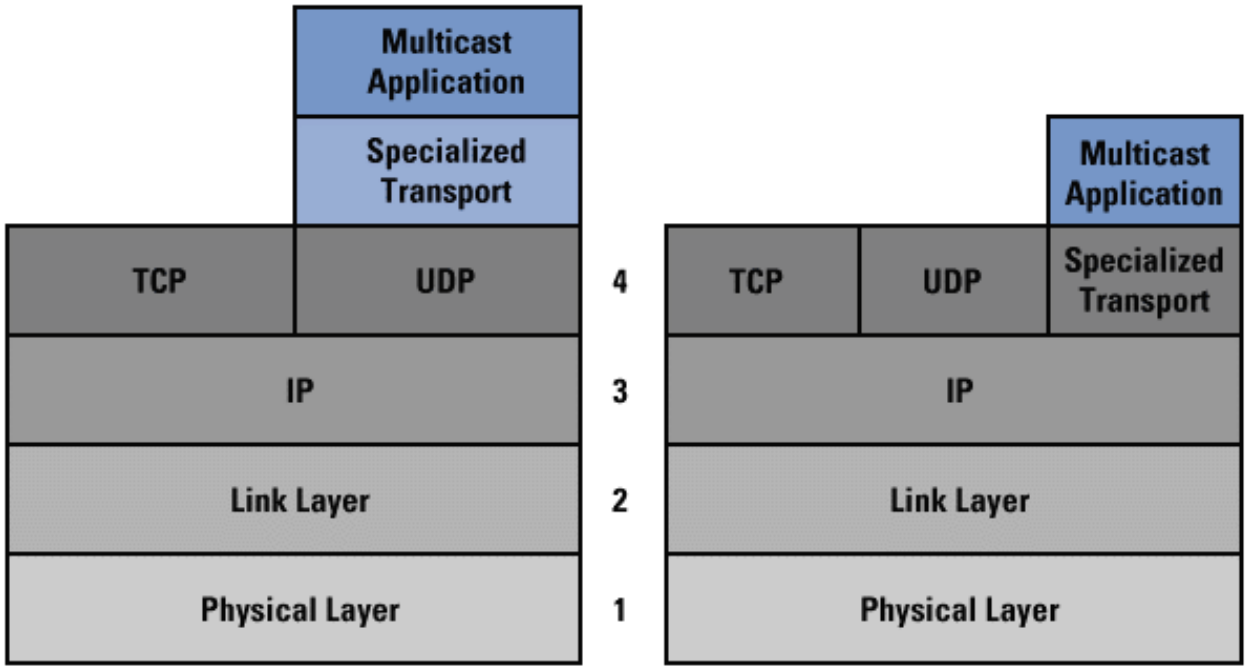
Se bazeaza pe

- Adrese de clasa D pentru transmitere pachete
- Internet Group Management Protocol (IGMP) pentru gestiunea grupurilor
- Rutere multicast
  - **transmit pachete multicast membrilor (best effort)**
  - **un nou host notifica ruterul la intrarea in grup**
  - **ruterul verifica periodic host-urile ramase in grup**



Transport - Multicast peste UDP sau IP

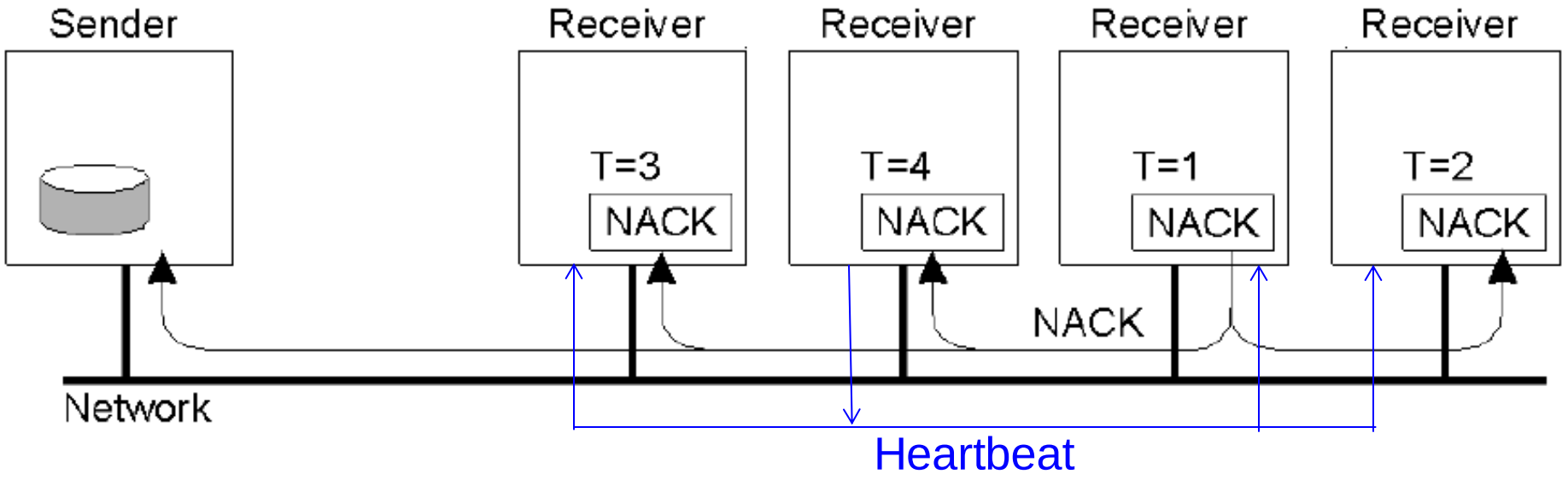
- Solutia TCP (corectia la transmitator) nu merge:
  - Multe ACK la transmitator - gatuire
  - Evidenta greoaie a setului de receptori
  - Conditii diferite la receptori diferiti (round-trip time, delay*bandwidth, legaturi supraincarcate diferit)
- Alta solutie: un Transport specializat peste UDP sau IP



Scalable Reliable Multicast - SRM

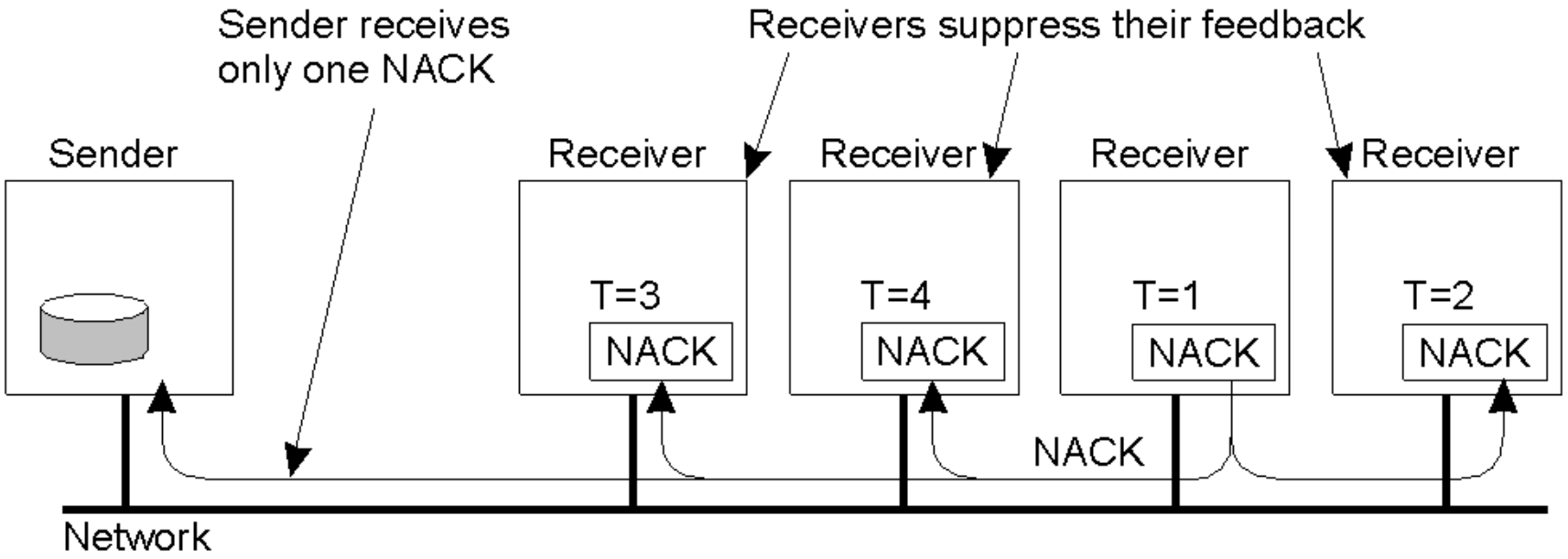
- Bazat pe IP multicast (ne-fiabil) la care:
  - Sursa transmite pe o adresa multicast fara sa cunoasca membrii grupului
  - Un receptor intra in grup sau iese din grup fara a afecta ceilalti membri
- Adauga caracteristici TCP
  - Fiabilitate capat la capat
  - Adaptare la conditiile retelei (dimensiune, trafic, topologie)
- Ideea - corectia la receptor
  - Fiecare membru al grupului raspunde de receptia corecta a mesajelor care ii sunt adresate

SRM – controlul cererilor



- Mesajele transmise de Sender poarta numere de secventa
- Se mai folosesc mesaje:
  - *Heartbeat* (intre receptori) – informeaza despre numerele de secventa ale mesajelor primite
  - *NACK* (spre Sender) – cerere retransmitere
  - *Repair* (de la Sender) – retransmitere din cache

Optimizari



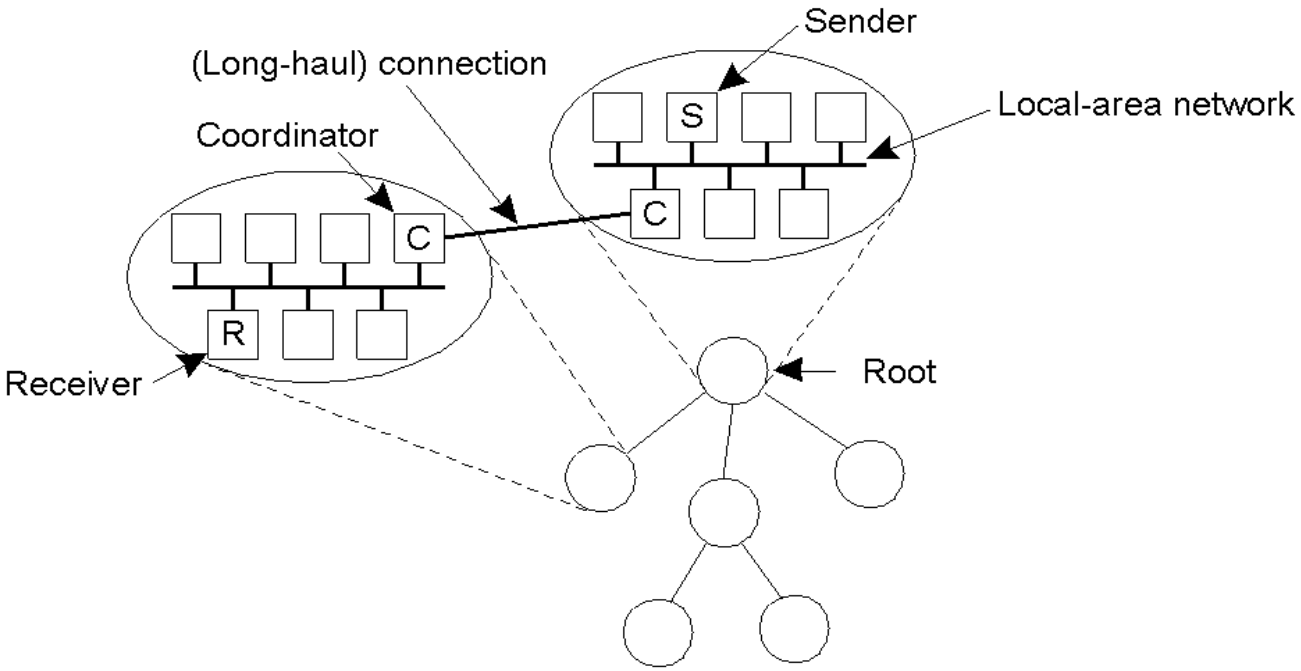
Minimizare numar *NAK – un singur NAK ajunge la transmitator*

- Transmitere multicast NAK cu intarzieri aleatoare (diferite de la un receptor la altul)
- Procesele care primesc un NAK si nu au transmis anuleaza propriul NAK
- Problema: planificarea intarzierilor pentru a asigura un singur NAK

Optimizari

- Evitare consum resurse pentru mesaje *Repair*
  - Retransmitere punct la punct doar pentru procese care nu au mesajul
  - Retransmitere multicast pe un alt grup, doar pentru procesele care au pierdut acelasi mesaj m
    - procesele care nu au primit m se alatura acestui grup
    - evita retransmiterea catre procese care au primit corect mesajul
    - dezavantaj - solutia cere un management foarte eficient al grupurilor, ceeace este greu de realizat
- Accelerare proces – recuperare locala
  - Retransmitere facuta de un membru "local"

Multicast fiabil ierarhic



Grupul este impartit in subgrupuri cu **coordonatori locali** care

- primesc mesaj de la Sender si retransmit mesajele catre receptori.
- primesc cererile de retransmitere de la receptori si transmit un singur mesaj la Sender

Problema: constructia dinamica a arborelui

- utilizare arbori multicast din reteaua suport
- solutie la nivelul aplicatiei (overlay network)

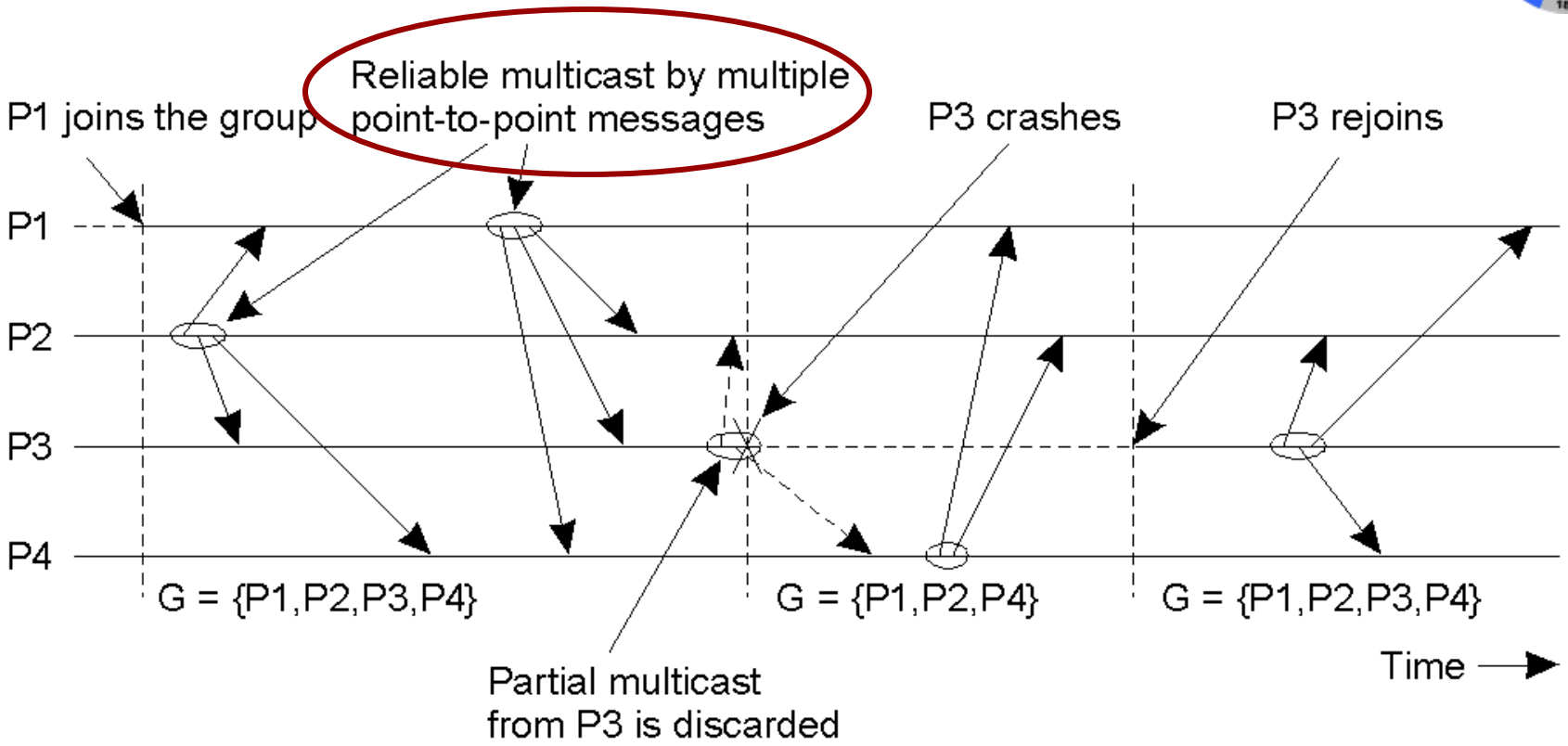
Multicast Atomic

Multicast fiabil chiar daca **unele procese se defecteaza**

Problema multicast atomic

- un mesaj este:
  - fie livrat tuturor proceselor din grup
  - fie nelivrat
    - acceptabil cand transmitatorul cade
    - similar cu caderea transmitatorului inaintea transmiterii mesajului
  - in plus, mesajele sunt livrate in **aceeasi ordine** tuturor proceselor
- **Solutia** tine cont de instabilitatea grupului, pentru care:
  - foloseste setul de procese din grup "vazut" de transmitator cand a trimis mesajul - **group view**

Virtual Synchrony



Toate procesele din set au acelasi view

Un group view se poate schimba

Toate operatiile multicast **au loc intre schimbarile de view.**

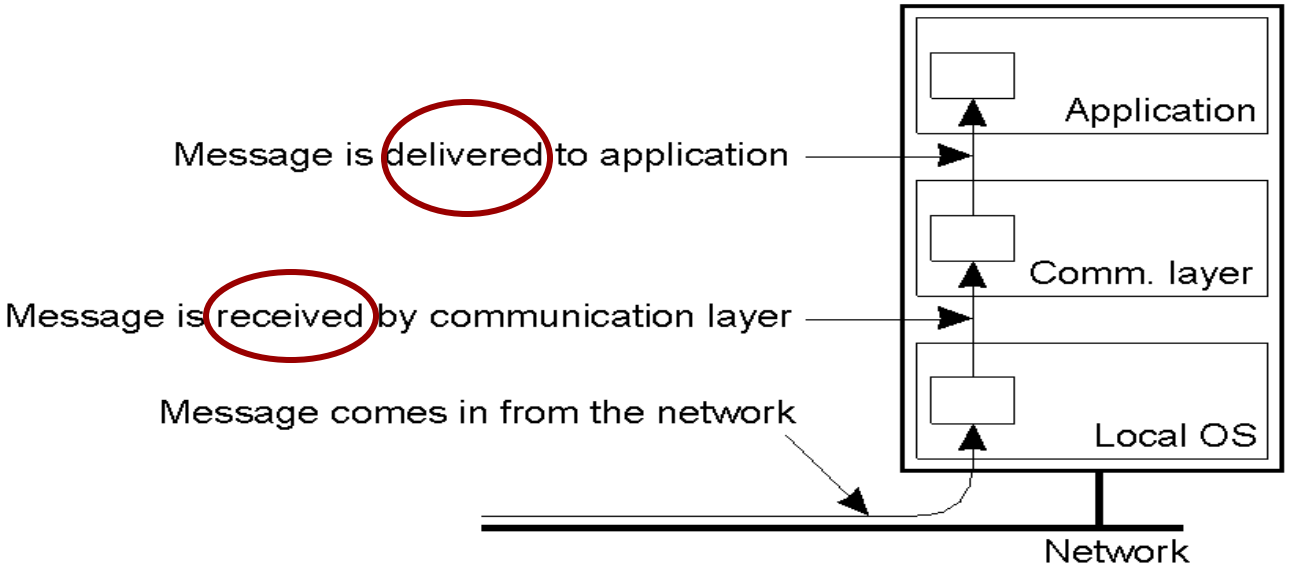
Principiul Virtual Synchrony

- Anunt schimbare group view (**G**)
  - se face prin transmiterea multicast a unui mesaj **vc – view change**

- Principiul **multicast-ului sincron virtual**
  - Daca m este transmis proceselor din G si, inainte de terminare, se transmite **vc** atunci
    - toate procesele din G primesc **m** inainte de **vc**,
    - sau nici un proces nu primeste **m**
  - Mesajul **m** transmis lui G nu poate fi livrat **decat** proceselor din G.

Implementare Virtual Synchrony (1)

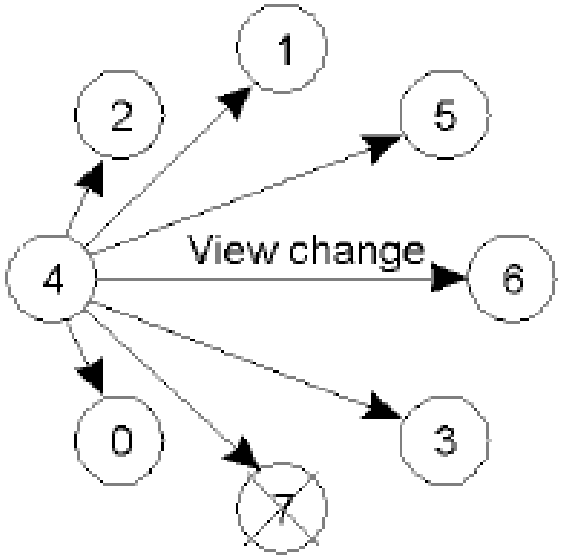
- Utilizeaza **comunicarea fiabila punct-la-punct** (TCP)
  - multicast = trimitere secventiala mesaj **m** fiecarui membru din grup
- Mesajele sunt **receptionate** de **nivelul comunicare** in ordinea primirii de la Local OS
- si sunt **livrate** aplicatiei intr-o ordine diferita (FIFO, cauzala), aceeasi pentru toate procesele



Implementare Virtual Synchrony (2)

- Obiectivul adoptat la implementare
  - un mesaj trimis unui group view G trebuie livrat tuturor proceselor valide din G inainte de urmatoarea schimbare de view
- Probleme
  - **vc** apare **inainte** ca mesajul **m** sa fie livrat tuturor proceselor din G
  - daca **transmitatorul cade**, cum iau mesajul procesele care nu l-au primit?
- Solutia foloseste doua categorii de mesaje:
  - mesaj **stabil** (a fost **primit** de toate procesele din grup): poate fi **livrat**
  - mesaj **instabil**: pastrat de procesele care l-au primit;
    - cand transmitatorul cade - unul din procese il va trimite celorlalte

Implementare Virtual Synchrony (3)

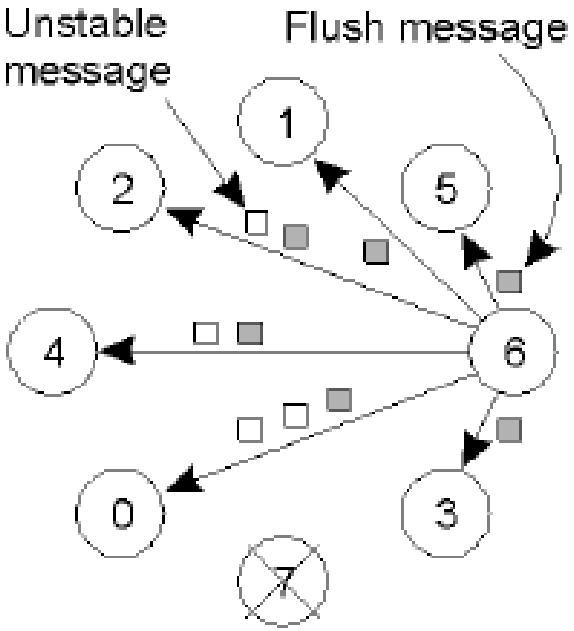


Un proces initiaza un view change cand

- primeste un mesaj provenind de la un alt proces care vrea sa se alature grupului,
- sa paraseasca grupul
- sau detecteaza defectarea unui proces in grupul curent

In figura, procesul 4 observa ca 7 a cazut si trimite un mesaj view change celorlalte procese, urmand ca 7 sa fie eliminat din group view

Implementare Virtual Synchrony (4)



Inainte de trecerea la noul group view G_{i+1}, mesajele instabile din grupul curent G_i trebuie facute stabile (adica transmise tuturor proceselor valide din G_i)

Ex, procesul 6 (care a primit view change) trimite proceselor valide toate mesajele instabile si le schimba starea in stabile

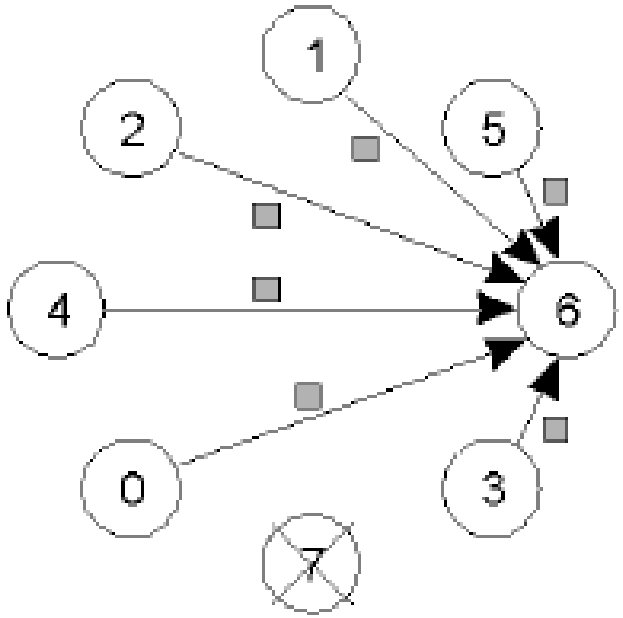
Apoi, procesul trimite un mesaj flush (nu mai are mesaje instabile)

Celelalte procese procedeaza la fel

Se asigura ca fiecare mesaj care a fost primit de cel putin un proces din G_i va fi primit de oricare alt proces valid din G_i

- alternativa – doar procesul 6 (coordonator) difuzeaza mesajele

Implementare Virtual Synchrony (5)



Procesul 6 **instaleaza** noul **view** cand primeste mesaje flush de la toate celelalte procese

Fiecare proces valid procedeaza ca 6

Ordinea livrarii mesajelor multicast

a) nici una

b) **FIFO** – mesajele de la un **acelasi proces** sunt livrate in ordinea transmiterii

Process P1	Process P2	Process P3	Process P4
sends m1	receives m1	receives m3	sends m3
sends m2	receives m3	receives m1	sends m4
	receives m2	receives m2	
	receives m4	receives m4	

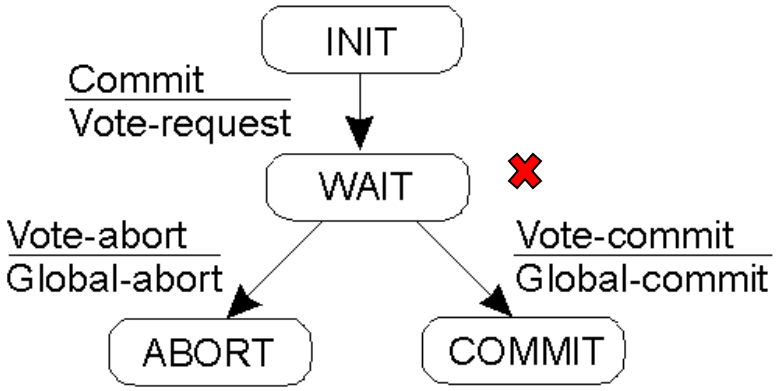
c) **cauzala** – daca **m1** precede cauzal **m3**, **m1** este primit inaintea lui **m3**

Process P1	Process P2	Process P3	Process P4
sends m1	receives m1	receives m1	sends m3
sends m2	receives m3	receives m3	sends m4
	receives m2	receives m4	
	receives m4	receives m2	

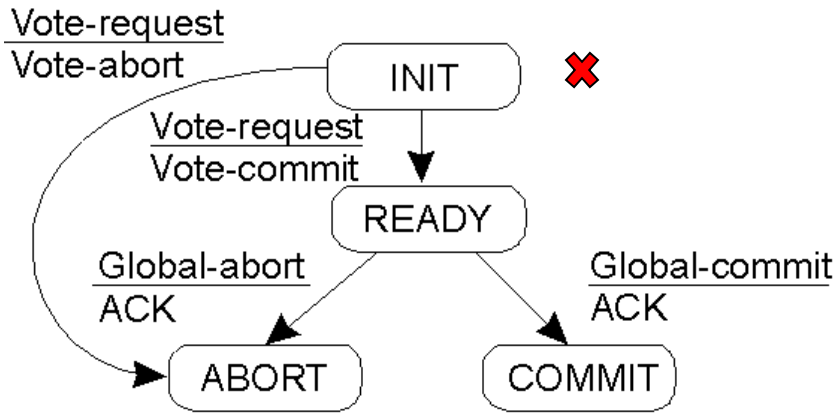
Distributed commit

- Definitie: data fiind o operatie distribuita proceselor unui grup, se asigura ca
  - sau fiecare proces din grup executa operatia
  - sau nici un proces nu o executa
- Solutie: folosirea unui coordonator
- Trei variante
  - one-phase commit – nu poate trata caderea unui proces
  - two-phase commit – nu poate trata caderea coordonatorului
  - three-phase commit

Two-Phase Commit



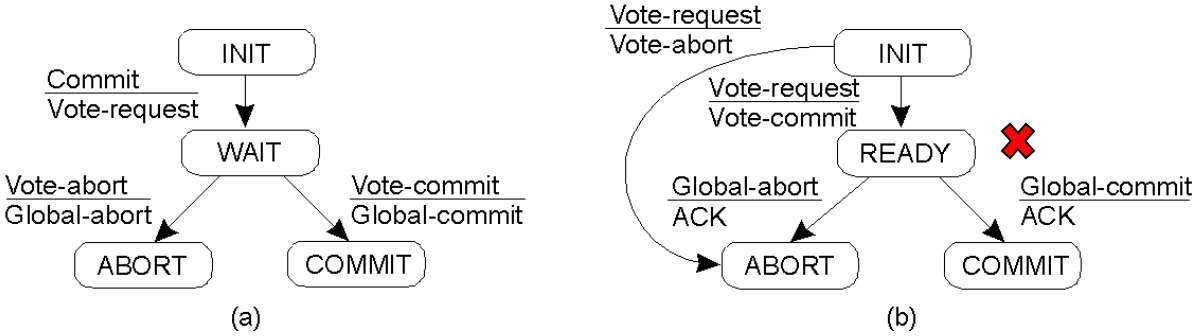
Masina de stari ptr **coordonator**



Masina de stari ptr un **participant**

- Coordonator este procesul care initiaza operatia
- Exista stari in care coord (WAIT) sau participant (INIT, READY) asteapta
- Pentru evitarea blocarilor se folosesc timeout-uri
- Comportare participant la timeout in INIT ???
  - trimite VOTE_ABORT
- si coordonator la timeout in WAIT ???
  - trimite GLOBAL_ABORT

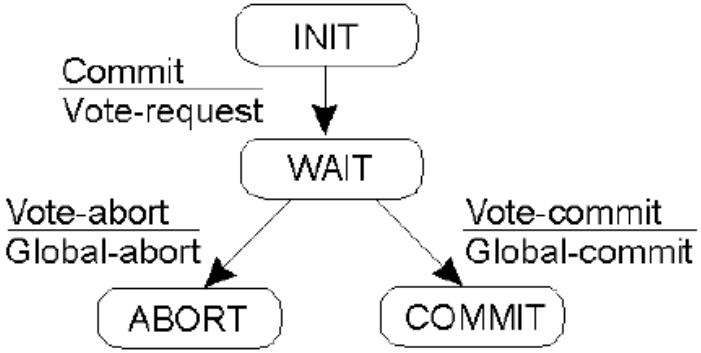
Comportare participant la timeout in READY



Participant *P* in starea *READY* contacteaza, la timeout, un alt participant *Q*.

Starea lui Q	**Actiunile lui P**	**Justificare**
COMMIT	Trece in COMMIT	P a pierdut GLOBAL_COMMIT
ABORT	Trece in ABORT	P a pierdut GLOBAL_ABORT
INIT	Trece in ABORT	Coord a crapat inainte de trimitere VOTE_REQUEST tuturor participantilor. La timeout, Q va trece si el in ABORT
READY	Contacteaza alt participant	Daca toti READY atunci asteapta recuperare coordonator

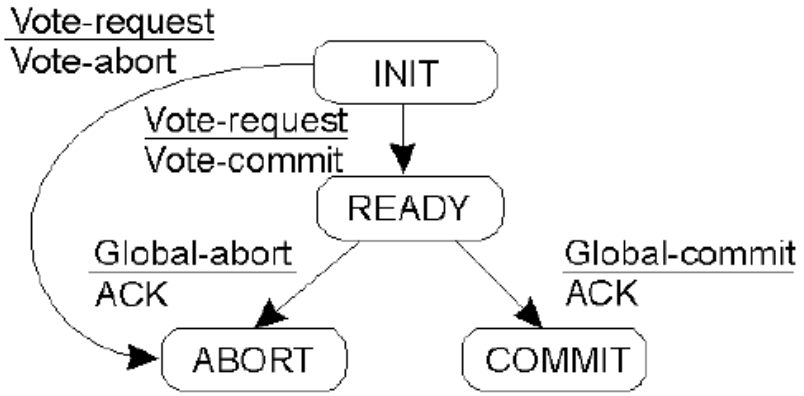
Logare stare la Coordonator




```
write START_2PC to local log;
multicast VOTE_REQUEST to all participants;
while not all votes have been collected {
    wait for any incoming vote;
    if timeout {
        write GLOBAL_ABORT to local log;
        multicast  GLOBAL_ABORT to all participants;
        exit;}
    record vote;                    /*inregistreaza un vot*/
}
if all participants sent VOTE_COMMIT and coordinator votes COMMIT{
    write GLOBAL_COMMIT to local log;
    multicast GLOBAL_COMMIT to all participants;}
else {
    write GLOBAL_ABORT  to local log;
    multicast GLOBAL_ABORT to all participants;
}
```

Logare stare la participant

```
write INIT to local log;
wait for VOTE_REQUEST from coordinator;
if timeout {
    write VOTE_ABORT to local log;
    exit;
}
if participant votes COMMIT {
    write VOTE_COMMIT to local log;
    send VOTE_COMMIT to coordinator;
    wait for DECISION from coordinator;
    if timeout {
        multicast DECISION_REQUEST to other participants;
        wait until DECISION is received; /* remain blocked */
        write DECISION to local log;
    }
    if DECISION == GLOBAL_COMMIT
        write GLOBAL_COMMIT to local log;
    else if DECISION == GLOBAL_ABORT
        write GLOBAL_ABORT to local log;
} else {
    write VOTE_ABORT to local log;
    send  VOTE_ABORT to coordinator;
}
```





Participant: tratarea cererilor de la alti participanti.

actions for handling decision requests: /* executed by separate thread */

```
while true {
   wait until any incoming DECISION_REQUEST is received; /* remain blocked */
   read most recently recorded STATE from the local log;
   if STATE == GLOBAL_COMMIT
      send GLOBAL_COMMIT to requesting participant;
   else if STATE == INIT or STATE == GLOBAL_ABORT
      send GLOBAL_ABORT to requesting participant;
   else       /* STATE == READY */
      skip;  /* participant remains blocked */

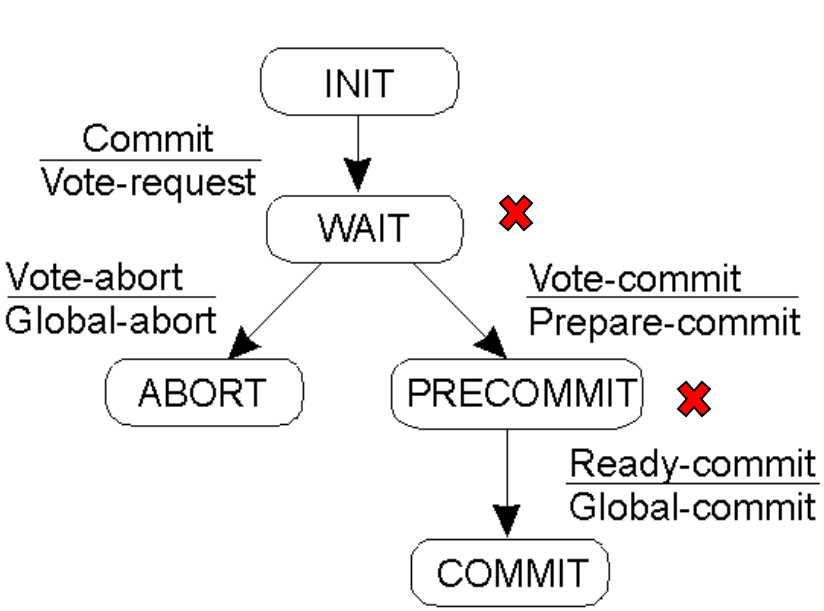
}
```

Participantii nu pot decide cand:

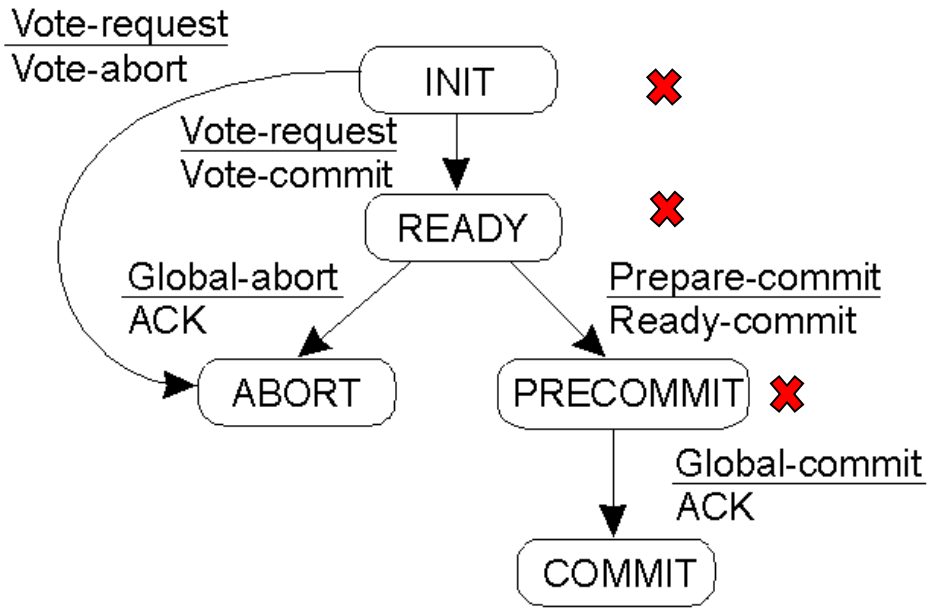
- toti participantii au primit si procesat VOTE_REQUEST
  deci STATE == READY
- intre timp coordonatorul a crapat si nu trimite decizia

Corespunde alternativei **skip** din actiunile participantilor

Three-Phase Commit

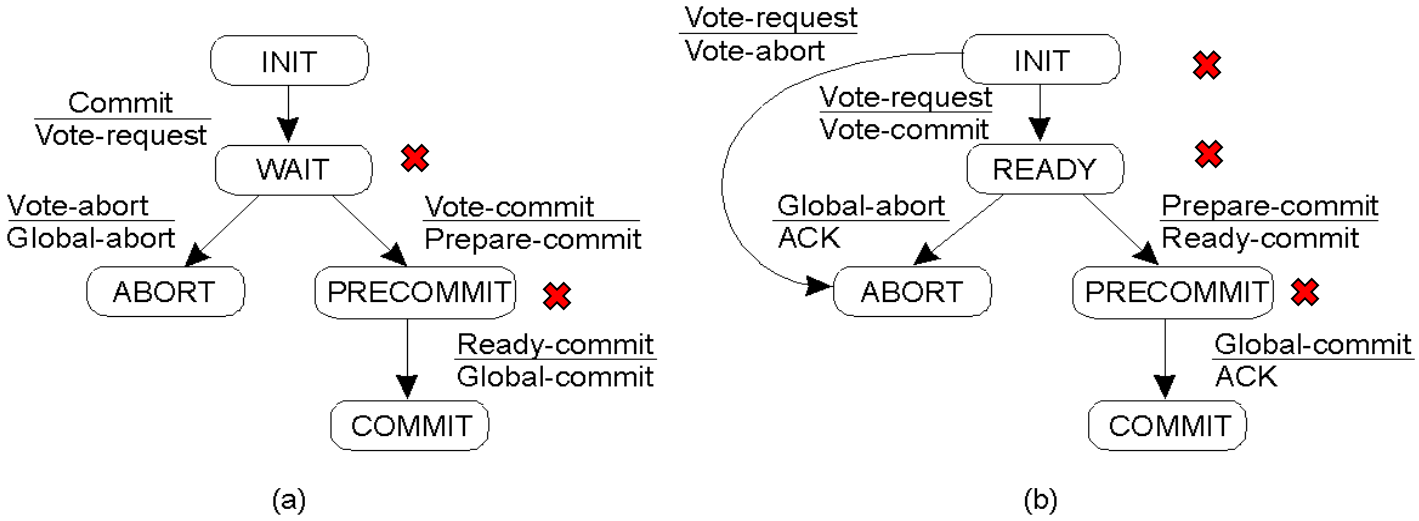


(a)



(b)

a) Masina de stari pentru coordonator in 3PC
b) Masina de stari pentru un participant



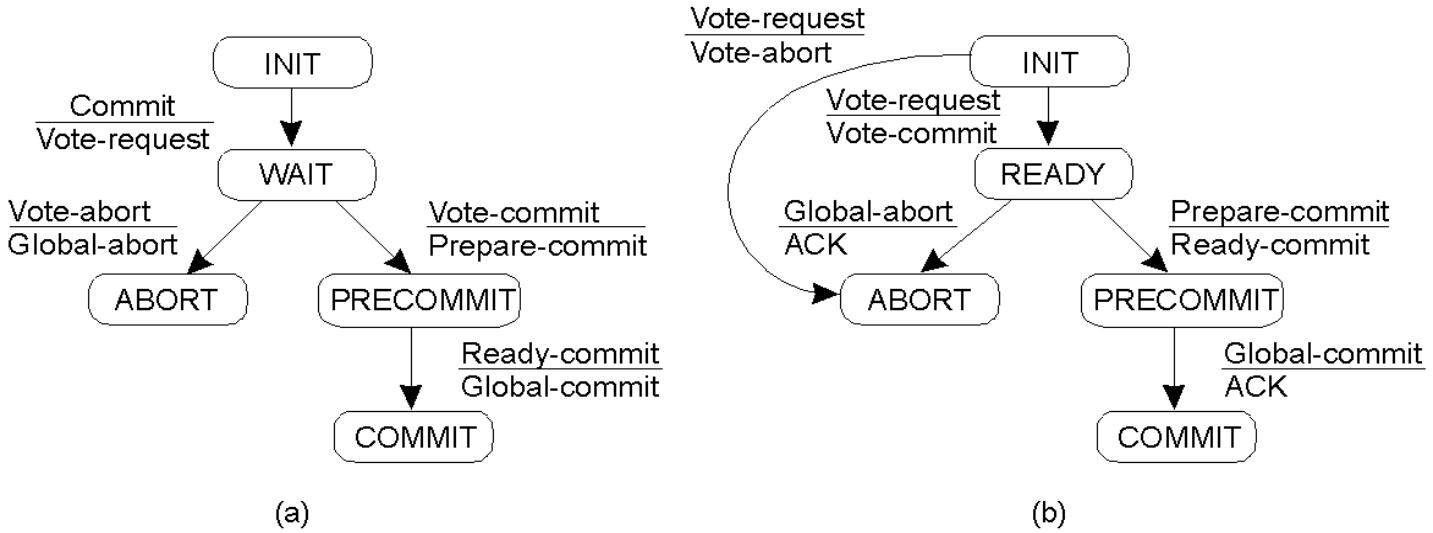
Actiuni la **timeout**

Principiul: pe calea spre COMMIT, coordonatorul si participantii **nu difera prin mai mult de o tranzitie**

Coordonator in WAIT –> la timeout, Global-abort (un participant a cazut)

Coordonator in PRECOMMIT –> Global-commit (un participant a cazut dar el a votat pentru comitere si, la recuperare va comite)

Participant in INIT –> Vote-abort (coordonatorul a cazut)



Actiuni la **timeout**

Participant P in READY sau PRECOMMIT

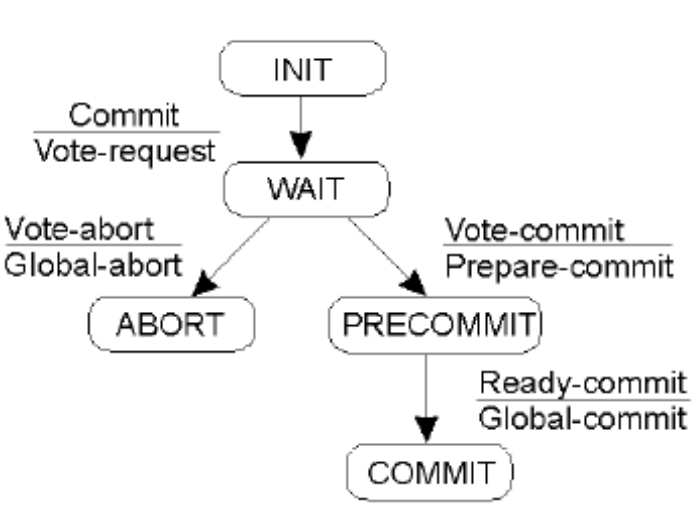
P presupune coord. a cazut → contacteaza alti participanti Q

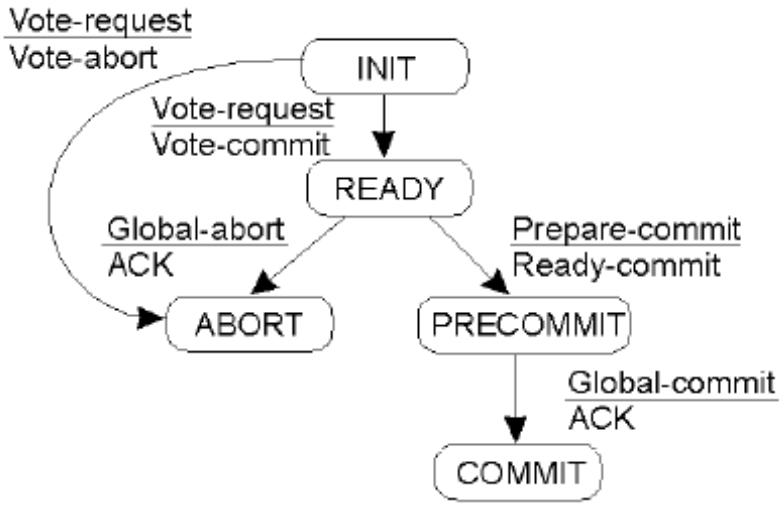
un Q in COMMIT (P in PRECOMMIT) → P trece in COMMIT

un Q in ABORT → P trece in ABORT

toti in PRECOMMIT (inclusiv P) → P trece in COMMIT

un Q in INIT (coord si nici un participant nu sunt in PRECOMMIT) → P aborteaza





Daca doar unii participanti pot fi contactati

- *toti contactatii* sunt in PRECOMMIT si majoritari → P trece in COMMIT
  - un proces cazut va recupera in READY, PRECOMMIT sau COMMIT; procedura de recuperare il va conduce in COMMIT
- *toti contactatii* sunt in READY si formeaza majoritatea → P aborteaza
  - un participant cazut va recupera in INIT, ABORT si va aborta
  - sau in PRECOMMIT (NU in COMMIT) si la recuperare, va aborta, neprimind Global-commit si majoritatea participantilor abortand

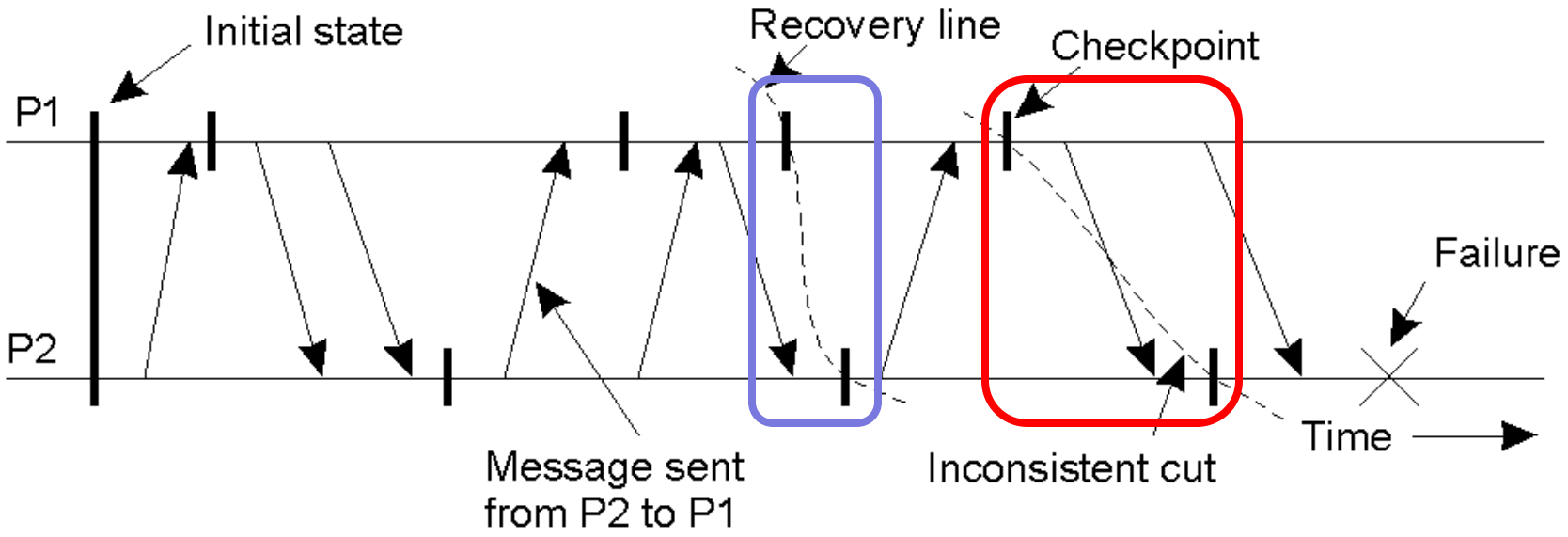
Concluzie: procesele supravietuitoare pot lua intotdeauna o decizie (spre deosebire de two-phase commit)

Recuperarea

- Sistemul este adus intr-o stare corecta
- Tipuri
  - inpoi (backward) – aduce sistemul intr-o stare anterioara
  - inainte (forward) – gaseste o stare noua din care sistemul poate continua operarea
- Recuperare inapoi
  - mai folosita
  - tehnici
    - checkpointing
    - message logging (folosita in combinatie cu checkpointing)
  - Recuperarea este mai complicata in sist distribuite
    - procesele trebuie sa identifice o stare consistenta

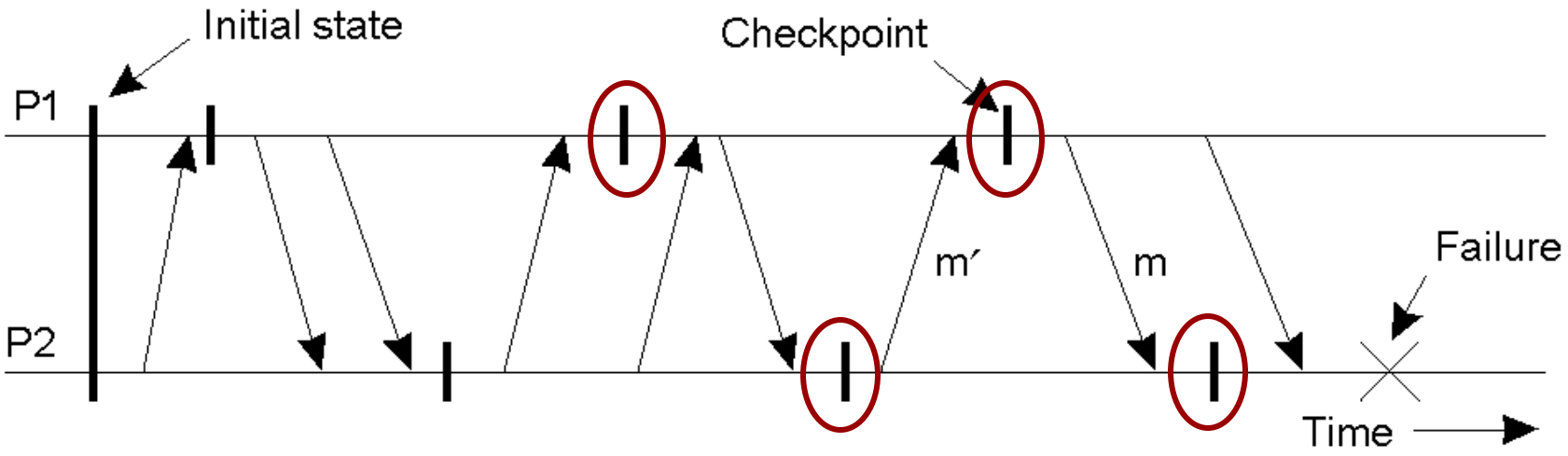
Checkpointing

- **Stare globala consistenta** = instantaneu distribuit
  - constituita din starile proceselor salvate in **memorii stabile** locale
  - constangere dificila pentru **mesaje**
    - daca P1 a inregistrat o receptie de mesaj starea trebuie sa includa o transmitere anterioara intr-un alt proces P2
- Linie de recuperare = cel mai recent instantaneu distribuit
  (cea mai recenta taietura consistenta)



Checkpointing Independent

- Fiecare proces salveaza starea periodic, independent de alte procese
- Linie de recuperare greu de gasit:
  - Efectul de domino

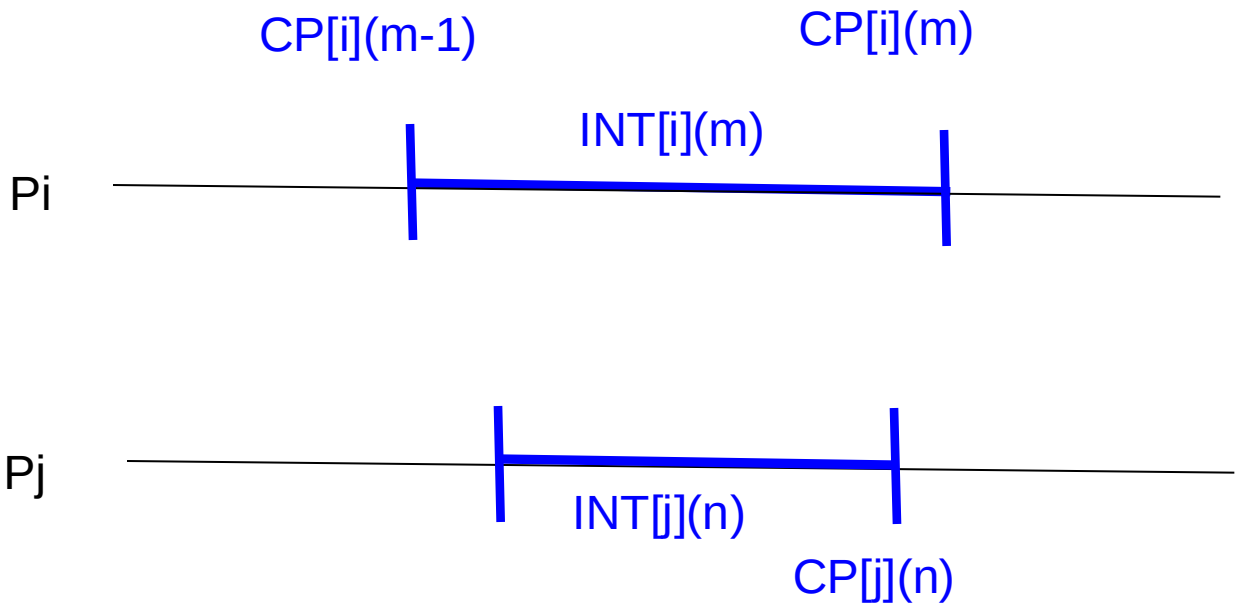


Pentru recuperare se inregistreaza dependentelor intre checkpoint-urile luate de diferite procese

Dependente intre checkpoints

Tine cont de mesajele trimise in intervalele dintre checkpoint-uri:

- *CPi(m)* denota checkpoint *m* al procesului *Pi*
- *INTi(m)* intervalul intre *CPi(m-1) si CPi(m)*.
- Ideea: Inregistreaza, la CP, dependenta intervalelor



Protocolul

Protocol:

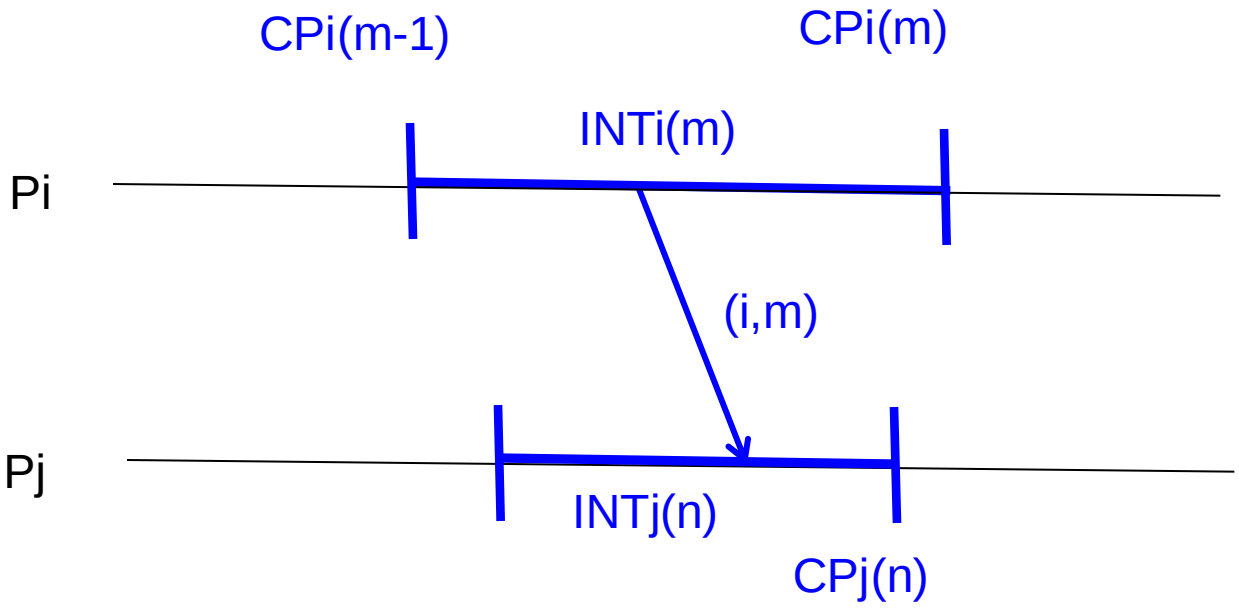
Pi trimite mesaj in *INTi(m)* si adauga (i,m) in mesaj

Pj primeste mesaj in *INTj(n)*

inregistreaza dependenta *INTi(m)* → *INTj(n)*

la checkpoint CPj(n): salveaza dependenta (*INTi(m)* → *INTj(n)*)

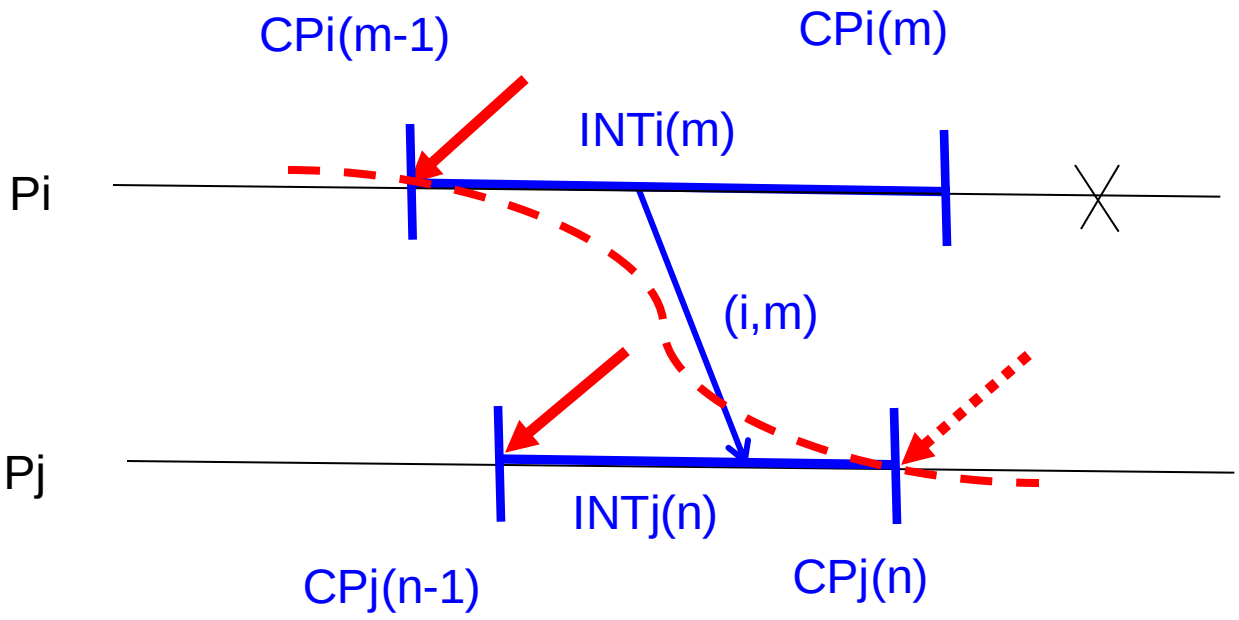
in memoria stabila



Consistenta

Daca *Pi* revine la *CPi(m-1)* => *Pj* revine la un CP anterior primirii mesajelor trimise de Pi in intervalul INTi(m), adica la CPj(n-1)

Daca prin aceasta starile nu sunt consistente, sunt necesarealte reveniri



Checkpointing Coordonat

Solutie simpla: protocol cu blocare in doua faze:

Coordonator: trimite multicast cerere *checkpoint*
Participant: primeste o cerere *checkpoint*
face checkpoint
pune intr-o coada mesajele care ii sunt pasate de aplicatia
pe care o executa, pentru a fi trimise
raporteaza ca a facut checkpoint
Coordonator: asteapta toate confirmarile de checkpoint
difuzeaza *checkpoint done*
Participant: primeste *checkpoint done*
continua

Solutia conduce la o stare globala consistenta:

- mesajele pasate de aplicatii pentru trimitere sunt pastrate in coada pana dupa checkpoint done
- in checkpoint-ul local nu vor fi inregistrate mesaje primite dupa cererea de checkpoint



Logarea Mesajelor

- Salvarea starii proceselor este costisitoare
  - Checkpoint se foloseste la intervale mari de timp
    - toate calculele de la ultimul checkpoint trebuie refacute – nu este convenabil
  - starea consistenta globala se poate reface prin "rejucarea" mesajelor logate de la ultimul checkpoint
- Protocol
  - periodic, fiecare proces salveaza starea sa locala
  - procesul logheaza mesajele pe care le-a primt dupa integistrarea starii
  - cand procesul cade, se creeaza un nou proces in locul lui
  - noului proces i se da starea locala si
  - i se trimit mesajele logate in ordinea originala

Procese orfane

- Cerinta pentru protocoalele de recuperare:
  - dupa recuperare, starea procesului sa fie consistenta cu a celorlalte procese
  - Consistenta = sa nu existe **procese orfane**
  - Def. Un proces R este orfan daca rezista caderii unui alt proces Q dar a carui stare este inconsistenta cu starea lui Q dupa recuperare

- Solutia: model de executie determinist pe bucati
  - executie = secventa de intervale
  - fiecare interval incepe cu un eveniment nedeterminist (ex. receptia unui mesaj)
  - in interval executia este determinista si poate fi rejucata de un proces
- Concluzie: in vederea recuperarii, inregistram doar evenimentele nedeterministe

Cand trebuie inregistrate mesajele?

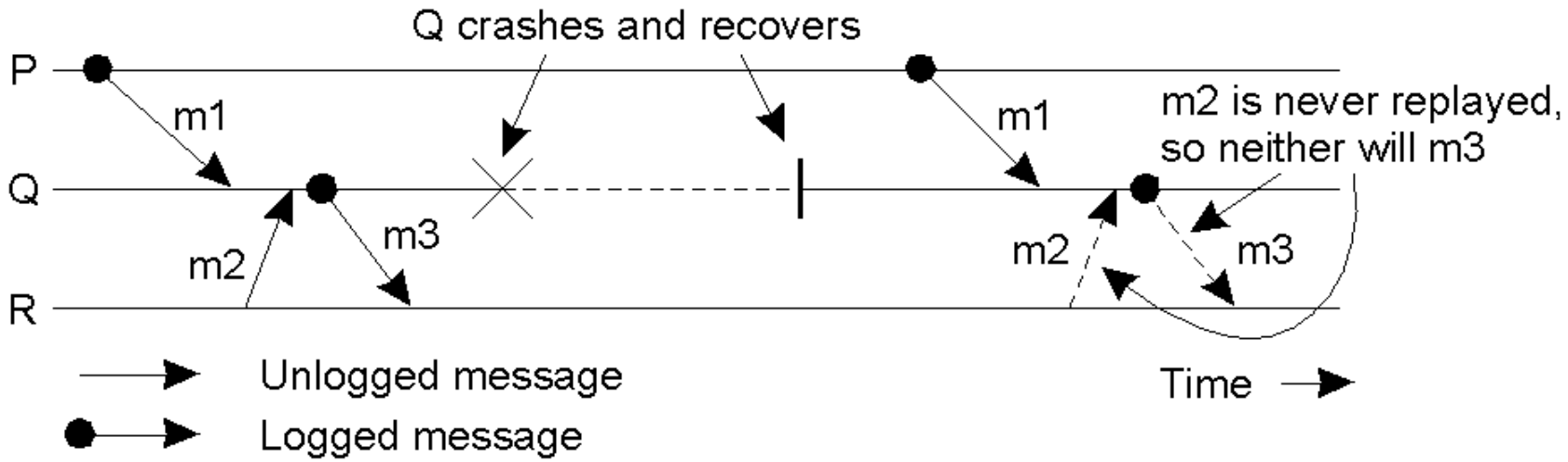
Ex. Q a primit si livrat m1 si m2; m1 este logat; m2 nu este logat

Q transmite m3 lui R; R primeste si livreaza m3

Q cade

La reluare se rejoaca doar mesajele pentru recuperarea lui Q

- Se rejoaca m1 (logat); m2 (nelogat) nu este rejucat ➔ m3 de asemena
  - Pentru R: m3 este primit de la Q;
  - pentru Q recuperat: m3 (ne-rejucat) nu apare ca transmis
- ➔ stari inconsistente pentru Q si R ➔ R devine orfan - are un mesaj care (aparent) nu a fost transmis



Informatii logate

- **HDR[m]**: antet mesaj *m* = *(*sursa, destinatia, numar secventa, numar livrare)
  - Antetul contine toate info necesare retransmiterii mesajului si livrarii sale in ordinea corecta;
  - datele sunt reproduse de aplicatie
- Mesaj *m* este **stabil** cand nu mai poate fi pierdut (a fost pus in memoria stabila).
  - poate fi "rejucat" pentru recuperare.
- **DEP[m]**: multimea proceselor la care s-a livrat *m* si proceselor la care s-a livrat *m'* dependent cauzal de livrarea lui *m*
- **COPY[m]**: procesele care au o copie a lui *m* in memoria volatila (inca nu in memoria stabila locala).
  - Cand Q livreaza m el devine membru al lui COPY[m].
  - Un proces din COPY[m] poate oferi o copie a lui m pentru a fi utilizata in rejucarea transmisiei lui m (daca procesul nu a cazut!)
- ***C*** colectia proceselor cazute

Scheme de logging (2)

Ipoteza: intr-un sistem distribuit, unele procese s-au defectat, dar R supravietuieste defectelor

R este **orfan** daca este dependent de un mesaj **m** si nu se poate rejuca transmisia lui **m**

adica – R este in DEP[m] si COPY[*m*] este inclus in C.

Problema: evitare orfani

Daca procesele din COPY(m) au cazut, sa nu existe procese in DEP(m)

Se poate asigura daca, atunci cand un proces devine dependent de **m**, el sa aiba o copie a lui **m**

altfel spus, daca procesul devine membru al lui DEP[m] atunci devine membru si al lui COPY[m]

Solutii

Protocol pesimist: pentru fiecare mesaj *non-stabil m*, exista cel mult un proces dependent de *m*, adica |DEP[m]|<=1

Consecinta: daca P primeste *m*, el il face stabil (scrie in memoria stabila) inainte de a trimite mesajul urmator

Protocol optimist: pentru fiecare mesaj non-stabil *m*, daca COPY[m] este inclus in C, atunci DEP[m] este inclus de asemena in C

Consecinta: actioneaza dupa o "cadere" - fiecare proces orfan R este intors la o stare in care R nu este in DEP[m].

complicat de implementat

Studiu de caz: Dynamo (*)

Modul de operare

- infrastructura Amazon cuprinde milioane de componente
- in orice moment numarul de componente defecte este semnificativ
- tolerarea defectelor este modul normal de operare

Solutia Dynamo – depozit de date cu disponibilitate ridicata

- model simplu de date, cheie-valoare
- replicarea datelor conform model eventual consistency
  - propagarea schimbarilor la replici se face in background
  - suporta deconectari de retea si actualizari concurente ale unor replici diferite
    - creste disponibilitatea la scriere a depozitelor de date (always writeable)
  - permite tratarea conflictelor de catre aplicatie

(*) G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall and W. Vogels. Dynamo: Amazon's Highly Available Key-value Store

Topici discutate

- Interfata depozitului de date
- Partitionarea datelor
- Replicarea
- Versionarea datelor
- Executia operatiilor
- Tratarea defectelor tranzitorii
- Tratarea defectelor permanente

Interfata

Interfata are doua operatii

- put (key, context, object)
  - determina nodurile (virtuale) unde trebuie stocate replicile obiectului si scrie in depozit obiectul si contextul
  - nodurile sunt gasite printr-un algoritm de complexitate logaritmica
  - contextul include, printre altele, versiunea obiectului
- get (key)
  - localizeaza replicile (atentie, diferit de modelul clasic de citire a unei singure replici!) asociate cu key si intoarce un singur obiect sau o lista de obiecte conflictuale plus un context

Partitionarea datelor

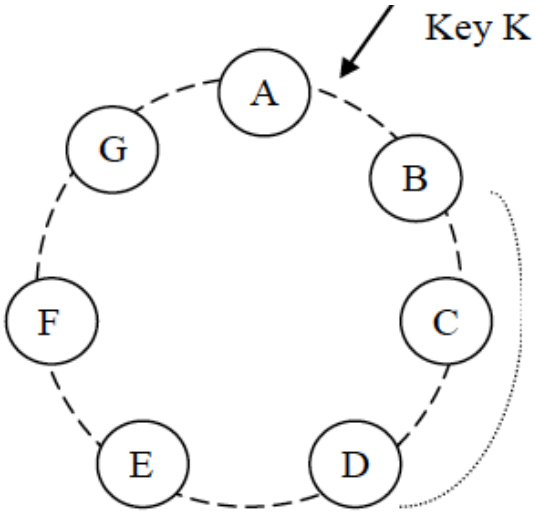
Depozitul de date consta din mai multe noduri fizice, fiecare incluzand mai multe noduri virtuale

Datele sunt repartizate unui set de noduri virtuale (de stocare)

- foloseste o schema de hashing consistent cu spatiul valorilor organizat logic in inel
- fiecare nod virtual este identificat printr-o valoare din acest spatiu, reprezentand pozitia nodului pe inel
- fiecare data este identificata printr-o cheie; hash-ul acestei chei reprezinta pozitia datei pe inel
- data este stocata in nodul cu

  cea mai mica pozitie >= pozitia datei,

  numit nod coordonator al cheii

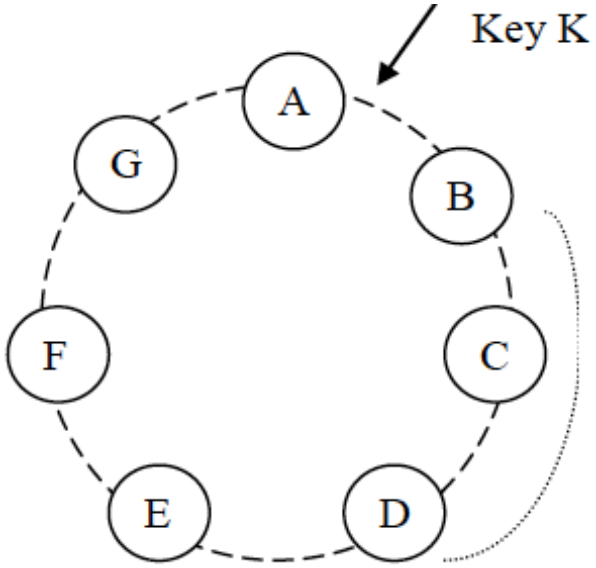


In figura, nodul virtual B depoziteaza cheile K din domeniul (A,B]

Replicarea

- o data este replicata in N noduri virtuale (N configurabil)
- schema: fiecare cheie este asignata coordonatorului care memoreaza datele corespunzatoare cheii si face replicarea lor la alte N-1 noduri virtuale; de ex.
  - daca N = 3, nodul coordonator B face replicarea cheii K la C si D
  - (B, C, D) alcatuiesc lista de preferinte a cheii K
- pentru toleranta la defecte
  - lista de preferinte include peste N noduri
  - nodurile virtuale trebuie sa fie situate

  in noduri fizice distincte (se “sar” nodurile virtuale care nu respecta conditia)



Versionarea

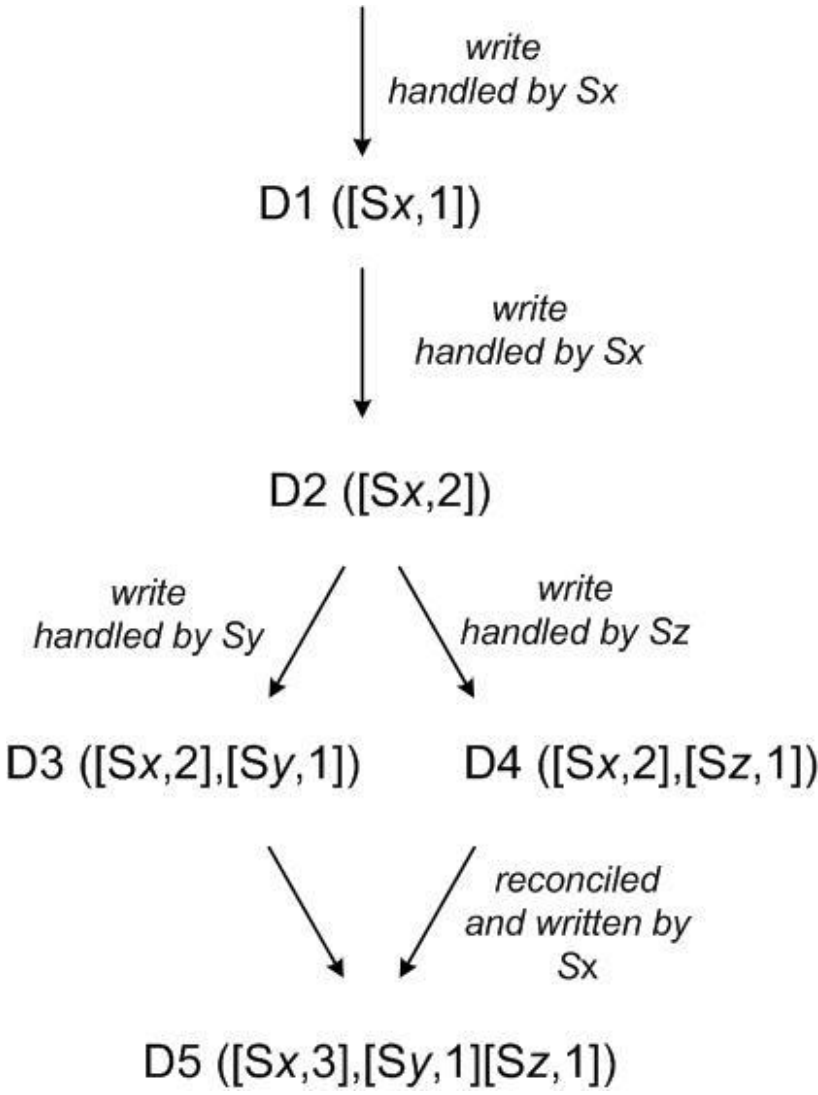
- Dynamo foloseste “eventual consistency” in care actualizarile sunt propagate asincron replicilor
  - o operatie **put** reda controlul apelantului inainte ca actualizarea sa se fi aplicat tuturor replicilor
  - consecinta: o operatie **get** ulterioara poate returna un obiect care nu are ultimele actualizari
- Ex. Amazon cere ca orice operatie “add to Cart” sa nu fie uitata sau rejectata
  - daca cea mai recenta replica a cosului nu este disponibila, operatia se face pe o replica mai veche → pot aparea conflicte intre replici
  - reconcilierea urmeaza sa se faca la o citire (get) ulterioara
- Amazon trateaza rezultatul fiecarei modificari (put) a unei date ca o noua versiune

Vectori de timp

- Pentru a reprezenta versiunile, se folosesc vectori de timp
  - vector de timp = lista de perechi [nod, contor]
    - nodul care a generat versiunea
    - numarul versiunii (local nodului)
  - la o scriere, clientul specifica versiunea pe care o actualizeaza, pe care o cunoaste din contextul primit la o citire anterioara
    - obiectul actualizat capata o noua versiune
- versiuni dependente cauzal
  - daca vectorul de timp al versiunii unui obiect, A1 este mai mic sau egal cu cel al versiunii A2 atunci A1 este un stramos al lui A2 si poate fi ignorat
  - altfel, versiunile se considera in conflict si trebuie reconciliate

Un exemplu

- versiunea D1 produsa de nodul Sx este stramos al versiunii D2 (a aceluiasi obiect)
- scrieri concurente conduc la doua versiuni diferite D3 si D4, ambele dependente de D2 dar conflictuale (sunt pe ramuri diferite ale arborelui de dependente)
- la citire, ambele versiuni sunt oferite ca rezultat
- clientul reconciliaza D3 si D4 (coordonat de Sx) si produce o singura versiunea D5 (reducand cele doua ramuri la una singura)



Executia operatiilor get si put

Oricare nod este eligibil pentru a primi de la client invocarile operatiilor get si put pentru orice cheie

De regula, o invocare este gestionata de nodul coordonator pentru cheia k specificata in invocare

In cazul in care coordonatorul este inaccesibil, (partitionari ale retelei) invocarea (get sau put) este pasata unui alt nod din lista de preferinte

Executia operatiei invocate implica primele N noduri din lista de preferinte (ocolind nodurile defecte sau inaccesibile)

Consistenta operatiilor get si put

Pentru a pastra consistenta replicilor, executia operatiilor put si get se face conform modelului de cvorum

- operatia presupune ca
  - minimum N_W noduri participa la operatia de scriere (put) sau
  - minimum N_R noduri participa la operatia de citire (get)
  - cu conditia suplimentara $N_R + N_W > N$

La put (key, context, object), coordonatorul (sau inlocuitorul)

  - genereaza vectorul de timp al noii versiuni si scrie local noua versiune
  - trimite versiunea catre primele N noduri tangibile din lista de preferinte
  - operatia se termina cand raspund N_W-1 noduri

Consistenta operatiilor get si put (2)

La get (key), coordonatorul

- cere toate versiunile de date pentru key de la primele N noduri tangibile din lista de preferinte
- asteapta N_R-1 raspunsuri pentru a trimite rezultatul la client
- trimite toate versiunile necorelate cauzal
- versiunile divergente sunt reconciliate
- versiunea rezultata din reconciliere inlocuieste versiunile curente in depozit

Tratarea defectelor tranzitorii

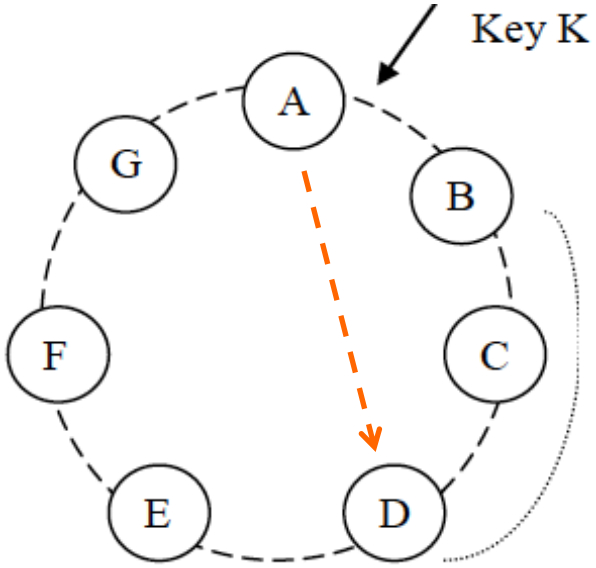
Abordarea cvorum nu functioneaza la defectarea nodurilor sau la partitionarea retelei

Solutia: model cvorum “relaxat”

- operatiile se executa pe primele N noduri “sanatoase” din lista de preferinte
- pentru N=3 si nod A cazut temporar, o replica destinata lui A este trimisa lui D
- D asociaza un hint – “locul replicii este A”
- pastreaza replica intr-o BD separata
- cand A revine, D ii transmite replica

Pentru defecte la nivel data center

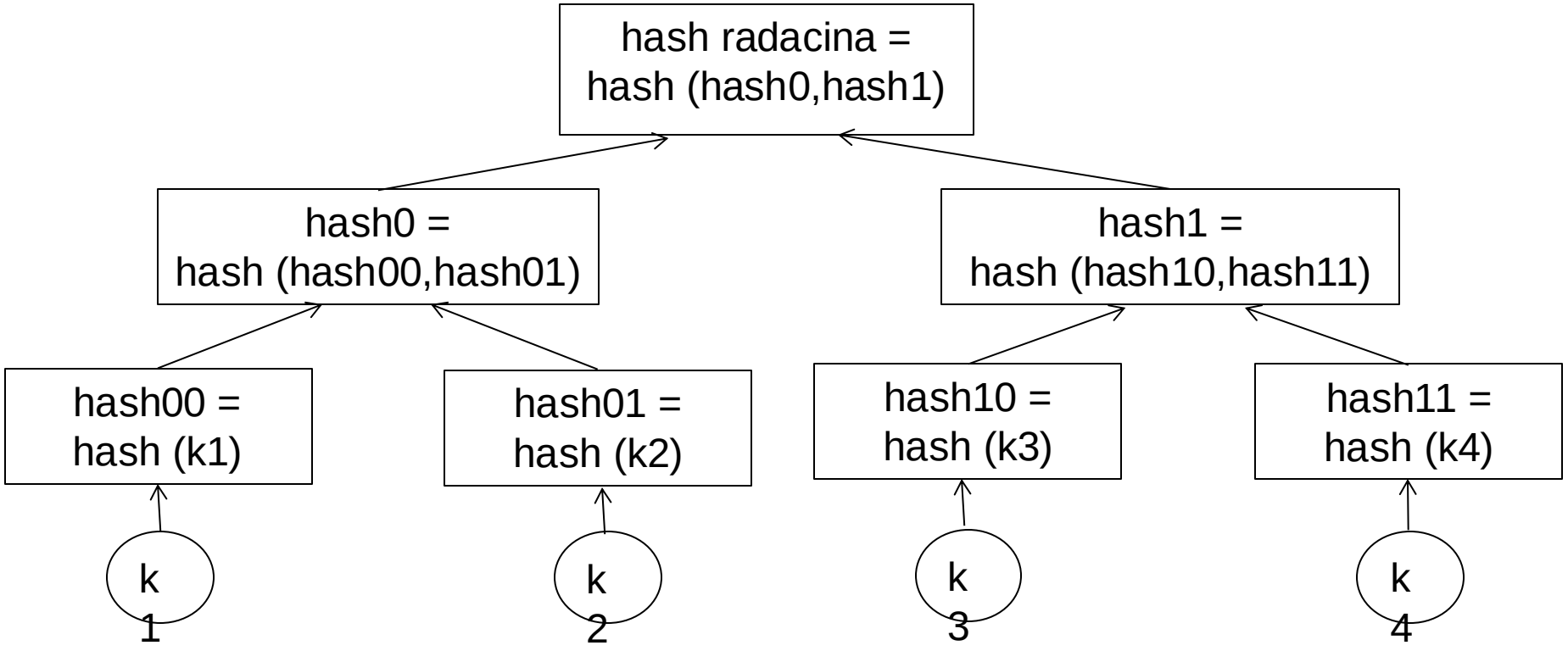
- lista de preferinte include noduri din diferite data centere
- data centere sunt conectate prin legaturi de mare capacitate



Tratarea defectarilor permanente

Protocol de sincronizare descentralizata a replicilor bazat pe Merkle trees

- frunzele sunt hash-uri de chei
- celelalte noduri sunt hash-uri ale fiilor



Algoritm anti-entropie pentru sincronizarea replicilor

Fiecare nod pastreaza un arbore Merkle separat pentru fiecare set de chei acoperite de un nod virtual

1. doua noduri schimba intre ele hash-urile din radacina arborilor Merkle corespunzatoare unui set de chei comune
2. daca nu sunt diferite → STOP (nu au nevoie de sincronizare)
3. daca sunt diferite → listele hash de la nivelul inferior sunt schimbate intre noduri
4. pentru fiecare pereche de hash-uri din cele doua liste ale nodurilor se repeta operatiile 2 si 3
5. operatiile continua pana se ajunge la frunzele arborelui Merkle, cand se identifica cheile desincronizate